

ZYNQ 开发平台 Linux 驱动教程

2024/2/28 10:31:13



版权声明

Copyright ©2012-2019 芯驿电子科技（上海）有限公司

公司网址:

[Http://www.alinx.com.cn](http://www.alinx.com.cn)

技术论坛:

<http://www.heijin.org>

官方旗舰店:

<http://alinx.jd.com>

邮箱:

avic@alinx.com.cn

电话:

021-67676997

传真:

021-37737073

ALINX 微信公众号:



我们承诺本教程并非一劳永逸，固守不变的文档。我们会根据论坛上大家的反馈意见，以及实际的开发实践经验积累不断的修正和优化教程

文档修订记录:

版本	时间	描述
1.01	2024/1/12	2017v1.06 版本更新至 2023v1.01 版本

前言

首先感谢大家购买芯驿电子科技（上海）有限公司出品的 ZYNQ 的开发板！您对我们和我们产品的支持和信任,给我们增添了永往直前的信心和勇气。

“播下一粒种子，收获一片森林”，更是芯驿电子科技（上海）有限公司的美好愿望，同时我们会在黑金动力社区 <http://www.heijin.org>和大家一起讨论，一起学习，一起进步，一起成长。

通过前面的教程我们熟悉了 ZYNQ 开发的基本流程，学习了 Vitis 下 ZYNQ 程序的开发，但是很多应用可能需要使用嵌入式 Linux，由于 Linux 系统庞大，芯驿电子科技（上海）有限公司只能抛砖引玉，把最常用的知识点收集分享给大家。

目录

版权声明	2
前言	4
目录	5
第一章 字符设备	12
1.1 Linux 驱动程序概述	12
1.1.1 Linux 应用程序与驱动程序的关系	12
1.1.2 Linux 驱动程序的分类	12
1.2 字符设备的开发步骤	13
1.2.1 实现系统调用	13
1.2.2 驱动模块的加载与卸载	14
1.2.3 字符设备的注册与注销	14
1.2.4 Linux 设备号	15
1.2.5 实现设备操作函数	15
1.2.6 添加驱动描述信息	15
1.3 字符设备驱动开发实验	16
1.3.1 查看硬件原理图以及数据手册	16
1.3.2 编写字符设备驱动程序	17
1.3.3 在 Petalinux 定制系统中添加新的驱动	20
1.3.4 使用 Ubuntu 虚拟机中的 QT 编写测试 APP	22
1.3.5 运行测试	27
第二章 字符设备的新写法	30
2.1 设备号管理	30
2.2 新的注册方法	31
2.1.1 字符设备数据结构	31
2.1.2 cdev 结构体初始化	31
2.1.3 新的注册和注销函数	31
2.3 自动创建设备文件	32
2.3.1 mdev	32
2.3.2 类的创建和删除	32
2.3.3 设备节点的创建和删除	33
2.4 字符设备新驱动实验	34
2.4.1 查看硬件原理图以及数据手册	34
2.4.2 编写字符设备驱动程序	34
2.4.3 编写测试 APP	37
2.4.5 运行测试	37
第三章 设备树和 of 函数	39

3.1 设备树	39
3.1.1 DTS、DTB 和 DTSI	39
3.1.2 节点	40
3.1.3 属性	40
3.1.4 在文件系统中查看设备树	42
3.1.5 修改设备树	42
3.2 of 函数	42
3.2.1 查找节点的 of 函数	43
3.2.2 提取属性的 of 函数	43
3.3 设备树下的 led 驱动实验	45
3.3.1 原理图	45
3.3.2 修改设备树	45
3.3.3 驱动程序	45
3.3.4 测试程序	48
3.3.5 运行测试	48
第四章 pinctrl 和 gpio 子系统	50
4.1 pinctrl 子系统	50
4.1.1 zynq 平台 pinctrl 子系统的设备树节点和属性	50
4.1.2 pinctrl 子系统的使用	52
4.2 gpio 子系统	52
4.2.1 设备树中的 gpio	53
4.2.2 gpio 接口函数	53
4.3 实验	54
4.3.1 原理图	54
4.3.2 设备树	54
4.3.3 驱动代码	55
4.3.4 测试代码	58
4.3.5 运行测试	58
第五章 并发的处理	60
5.1 Linux 中的并发	60
5.2 Linux 对并发的处理	60
5.2.1 原子操作	60
5.2.2 锁机制	61
5.2.3 信号量	62
5.3 实验	63
5.3.1 原理图	63
5.3.2 设备树	63
5.3.3 驱动代码	63

5.3.4 测试代码	68
5.4.5 运行测试	69
第六章 gpio 输入	71
6.1 原理图	71
6.2 设备树	71
6.3 驱动代码	72
6.4 测试代码	75
6.5 运行测试	76
第七章 定时器	78
7.1 Linux 内核中的定时器	78
7.1.1 系统频率	78
7.1.2 节拍数	79
7.1.3 内核定时器及相关函数	80
7.2 实验	81
7.2.1 原理图	81
7.2.2 设备树	81
7.2.3 驱动程序	81
7.2.4 测试程序	84
7.3.5 运行测试	85
第八章 中断	87
8.1 Linux 中断框架简介	87
8.1.1 接口函数	87
8.1.2 Linux 的下半部机制	88
8.1.3 设备树中的中断	91
8.2 实验	92
8.2.1 原理图	92
8.2.2 设备树	92
8.2.3 驱动程序	92
8.2.4 测试程序	96
8.2.5 运行测试	96
第九章 阻塞 IO	97
9.1 阻塞和非阻塞、同步和异步与 IO 操作	97
9.2 阻塞 IO	97
9.2.1 等待队列	97
9.3 实验	99
9.3.1 原理图	99
9.3.2 设备树	99
9.3.3 驱动程序	99

9.3.4 测试程序	104
9.3.5 运行测试	104
第十章 非阻塞 IO	106
10.1 Linux 中的 NIO	106
10.1.1 应用程序中的轮询方法	106
10.1.2 驱动程序中的 poll 函数	109
10.2 实验	109
10.2.1 原理图	109
10.2.2 设备树	109
10.2.3 驱动代码	109
10.2.4 测试代码	114
10.2.5 运行测试	115
第十一章 异步 IO	117
11.1 Linux 中的异步 IO	117
11.1.1 信号	117
11.1.2 应用程序中信号的使用	117
11.1.3 驱动程序中信号的实现	118
11.2 实验	118
11.2.1 原理图	118
11.2.2 设备树	119
11.2.3 驱动代码	119
11.2.4 测试代码	124
11.2.5 运行测试	125
第十二章 platform 平台设备	127
12.1 驱动分离	127
12.2 platform 模型	128
12.2.1 platform_bus	128
12.2.2 platform_driver	129
12.2.3 platform_device	130
12.3 实验	131
12.3.1 原理图	131
12.3.2 设备树	131
12.3.3 驱动程序	131
12.3.4 测试程序	136
12.3.5 运行测试	136
第十三章 platform 与设备树	138
13.1 设备树下的 platform	138
13.2 实验	138
13.2.1 原理图	139

13.2.2 设备树	139
13.2.3 驱动程序	139
13.2.4 测试程序	143
13.2.5 运行测试	143
第十四章 MISC 设备驱动	145
14.1 MISC 设备简介	145
14.2 MISC 框架的使用	145
14.3 实验	146
14.3.1 原理图	146
14.3.2 设备树	146
14.3.3 驱动代码	146
14.3.4 测试代码	151
14.3.5 运行测试	151
第十五章 input 子系统	152
15.1 input 子系统简介	152
15.1.1 input 子系统驱动框架的使用	152
15.1.2 input 子系统应用程序的使用	155
15.3 实验	155
15.3.1 原理图	155
15.3.2 设备树	155
15.3.3 驱动程序	155
15.3.4 测试程序	159
15.3.5 运行测试	161
第十六章 pwm 驱动	162
16.1 zynq 上的 pwm 实现	162
16.1.1 修改 vivado 工程	162
16.1.2 控制 pwm 输出的方法	163
16.2 Linux 中的 pwm 框架	165
16.2.1 pwm_device	165
16.2.1 pwm_chip	166
16.2.1 示例	167
16.3 实验	169
16.3.1 原理图	169
16.3.2 设备树	169
16.3.3 驱动程序	170
16.3.4 测试程序	173
16.3.5 运行测试	174
第十七章 I2C 驱动	175
17.1 I2C 驱动框架	175

17.1.1 I2C 控制器驱动	175
17.1.2 I2C 设备驱动	176
17.1.3 I2C 设备驱动的实现流程	178
17.3 实验	180
17.3.1 原理图	180
17.3.2 设备树	183
17.3.3 驱动程序	184
17.3.4 测试程序	188
17.3.5 运行测试	189
第十八章 USB 驱动	191
18.1 usb 识别过程	191
18.2 usb 的传输	191
18.2.1 usb 的传输模式	191
18.2.2 端点	192
18.2.3 管道	192
18.3 usb 总线驱动	192
18.4 usb 设备驱动	193
18.5 urb 请求块	194
18.6 实验	196
18.6.1 原理图	196
18.6.2 设备树	196
18.6.3 驱动程序	196
18.6.4 运行测试	199
第十九章 SPI 驱动	203
19.1 SPI 控制器驱动	203
19.2 SPI 设备驱动	203
19.3 SPI 设备驱动中数据收发处理	206
19.4 实验	207
19.4.1 原理图	207
19.4.2 设备树	207
19.4.3 驱动程序	207
19.4.4 测试程序	212
19.4.5 运行测试	212
第二十章 串口驱动	216
20.1 uart 驱动框架	216
20.2 xilinx 的 uart 驱动	218
第二十一章 块设备驱动	223
21.1 块设备简介	223

21.2 内核中的块设备访问	223
21.3 块设备框架	225
21.3.1 gendisk 结构体	226
21.3.2 request_queue	227
21.4 实验	227
21.1.1 驱动程序	227
21.1.2 运行测试	230
第二十二章 网卡驱动	233
22.1 Linux 网卡驱动	233
22.1.1 初始化	233
22.1.2 发包	235
22.1.3 收包	237
22.2 实验	237
21.1.1 驱动程序	237
21.1.2 运行测试	239
第二十三章 DMA 驱动	241
23.1 DMA 简介	241
23.2 DMAC	241
23.2.1 DMA channels	241
23.2.2 DMA request line	241
23.2.3 transfer size、transfer wide、burst size	241
23.2.4 scatter-gather	241
23.3 PL330	242
23.4 linux 中的 DMA	252
23.4.1 DMA controller 驱动框架	252
23.4.2 DMA client 驱动框架	257
第二十四章 多点触摸屏驱动	262
24.1 input 子系统下的多点触摸屏	262
24.2 多点触摸屏与 SoC 的接口	263
24.3 vivado 工程与 petalinux	263
24.4 原理图	264
24.5 设备树	264
24.6 多点触摸屏驱动程序	265
第二十五章 LCD 驱动	277
25.1 framebuffer 框架	277
25.2 VDMA	278
25.3 DRM 框架	280

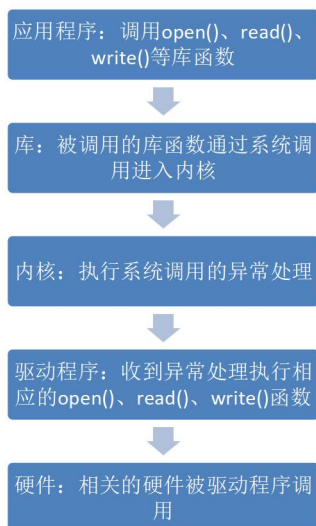
第一章 字符设备

1.1 Linux 驱动程序概述

在动手写程序之前，先了解一下基本的 Linux 设备及驱动程序相关的知识。

1.1.1 Linux 应用程序与驱动程序的关系

Linux 应用程序调用驱动程序的步骤大致如下：



- 1) 应用程序调用库函数提供的 `open()` 函数打开某个设备文件；
- 2) 库根据 `open()` 函数的输入参数引起 CPU 异常，进入内核；
- 3) 内核的异常处理函数根据输入参数找到相应的驱动程序，返回文件句柄给库，库函数再返回给应用程序；
- 4) 应用程序再使用得到的文件句柄调用 `write()`、`read()` 等函数发出控制指令；
- 5) 库根据 `write()`、`read()` 等函数的输入参数引起 CPU 异常，进入内核；
- 6) 内核的异常处理函数根据输入参数调用相应的驱动程序执行相应的操作。

步骤一中 `open()` 函数打开的设备文件，是在驱动加载成功后再目录 `/dev` 中生成的相关文件，是应用程序调用对应硬件的入口。

在这个过程中应用程序涉及到的 `open()`、`read()`、`write()` 等由库提供的接口函数成为系统调用，他们通过执行某条指令引发异常进入内核，是应用程序操作硬件沟通的途径。应用程序执行系统调用后，最终会使用驱动函数中对应的函数，而驱动函数中的 `open()`、`read()`、`write()` 等函数则需要驱动开发人员去实现。

应用程序运行于用户空间，驱动程序运行于内核空间。Linux 系统可以通过 MMU 限制应用程序运行在某个内存块中，以避免这个应用程序出错导致整个系统崩溃。而运行于内核空间的驱动程序是系统的一部分，驱动程序出错有可能牵连整个系统，因此在实现驱动时要格外小心。

1.1.2 Linux 驱动程序的分类

驱动可分为三类：字符设备驱动、块设备驱动、网络设备驱动。

字符设备即是按字节访问的设备。比如以一个字节的收发数据的串口，就是典型的字符设

备。还有较为简单简单的我们接下来会讲的 LED 灯驱动，相对复杂的如音频都属于字符设备，字符设备在 Linux 外设中占比最大。

块设备的特点是他们是按一定格式存取数据，具体的格式有文件系统决定。他们通常以存储设备为主如 EMMC、FLASH、SD 卡、EEPROM 等。存储设备的特点是以存储块为基础，因此得名块设备。

对于应用程序来说，块设备和字符设备访问方式基本无差，即驱动程序实现系统调用，应用程序通过设备文件（比如/dev/mtd0、/dev/i2c-0 等）来调用对应的库函数。当然这需要驱动程序开发者来保证，即再实现块设备驱动时将数据按一定格式组织成块后再进行读写。另外，块设备还需要向内核其他部件提供一些接口，使得块设备上可以存放文件系统挂接块设备。因此块设备驱动的实现相比字符设备驱动要复杂得多。

网络设备比较特殊，应用程序和网络设备驱动间的通信与上面两种设备完全不同，由库和内核提供的一套数据包传输函数替代了 open()、read()、write()等函数。但是网络设备也很好理解和辨认，WIFI 或者有线接口都属于网络设备。

设备驱动中还会出现一个驱动包含多种设备驱动的情况，如 SPI（字符设备）连接的 FLASH（块设备）、USB 接口（字符设备）的 WIFI（网络设备）。接下来我们会循序渐进的学习各种设备的驱动，由浅入深，先从最简单的字符设备开始。

1.2 字符设备的开发步骤

Linux 驱动程序是有框架的，现有框架方便我们增加新的驱动程序，熟悉现有框架之后，在现有驱动程序上稍微修改就能得到新设备的驱动，而学习现有框架就是我们的目标之一。

- 1) 熟悉框架之后，实现设备驱动的步骤大致如下：
- 2) 查看原理图以及数据手册了解设备操作方法；
- 3) 修改设备树文件；
- 4) 套用与设备相近的框架，或找到内核中相似设备的驱动代码直接修改，实现驱动程序的初始化以及操作函数；
- 5) 将驱动编译进内核或单独编译加载驱动；
- 6) 编写应用程序测试驱动程序。

不难看出上述步骤中重点以及难点是步骤③，下面我们细说字符设备驱动中步骤③中需要实现的内容。

1.2.1 实现系统调用

系统调用即设备操作函数，是字符设备驱动的核心。在内核文件 include/linux/fs.h 中定义了数据结构 file_operations，涵括了所有内核驱动操作函数，内容如下：

```
1. struct file_operations {
2.     struct module *owner;
3.     loff_t (*llseek) (struct file *, loff_t, int);
4.     ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
5.     ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
6.     ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
7.     ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
8.     int (*iterate) (struct file *, struct dir_context *);
9.     int (*iterate_shared) (struct file *, struct dir_context *);
10.    unsigned int (*poll) (struct file *, struct poll_table_struct *);
```

```

11.     long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
12.     long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
13.     int (*mmap) (struct file *, struct vm_area_struct *);
14.     int (*open) (struct inode *, struct file *);
15.     int (*flush) (struct file *, fl_owner_t id);
16.     int (*release) (struct inode *, struct file *);
17.     int (*fsync) (struct file *, loff_t, loff_t, int datasync);
18.     int (*fasync) (int, struct file *, int);
19.     int (*lock) (struct file *, int, struct file_lock *);
20.     ssize_t (*sendpage) (struct file *, struct page *, int, size_t, loff_t *, int);
21.     unsigned long (*get_unmapped_area)(struct file *, unsigned long, unsigned long, unsigned
    long, unsigned long);
22.     int (*check_flags)(int);
23.     int (*flock) (struct file *, int, struct file_lock *);
24.     ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t *, size_t, unsig
    ned int);
25.     ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info *, size_t, unsig
    ned int);
26.     int (*setlease)(struct file *, long, struct file_lock **, void **);
27.     long (*fallocate)(struct file *file, int mode, loff_t offset, loff_t len);
28.     void (*show_fdinfo)(struct seq_file *m, struct file *f);
29.     #ifndef CONFIG_MMU
30.     unsigned (*mmap_capabilities)(struct file *);
31.     #endif
32.     ssize_t (*copy_file_range)(struct file *, loff_t, struct file *, loff_t, size_t, unsig
    ned int);
33.     int (*clone_file_range)(struct file *, loff_t, struct file *, loff_t, u64);
34.     ssize_t (*dedupe_file_range)(struct file *, u64, u64, struct file *, u64);
35. }

```

简单介绍一下接下来的实验中会用到的成员和函数：

- 1) 成员变量 **owner** 一般设置为 **THIS_MODULE**。
- 2) **open()** 函数已经提到多次，用于打开设备文件。
- 3) **write()** 函数用于向设备写入或发送数据。
- 4) **read()** 函数用于从设备读取数据。
- 5) **release()** 函数用于关闭设备文件，与应用程序中的 **close()** 函数相对应。

1.2.2 驱动模块的加载与卸载

驱动程序完成后，可以选择编译进内核，或单独编译后使用命令 **insmod** 加载到系统中。后者对于驱动开发而言方便很多。驱动程序单独编译后，会得到后缀为 **.ko** 的驱动模块文件，比如加载名为 **led.ko** 的驱动模块，就使用命令 **insmod led.ko** 即可。当然有加载就有卸载，卸载命令为 **rmmod**，卸载驱动预加载驱动稍有区别，不是 **rmmod led.ko** 了。可以先使用 **lsmod** 命令来查看现已加载的驱动，再使用 **rmmod** 命令加空格再加上 **lsmod** 命令罗列出的你想卸载的驱动名称即可。

在实现驱动时，需要注册这两个操作函数，注册函数为：

```
module_init(xxx_init);    //注册模块加载函数
```

```
module_exit(xxx_exit);    //注册模块卸载函数
```

xxx_init 是驱动入口函数，**xxx_exit** 是驱动出口函数。当执行 **insmod** 命令时，**xxx_init** 函数就会被调用。**xxx_exit** 函数同理。这两个函数原型也很简单，到后面编码时再说。

1.2.3 字符设备的注册与注销

驱动入口函数是驱动加载时执行的第一个函数，我们需要抓住这个机会在入口函数中实现驱动的初始化即注册字符设备。注册和注销字符设备的函数分别为：

```
int register_chrdev(unsigned int major, const char *name, const struct file_operations
    *fops);
void unregister_chrdev(unsigned int major, const char *name);
```

两个函数总共涉及到三个输入参数：

major：主设备号，linux 系统下每个设备都有一个设备号，主设备下还有子设备号 minor；

name：设备名；

fops：file_operations 型指针，设备操作函数合集变量。

1.2.4 Linux 设备号

每个设备文件都有主次设备号 major，主设备号是唯一的，每个主设备下有次设备号 minor，次设备号在这个主设备下也是唯一的。在 Linux 系统下输入 cat /proc/devices 命令可以查看已被注册的主设备号。

1.2.5 实现设备操作函数

在接下来的点亮 led 实验中，会用到那些设备操作函数呢，先看一下点亮 led 的步骤。

首先我们要初始化 led 所连接的 IO，使能时钟、设置 IO 为输出等，open()函数是应用程序关联设备文件的开始，设备的初始化可以放在 open()函数中。实现了 open()函数必然就要实现对应的 release()函数。IO 初始化完成后，我们需要点亮 led，实际上就是往 IO 对应的寄存器内写入对应的值，写入自然就是实现 write()函数了。相应的，也可以实现 read()函数来读取 IO 当前的状态。

1.2.6 添加驱动描述信息

在驱动代码中我们需要添加 LICENSE 信息和作者信息等驱动描述信息，其中 LICENSE 是必须添加的，方法如下：

```
MODULE_LICENSE("GPL");
```

现在我们已经知道了注册驱动入口和出口函数、注册和注销字符设备、需要实现的设备操作函数，大致可以写出如下的框架：

```
1.  /* 驱动名称 */
2.  #define DEVICE_NAME      "gpio_leds"
3.  /* 驱动主设备号 */
4.  #define GPIO_LED_MAJOR   200
5.
6.  /* open 函数实现，对应到 Linux 系统调用函数的 open 函数 */
7.  static int gpio_leds_open(struct inode *inode_p, struct file *file_p)
8.  {
9.      return 0;
10. }
11.
12. /* write 函数实现，对应到 Linux 系统调用函数的 write 函数 */
13. static ssize_t gpio_leds_write(struct file *file_p, const char __user *buf, size_t len, loff_t *
    loff_t_p)
14. {
15.     return 0;
16. }
17.
18. /* release 函数实现，对应到 Linux 系统调用函数的 close 函数 */
19. static int gpio_leds_release(struct inode *inode_p, struct file *file_p)
20. {
21.     return 0;
22. }
23.
24. /* file_operations 结构体声明，是上面 open、write 实现函数与系统调用函数对应的关键 */
25. static struct file_operations gpio_leds_fops = {
26.     .owner    = THIS_MODULE,
27.     .open     = gpio_leds_open,
28.     .write    = gpio_leds_write,
29.     .release  = gpio_leds_release,
30. };
31.
32. /* 模块加载时会调用的函数 */
```



```

33. static int __init gpio_led_init(void)
34. {
35.     int ret;
36.
37.     /* 通过模块主设备号、名称、模块带有的功能函数(及 file_operations 结构体)来注册模块 */
38.     ret = register_chrdev(GPIO_LED_MAJOR, DEVICE_NAME, &gpio_leds_fops);
39.     if (ret < 0)
40.     {
41.         return ret;
42.     }
43.     else
44.     {
45.     }
46.     }
47.     return 0;
48. }
49.
50. /* 卸载模块 */
51. static void __exit gpio_led_exit(void)
52. {
53.     /* 注销模块, 释放模块对这个设备号和名称的占用 */
54.     unregister_chrdev(GPIO_LED_MAJOR, DEVICE_NAME);
55. }
56.
57. /* 注册模块入口和出口函数 */
58. module_init(gpio_led_init);
59. module_exit(gpio_led_exit);
60.
61. /* 添加 LICENSE 信息 */
62. MODULE_LICENSE("GPL");

```

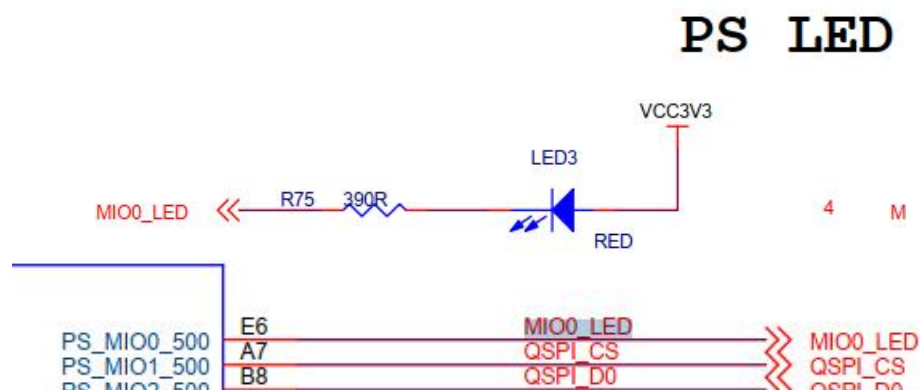
这里我把驱动命名为"gpio_leds"。设备号设置为 200, 我这里 200 号设备号没有被占用, 实际实验要根据自身实际情况更改。

1.3 字符设备驱动开发实验

现在我们已经大致了解了字符设备的实现方法, 接下来就一步一步的去实现。我们的目标是编写开发板上"PS LED3"这个 led 的设备驱动, 通过驱动程序能点亮、熄灭这个 led。

1.3.1 查看硬件原理图以及数据手册

打开开发板原理图, 查看 PS LED3 的连接方式。



可知目标 led 连接在开发板 PS 端的 MIO0 引脚上。

再打开数据手册《ug585-Zynq-7000-TRM.pdf》。根据关键词 MIO 我们能找到一张 MIO 对应表再 2.4.4 节, led 的操作对应的引脚功能为 GPIO, MIO0 对应 GPIO0。我们再到寄存器设置"Appendix B: Register Details"章节中找到 GPIO 的寄存器设置。

B.19 General Purpose I/O (gpio)

Module Name	General Purpose I/O (gpio)
Software Name	XGPIOPS
Base Address	0xE000A000 gpio
Description	General Purpose Input / Output

GPIO 寄存器的基地址为 0xE000A000，对于 GPIO 的设置，需要使能、设置方向以及控制，对应找到使能寄存器 0xE000A208、方向寄存器 0xE000A204、控制（数据）寄存器 0xE000A040。另外还需要使能 GPIO 时钟，找到 AMBA 外设时钟使能寄存器 0xF800012C。

1.3.2 编写字符设备驱动程序

上面我们已经得到了 led 驱动的大体框架，接下来要做的实际就是晚上设备操作函数中的内容，也是再对应的函数中设置相关的寄存器。在 open()函数中实现 led 初始化，在 write()函数中实现 led 的控制，在 release()函数中完成 led 的使能。最终的代码如下：

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/fs.h>
4. #include <linux/init.h>
5. #include <linux/types.h>
6. #include <linux/errno.h>
7. #include <linux/cdev.h>
8. #include <linux/device.h>
9. #include <asm/uaccess.h>
10. #include <asm/io.h>
11.
12. /* 驱动名称 */
13. #define DEVICE_NAME      "gpio_leds"
14. /* 驱动主设备号 */
15. #define GPIO_LED_MAJOR   200
16.
17. /* gpio 寄存器虚拟地址 */
18. static unsigned int gpio_add_minor;
19. /* gpio 寄存器物理基地址 */
20. #define GPIO_BASE        0xE000A000
21. /* gpio 寄存器所占空间大小 */
22. #define GPIO_SIZE        0x1000
23. /* gpio 方向寄存器 */
24. #define GPIO_DIRM_0      (unsigned int *) (0xE000A204 - GPIO_BASE + gpio_add_minor)
25. /* gpio 使能寄存器 */
26. #define GPIO_OEN_0       (unsigned int *) (0xE000A208 - GPIO_BASE + gpio_add_minor)
27. /* gpio 控制寄存器 */
28. #define GPIO_DATA_0      (unsigned int *) (0xE000A040 - GPIO_BASE + gpio_add_minor)
29.
30. /* 时钟使能寄存器虚拟地址 */
31. static unsigned int clk_add_minor;
32. /* 时钟使能寄存器物理基地址 */
33. #define CLK_BASE         0xF8000000
34. /* 时钟使能寄存器所占空间大小 */
35. #define CLK_SIZE         0x1000
36. /* AMBA 外设时钟使能寄存器 */
37. #define APER_CLK_CTRL    (unsigned int *) (0xF800012C - CLK_BASE + clk_add_minor)
38.
39. /* open 函数实现，对应到 Linux 系统调用函数的 open 函数 */
40. static int gpio_leds_open(struct inode *inode_p, struct file *file_p)
41. {
42.     /* 把需要修改的物理地址映射到虚拟地址 */
43.     gpio_add_minor = (unsigned int) ioremap(GPIO_BASE, GPIO_SIZE);
44.     clk_add_minor = (unsigned int) ioremap(CLK_BASE, CLK_SIZE);
45.
46.     /* MIO_0 时钟使能 */
47.     *APER_CLK_CTRL |= 0x00400000;
```

```
48.  /* MIO_0 设置成输出 */
49.  *GPIO_DIRM_0 |= 0x00000001;
50.  /* MIO_0 使能 */
51.  *GPIO_OEN_0 |= 0x00000001;
52.
53.  printk("gpio_test module open\n");
54.
55.  return 0;
56. }
57.
58.
59. /* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
60. static ssize_t gpio_leds_write(struct file *file_p, const char __user *buf, size_t len, loff_t *
    loff_t_p)
61. {
62.     int rst;
63.     char writeBuf[5] = {0};
64.
65.     printk("gpio_test module write\n");
66.
67.     rst = copy_from_user(writeBuf, buf, len);
68.     if(0 != rst)
69.     {
70.         return -1;
71.     }
72.
73.     if(1 != len)
74.     {
75.         printk("gpio_test len err\n");
76.         return -2;
77.     }
78.     if(1 == writeBuf[0])
79.     {
80.         *GPIO_DATA_0 &= 0xFFFFFFF;
81.         printk("gpio_test ON\n");
82.     }
83.     else if(0 == writeBuf[0])
84.     {
85.         *GPIO_DATA_0 |= 0x00000001;
86.         printk("gpio_test OFF\n");
87.     }
88.     else
89.     {
90.         printk("gpio_test para err\n");
91.         return -3;
92.     }
93.
94.     return 0;
95. }
96.
97. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
98. static int gpio_leds_release(struct inode *inode_p, struct file *file_p)
99. {
100.     printk("gpio_test module release\n");
101.     return 0;
102. }
103.
104. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
105. static struct file_operations gpio_leds_fops = {
106.     .owner    = THIS_MODULE,
107.     .open     = gpio_leds_open,
108.     .write    = gpio_leds_write,
109.     .release  = gpio_leds_release,
110. };
111.
112. /* 模块加载时会调用的函数 */
113. static int __init gpio_led_init(void)
114. {
115.     int ret;
116.
117.     /* 通过模块主设备号、名称、模块带有的功能函数(及 file_operations 结构体)来注册模块 */
118.     ret = register_chrdev(GPIO_LED_MAJOR, DEVICE_NAME, &gpio_leds_fops);
119.     if (ret < 0)
120.     {
121.         printk("gpio_led_dev_init_ng\n");
122.         return ret;
123.     }
124.     else
```

```

125. {
126.     /* 注册成功 */
127.     printk("gpio_led_dev_init_ok\n");
128. }
129. return 0;
130. }
131.
132. /* 卸载模块 */
133. static void __exit gpio_led_exit(void)
134. {
135.     /* 释放对虚拟地址的占用 */
136.     iounmap((unsigned int *)gpio_add_minor);
137.     iounmap((unsigned int *)clk_add_minor);
138.     /* 注销模块, 释放模块对这个设备号和名称的占用 */
139.     unregister_chrdev(GPIO_LED_MAJOR, DEVICE_NAME);
140.
141.     printk("gpio_led_dev_exit_ok\n");
142. }
143.
144. /* 标记加载、卸载函数 */
145. module_init(gpio_led_init);
146. module_exit(gpio_led_exit);
147.
148. /* 驱动描述信息 */
149. MODULE_AUTHOR("Alinx");
150. MODULE_ALIAS("gpio_led");
151. MODULE_DESCRIPTION("GPIO LED driver");
152. MODULE_VERSION("v1.0");
153. MODULE_LICENSE("GPL");

```

49 行出现的 `printk()` 函数, 是内核态输出字符串到控制台的函数, 这里用于调试, 相当于应用程序中的 `printf()`。 `printk()` 函数存在消息级别, 定义在头文件 `include/linux/kern_levels.h` 中, 如下:

```

1. #define KERN_EMERG KERN_SOH "" /* system is unusable */
2. #define KERN_ALERT KERN_SOH "" /* action must be taken immediately */
3. #define KERN_CRIT KERN_SOH "" /* critical conditions */
4. #define KERN_ERR KERN_SOH "" /* error conditions */
5. #define KERN_WARNING KERN_SOH "" /* warning conditions */
6. #define KERN_NOTICE KERN_SOH "" /* normal but significant condition */
7. #define KERN_INFO KERN_SOH "" /* informational */
8. #define KERN_DEBUG KERN_SOH "" /* debug-level messages */

```

其中 0 的优先级最高, 7 的优先级最低。如果要设置消息级别, 可设置如下:

```
printk(KERN_INFO"gpio_test module open\n");
```

如果不设置消息级别, 那么 `printk()` 会采用默认级别 `MESSAGE_LOGLEVEL_DEFAULT`。只有消息级别高于头文件 `include/linux/printk.h` 中定义的宏 `CONSOLE_LOGLEVEL_DEFAULT`, 消息才会被打印。

39 行出现的函数 `ioremap()`, 用于把物理地址映射到虚拟地址。在 Linux 中由于 MMU 内存映射的关系, 我们无法直接操作物理地址, 而需要吧物理地址映射到虚拟地址上再操作对应的虚拟地址。 `ioremap()` 定义在头文件 `arch/arm/include/asm/io.h` 中, 如下:

```
#define ioremap(cookie,size) __arm_ioremap((cookie), (size), MT_DEVICE)
```

`cookie` 指代物理地址, `size` 为需要映射的地址长度。39 行的代码即是把 GPIO 寄存器基地址及范围映射到虚拟地址并赋值给全局变量 `gpio_add_minor`, 之后可以直接对变量 `gpio_add_minor` 进行读写操作。

97 行的 `iounmap()` 函数是与 `ioremap()` 相对的释放虚拟地址函数。输入参数为 `ioremap()` 函数返回得到的虚拟地址首地址, 比如这里的 `gpio_add_minor`。

剩下的就是寄存器操作了。对于虚拟地址的操作, 我们这么里都是直接通过指针访问的,

但是 Linux 有推荐的读写方法，而不是使用指针，如下：

读函数：

```
u8 readb(const volatile void __iomem *addr);
u16 readw(const volatile void __iomem *addr);
u32 readl(const volatile void __iomem *addr);
```

写函数：

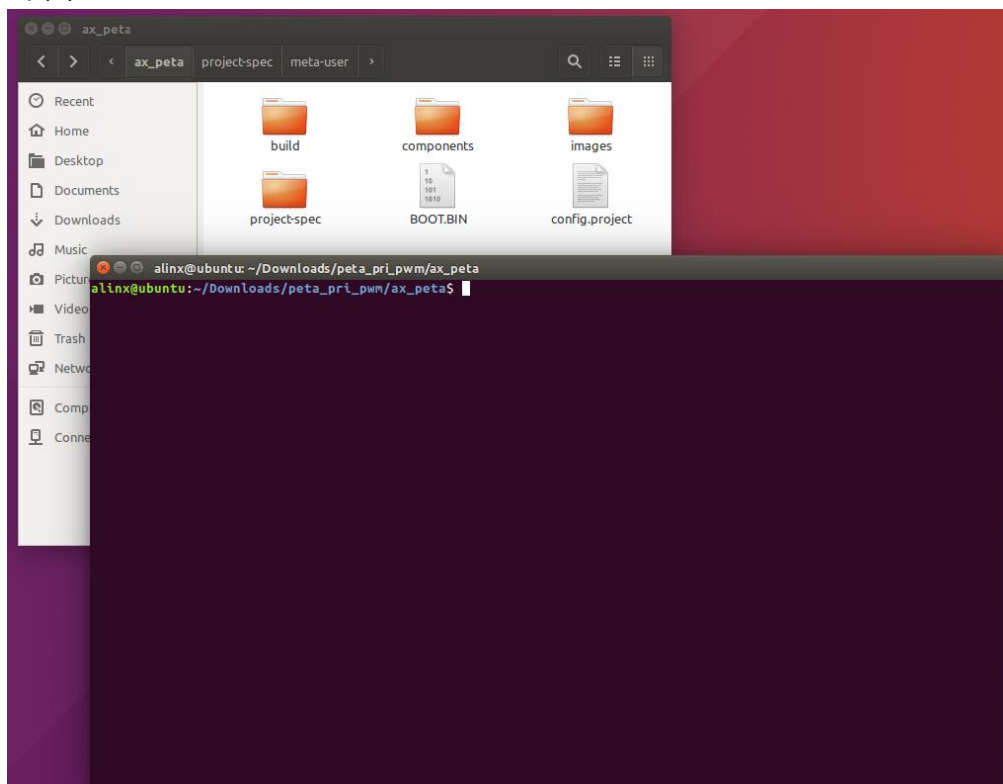
```
void writb(u8 value, volatile void __iomem *addr);
void writew(u16 value, volatile void __iomem *addr);
void writel(u32 value, volatile void __iomem *addr);
```

value 为需要写入的值，addr 为需要操作的地址。

56 行的 write() 函数中通过判断用户输入的 __buf 值为 0 还是 1 相应的执行点灯和灭灯的操作。

1.3.3 在 Petalinux 定制系统中添加新的驱动

petalinux 定制 Linux 系统的方法可以参考教程“course_s4_ZYNQ 那些事儿-Linux 实验篇 V1.06 “开发平台基础教程”的第三章到第五章。在得到定制系统后，打开终端，进入定制系统的根目录，如下图：



ax_peta 是我用 petalinux 定制得到的系统工程根目录。

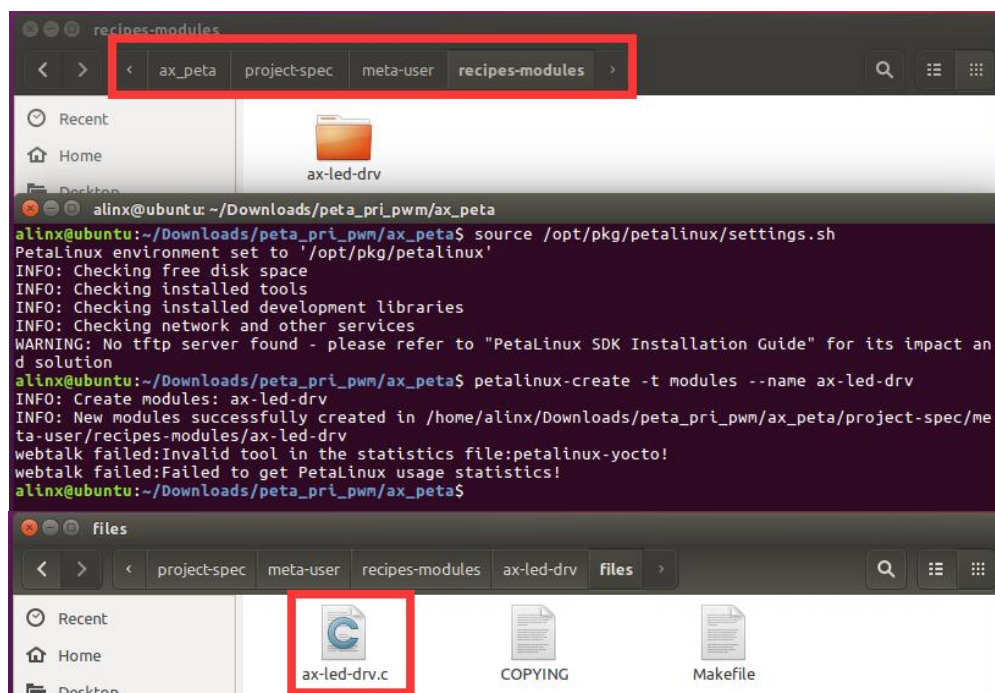
1) 先在终端中输入下面的命令设置 petalinux 环境变量：

```
source /opt/pkg/petalinux/settings.sh
```

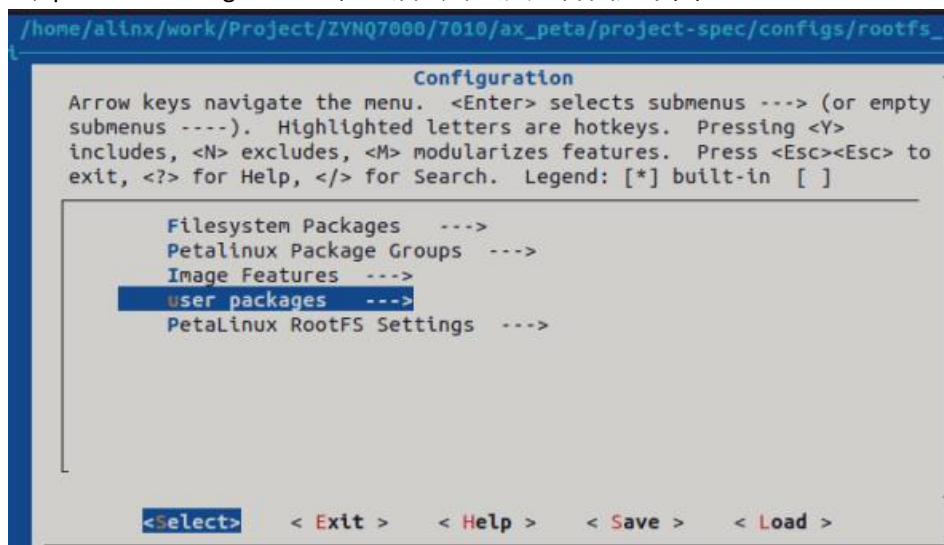
2) 再输入下面的命令添加新的驱动：

```
petalinux-create -t modules --name ax-led-driv
```

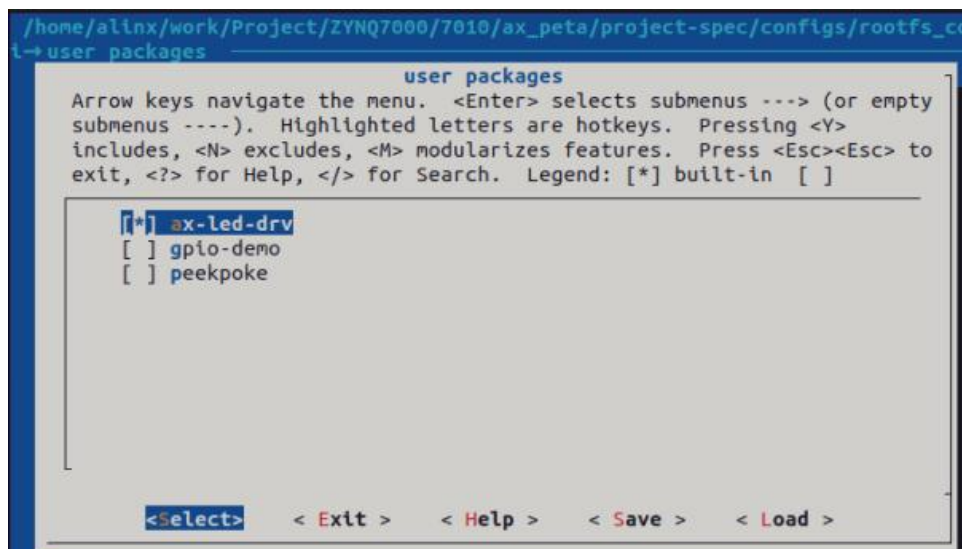
3) 查看目录.../ax_peta/project-spec/meta-user/recipes-modules，下面有了名为 ax-led-driv 文件夹，再进入目录.../ax_peta/project-spec/meta-user/recipes-modules/ax-led-driv/files，目录下名为 ax-led-driv.c 的文件就是 petalinux 帮我们新建的驱动文件。



- 4) 打开这个 c 文件，把我们先前实现的 led 驱动代码粘贴进去保存退出,可以将资料中提供的的驱动源代码直接复制到该文件中。打开终端到先前的 ax_peta 目录下，输入命令 petalinux-config -c rootfs，之后会弹出交互界面如下图：



- 5) 按上下方向键移动到 module 选项，按空格键进入新的界面如下：



选项中的 ax-led-drv 就是我们新加的驱动，按空格选择他，前面的方括号中会出现星号[*]。按左右方向键移动选项到<Save>，按回车保存设置，之后选择<Exit>退出界面。之后在编译 petalinux 工程时，这个驱动就会被编译。

6) 退出交互界面后，在终端输入命令：

```
petalinux-build
```

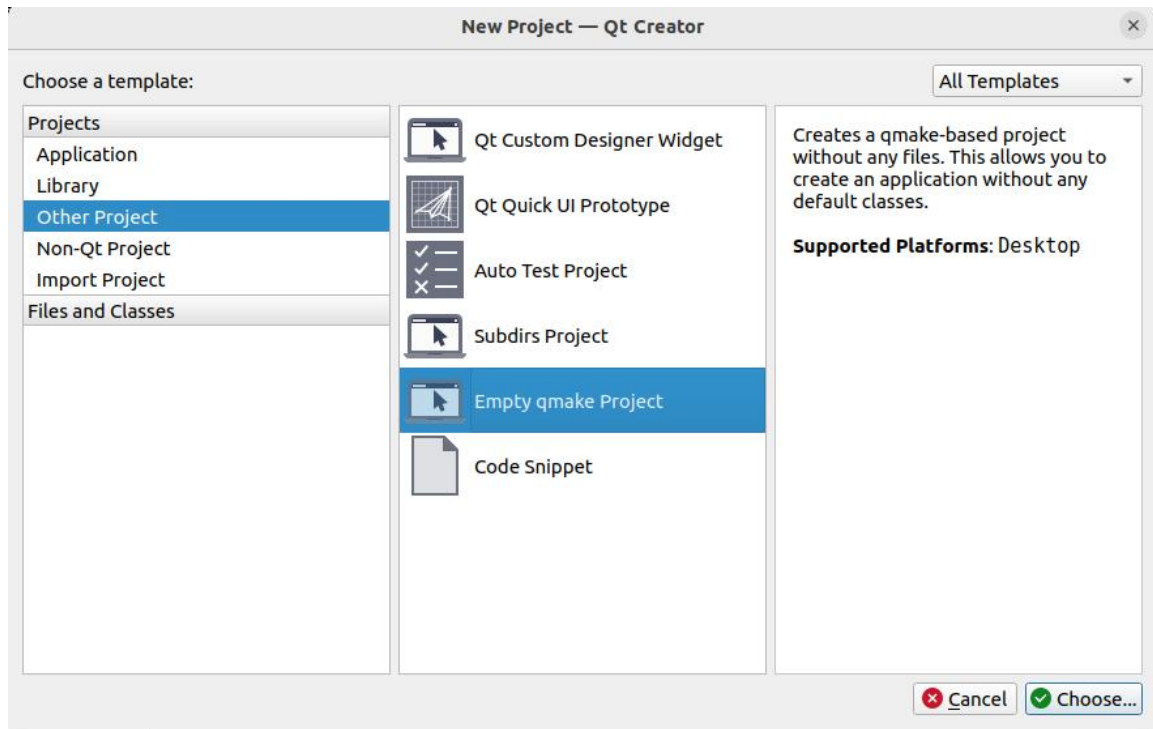
编译 petalinux 工程，编译完成后在 ax_peta 目录下搜索“ax-led-drv.ko”，就是需要的驱动模块文件，保存好，稍后我们会在系统中加载这个驱动模块。



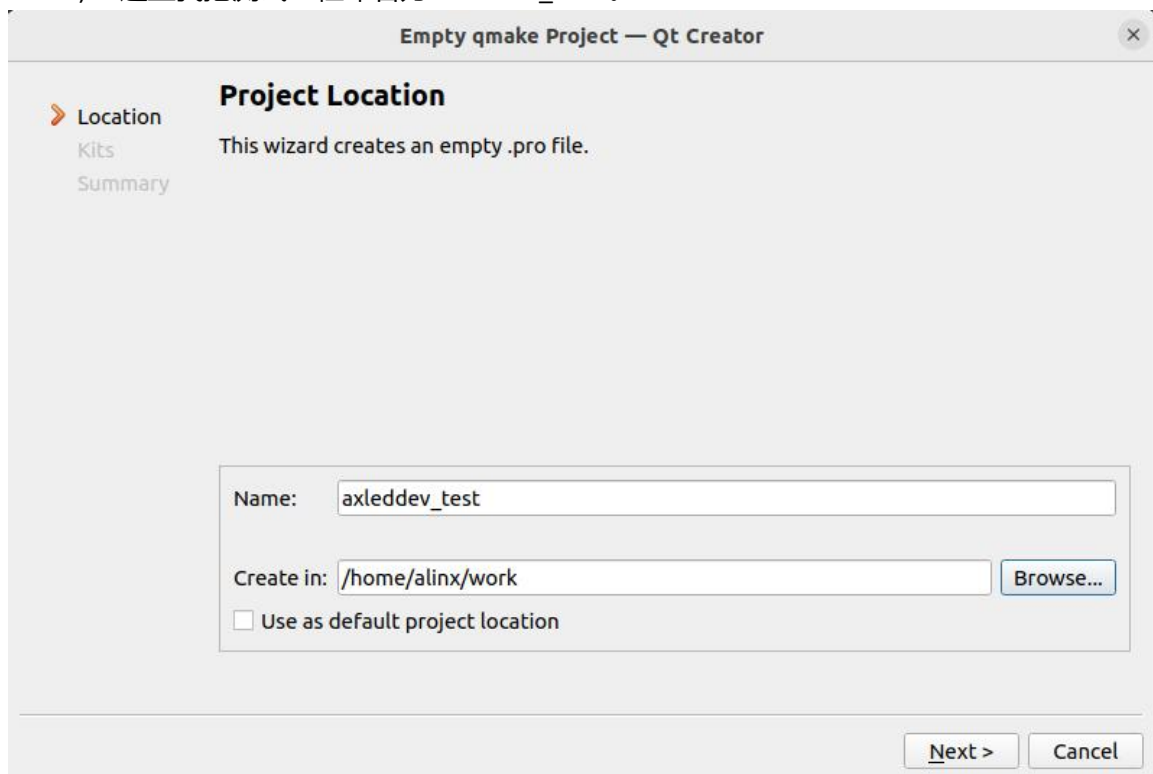
1.3.4 使用 Ubuntu 虚拟机中的 QT 编写测试 APP

驱动代码完成之后，需要测试他的功能。为了方便测试，这里我们使用 Ubuntu 虚拟机中的 QT 来编写测试程序。QT 在 ubuntu 虚拟机中的应用方法可参考教程“course_s4_ZYNQ 那些事儿-Linux 实验篇 V1.06”的第十一章。

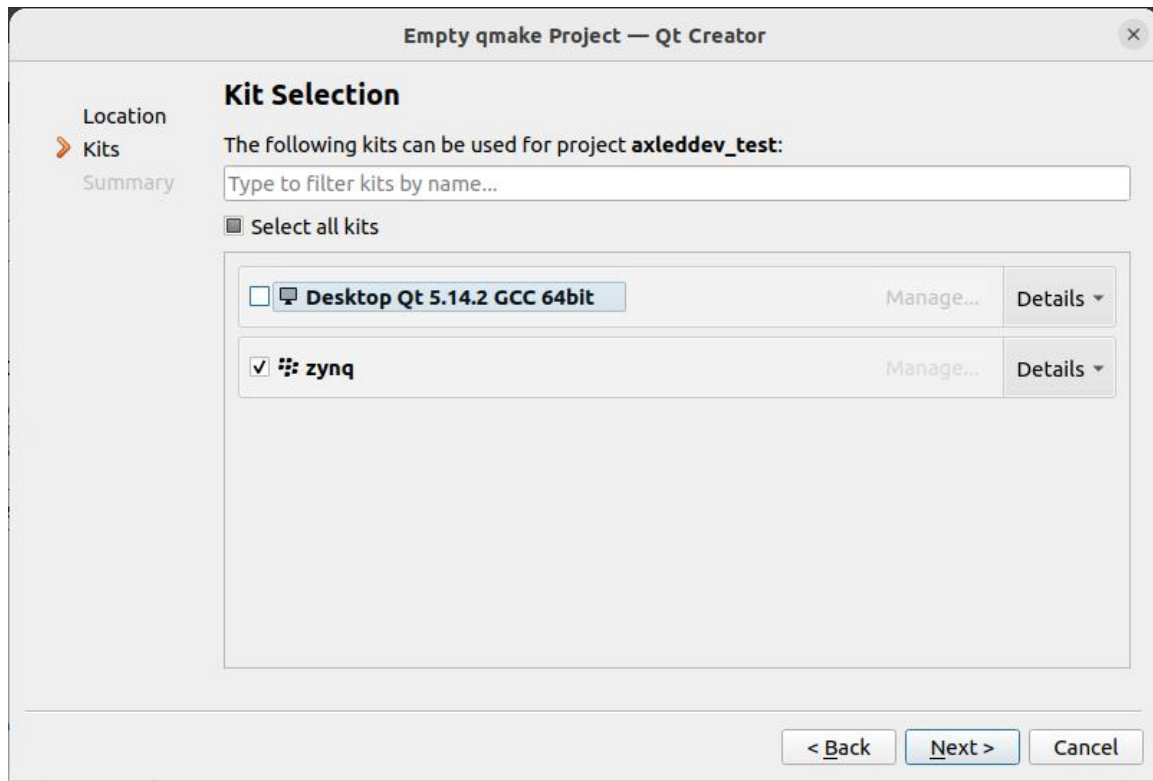
1) 打开 QT，点击 New Project 按钮，创建新的空工程如下图：



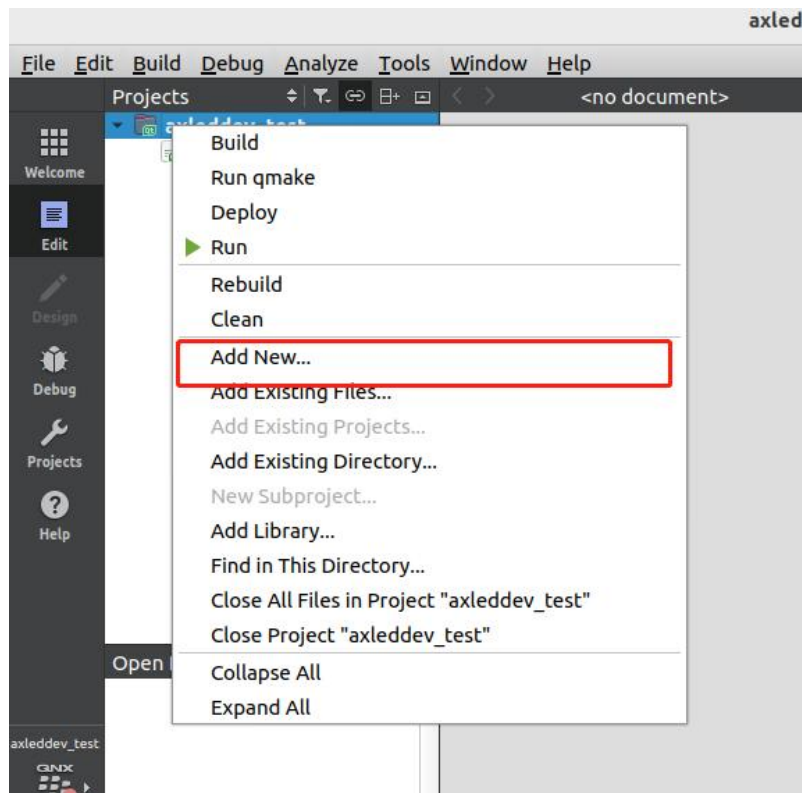
2) 这里我把测试工程命名为“axleddev_test”。



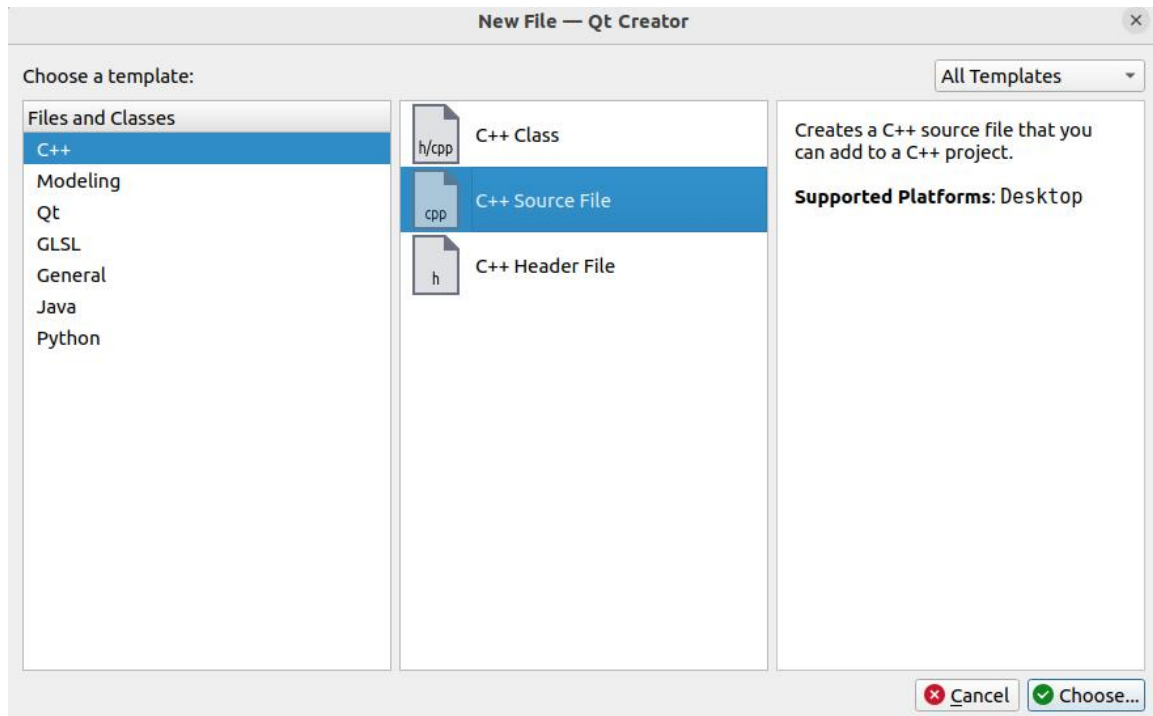
3) 选择 ZYNQ, 如果你这里没有这个选项, 请先参考教程“course_s4_ZYNQ 那些事儿-Linux 实验篇 V1.06”的第十一章来设置 QT 的交叉编工具链。



- 4) 打开新建的工程，在工程目录上右击，点击 Add New 选项。



- 5) 添加 C++ Source File, 命名为 main.c。



6) 打开 main.c, 输入下面的代码:

```
1. #include <stdio.h>
2. #include <string.h>
3. #include <unistd.h>
4. #include <fcntl.h>
5.
6. int main(int argc, char **argv)
7. {
8.     int fd;
9.     char buf;
10.
11.     if(3 != argc)
12.     {
13.         printf("none para\n");
14.         return -1;
15.     }
16.
17.     fd = open(argv[1], O_RDWR);
18.     if(fd < 0)
19.     {
20.         printf("Can't open file %s\r\n", argv[1]);
21.         return -1;
22.     }
23.
24.     if(!strcmp("on",argv[2]))
25.     {
26.         printf("ps_led1 on\n");
27.         buf = 1;
28.         write(fd, &buf, 1);
29.     }
30.     else if(!strcmp("off",argv[2]))
31.     {
32.         printf("ps_led1 off\n");
33.         buf = 0;
34.         write(fd, &buf, 1);
35.     }
36.     else
37.     {
38.         printf("wrong para\n");
39.         return -2;
40.     }
41.
42.     close(fd);
43.     return 0;
44. }
```

应用程序中的系统调用函数, 与内核驱动中的函数格式稍有区别。

17 行的 open()函数原型为:

```
int open (const char * __file, int __oflag, ...);
```

参数说明如下:

__file: 设备文件。我们通过运行程序时的第二个参数来输入设备文件名。

__oflag: 为文件打开模式, 有三项必选其一:

```
#define O_RDONLY    00    //只读
#define O_WRONLY    01    //只写
#define O_RDWR      02    //读写
```

返回值: 如果文件打开成功, 则返回文件句柄。

28 行的 write()函数原型为:

```
ssize_t write (int __fd, const void * __buf, size_t __n);
```

参数说明如下:

__fd: open()函数返回的文件句柄。

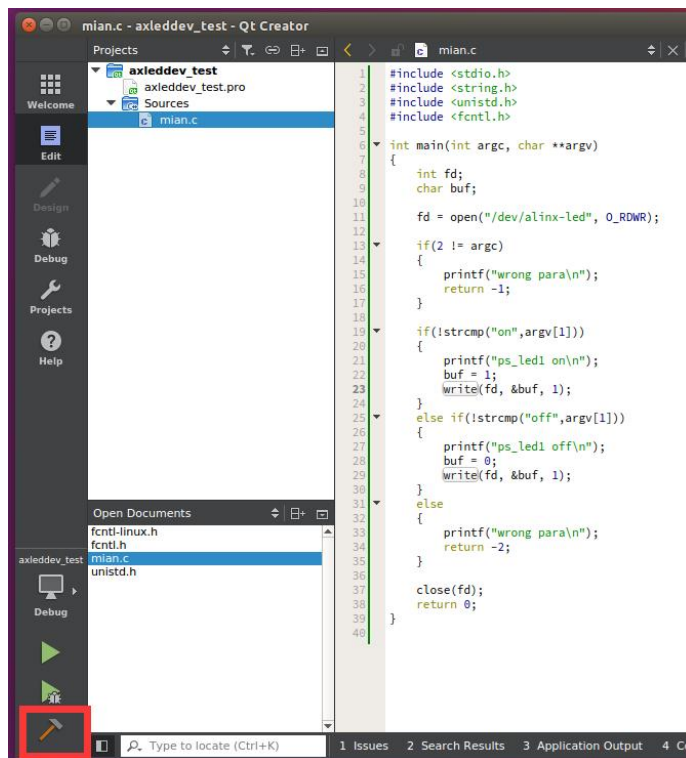
__buf: 需要写入的数据首地址。

__n: 需要写入的数据长度。

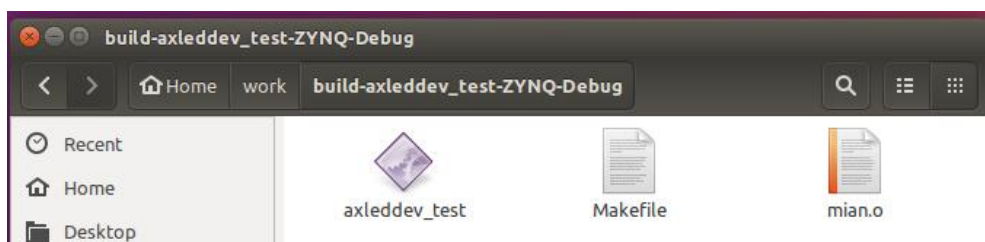
这里 write()函数的调用方式与先前驱动中的实现相对应, 仅输入 0 或者 1 两个值, 来通知驱动做电灯还是灭灯。

42 行在驱动文件使用完之后调用 close()函数释放文件句柄, 输入参数即文件句柄。执行 close()函数后, 最终驱动中实现的 releasae()函数会被执行。

7) 测试代码完成后, 点击 QT 左下角的小锤子编译工程, 如下图:



8) 编译完成后, 在和这个 QT 工程同级及目录下找到编译生成的文件夹, 里面的 axleddev_test 是我们开发板可运行的可执行文件。这个程序想要达成的目标是输入 on 时点灯, 输入 off 灭灯。

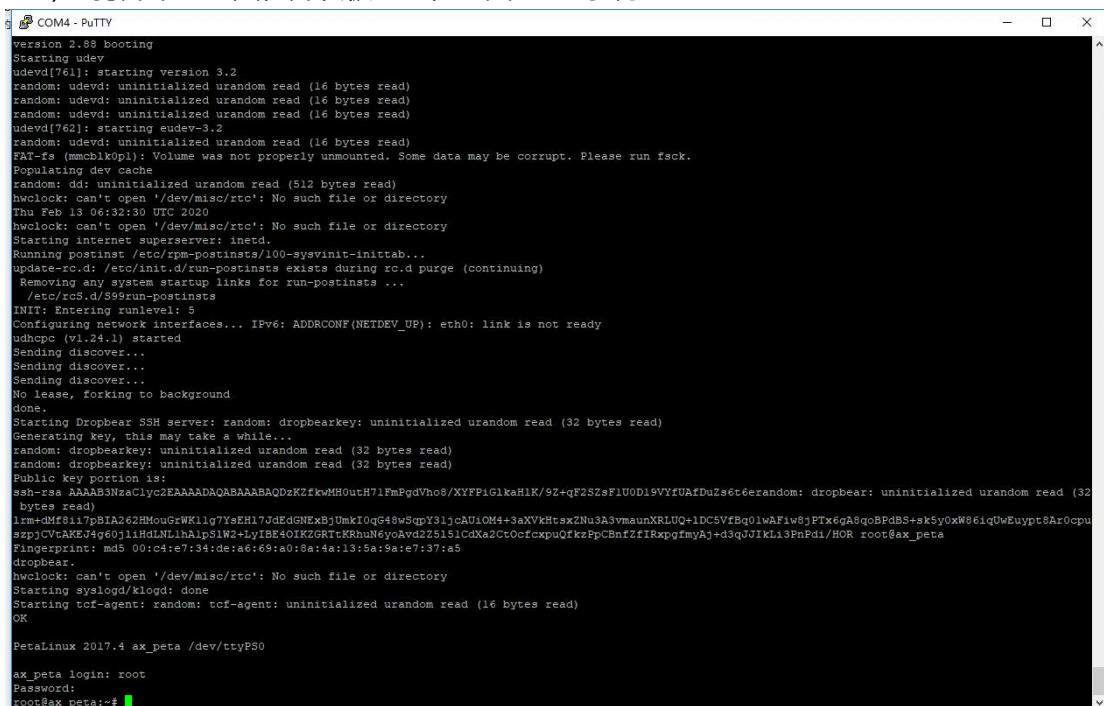


1.3.5 运行测试

现在驱动模块和测试程序都有了，可以开始测试了。

注意此时开发板中应该运行的是与我们创建驱动相同的 petalinux 工程编译出来的 linux 系统。

- 1) 打开串口工具，开发板上电，登陆 linux 系统。



- 2) 输入下列命令，用 NFS 服务挂载 ubuntu 的工作路径到开发板的/mnt 路径下。关于 NFS 服务的使用可参考教程“_course_s4_ZYNQ 那些事儿-Linux 实验篇 V1.06”的第四章。

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
```

其中 192.168.1.107 是我 ubuntu 虚拟机的 ip，/home/alinx/work 是我 ubuntu 的工作路径。这两个需要根据实际情况修改调整。

- 3) 把先前得到的驱动模块 ax-led-drv.ko 和可执行文件 axleddev_test 放到工作路径中，也就是我这里的/home/alinx/work。
- 4) 加载驱动模块 Insmod ax-led-drv.ko。控制台打印出 gpio_led_dev_init_ok，加载成功。

```
root@ax_peta:/mnt# insmod ax-led-drv.ko
ax_led_drv: loading out-of-tree module taints kernel.
gpio_led_dev_init_ok
```

- 5) 之前我们提到过使用命令 `cat /proc/devices` 来查看已被使用的设备号, 现在我们的设备驱动加载成功了, 同样可以用这个命令查看有没有成功注册到设备号。确实已经存在了。

```
root@ax_peta:/mnt# cat /proc/devices
Character devices:
 1 mem
 4 /dev/vc/0
 4 tty
 5 /dev/tty
 5 /dev/console
 5 /dev/ptmx
 7 vcs
10 misc
13 input
21 sg
29 fb
81 video4linux
89 i2c
90 mtd
116 alsa
128 ptm
136 pts
180 usb
200 gpio_leds
245 uio
246 xdevcfg
247 watchdog
248 iio
```

- 6) 至此我们还缺少设备文件, 创建设备文件的命令为:

```
mknode /dev/xxx type major minor
```

/dev/xxx: xxx 为设备文件名称, 统一放在 /dev 路径下, 这里要和我们应用程序中的文件名保持一致即为 /dev/alinx-led。

type: 设备类型, 字符设备为 c。

major: 主设备号, 要与我们调用的驱动一直, 应用程序通过 `open()` 函数来关联设备文件, 设备文件通过主设备号来关联驱动程序, 最终实现应用程序和驱动相互关联。所以这里这里主设备号为 200。

minor: 次设备号, 一般从零开始, 这里填 0 即可。

输入命令:

```
mknode /dev/alinx-led c 200 0
```

再执行命令 `ls /dev`, 就能查到我们需要的设备文件了。

- 7) 设备文件也准备好了, 接下来就可以运行测试程序了, 点亮命令如下:

```
./build-axleddev_test-ZYNQ-Debug/axleddev_test /dev/alinx-led on
root@ax_peta:/mnt# ./build-axleddev_test-ZYNQ-Debug/axleddev_test /dev/alinx-led on
gpio_test module open
ps_led1 on
gpio_test module write
gpio_test ON
gpio_test module release
root@ax_peta:/mnt#
```

Led 被点亮, 调试使用的打印信息也输出成功。

8) 熄灭 led:

```
./build-axleddev_test-ZYNQ-Debug/axleddev_test /dev/alinx-led off
```

9) 卸载驱动模块也测试一下:

```
rmmod ax_led_drv
```

```
root@ax_peta:/mnt# rmmod ax_led_drv  
gpio_led_dev_exit_ok
```

第二章 字符设备的新写法

2.1 设备号管理

在上一章中的实验中，设备号直接在驱动代码中写死了。这样做会带来很多麻烦：

- 1) 编译驱动代码前，必须要先查看目标系统中设备号的占用情况；
- 2) 更换设备后原先驱动中写死的设备号就可能已被占用；
- 3) 原先的驱动注册函数 `register_chrdev()` 输入参数中仅有主设备号而没有次设备号，这意味着一个设备就会占用所有的次设备号，十分浪费。

针对这些问题，Linux 内核提出了新的字符设备注册方法，并由内核来管理设备号。新增两个设备号注册函数。

- 1) 当驱动程序需要给定主设备号时，使用函数来注册设备号：

```
int register_chrdev_region(dev_t from, unsigned count, const char *name);
```

输入参数说明：

from：需要申请的起始设备号，`dev_t` 类型，它取代了原先的主设备号和次设备号，在需要指定主次设备号的情况下，可以通过方法 `from=MKDEV(major, minor)` 来获取他的值。

count：需要申请设备号的个数，一般只要申请一个。

name：设备名。

示例：

```
1. int major = 200;    //主设备号指定为 200
2. int minor = 0;      //次设备号为 0
3. dev_t devid = MKDEV(major, minor);    //通过主次设备号获得设备号
4. /* 向内核注册设备号 */
5. register_chrdev_region(devid, 1, "xxx-dev");
```

- 2) 驱动程序不需要指定主设备号时，使用函数：

```
int alloc_chrdev_region(dev_t *dev, unsigned baseminor, unsigned count, const char *name)
```

输入参数说明：

dev：设备号指针，不指定主次设备号的情况下，设备号由内核分配，因此传入指针来获取设备号，注册成功后可以通过方法 `major = MAJOR(*dev)` 和 `minor = MINOR(*dev)` 分别获取主次设备号，如果不需要用到主次设备号，不获取也可以。

baseminor：次设备号起始地址。

count：需要申请设备号的个数，一般只要申请一个。

name：设备名。

示例：

```
1. dev_t devid;    //设备号
2. /* 申请设备号 */
3. alloc_chrdev_region(&devid, 0, 1, "xxx-dev");
```

- 3) 注销设备号只要使用同一个函数：

```
void unregister_chrdev_region(dev_t from, unsigned count);
```

输入参数含义和注册函数一致。

示例：

```
1. dev_t devid;    //设备号
2. /* 申请设备号 */
3. alloc_chrdev_region(&devid, 0, 1, "xxx-dev");
4. /* 注销设备号 */
5. unregister_chrdev_region(devid, 1);
```

2.2 新的注册方法

上面提到过注册函数 `register_chrdev()` 存在问题。和新的设备号管理方式相对应，现在摒弃 `register_chrdev()`，使用一套新的方法来注册字符设备。

2.1.1 字符设备数据结构

使用 `cdev` 结构体来定义一个字符设备，他定义在 `include/linux/cdev.h` 中，具体如下：

```
1. struct cdev {
2.     struct kobject kobj;
3.     struct module *owner;
4.     const struct file_operations *ops;
5.     struct list_head list;
6.     dev_t dev;
7.     unsigned int count;
8. };
```

重要成员变量：

owner：一般设置为 `THIS_MODULE`；

ops：设备操作函数指针；

dev：设备号。

2.1.2 cdev 结构体初始化

`cdev` 结构体变量定义后需要用 `cdev_init()` 函数初始化，函数原型为：

```
void cdev_init(struct cdev *cdev, const struct file_operations *fops)
```

cdev：字符设备结构体指针；

fops：设备操作函数集合结构体指针。

使用示例：

```
1. /* 字符设备 */
2. struct cdev ax_cdev = {
3.     .owner = THIS_MODULE,
4. };
5.
6. /* 设备操作函数 */
7. static struct file_operations ax_fops = {
8.     .owner = THIS_MODULE,
9.     .open..... /* 此处有省略 */
10. };
11.
12. /* ax_cdev 变量初始化 */
13. cdev_init(&ax_cdev, &ax_fops);
```

2.1.3 新的注册和注销函数

初始化字符结构体变量后，便可以使用这个变量来向 Linux 系统注册字符设备。使用新的注册函数 `cdev_add`，原型为：

```
int cdev_add(struct cdev *p, dev_t dev, unsigned count);
```


p: 上面初始化后的字符设备结构体变量;

dev: 设备号;

count: 需要添加的设备数量。

注册函数变了, 注销卸载函数也一样, 不再使用 `unregister_chrdev()` 函数, 改用 `cdev_del()`

函数, 原型:

```
void cdev_del(struct cdev *p);
```

输入参数即为字符设备结构体变量。

结合上设备号, 补充一下 2.2.2 中的示例:

```
1.  /* 字符设备 */
2.  struct cdev ax_cdev = {
3.      .owner = THIS_MODULE,
4.  };
5.
6.  /* 设备操作函数 */
7.  static struct file_operations ax_fops = {
8.      .owner = THIS_MODULE,
9.      .open.....
10. };
11.
12. /* 设备号 */
13. cdev_t devid;
14. /* 申请设备号 */
15. alloc_chrdev_region(&devid, 0, 1, " xxx-dev");
16.
17. /* ax_cdev 变量初始化 */
18. cdev_init(&ax_cdev, &ax_fops);
19. /* 注册字符设备 */
20. cdev_add(&ax_cdev, devid, 1);
21. ..... /* 此处有省略 */
22. /* 卸载字符设备 */
23. cdev_del(&ax_cdev);
```

2.3 自动创建设备文件

2.3.1 mdev

`mdev` 是一个用户程序, 是 `udev` 的简化版。它可以检测并根据系统中硬件设备状态来创建或者删除设备文件。在加载驱动模块后, 会自动在 `/dev` 目录下创建设备节点文件, 卸载驱动模块后设备节点也会自动删除。接下来看看如何实现。

2.3.2 类的创建和删除

创建设备前需要先创建类, 设备是在类下面创建的。类的结构体 `struct class` 结构体定义在 `include/linux/device.h` 中, 需要使用函数 `class_create()` 来创建。`class_create()` 是个宏定义:

```
1. #define class_create(owner, name) \
2. ({ \
3.     static struct lock_class_key __key; \
4.     __class_create(owner, name, &__key); \
5. })
6.
7. struct class *__class_create(struct module *owner, const char *name, struct lock_class_key *key)
```

展开后可以看出:

输入参数:

owner: 至今出现的 `owner` 值都是 `THIS_MODULE`, 这里任然不例外;

name: 类的名字。

返回值： struct class 类型的结构体指针。

卸载驱动程序时需要删除类，使用函数 class_destroy()，原型如下：

```
void class_destroy(struct class *cls);
```

cls 为需要删除的类。

2.3.3 设备节点的创建和删除

创建类后，使用 device_create()函数在类下面创建设备，原型为：

```
1. struct device *device_create(struct class *class,  
2.                             struct device *parent,  
3.                             dev_t devt,  
4.                             void *drvdata,  
5.                             const char *fmt, ...)
```

参数说明：

class：上节介绍的类，设备会在这个类下创建；

parent：父设备，没有父设备的话填 NULL；

devt：设备号；

drvdata：设备可能会用到的数据，没有的话填 NULL；

fmt：设备名，比如当 fmt=axled 时，创建设备后就会生成/dev/axled 文件。

删除设备函数为：

```
void device_destroy(struct class *class, dev_t devt);
```

输入参数含义和上面一样。

自动创建设备节点的实现一般放在驱动入口函数中，结合上一节类的创建以及设备号，自动创建设备节点的实现示例如下：

```
1. struct class *class;      /* 类 */  
2. struct device *device;    /* 设备 */  
3. dev_t devid;              /* 设备号 */  
4.  
5. /* 驱动入口函数 */  
6. static int __init xxx_init(void)  
7. {  
8.     .....  
9.     /* 申请设备号 */  
10.    alloc_chrdev_region(&devid, 0, 1, "xxx-dev");  
11.    .....  
12.    /* 创建类 */  
13.    class = class_create(THIS_MODULE, "xxx");  
14.    /* 创建设备 */  
15.    device = device_create(class, NULL, devid, NULL, "xxx");  
16.  
17.    return 0;  
18. }  
19.  
20. /* 驱动出口函数 */  
21. static void __exit led_exit(void)  
22. {  
23.     /* 删除设备 */  
24.     device_destroy(class, devid);  
25.     /* 删除类 */  
26.     class_destroy(class);  
27.     /* 注销字符设备 */  
28.     unregister_chrdev_region(devid, 1);  
29. }  
30.  
31. module_init(led_init);  
32. module_exit(led_exit);
```

2.4 字符设备新驱动实验

现在了解字符设备驱动的新写法相关的要点，接下来就动手尝试，这章我们要达成的目标和上一章相同，编写开发板上“PS LED1”这个 led 的设备驱动，通过驱动程序能点亮、熄灭这个 led。

2.4.1 查看硬件原理图以及数据手册

和上一章 1.3.1 节的内容相同。

2.4.2 编写字符设备驱动程序

使用 petalinux 创建新的驱动，方法也和上一章相同，之后重复的步骤就略过了。这里我新建了名为 ax-newled-drv 的驱动。打开文件 ax-newled-drv.c，输入以下内容：

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/fs.h>
4. #include <linux/init.h>
5. #include <linux/ide.h>
6. #include <linux/types.h>
7. #include <linux/errno.h>
8. #include <linux/cdev.h>
9. #include <linux/device.h>
10. #include <asm/uaccess.h>
11. #include <asm/io.h>
12. /* 设备节点名称 */
13. #define DEVICE_NAME      "gpio_leds"
14. /* 设备号个数 */
15. #define DEVID_COUNT      1
16. /* 设备个数 */
17. #define DEVICE_COUNT     1
18. /* 主设备号 */
19. #define MAJOR
20. /* 次设备号 */
21. #define MINOR            0
22.
23. /* gpio 寄存器虚拟地址 */
24. static unsigned int gpio_add_minor;
25. /* gpio 寄存器物理基地址 */
26. #define GPIO_BASE        0xE000A000
27. /* gpio 寄存器所占空间大小 */
28. #define GPIO_SIZE        0x1000
29. /* gpio 方向寄存器 */
30. #define GPIO_DIRM_0      (unsigned int *)(0xE000A204 - GPIO_BASE + gpio_add_minor)
31. /* gpio 使能寄存器 */
32. #define GPIO_OEN_0       (unsigned int *)(0xE000A208 - GPIO_BASE + gpio_add_minor)
33. /* gpio 控制寄存器 */
34. #define GPIO_DATA_0      (unsigned int *)(0xE000A040 - GPIO_BASE + gpio_add_minor)
35.
36. /* 时钟使能寄存器虚拟地址 */
37. static unsigned int clk_add_minor;
38. /* 时钟使能寄存器物理基地址 */
39. #define CLK_BASE          0xF8000000
40. /* 时钟使能寄存器所占空间大小 */
41. #define CLK_SIZE          0x1000
42. /* AMBA 外设时钟使能寄存器 */
43. #define APER_CLK_CTRL    (unsigned int *)(0xF800012C - CLK_BASE + clk_add_minor)
44.
45. /* 把驱动代码中会用到的数据打包进设备结构体 */
46. struct alinx_char_dev{
47.     dev_t      devid;        //设备号
48.     struct cdev cdev;        //字符设备
49.     struct class *class;     //类
50.     struct device *device;    //设备节点
51. };
52. /* 声明设备结构体 */
53. static struct alinx_char_dev alinx_char = {
54.     .cdev = {
55         .owner = THIS_MODULE,
56     },
57. };
58.
```

```
59. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
60. static int gpio_leds_open(struct inode *inode_p, struct file *file_p)
61. {
62.     /* 把需要修改的物理地址映射到虚拟地址 */
63.     gpio_add_minor = (unsigned int)ioremap(GPIO_BASE, GPIO_SIZE);
64.     clk_add_minor = (unsigned int)ioremap(CLK_BASE, CLK_SIZE);
65.
66.     /* MIO_0 时钟使能 */
67.     *APER_CLK_CTRL |= 0x00400000;
68.     /* MIO_0 设置成输出 */
69.     *GPIO_DIRM_0 |= 0x00000001;
70.     /* MIO_0 使能 */
71.     *GPIO_OEN_0 |= 0x00000001;
72.
73.     printk("gpio_test module open\n");
74.
75.     return 0;
76. }
77.
78.
79. /* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
80. static ssize_t gpio_leds_write(struct file *file_p, const char __user *buf, size_t len, loff_t *loff_t
    _p)
81. {
82.     int rst;
83.     char writeBuf[5] = {0};
84.
85.     printk("gpio_test module write\n");
86.
87.     rst = copy_from_user(writeBuf, buf, len);
88.     if(0 != rst)
89.     {
90.         return -1;
91.     }
92.
93.     if(1 != len)
94.     {
95.         printk("gpio_test len err\n");
96.         return -2;
97.     }
98.     if(1 == writeBuf[0])
99.     {
100.         *GPIO_DATA_0 &= 0xFFFFFEE;
101.         printk("gpio_test ON\n");
102.     }
103.     else if(0 == writeBuf[0])
104.     {
105.         *GPIO_DATA_0 |= 0x00000001;
106.         printk("gpio_test OFF\n");
107.     }
108.     else
109.     {
110.         printk("gpio_test para err\n");
111.         return -3;
112.     }
113.
114.     return 0;
115. }
116.
117. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
118. static int gpio_leds_release(struct inode *inode_p, struct file *file_p)
119. {
120.     printk("gpio_test module release\n");
121.     return 0;
122. }
123.
124. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
125. static struct file_operations ax_char_fops = {
126.     .owner    = THIS_MODULE,
127.     .open     = gpio_leds_open,
128.     .write    = gpio_leds_write,
129.     .release  = gpio_leds_release,
130. };
131.
132. /* 模块加载时会调用的函数 */
133. static int __init gpio_led_init(void)
134. {
135.     /* 注册设备号 */
136.     alloc_chrdev_region(&alinx_char.devid, MINOR, DEVID_COUNT, DEVICE_NAME);
137.
138.     /* 初始化字符设备结构体 */
139.     cdev_init(&alinx_char.cdev, &ax_char_fops);
140. }
```

```
141.  /* 注册字符设备 */
142.  cdev_add(&alinx_char.cdev, alinx_char.devid, DEVICE_COUNT);
143.
144.  /* 创建类 */
145.  alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
146.  if(IS_ERR(alinx_char.class))
147.  {
148.      return PTR_ERR(alinx_char.class);
149.  }
150.
151.  /* 创建设备节点 */
152.  alinx_char.device = device_create(alinx_char.class, NULL,
153.                                   alinx_char.devid, NULL,
154.                                   DEVICE_NAME);
155.  if (IS_ERR(alinx_char.device))
156.  {
157.      return PTR_ERR(alinx_char.device);
158.  }
159.
160.  return 0;
161. }
162.
163. /* 卸载模块 */
164. static void __exit gpio_led_exit(void)
165. {
166.     /* 释放对虚拟地址的占用 */
167.     iounmap((unsigned int *)gpio_add_minor);
168.     iounmap((unsigned int *)clk_add_minor);
169.     /* 注销字符设备 */
170.     cdev_del(&alinx_char.cdev);
171.
172.     /* 注销设备号 */
173.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
174.
175.     /* 删除设备节点 */
176.     device_destroy(alinx_char.class, alinx_char.devid);
177.
178.     /* 删除类 */
179.     class_destroy(alinx_char.class);
180.
181.     printk("gpio_led_dev_exit_ok\n");
182. }
183.
184. /* 标记加载、卸载函数 */
185. module_init(gpio_led_init);
186. module_exit(gpio_led_exit);
187.
188. /* 驱动描述信息 */
189. MODULE_AUTHOR("Alinx");
190. MODULE_ALIAS("gpio_led");
191. MODULE_DESCRIPTION("NEW GPIO LED driver");
192. MODULE_VERSION("v1.0");
193. MODULE_LICENSE("GPL");
```

与上一章相比，改动的部分加粗了，除了新增了一些宏定义和参数声明之外，改动的只有入口函数和出口函数两个地方。

13~21 行，新增了几个宏定义，`DEVICE_NAME` 表示设备名，也是设备节点名，最终我们要到 `/dev` 目录中寻找这个名字的设备文件。

`DEVID_COUNT` 和 `DEVICE_COUNT` 分别表示设备号数和设备节点数。一般一个设备对应一个驱动，所以在一个驱动代码中就只有一个设备，所以这里都为 1。

`MAJOR` 主设备号，这里我们通过函数获取设备号，主设备号不需要自己设置。`MAJOR` 次设备号一般从 0 开始。

46~57 行，新建了一个结构体类型，并用这个类型声明了一个变量。把之后会用到的设备号、字符设备、类、设备节点等数据类型都打包进这个结构体中。驱动开发中通常在 `open` 函数中将文件(struct file 结构体)的私有数据(private_data 元素)指向设备结构体，在 `read()`、`write()`、`ioctl()`等函数通过 `private_data` 访问数据 设备结构体。举例如下：

```
1.  /* 定义一个设备结构体 */
2.  struct char_dev {
3.      .....
4.      int value;
5.  };
6.  /* 声明设备结构体 */
7.  static struct char_dev charDev_t = {
8.      .cdev = {
9.          .owner = THIS_MODULE,
10.     },
11. };
12.
13. static int xxx_open(struct inode *inode, struct file *filp)
14. {
15.     filp->private_data = &charDev_t;
16.     .....
17.     return 0;
18. }
19.
20. static ssize_t xxx_read(struct file *filp, char __user *buf, size_t cnt, loff_t *offt)
21. {
22.     int value = 0;
23.
24.     struct char_dev *dev = (struct char_dev *)filp->private_data;
25.
26.     value = dev->value;
27.     .....
28. }
```

对于这种一般性的规范，慢慢去习惯就好了，一开始不这么写也没问题。

140~162 行是关键的地方，在驱动入口函数中，把本章先后讲到的知识点申请设备号、初始化并注册字符设备、创建类和设备节点全部结合。

171~180 行出口函数中就是做与入口函数中注册创建相对应的注销和删除。

结合上面几节的内容，这个新驱动代码不难理解。完成后在 ubuntu 虚拟机重编译得到驱动模块文件 ax-newled-dev.ko。

2.4.3 编写测试 APP

测试 APP 与上一章 1.3.4 节内容一致，可以直接使用上一章的测试程序。

2.4.5 运行测试

测试方法也与之前一样，给开发板上电，并挂在虚拟机的工作目录到开发板/mnt 路径。

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
```

1) 先加载驱动，执行命令：

```
insmod ax-newled-dev.ko
```

```
ax_newled_dev: loading out-of-tree module taints kernel.
root@ax_peta:/mnt#
```

2) 驱动加载成功，再看看设备文件有没有创建成功，执行命令：

```
ls /dev
```

```

root@ax_peta:/mnt# ls /dev
block          loop-control   mtd2ro         ram13
bus            loop0          mtd3           ram14
char           loop1          mtd3ro         ram15
console        loop2          mtblock0       ram2
cpu_dma_latency loop3          mtblock1       ram3
disk           loop4          mtblock2       ram4
fd             loop5          mtblock3       ram5
full           loop6          network_latency ram6
gpio_eds       loop7          network_throughput ram7
gpiochip0      mem           null           ram8
gpiochip1      memory_bandwidth port           ram9
gpiochip2      mmcblk0       psaux          random
i2c-0          mmcblk0p1     ptmx           shm
i2c-1          mtab          pts            snd
l10:device0    mtd0          ram0           stderr
initctl        mtd0ro        ram1           stdin
input          mtd1          ram10          stdout
kmsg           mtdlro        ram11          tty
log            mtd2          ram12          tty0
root@ax_peta:/mnt#

```

3) 设备节点文件已经存在了, 可以使用测试 APP 来试试驱动程序了, 执行下列命令:

```

cd ./build-axleddev_test-ZYNQ-Debug/
./axleddev_test /dev/gpio_eds on
root@ax_peta:/mnt# cd ./build-axleddev_test-ZYNQ-Debug/
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# ./axleddev_test /dev/gpio_eds on
gpio_test module open
ps_led1 on
gpio_test module write
gpio_test ON
gpio_test module release

```

4) led 被点亮, 最后在测试卸载驱动, 执行命令:

```
rmmod ax-newled-dev
```

如果不确定驱动名称, 可以先执行 lsmod 命令查看。

```

root@ax_peta:/mnt# rmmod ax-newled-dev
gpio_led_dev_exit ok
root@ax_peta:/mnt#

```

5) 删除设备后, 再确认设备节点文件有没有被删除:

```
ls /dev/gpio_led
```

```

root@ax_peta:/mnt# ls /dev/gpio_led
ls: /dev/gpio_led: No such file or directory
root@ax_peta:/mnt#

```

没有问题, 试验成功。

第三章 设备树和 of 函数

3.1 设备树

ARM Linux3.0 以前是没有应用设备树的,那时 ARM Linux 使用者们用 c 语言的数据结构来描述板子上的设备,每一个设备就对应到一个描述文件。ARM 的板子种类不计其数,而 Linux 社区为了保证内核源码的通用性,把这些描述文件全部塞了进去,导致 Linux 内核充满了垃圾代码,震怒了 Linux 之父 Linus。之后 ARM 社区就学习 PowerPC 引入了设备树来替代以前的方法。

设备树除了没有用于代码之外还有其他优势。在驱动对应的硬件有变动时,不需要重新编译内核或驱动程序,只需要单独编译设备树文件,在启动板卡时把设备树便宜结果传给内核即可。(遗憾的是,这个优势在 petalinux 中没有得到体现,petalinux 编译出来的结果中,u-boot、内核和设备树被一并打包进了 image.ub 里,而不是单独提供的。所以变更设备树时,往往会使用 petalinux-build 命令把内核整体重新编译。其实通过前面两章可以发现,在更改或者新增驱动时,也是整体编译内核的。实际上单独编译驱动是可以的,但是,为了统一操作步骤就都用 petalinux-build 命令了。)

设备树即树状的设备结构,“树”是个形象的比喻,以系统总线为主干,其他挂在在系统总线上的如 I2C、SPI、GPIO 控制器等设备为分支,而这些挂载在主干上的分支又有他们自身挂载的设备,如 I2C 上挂载了 EEPROM、RTC 等设备,这样的枝干分叉结构就如同树一般。

3.1.1 DTS、DTB 和 DTSI

通常我们编辑的设备树文件扩展名为.dtc(device tree source),而内核读取的设备树文件为编译之后得到的二进制文件.dtb(device tree binary large object)。DTB 文件我们也可以直接修改,但是二进制文件修改很不方便,也不是设备树设计的初衷,这里就重点说 DTS 文件。

DTS 文件内容乍一看有点像 JSON 数据结构,大括号里大括号。当然肯定是有差别的,DTS 有自己的一套语法规则,详情可参考官方文档“devicetree-specification-v0.2.pdf”。

先来看一段 DTS 文件内容,文件 arch/arm/boot/dts/zynq-zc702.dts 的节选:

```
1. #include "zynq-7000.dtsi"
2.
3. / {
4.     model = "Zynq ZC702 Development Board";
5.     compatible = "xlnx,zynq-zc702", "xlnx,zynq-7000";
6.
7.     .....
8.
9.     memory@0 {
10.         device_type = "memory";
11.         reg = <0x0 0x40000000>;
12.     };
13.
14.     .....
15.
16.     gpio-keys {
17.         compatible = "gpio-keys";
18.         #address-cells = <1>;
19.         #size-cells = <0>;
20.         autorepeat;
21.         sw14 {
22.             label = "sw14";
23.             gpios = <&gpio0 12 0>;
24.             linux,code = <108>; /* down */
25.             wakeup-source;
```



```
26.         autorepeat;
27.     };
28.
29.     .....
30. };
31. };
```

结合这段代码，来大致了解一下 DTS 的语法格式。

第 1 行是包含头文件的语句。和 c 语言一样，DTS 文件使用 `#include` 来引用头文件，可以用 `.h`、`.dts` 以及 `.dtsi` 文件。`“.dtsi”` 文件是专门用于编写 DTS 头文件的文件，语法与 DTS 文件相同，是编写 DTS 头文件的首选。

3.1.2 节点

1) 通用节点格式

DTS 文件中，设备用节点来表示，格式为：

```
1. [label:] node-name[@unit-address] {
2.     [properties definitions]
3.     [child nodes]
4. };
```

[] 中的部分是非必要项。

node-name 是设备节点名称，根节点用 `"/"` 表示。第三行的 `"/"` 即是设备 "Zynq ZC702 Development Board" 的根节点。第 9 行的 `memory`、第 11 行的 `gpio-keys`、第 21 行的 `sw14` 都是设备节点名称。

unit-address 一般是指代设备地址或寄存器首地址，如代码中第 9 行的 `memory@0`。如果设备没有设备地址或寄存器就不用写。

label 是设备别称，用于便捷访问节点。如一个名称为 `slcr@f8000000` 的节点，正常要访问的话，需要用名称 `slcr@f8000000` 去访问，如果给这个节点加上标签 `slcr1: slcr@f8000000`，访问节点只需要使用 `&slcr1` 即可。

{ } 里的是节点的内容，节点内容有两类，`[properties definitions]` 是节点属性，后面再讲解。`[child nodes]` 是这个挂在这个设备节点上的子节点。子节点的格式和上面介绍的一样。

2) 特殊节点 aliases

`aliases` 节点用于定义别名，作用于 `label` 标签相似，格式如下：

```
1. aliases {
2.     ethernet0 = "&gem0";
3.     serial0 = "&uart1";
4. };
```

之后便可以使用 `&gem` 来访问节点 `ethernet`。

3.1.3 属性

节点属性 `[properties definitions]` 有 4 种形式：

1) [label:]property-name;

属性为空值。如示例代码中 20 行和 26 行的 `autorepeat`。

2) [label:]property-name = <arrays of cells>;

用 `<>` 括起来的值内容是 32 位数据的合集。如示例代码 11 行的 `reg = <0x0 0x40000000>`。

3) [label:]property-name = "string";

用""包含的表示字符串, 如第五行的 compatible = "xlnx,zynq-zc702", "xlnx,zynq-7000".

4) [label:]property-name = [bytestring];

用[]括起来的表示字符序列, 这个比较少见, 手动举个例子, 假设有属性表示为 memory-addr = [0011223344];, 这个就等同于 memory-addr = [00 11 22 33 44];他的值是 5 个 byte 型的数组成的序列, 并且这个这个 byte 型的数是两位 16 进制数组成的。

属性可以用户自定义, 也有很多标准属性, 下面介绍几种常见的标准属性:

1) compatible 属性

compatible 属性也叫兼容性, 他的值是字符串列表, 是设备和驱动关联的关键, 在驱动代码中定义一个 OF 匹配表来和 compatible 属性对应。如示例代码 17 行, 节点 gpio-keys 的 compatible = "gpio-keys", 那么在对应的驱动代码中, 就会有以下定义来与之对应:

```
1. static const struct of_device_id gpiokeys_ids[] = {
2.     { .compatible = "gpio-keys", },
3.     { /* sentinel */ }
4. };
```

struct of_device_id 是 OF 匹配表的数据类型。当驱动程序 OF 匹配表中的 compatible 值与设备树节点中的 compatible 对应时, 这个节点就会使用这个驱动程序。不过在目前我们只用到了字符设备的框架, 这个框架还用不上 OF 匹配表, 等后面我们用到总线设备模型时, 会再讲到 OF 匹配表。

根节点中的 compatible 属性表示这个板子兼容哪些平台。一般有两个值, 前者表示板子的型号, 后者表示使用的芯片。

2) model 属性

model 属性的值也是字符串, 一般用来表示板子的名称, 与根节点中的 compatible 属性类似。

3) #address-cells、#size-cells 和 reg 属性

#address-cells 属性表示当前节点子节点的 reg 属性中, 使用多少个 u32 整数来描述地址 address。

#size-cells 属性表示当前节点子节点的 reg 属性中, 使用多少个 u32 整数来描述大小 length。

reg 属性一般用于描述某个外设的寄存器地址范围信息, 格式为 reg = <address1 length1 address2 length2 address3 length3.....>。父节点中的#address-cells 属性即指代 reg 属性中 address 的大小, #size-cells 即指代 reg 属性中 length 的大小。

这个在示例代码中也没体现, 为了便于理解, 举个极端一点例子:

```
1. ax-parent {
2.     #address-cells = <2>;
3.     #size-cells = <1>;
4.     .....
5.
6.     ax-son {
7.         reg = <0x00000001 0x00000002 0x00000003
8.             0x00000004 0x00000005 0x00000006>;
9.         .....
10.    }
11. }
```

子节点 ax-son 的 reg 属性的有 6 个 u32 的数据, 父节点 ax-parent 中#address-cells 等于 2,

因此子节点 ax-son 的 reg 属性的值中表示 address 的有两个 u32 的数,即 0x00000001、0x00000002 这两个数据都是 address 的值。同理#size-cells 等于 1, 所以 length1 的值仅有一个 u32 的数据等于 0x00000003。而后面的三个 u32 数据则是 address2 和 length2 的值。

4) device_type 属性

这个属性现在只能用于 cpu 节点或者 memory 节点。在 cpu 节点中 device_type = “cpu”, 在 memory 节点中 device_type = “memory”。

5) phandle 属性

phandle 属性的取值必须是唯一的, 他的作用与 label 标签相似, 用来引用节点。

```
1. ax-node-1 {
2.     phandle = <1>;
3.     interrupt-controller;
4. }
5.
6. ax-node-2 {
7.     interrupt-parent = <1>;
8. }
```

在节点 ax-node-1 中有 phandle 属性为<1>, 在 ax-node-2 中 interrupt-parent 属性需要指定父节点, 赋值为<1>即可。

标准属性先介绍这些, 还有一些特殊的以后用到再说。

3.1.4 在文件系统中查看设备树

内核启动时会解析 DTB 文件中的节点信息, 并在根文件系统的 /proc/devicetree 目录下创建一个节点对应的文件夹。

```
root@ax_peta:~# cd /proc/device-tree
root@ax_peta:/proc/device-tree# ls
#address-cells  alinxled      chosen          fpga-full      pmu@f8891000
#size-cells    alinxpwm      compatible     memory         usb_phy@0
aliases        amba          cpus           model
alinxkey       amba_pl      fixedregulator name
```

进入到对应的节点中, 能查看到节点的各个属性。

```
root@ax_peta:/proc/device-tree# cd memory/
root@ax_peta:/proc/device-tree/memory# ls
device_type  name      reg
root@ax_peta:/proc/device-tree/memory# cat device_type
memoryroot@ax_peta:/proc/device-tree/memory#
```

3.1.5 修改设备树

设备树本身有一定的标准, 不同的芯片厂家对于设备树有一些不同的自定义标准, 我们在修改设备树时, 有些需要遵循这些标准, 但我们却不知道标准时, 可以在内核源码目录 /Documentation/devicetree/bindings 中查看说明和指导。如果找不到, 那就只能咨询芯片厂家了。

3.2 of 函数

Linux 内核提供了 of 函数来让我们获取设备树中的信息, of 之名来自这些函数的前缀“of_”。of 函数的原型定义在内核目录 include/linux/of.h 中。这节我们介绍一些常用的 of 函数, 没有介绍的等用到的时候再去了解也不迟。

3.2.1 查找节点的 of 函数

1) device_node 结构体

device_node 结构体也定义在 include/linux/of.h 中，作为查找节点的 of 函数的返回值，用于给内核描述设备节点。

2) of_find_node_by_name 函数

of_find_node_by_name ()通过节点名查找节点，函数原型为：

```
struct device_node *of_find_node_by_name(struct device_node *from, const char *name);
```

参数说明：

from：从这个节点开始查找，输入 NULL 时从根节点开始查找。

name：目标节点名称。

返回值：找到目标节点返回 device_node 结构体。没有找到时返回 NULL。

3) of_find_node_by_type 函数

of_find_node_by_type ()通过 device_type 属性查找节点，函数原型为：

```
struct device_node *of_find_node_by_type(struct device_node *from, const char *type);
```

参数说明：

from：从这个节点开始查找，输入 NULL 时从根节点开始查找。

type：device_type 属性值。

返回值：找到目标节点返回 device_node 结构体。没有找到时返回 NULL。

4) of_find_compatible_node 函数

of_find_compatible_node ()通过 device_type 和 compatible 属性查找节点，函数原型为：

```
struct device_node *of_find_compatible_node(struct device_node *from, const char *type, const char *compatible);
```

参数说明：

from：从这个节点开始查找，输入 NULL 时从根节点开始查找。

type：device_type 属性值，输入 NULL 时忽略。

compatible：compatible 属性值。

返回值：找到目标节点返回 device_node 结构体。没有找到时返回 NULL。

5) of_find_node_by_path 函数

of_find_node_by_path ()通过节点路径查找节点，函数原型为：

```
struct device_node *of_find_node_by_path(const char *path);
```

参数说明：

path：节点的完整路径，以 3.1.1 节中的示例代码为例，21 行的 sw14 设备的完整路径为 /gpio-keys/sw14。

返回值：找到目标节点返回 device_node 结构体。没有找到时返回 NULL。

3.2.2 提取属性的 of 函数

1) property 结构体

property 结构体同样也定义在 include/linux/of.h 中，作为提取属性的 of 函数的返回值，用

于给内核描述节点属性。

2) of_find_property 函数

of_find_property 函数通过设备节点、属性名、属性值的大小查找属性，函数原型：

```
property *of_find_property(const struct device_node *np, const char *name, int *len);
```

参数说明：

np：设备节点。

name：目标属性名。

len：目标属性值的长度。

返回值：目标属性。

3) of_property_read_u32_array 函数

of_property_read_u32_array()用于获取有多个值的属性的多个数据，函数名以及输入函数中的 u32 代表目标属性单个值的大小，可替换为 u8、u16、u32。函数原型：

```
int of_property_read_u32_array(const struct device_node *np, const char *propname, u32 *out_values, size_t size);
```

参数说明：

np：设备节点。

propname：目标属性名。

out_values：读取到的数据指针，读取到的数据会保存到这个地址中。

size：要读取数据数量。

返回值：

0：读取成功；

-EINVAL：属性不存在；

-ENODATA：属性无数据；

-EOVERFLOW：属性值数据数量小于 size。

4) of_property_read_u32 函数

of_property_read_u32()用于获取只有单个值的属性数据，函数名以及输入函数中的 u32 代表目标属性单个值的大小，可替换为 u8、u16、u32。函数原型：

```
int of_property_read_u32(const struct device_node *np, const char *propname, u32 *out_value);
```

参数说明：

np：设备节点。

propname：目标属性名。

out_values：目标数据指针，读取到的数据会保存到这个地址中。

返回值：

0：读取成功；

-EINVAL：属性不存在；

-ENODATA：属性无数据。

5) of_property_read_string 函数

of_property_read_string 函数用于读取属性中字符串值，函数原型如下：

```
int of_property_read_string(struct device_node *np, const char *propname, const char
```

```
**out_string);
```

参数说明：

np：设备节点。

prname：目标属性名。

out_string：目标字符串指针，读取到的字符串会保存到该地址。

返回值：返回 0 读取成功。

3.3 设备树下的 led 驱动实验

通过上面两节大概了解设备树和 of 函数之后，光看书面上的东西很难理解深刻，还是得通过实验来深入理解。这节课还是使用简单的 led 设备来测试，不会涵盖上面讲的所有，但是在以后的实验中，会一直用到设备树，所以不用着急，在之后的实验中慢慢去掌握就行了。

3.3.1 原理图

和章节 1.3.1 的内容相同。

3.3.2 修改设备树

petalinux 的工程文件中提供了让我们修改的设备树文件，在工程目录“ax_peta/project-spec/meta-user/recipes-bsp/device-tree/files”中，ax_peta 是我的 petalinux 工程名，需要根据自身实际情况修改。

打开文件“system-conf.dtsi”，在根节点下添加以下节点内容：

```
1. alinxled {
2.     compatible = "alinxled";
3.     reg = <
4.         0xE000A204 0x04 /* gpio 方向寄存器 */
5.         0xE000A208 0x04 /* gpio 使能寄存器 */
6.         0xE000A040 0x04 /* gpio 控制寄存器 */
7.         0xF800012C 0x04 /* AMBA 外设时钟使能寄存器 */
8.     >;
9. };
```

节点名称为 alinxled，兼容性属性值为“alinxled”，reg 中的值即为在前两次实验中用到的 led 相关的寄存器。

如果你的“system-conf.dtsi”文件是空的，那就自己写一个根目录，再把上面的内容放进根目录即可。

3.3.3 驱动程序

使用 petalinux 新建名为“ax-dtled-drv”的驱动程序，别忘了用 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-dtled-drv.c 文件中输入下面的代码：

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/fs.h>
4. #include <linux/init.h>
5. #include <linux/types.h>
6. #include <linux/errno.h>
7. #include <linux/cdev.h>
8. #include <linux/of.h>
9. #include <linux/device.h>
```

```
10. #include <asm/uaccess.h>
11. #include <asm/io.h>
12. /* 设备节点名称 */
13. #define DEVICE_NAME "gpio_leds"
14. /* 设备号个数 */
15. #define DEVID_COUNT 1
16. /* 驱动个数 */
17. #define DRIVE_COUNT 1
18. /* 主设备号 */
19. #define MAJOR
20. /* 次设备号 */
21. #define MINOR 0
22.
23. /* gpio 寄存器虚拟地址 */
24. static u32 *GPIO_DIRM_0;
25. /* gpio 使能寄存器 */
26. static u32 *GPIO_OEN_0;
27. /* gpio 控制寄存器 */
28. static u32 *GPIO_DATA_0;
29. /* AMBA 外设时钟使能寄存器 */
30. static u32 *APER_CLK_CTRL;
31.
32. /* 把驱动代码中会用到的数据打包进设备结构体 */
33. struct alinx_char_dev{
34.     dev_t devid; //设备号
35.     struct cdev cdev; //字符设备
36.     struct class *class; //类
37.     struct device *device; //设备
38.     struct device_node *nd; //设备树的设备节点
39. };
40. /* 声明设备结构体 */
41. static struct alinx_char_dev alinx_char = {
42.     .cdev = {
43.         .owner = THIS_MODULE,
44.     },
45. };
46.
47. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
48. static int gpio_leds_open(struct inode *inode_p, struct file *file_p)
49. {
50.     /* MIO_0 时钟使能 */
51.     *APER_CLK_CTRL |= 0x00400000;
52.     /* MIO_0 设置成输出 */
53.     *GPIO_DIRM_0 |= 0x00000001;
54.     /* MIO_0 使能 */
55.     *GPIO_OEN_0 |= 0x00000001;
56.     printk("gpio_test module open\n");
57.     return 0;
58. }
59.
60. }
61.
62.
63. /* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
64. static ssize_t gpio_leds_write(struct file *file_p, const char __user *buf, size_t len, loff_t *
loff_t_p)
65. {
66.     int rst;
67.     char writeBuf[5] = {0};
68.     printk("gpio_test module write\n");
69.     rst = copy_from_user(writeBuf, buf, len);
70.     if(0 != rst)
71.     {
72.         return -1;
73.     }
74.     if(1 != len)
75.     {
76.         printk("gpio_test len err\n");
77.         return -2;
78.     }
79.     if(1 == writeBuf[0])
80.     {
81.         *GPIO_DATA_0 &= 0xFFFFF0;
82.         printk("gpio_test ON\n");
83.     }
84. }
```

```
86.     }
87.     else if(0 == writeBuf[0])
88.     {
89.         *GPIO_DATA_0 |= 0x00000001;
90.         printk("gpio_test OFF\n");
91.     }
92.     else
93.     {
94.         printk("gpio_test para err\n");
95.         return -3;
96.     }
97.
98.     return 0;
99. }
100.
101. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
102. static int gpio_leds_release(struct inode *inode_p, struct file *file_p)
103. {
104.     printk("gpio_test module release\n");
105.     return 0;
106. }
107.
108. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
109. static struct file_operations ax_char_fops = {
110.     .owner    = THIS_MODULE,
111.     .open     = gpio_leds_open,
112.     .write    = gpio_leds_write,
113.     .release  = gpio_leds_release,
114. };
115.
116. /* 模块加载时会调用的函数 */
117. static int __init gpio_led_init(void)
118. {
119.     /* 用于接受返回值 */
120.     u32 ret = 0;
121.     /* 存放 reg 数据的数组 */
122.     u32 reg_data[10];
123.
124.     /* 通过节点名称获取节点 */
125.     alinx_char.nd = of_find_node_by_name(NULL, "alinxled");
126.     /* 4、获取 reg 属性内容 */
127.     ret = of_property_read_u32_array(alinx_char.nd, "reg", reg_data, 8);
128.     if(ret < 0)
129.     {
130.         printk("get reg failed!\r\n");
131.         return -1;
132.     }
133.     else
134.     {
135.         /* do nothing */
136.     }
137.
138.     /* 把需要修改的物理地址映射到虚拟地址 */
139.     GPIO_DIRM_0 = ioremap(reg_data[0], reg_data[1]);
140.     GPIO_OEN_0  = ioremap(reg_data[2], reg_data[3]);
141.     GPIO_DATA_0 = ioremap(reg_data[4], reg_data[5]);
142.     APER_CLK_CTRL = ioremap(reg_data[6], reg_data[7]);
143.
144.     /* 注册设备号 */
145.     alloc_chrdev_region(&alinx_char.devid, MINOR, DEVID_COUNT, DEVICE_NAME);
146.
147.     /* 初始化字符设备结构体 */
148.     cdev_init(&alinx_char.cdev, &ax_char_fops);
149.
150.     /* 注册字符设备 */
151.     cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
152.
153.     /* 创建类 */
154.     alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
155.     if(IS_ERR(alinx_char.class))
156.     {
157.         return PTR_ERR(alinx_char.class);
158.     }
159.
160.     /* 创建设备节点 */
161.     alinx_char.device = device_create(alinx_char.class, NULL,
162.                                     alinx_char.devid, NULL,
163.                                     DEVICE_NAME);
```



```
164.     if (IS_ERR(alinx_char.device))
165.     {
166.         return PTR_ERR(alinx_char.device);
167.     }
168.
169.     return 0;
170. }
171.
172. /* 卸载模块 */
173. static void __exit gpio_led_exit(void)
174. {
175.     /* 注销字符设备 */
176.     cdev_del(&alinx_char.cdev);
177.
178.     /* 注销设备号 */
179.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
180.
181.     /* 删除设备节点 */
182.     device_destroy(alinx_char.class, alinx_char.devid);
183.
184.     /* 删除类 */
185.     class_destroy(alinx_char.class);
186.
187.     /* 释放对虚拟地址的占用 */
188.     iounmap(GPIO_DIRM_0);
189.     iounmap(GPIO_OEN_0);
190.     iounmap(GPIO_DATA_0);
191.     iounmap(APER_CLK_CTRL);
192.
193.     printk("gpio_led_dev_exit_ok\n");
194. }
195.
196. /* 标记加载、卸载函数 */
197. module_init(gpio_led_init);
198. module_exit(gpio_led_exit);
199.
200. /* 驱动描述信息 */
201. MODULE_AUTHOR("Alinx");
202. MODULE_ALIAS("gpio_led");
203. MODULE_DESCRIPTION("DEVICE TREE GPIO LED driver");
204. MODULE_VERSION("v1.0");
205. MODULE_LICENSE("GPL");
```

和上一章有区别的地方加粗了。

主要的改动集中在入口函数的 120~137 行。

126 行使用 of_find_node_by_name 函数通过节点名称获取节点, 因为 alinxled 节点挂在在根目录下, 所以第一个参数输入 NULL。

128 行, 在获取到节点后, 再获取节点中的 reg 属性的数据, 因为 reg 属性中存放着我们需要的寄存器地址和大小。总共 4 个地址 4 个 size, 因此数据总数为 8。

其他的操作与上一章基本相同。

因为修改了设备树, 在 3.1 节我们就说过, petalinux 在修改设备树后, 会编译出新的 BOOT.bin 和 image.ub。所以别忘了在编译完成后, 把新的 BOOT.bin 和 image.ub 拷贝到 SD 中, 并重启开发板。

3.3.4 测试程序

测试 APP 与章节 1.3.4 内容一致, 可以直接使用。

3.3.5 运行测试

因为 APP 相同, 所以测试方法任然相同, 只要能成功点亮 led 就成功了。

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
random: fast init done
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ax-dtled-dev.ko
ax_dtled_dev: loading out-of-tree module taints kernel.
root@ax_peta:/mnt# cd ./build-axleddev_test-ZYNQ-Debug/
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# ./axleddev_test /dev/gpio_leds on
gpio_test module open
ps_led1 on
gpio_test module write
gpio_test ON
gpio_test module release
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# ./axleddev_test /dev/gpio_leds off
gpio_test module open
ps_led1 off
gpio_test module write
gpio_test OFF
gpio_test module release
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug#
```

另外在系统运行之后，查看以下/proc/device-tree 路径中有没有我们添加的 alinxled 节点，并核对内容。

```
root@ax_peta:~# cd /proc/device-tree
root@ax_peta:/proc/device-tree# ls
#address-cells  amba          cpus              model
#size-cells     amba_pl        fixedregulator   name
aliases         chosen          fpga-full        pmu@f8891000
alinxled        compatible      memory           usb_phy@0
root@ax_peta:/proc/device-tree# cd ./alinxled/
root@ax_peta:/proc/device-tree/alinxled# ls
compatible  name          reg
root@ax_peta:/proc/device-tree/alinxled# cat ./compatible
alinxledroot@ax_peta:/proc/device-tree/alinxled# cat ./name
alinxledroot@ax_peta:/proc/device-tree/alinxled#
```

第四章 pinctrl 和 gpio 子系统

在前面的实验中，我们操作设备都是直接操作寄存器的。虽然上一章使用了设备树，但也只是把寄存器信息放到设备树中，本质上都是一样的。直接操作寄存器来控制硬件资源对于编码的人来说很方便，然而对于阅读代码的人来说想必是很崩溃的。而且在设备树中写入大量的寄存器地址，不仅不易于阅读，在使用时也很容易出错。比如上一章中，仅是控制 led 就用了 4 个寄存器，而且寄存器地址在设备树中的顺序也没有规则，全是自定的，在驱动程序中读取地址也无法对应和确认，最终我们在设置寄存器时就很容易错位。

pinctrl 子系统来可以应对这个问题，同时针对我们实验中用到的 led 也就是 gpio 操作来说还需要结合 gpio 子系统。

4.1 pinctrl 子系统

Linux 内核提供 pinctrl 子系统来统一芯片厂商的 pin 脚管理方法，他会帮我们完成以下工作：

- 1) 从设备树中获取 pin 脚信息；
- 2) 设置 pin 脚的复用功能；
- 3) 配置 pin 脚的电器特性。

使用 pinctrl 子系统，只需要在设备树中配置好相关节点信息，剩余的工作全由 pinctrl 子系统完成，我们只要做甩手掌柜就行了。

4.1.1 zynq 平台 pinctrl 子系统的设备树节点和属性

不同的芯片 pin 脚配置方法有差别，相应的芯片厂家的 pinctrl 子系统实现也会有差别，但我们需要关心的仍然只有设备树的写法。直接来看一段示例：

```
1. pinctrl0: pinctrl@700 {
2.     compatible = "xlnx,pinctrl-zynq";
3.     reg = <0x700 0x200>;
4.     syscon = <&slcr>;
5.     pinctrl_uart1_default: uart1-default {
6.         mux {
7.             groups = "uart1_10_grp";
8.             function = "uart1";
9.         };
10.        conf {
11.            groups = "uart1_10_grp";
12.            slew-rate = <0>;
13.            io-standard = <1>;
14.        };
15.        conf-rx {
16.            pins = "MI049";
17.            bias-high-impedance;
18.        };
19.        conf-tx {
20.            pins = "MI048";
21.            bias-disable;
22.        };
23.    };
24.};
```

第 1 行的 pinctrl0: pinctrl@700 节点，就是 ZYNQ 中 pinctrl 子系统的节点。pinctrl0 的父节点为 slcr@f8000000，700 是相对 f8000000 的偏移，也就是说 pin 的寄存器起始地址为 0xf8000700。可以在数据手册中找到对应的寄存器说明，如下图：

B.28 System Level Control Registers (slcr)

Module Name	System Level Control Registers (slcr)
Base Address	0xF8000000 slcr
Description	System Level Control Registers
Version	1.0
Doc Version	1.3, based on 11/18/2010 SLCR_spec.doc
Vendor Info	Xilinx Zynq slcr

MIO_PIN_00	0x00000700	32	rw	0x00001601	MIO Pin 0 Control
MIO_PIN_01	0x00000704	32	rw	0x00001601	MIO Pin 1 Control
MIO_PIN_02	0x00000708	32	rw	0x00000601	MIO Pin 2 Control
MIO_PIN_03	0x0000070C	32	rw	0x00000601	MIO Pin 3 Control
MIO_PIN_04	0x00000710	32	rw	0x00000601	MIO Pin 4 Control

不过寄存相关的内容不是这里我们需要关系的对象，示例的 2、3、4 行是 pinctrl0 的三个属性，都是 xilinx 设置好的，也不用去关心。重点是第 5 行开始，在 pinctrl0 节点中添加的子设备节点 uart1-default，我们要通过学习这个子设备节点的配置方法，添加我们自己需要的设备节点。这个节点下面没有属性，而是有 4 个子节点：mux、conf、conf-rx、conf-tx。其中 mux 是配置 pin 脚的复用功能，其他三个是配置 pin 脚的电气特性，conf 中设置通用的特性，conf-rx 中设置输入引脚和相关特性，conf-tx 中设置输出引脚和相关特性。

mux 节点中，有两个必要属性：

groups：选择这个复用功能的 pin 脚分组列表。可选的值有：

ethernet0_0_grp, ethernet1_0_grp, mdio0_0_grp, mdio1_0_grp, qspi0_0_grp, qspi1_0_grp, qspi_fbclk, qspi_cs1_grp, spi0_0_grp - spi0_2_grp, spi0_X_ssY (X=0..2, Y=0..2), spi1_0_grp - spi1_3_grp, spi1_X_ssY (X=0..3, Y=0..2), sdio0_0_grp - sdio0_2_grp, sdio1_0_grp - sdio1_3_grp, sdio0_emio_wp, sdio0_emio_cd, sdio1_emio_wp, sdio1_emio_cd, smc0_nor, smc0_nor_cs1_grp, smc0_nor_addr25_grp, smc0_nand, can0_0_grp - can0_10_grp, can1_0_grp - can1_11_grp, uart0_0_grp - uart0_10_grp, uart1_0_grp - uart1_11_grp, i2c0_0_grp - i2c0_10_grp, i2c1_0_grp - i2c1_10_grp, ttc0_0_grp - ttc0_2_grp, ttc1_0_grp - ttc1_2_grp, swdt0_0_grp - swdt0_4_grp, gpio0_0_grp - gpio0_53_grp, usb0_0_grp, usb1_0_grp。

这些分组包含哪些 pin 脚，可以在 drivers/pinctrl/pinctrl-zynq.c 文件中找到相应的数组，比如示例中的 group“uart1_10_grp”，对应的数组为 static const unsigned int uart1_10_pins[] = {48, 49}，这个 group 包含的 pin 脚即为 pin48 和 pin49。

function：选择 pin 脚分组的复用功能。可选的值有：

ethernet0, ethernet1, mdio0, mdio1, qspi0, qspi1, qspi_fbclk, qspi_cs1, spi0, spi0_ss, spi1, spi1_ss, sdio0, sdio0_pc, sdio0_cd, sdio0_wp, sdio1, sdio1_pc, sdio1_cd, sdio1_wp, smc0_nor, smc0_nor_cs1, smc0_nor_addr25, smc0_nand, can0, can1, uart0, uart1, i2c0, i2c1, ttc0, ttc1, swdt0, gpio0, usb0, usb1。

示例中的设备节点名为 uart1-default，用作 uart1，所以 function 的值是 uart1。

需要什么功能，可以用什么引脚，可以从数据手册中查看，也可以直接在 drivers/pinctrl/pinctrl-zynq.c 文件中查看，很容易看出对应关系。

conf 节点可以分为多组，因为相同复用功能的 pin 组里每个 pin 的电器特性可能会不同，比

如示例中的 uart1, 有一个 pin 脚用作输出不需要设置偏置, 另一个用做输入需要设置成高阻。conf 节点的名字必须要包含“conf”这个关键词, 其他的关键词仅用于区分分组含义, 比如示例中的 conf-rx 是指这个 conf 节点适用于配置输入 pin 脚的, 命名为 conf-in 也是同样的效果, 甚至把示例中 rx 和 tx 交换, 对使用也没用影响, 仅用于含以上的区分。

conf 节点的必要属性为 **pin** 和 **group** 二选一。设置 pin 组时使用属性 **group**, 如示例中第 10 行的 conf 节点, 设置单个 pin 或多个不在一个 group 中的 pin 时使用属性 **pin**, 如 15 行的 conf-rx 和 19 行的 conf-tx。

group 属性设上面 mux 节点中的一样。

pin: 使用这个 conf 的 pin 脚列表。可选的值为: MIO0 ~ MIO53。

conf 节点还有一些可选属性:

io-standard: 设置 io 电平标准, 可选值为:

- 1: LVCMOS18
- 2: LVCMOS25
- 3: LVCMOS33
- 4: HSTL

bias-disable、**bias-high-impedance**、**bias-pull-up**: 设置 pin 脚偏置, 不需要赋值, 只需要在对应的 conf 节点中包含需要的偏置属性即可。

slew-rate: 设置转换速率, 等于 0 时为 slow, 等于 1 时为 fast。

low-power-disable、**low-power-enable**: 使能低功耗模式, 在对应的 conf 节点中包含需要的偏置属性即可。

还有其他的可选属性不就一一介绍了, 可以在内核源码目录

Documentation/devicetree/bindings/pinctrl/pinctrl-binding.txt 文件中, 搜索“[Generic pin configuration node content](#)”词条查看。

4.1.2 pinctrl 子系统的使用

在 pinctrl 节点中添加了设备子节点后, 系统还不回去初始化这个设备, 我们需要在客户设备(client device)节点中去调用他。举个例子:

```
1. device {
2.     pinctrl-names = "active", "idle";
3.     pinctrl-0 = <&state_0_node_a>;
4.     pinctrl-1 = <&state_1_node_a &state_1_node_b>;
5. };
```

device 是客户设备节点, 是调用 pinctrl 的设备。

pinctrl-0 是必选属性, 是这个设备对应 pin 的 pinctrl 节点应用列表, 如果这个设备对应了多个 pin, 那就增加属性 **pinctrl-0~n**, 如果一个 pin 对应了多个 pinctrl, 就如第 4 行的 pinctrl-1 属性一般附上多个 pinctrl 节点。

pinctrl-names 是下面 pinctrl0~n 的名称列表, 命名是自定的但要尽量易读并贴合实际。

4.2 gpio 子系统

pinctrl 子系统中, 如果把功能配置成 gpio 的话, 就需要用到 gpio 子系统了。gpio 子系统帮

我们实现了 gpio 初始化，并提供了一些接口函数给我们操作 gpio。我们要做的同样只是在设备树中配置好信息，然后了解一下接口函数的用法即可。

4.2.1 设备树中的 gpio

在 Documentation/devicetree/bindings/gpio/gpio-zynq.txt 文件中，介绍了 zynq 系列芯片 gpio 在设备数中的描述方式，我们在现有的 zynq 设备数中，也能找到 gpio 的默认配置，如下：

```
1. gpio0:gpio@e000a000 {
2.     #gpio-cells = <2>;
3.     compatible = "xlnx,zynq-gpio-1.0";
4.     clocks = <&clkc 42>;
5.     gpio-controller;
6.     interrupt-parent = <&intc>;
7.     interrupts = <0 20 4>;
8.     interrupt-controller;
9.     #interrupt-cells = <2>;
10.    reg = <0xe000a000 0x1000>;
11. };
```

这些属性都是必要属性。

interrupt-parent、interrupts、interrupt-controller、#interrupt-cells：这四个属性是中断相关的，等之后讲中断的时候再说。

clocks 属性时时钟设置相关，暂时使用默认配置即可。

#gpio-cells：指代引用这个 gpio 节点时需要两个 cell，他的值必须是 2，即引用节点时需要有两个 cell，第一个 cell 为 gpio 编号，第二个 cell 用于指顶可选参数，第二个 cell 在 zynq 中没有用到填 0 即可。举个例子：alinxled-gpios = <&gpio0 0 0>;，即客户设备的使用的 gpio0_mio0 这个 io。

compatible：兼容新，必须为"xlnx, zynq-gpio-1.0"或"xlnx, zynqmp-gpio-1.0"。

gpio-controller：指定该节点为 gpio 控制器。

reg：这组 gpio 的寄存器首地址和范围。

这里我们不需要修改什么，保持现有配置即可。

4.2.2 gpio 接口函数

gpio 子系统提供接口函数来代替我们直接对寄存器的操作把驱动程序再次分离分层。这节我们介绍一些常用的 gpio 接口函数。

1) of 函数扩展

在介绍 gpio 接口函数之前，先扩展一个 gpio 相关的 of 函数 of_get_named_gpio，用于通过设备树中节点属性如"gpio0 = <&gpio0 0 0 &gpio0 1 0>;"获取 gpio 编号，函数原型如下：

```
int of_get_named_gpio(struct device_node *np, const char *propname, int index);
```

参数说明：

np：设备节点。

propname：属性名称，从这个属性中获取 gpio 编号。

index：需要获取的 gpio 下标。

返回值：gpio 编号；返回负值则获取失败。

2) gpio_request()

在使用 gpio 做其他操作之前，需要先用 gpio_request()函数申请一个 io。至今为止接触到的 Linux 内核提供的接口，都是这种面向对象的思想，先声明一个对象，再操作这个对象。

函数原型:

```
int gpio_request(unsigned int gpio, const char *label);
```

参数说明:

gpio: 需要申请的 gpio 标号, 通过上面介绍的 of_get_named_gpio 函数获取。

label: 申请到的 gpio 的标签。

返回值: 0 表示成功, 其他表示失败。

3) gpio_free()

释放 gpio, 与 gpio_request()相对, 原型:

```
void gpio_free(unsigned int gpio);
```

gpio: 需要释放的 gpio 标号。

4) gpio_direction_input()

设置 gpio 为输入, 函数原型:

```
int gpio_direction_input(unsigned gpio);
```

gpio: 需要设置的 gpio 标号。

返回值: 0 表示成功, 负值表示失败。

5) gpio_direction_output()

设置 gpio 为输出, 函数原型:

```
int gpio_direction_output(unsigned gpio, int value);
```

gpio: 需要设置的 gpio 标号。

value: 输出的默认电平。

返回值: 0 表示成功, 负值表示失败。

6) gpio_get_value()

读取 gpio 的电平, 原型:

```
int __gpio_get_value(unsigned gpio);
```

gpio: 需要读取的 gpio 标号。

返回值: 读取到的电平, 负值表示失败。

7) gpio_set_value()

设置 gpio 的电平, 原型:

```
void __gpio_set_value(unsigned gpio, int value);
```

gpio: 需要操作的 gpio 标号。

value: 需要写入的电平。

4.3 实验

必要的知识都了解了, 接下来通过实验来验证。仍然使用 led 来测试。

4.3.1 原理图

和章节 1.3.1 的内容相同。

4.3.2 设备树

打开文件“system-conf.dtsi”, 在根节点下添加以下节点内容:

```
1.  ambal {
2.      gpio@e000a000 {
3.          compatible = "xlnx,zynq-gpio-1.0";
```



```

4.     #gpio-cells = <0x2>;
5.     clocks = <0x1 0x2a>;
6.     gpio-controller;
7.     interrupt-controller;
8.     #interrupt-cells = <0x2>;
9.     interrupt-parent = <0x4>;
10.    interrupts = <0x0 0x14 0x4>;
11.    reg = <0xe000a000 0x1000>;
12.    };
13.
14.    slcr@f8000000 {
15.        pinctrl@700 {
16.            pinctrl_led_default: led-default {
17.                mux {
18.                    groups = "gpio0_0_grp";
19.                    function = "gpio0";
20.                };
21.
22.                conf {
23.                    pins = "MIO0";
24.                    io-standard = <1>;
25.                    bias-disable;
26.                    slew-rate = <0>;
27.                };
28.            };
29.        };
30.    };
31. };
32.
33. alinxled {
34.     compatible = "alinx-led";
35.     pinctrl-names = "default";
36.     pinctrl-0 = <&pinctrl_led_default>;
37.     alinxled-gpios = <&gpio0 0 0>;
38. };

```

第 2~12 行是 gpio 节点，这个内容其实是 zynq 设备树中已有的，内容 4.2.1 节讲过了。

第 16~28 行我们新增的 pin 设备节点，用于配置 led 用到的 pin 脚。led 链接的 io 是 MIO0，所在分组为"gpio0_0_grp"，功能就是 gpio0。因为只有一个引脚，所以只要一个 conf 节点就行了，转换率设置为 slow，电平标准为 1.8，偏置失能，配置目标为"MIO0"。

第 33 行则是我们的 led 设备节点，对于 gpio 子系统和 pinctrl 子系统来说，就是客户设备(client device)，通过属性 pinctrl-0 = <&pinctrl_led_default>调用 pin 节点，通过属性 alinxled-gpios = <&gpio0 0 0>来绑定 gpio。这个属性的名字 alinxled-gpios 需要记号，稍后我们要通过这个名字来获取 gpio 标号。

4.3.3 驱动代码

使用 petalinux 新建名为"ax-pinioled-drv"的驱动程序，并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-pinioled-drv.c 文件中输入下面的代码：

```

1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/fs.h>
4. #include <linux/init.h>
5. #include <linux/types.h>
6. #include <linux/errno.h>
7. #include <linux/cdev.h>
8. #include <linux/of.h>
9. #include <linux/of_address.h>
10. #include <linux/of_gpio.h>
11. #include <linux/device.h>
12. #include <linux/delay.h>
13. #include <linux/init.h>
14. #include <linux/gpio.h>
15. #include <asm/uaccess.h>
16. #include <asm/mach/map.h>

```

```
17. #include <asm/io.h>
18.
19. /* 设备节点名称 */
20. #define DEVICE_NAME      "gpio_leds"
21. /* 设备号个数 */
22. #define DEVID_COUNT      1
23. /* 驱动个数 */
24. #define DRIVE_COUNT      1
25. /* 主设备号 */
26. #define MAJOR1
27. /* 次设备号 */
28. #define MINOR1          0
29. /* LED 点亮时输入的值 */
30. #define ALINX_LED_ON     1
31. /* LED 熄灭时输入的值 */
32. #define ALINX_LED_OFF   0
33.
34.
35. /* 把驱动代码中会用到的数据打包进设备结构体 */
36. struct alinx_char_dev{
37.     dev_t      devid;          //设备号
38.     struct cdev cdev;          //字符设备
39.     struct class *class;       //类
40.     struct device *device;      //设备
41.     struct device_node *nd;     //设备树的设备节点
42.     int         alinx_led_gpio; //gpio 号
43. };
44. /* 声明设备结构体 */
45. static struct alinx_char_dev alinx_char = {
46.     .cdev = {
47.         .owner = THIS_MODULE,
48.     },
49. };
50.
51. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
52. static int gpio_leds_open(struct inode *inode_p, struct file *file_p)
53. {
54.     /* 设置私有数据 */
55.     file_p->private_data = &alinx_char;
56.     printk("gpio_test module open\n");
57.     return 0;
58. }
59.
60.
61. /* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
62. static ssize_t gpio_leds_write(struct file *file_p, const char __user *buf, size_t len, loff_t *
loff_t_p)
63. {
64.     int retvalue;
65.     unsigned char databuf[1];
66.     /* 获取私有数据 */
67.     struct alinx_char_dev *dev = file_p->private_data;
68.     retvalue = copy_from_user(databuf, buf, len);
69.     if(retvalue < 0)
70.     {
71.         printk("alinx led write failed\n");
72.         return -EFAULT;
73.     }
74.
75.     if(databuf[0] == ALINX_LED_ON)
76.     {
77.         gpio_set_value(&alinx_char.alinx_led_gpio, !!0);
78.     }
79.     else if(databuf[0] == ALINX_LED_OFF)
80.     {
81.         gpio_set_value(&alinx_char.alinx_led_gpio, !!1);
82.     }
83.     else
84.     {
85.         printk("gpio_test para err\n");
86.     }
87.
88.     return 0;
89. }
90.
91. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
92. static int gpio_leds_release(struct inode *inode_p, struct file *file_p)
```

```
93. {
94.     printk("gpio_test module release\n");
95.     return 0;
96. }
97.
98. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
99. static struct file_operations ax_char_fops = {
100.     .owner    = THIS_MODULE,
101.     .open     = gpio_leds_open,
102.     .write    = gpio_leds_write,
103.     .release  = gpio_leds_release,
104. };
105.
106. /* 模块加载时会调用的函数 */
107. static int __init gpio_led_init(void)
108. {
109.     /* 用于接受返回值 */
110.     u32 ret = 0;
111.
112.     /* 获取设备节点 */
113.     alinx_char.nd = of_find_node_by_path("/alinxled");
114.     if(alinx_char.nd == NULL)
115.     {
116.         printk("alinx_char node not find\n");
117.         return -EINVAL;
118.     }
119.     else
120.     {
121.         printk("alinx_char node find\n");
122.     }
123.
124.     /* 获取节点中 gpio 标号 */
125.     alinx_char.alinx_led_gpio = of_get_named_gpio(alinx_char.nd, "alinxled-gpios", 0);
126.     if(alinx_char.alinx_led_gpio < 0)
127.     {
128.         printk("can not get alinxled-gpios");
129.         return -EINVAL;
130.     }
131.     printk("alinxled-gpio num = %d\n", alinx_char.alinx_led_gpio);
132.
133.     /* 申请 gpio 标号对应的引脚 */
134.     gpio_request(alinx_char.alinx_led_gpio, "alinxled");
135.     if(ret != 0)
136.     {
137.         printk("can not request gpio\n");
138.     }
139.
140.     /* 把这个 io 设置为输出 */
141.     ret = gpio_direction_output(alinx_char.alinx_led_gpio, 1);
142.     if(ret < 0)
143.     {
144.         printk("can not set gpio\n");
145.     }
146.
147.     /* 注册设备号 */
148.     alloc_chrdev_region(&alinx_char.devid, MINOR1, DEVID_COUNT, DEVICE_NAME);
149.
150.     /* 初始化字符设备结构体 */
151.     cdev_init(&alinx_char.cdev, &ax_char_fops);
152.
153.     /* 注册字符设备 */
154.     cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
155.
156.     /* 创建类 */
157.     alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
158.     if(IS_ERR(alinx_char.class))
159.     {
160.         return PTR_ERR(alinx_char.class);
161.     }
162.
163.     /* 创建设备节点 */
164.     alinx_char.device = device_create(alinx_char.class, NULL,
165.                                     alinx_char.devid, NULL,
166.                                     DEVICE_NAME);
167.     if (IS_ERR(alinx_char.device))
168.     {
169.         return PTR_ERR(alinx_char.device);
170.     }
```

```
171.
172.     return 0;
173. }
174.
175. /* 卸载模块 */
176. static void __exit gpio_led_exit(void)
177. {
178.     /* 释放 gpio */
179.     gpio_free(alinx_char.alinx_led_gpio);
180.
181.     /* 注销字符设备 */
182.     cdev_del(&alinx_char.cdev);
183.
184.     /* 注销设备号 */
185.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
186.
187.     /* 删除设备节点 */
188.     device_destroy(alinx_char.class, alinx_char.devid);
189.
190.     /* 删除类 */
191.     class_destroy(alinx_char.class);
192.
193.     printk("gpio_led_dev_exit_ok\n");
194. }
195.
196. /* 标记加载、卸载函数 */
197. module_init(gpio_led_init);
198. module_exit(gpio_led_exit);
199.
200. /* 驱动描述信息 */
201. MODULE_AUTHOR("Alinx");
202. MODULE_ALIAS("gpio_led");
203. MODULE_DESCRIPTION("PINCTRL AND GPIO LED driver");
204. MODULE_VERSION("v1.0");
205. MODULE_LICENSE("GPL");
```

与上一章驱动代码相比有却别的部分加粗了，主要是在入口函数中用 gpio 子系统获取 gpio 并初始化替换了原先获取寄存器再初始化的操作。

43 行在设备结构体中定义了 gpio 标号便于后续的获取操作。

55 行原先通过寄存器初始化 io 的操作删除了。

56 行在 open 函数中设置私有数据，之后在 write 函数中可以获取使用。

68 行获取私有数据。

79 和 83 行使用 gpio_set_value 函数来设置 gpio 的电平，用获取的私有数据应用 gpio 标号，第二个输入参数用!!二值化。

115 行获取/alinxled 节点。

127 行使用 of_get_named_gpio 函数，通过属性 alinxled-gpios 获取 gpio 标号。

136 行使用 gpio_request 函数申请标号对应的引脚。

143 行使用 gpio_direction_output 函数把 gpio 设置为输出。

181 行在卸载模块时，增加 gpio 释放操作。

4.3.4 测试代码

测试 APP 与章节 1.3.4 内容一致，可以直接使用。

4.3.5 运行测试

因为 APP 相同，所以测试方法任然相同，只要能成功点亮 led 就成功了。

```
root@ax_peta:/mnt# insmod ./ax-pinioled-dev.ko
ax_pinioled_dev: loading out-of-tree module taints kernel.
alinx_char mode find
alinxled-gpio num = 899
root@ax_peta:/mnt#
```

```
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# ./axleddev_test /dev/gpio_leds on
gpio_test module open
ps_led1 on
alinx led on
gpio_test module release
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug#
```

第五章 并发的处理

Linux 这样的多任务操作系统，都无法避免并发的问题。这一章我们简单了解并发和并发的处理方式。

5.1 Linux 中的并发

所谓并发就是指多个任务同时访问同一个资源，内存就是资源。内存出错，很有可能导致系统崩溃，所以必须要极力避免并发问题的发生。

并发访问产生的原因众多，比如多线程访问、抢占、中断等等，这些都是系统机制很难避免。要避免并发访问带来的问题，不如去识别出会被同时访问的共享资源以及会访问共享资源的程序即临界区，并做好保护。当然很多时候识别出需要保护的资源并非易事，这我们只能从不断的积累中去总结。

5.2 Linux 对并发的处理

识别出共享资源后，我们就可以使用 Linux 内核提供的保护共享资源机制来避免并发访问。这节我们介绍几种内核提供的并发处理方法。

5.2.1 原子操作

原子操作是指不能被分割或打断的操作，Linux 内核提供了原子变量和系列函数来实现原子操作。文件 include/linux/types.h 中，原子变量定义如下：

```
1. typedef struct {  
2.     int counter;  
3. } atomic_t;
```

定义原子变量的方式为：

```
1. atomic_t a = ATOMIC_INIT(x);
```

ATOMIC_INIT(x)是用于原子变量初始化的宏定义，x 是我们赋予的初值，不需要的初值的话可以省略。

原子变量的读写操作也要内核提供的接口函数，除了上面说的 ATOMIC_INIT(x)外，还有些常用的接口函数如下：

```
/* 读取 v 并且返回 */  
int atomic_read(atomic_t *v);  
/* 把 i 写入 v */  
void atomic_set(atomic_t *v, int i);  
/* v 的值减去 i */  
void atomic_add(int i, atomic_t *v);  
/* v 的值加上 i */  
void atomic_sub(int i, atomic_t *v);
```

```
/*自增 */
void atomic_inc(atomic_t *v);
/* v 自减 */
void atomic_dec(atomic_t *v);
/* v 自增并返回 */
int atomic_inc_return(atomic_t *v);
/* v 自减并返回 */
int atomic_dec_return(atomic_t *v);
```

当然还不止这些，除了对整型的操作，内核还提供了原子的位操作，等需要的时候再去了解吧。具体用法也放到之后的实验中再去分析。

5.2.2 锁机制

原子操作只能对整形变量和位起到保护作用，很多场景都不适用。比如需要保护结构多变的结构体变量时就需要用到其他的机制，比如这节要讲的锁机制。内核中的锁也用很多种，先介绍一种常用的自旋锁。

自旋锁的机制是，某个线程要访问共享资源时，需要先获取相应的锁即上锁，只要不释放这个锁即解锁，别的线程就无法获取锁也就无法访问共享资源。此时需要却没有获取到自旋锁的线程就会一直处于等待状态(阻塞)。线程的等待状态会浪费很多处理器资源，所以占用自旋锁的临界区要尽量的轻便，更不能调用或存在阻塞的函数或逻辑。

所是一个形象比喻，说到底，也就是个结构体，定义如下：

```
1. typedef struct spinlock {
2.     union {
3.         struct raw_spinlock rlock;
4.         #ifdef CONFIG_DEBUG_LOCK_ALLOC
5.         # define LOCK_PADSIZE (offsetof(struct raw_spinlock, dep_map))
6.         struct {
7.             u8 __padding[LOCK_PADSIZE];
8.             struct lockdep_map dep_map;
9.         };
10.    #endif
11.    };
12. } spinlock_t;
```

同样的有了数据结构，还得有相应的操作函数，常用的自旋锁接口函数有：

```
/*自旋锁初始化 */
int spin_lock_init(spinlock_t *lock);
/*获取自旋锁 */
void spin_lock(spinlock_t *lock);
/*释放自旋锁 */
void spin_unlock(spinlock_t *lock);
/*获取自旋锁，没有获取到则返回 0 */
int spin_trylock(spinlock_t *lock);
/*检查自旋锁是否已被获取，返回 0 则已被获取，其他值则未被获取 */
int spin_is_locked(spinlock_t *lock);
/*禁用本地中断，并获取自旋锁 */
void spin_lock_irq(spinlock_t *lock);
/*恢复本地中断，并释放自旋锁 */
```



```
void spin_unlock_irq(spinlock_t *lock);
/*保存中断状态，禁用本地中断，并获取自旋锁，flags 为中断状态 */
void spin_lock_irqsave(spinlock_t *lock, unsigned long flags);
/*恢复中断至保存的状态，恢复本地中断，并获释放旋锁 */
void spin_unlock_irqrestore(spinlock_t *lock, unsigned long flags;
```

使用自旋锁，需要注意：

- 1) **使用自旋锁的临界区必定不能进入休眠。**自旋锁被成功获取后，内核会停用抢占机制。假如现在有个临界区 A 获取了自旋锁，然后进入休眠主动放弃了 CPU 使用权，线程 B 开始运行。线程 B 的临界区也想访问共享数据，但是自旋锁已被占用，B 线程的临界区就一直处于等待自旋锁的状态，而此时内核抢占被禁止，临界区 A 无法获得主动权释放锁，从而发生了死锁。
- 2) **临界区内要避免中断抢占。**假设现在有线程获取了自旋锁，在释放之前，被中断抢占了，如果中断也需要获取共享数据，也去申请自旋锁，那中断就会进入等待状态，那就大事不妙了，直接死锁。这里要避免终端抢占的办法，就是使用上面介绍的 `spin_lock_irq` 函数和 `spin_lock_irqsave` 函数来申请自旋锁，禁用本地中断。
- 3) **临界区要尽量短。**原因上面有解释过了。

使用示例：

```
1. spinlock_t lock;
2. spin_lock (&gpioled.lock); //上锁
3. /* 临界区 */
4. spin_unlock (&gpioled.lock); //解锁
```

上锁和解锁中间就是临界区。

5.2.3 信号量

与自旋锁相比，信号量有两个优势：

- 1) 信号量可以让等待信号量的线程进入休眠，减少 CPU 的占用；
- 2) 信号量支持对个线程同时访问共享资源。

信号量结构体定义如下：

```
1. struct semaphore {
2.     raw_spinlock_t lock;
3.     unsigned int count;
4.     struct list_head wait_list;
5. };
```

元素 `count` 即指信号量支持同时访问共享资源的线程数。

常用的信号量接口函数有：

```
/* 初始化信号量，设置信号值(同时访问个数)为 val */
void sema_init(struct semaphore *sem, int val);
/* 获取信号量，失败时会进入休眠且不可以被信号打断，不能用于中断 */
void down(struct semaphore *sem);
/* 获取信号量，不会进入休眠，成功返回 0 */
int down_trylock(struct semaphore *sem);
/* 获取信号量，失败时会进入休眠但可以被信号打断，不能用于中断，休眠被打
```

```
断时返非 0 值 */
int down_interruptible(struct semaphore *sem);
/* 释放信号量 */
void up(struct semaphore *sem);
```

使用信号量要注意的地方：

- 1) 线程休眠后会切换线程，如果占用信号量的临界区很短，导致频繁切换线程，也会带来大量的开销，因此与自旋锁相反，**信号量不适用于共享资源使用时间很短的场合**；
- 2) 中断不能休眠，因此**中断中不能使用会进入休眠的方式申请信号量**；
- 3) 信号量**允许多个线程同时访问共享资源时(count > 1)**，**不可用于互斥访问**(互斥访问指一次只能有一个线程访问共享资源)。

使用示例：

```
1. struct semaphore sem;
2. sema_init(&sem, 1);
3. down(&sem);
4. /* 临界区 */
5. up(&sem);
```

5.3 实验

5.3.1 原理图

和章节 1.3.1 的内容相同。

5.3.2 设备树

和上一章 4.3.2 相同。

5.3.3 驱动代码

以上一章的驱动代码为例，首先识别出共享资源。led 设备的驱动，那共享资源就是 led 设备了，如果有多个应用程序来调用这个驱动，最终就是 led 设备会被多个应用程序操作。再进一步的说，就是 led 设备节点，即/dev/gpio_leds 这个设备文件，在操作这个设备时，不能让别的应用程序再操作。而临界区就是从占用这个节点开始(即 open 函数打开/dev/gpio_leds 这个设备文件)到释放这个节点为止(close 设备文件)。

这章有三个机制要实验，但重复的代码很多，为了简便，我把三个机制都写在一个代码里。用宏定义开关来隔开。使用 petalinux 新建驱动名为“ax-concled-drv”，在 ax-concled-drv.c 中输入以下代码：

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/fs.h>
4. #include <linux/init.h>
5. #include <linux/types.h>
6. #include <linux/errno.h>
7. #include <linux/cdev.h>
8. #include <linux/of.h>
9. #include <linux/of_address.h>
10. #include <linux/of_gpio.h>
11. #include <linux/device.h>
12. #include <linux/delay.h>
13. #include <linux/init.h>
14. #include <linux/gpio.h>
15. #include <asm/uaccess.h>
```

```
16. #include <asm/mach/map.h>
17. #include <asm/io.h>
18.
19. /* 设备节点名称 */
20. #define DEVICE_NAME      "gpio_leds"
21. /* 设备号个数 */
22. #define DEVID_COUNT      1
23. /* 驱动个数 */
24. #define DRIVE_COUNT      1
25. /* 主设备号 */
26. #define MAJOR1
27. /* 次设备号 */
28. #define MINOR1          0
29. /* LED 点亮时输入的值 */
30. #define ALINX_LED_ON     1
31. /* LED 熄灭时输入的值 */
32. #define ALINX_LED_OFF    0
33.
34. /* 原子变量开关 */
35. #define ATOMIC_T_ON
36. /* 自旋锁开关 */
37. // #define SPINKLOCK_T_ON
38. /* 信号量开关 */
39. // #define SEMAPHORE_ON
40.
41. /* 把驱动代码中会用到的数据打包进设备结构体 */
42. struct alinx_char_dev{
43.     dev_t      devid;           //设备号
44.     struct cdev cdev;           //字符设备
45.     struct class *class;        //类
46.     struct device *device;       //设备
47.     struct device_node *nd;      //设备树的设备节点
48.     int         alinx_led_gpio;  //gpio 号
49.
50. #ifdef ATOMIC_T_ON
51.     atomic_t    lock;           //原子变量
52. #endif
53.
54. #ifdef SPINKLOCK_T_ON
55.     spinlock_t  lock;           //自旋锁变量
56.     int         source_status;  //资源占用状态
57. #endif
58.
59. #ifdef SEMAPHORE_ON
60.     struct semaphore lock;
61. #endif
62. };
63. /* 声明设备结构体 */
64. static struct alinx_char_dev alinx_char = {
65.     .cdev = {
66.         .owner = THIS_MODULE,
67.     },
68. };
69.
70. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
71. static int gpio_leds_open(struct inode *inode_p, struct file *file_p)
72. {
73.     /* 应用程序调用了 open 函数表示需要调用共享资源 */
74. #ifdef ATOMIC_T_ON
75.     /* 通过判断原子变量的值来判断资源的占用状态 */
76.     if (!atomic_read(&alinx_char.lock))
77.     {
78.         /* 若原子变量值为 0, 则资源没有被占用,
79.          * 此时把原子变量加 1, 表示之后资源就被占用了 */
80.         atomic_inc(&alinx_char.lock);
81.     }
82.     else
83.     {
84.         /* 否则资源被占用, 返回忙碌 */
85.         return -EBUSY;
86.     }
87. #endif
88.
89. #ifdef SPINKLOCK_T_ON
90.     /* 获取自旋锁 */
91.     spin_lock(&alinx_char.lock);
```

```
92.     /* 判断资源占用状态 */
93.     if(!alinx_char.source_status)
94.     {
95.         /* 为 0 则未被占用,
96.            此时把状态值加 1, 表示之后资源就被占用了 */
97.         alinx_char.source_status ++;
98.         /* 释放锁 */
99.         spin_unlock(&alinx_char.lock);
100.    }
101.    else
102.    {
103.        /* 释放锁 */
104.        spin_unlock(&alinx_char.lock);
105.        /* 否则资源被占用, 返回忙碌 */
106.        return -EBUSY;
107.    }
108. #endif
109.
110. #ifdef SEMAPHORE_ON
111.     /* 获取信号量 */
112.     down(&alinx_char.lock);
113. #endif
114.
115.     /* 设置私有数据 */
116.     file_p->private_data = &alinx_char;
117.     printk("gpio_test module open\n");
118.     return 0;
119. }
120.
121.
122. /* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
123. static ssize_t gpio_leds_write(struct file *file_p, const char __user *buf, size_t len, loff_t *
    loff_t_p)
124. {
125.     int retvalue;
126.     unsigned char databuf[1];
127.     /* 获取私有数据 */
128.     struct alinx_char_dev *dev = file_p->private_data;
129.
130.     retvalue = copy_from_user(databuf, buf, len);
131.     if(retvalue < 0)
132.     {
133.         printk("alinx led write failed\n");
134.         return -EFAULT;
135.     }
136.
137.     if(databuf[0] == ALINX_LED_ON)
138.     {
139.         gpio_set_value(dev->alinx_led_gpio, !!0);
140.     }
141.     else if(databuf[0] == ALINX_LED_OFF)
142.     {
143.         gpio_set_value(dev->alinx_led_gpio, !!1);
144.     }
145.     else
146.     {
147.         printk("gpio_test para err\n");
148.     }
149.
150.     return 0;
151. }
152.
153. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
154. static int gpio_leds_release(struct inode *inode_p, struct file *file_p)
155. {
156.     /* 应用程序调用 close 函数, 宣布资源已使用完毕 */
157. #ifdef ATOMIC_T_ON
158.     /* 原子变量恢复为 0, 表示资源已使用完毕 */
159.     atomic_set(&alinx_char.lock, 0);
160. #endif
161.
162. #ifdef SPINLOCK_T_ON
163.     /* 获取自旋锁 */
164.     spin_lock(&alinx_char.lock);
165.     /* 资源占用状态恢复为 0, 表示资源已使用完毕 */
166.     alinx_char.source_status = 0;
167.     /* 释放锁 */
```

```
168.     spin_unlock(&alinx_char.lock);
169. #endif
170.
171. #ifdef SEMAPHORE_ON
172.     /* 释放信号量 */
173.     up(&alinx_char.lock);
174. #endif
175.
176.     printk("gpio_test module release\n");
177.     return 0;
178. }
179.
180. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
181. static struct file_operations ax_char_fops = {
182.     .owner    = THIS_MODULE,
183.     .open     = gpio_leds_open,
184.     .write    = gpio_leds_write,
185.     .release  = gpio_leds_release,
186. };
187.
188. /* 模块加载时会调用的函数 */
189. static int __init gpio_led_init(void)
190. {
191.     /* 用于接受返回值 */
192.     u32 ret = 0;
193.
194. #ifdef ATOMIC_T_ON
195.     /* 设置原子变量为 0, 即资源为未被占用的状态 */
196.     atomic_set(&alinx_char.lock, 0);
197. #endif
198.
199. #ifdef SPINLOCK_T_ON
200.     /* 初始化自旋锁 */
201.     spin_lock_init(&alinx_char.lock);
202.     /* 初始化资源占用状态为 0, 意为资源没有被占用 */
203.     alinx_char.source_status = 0;
204. #endif
205.
206. #ifdef SEMAPHORE_ON
207.     /* 初始化信号量 */
208.     sema_init(&alinx_char.lock, 1);
209. #endif
210.
211.     /* 获取设备节点 */
212.     alinx_char.nd = of_find_node_by_path("/alinxled");
213.     if(alinx_char.nd == NULL)
214.     {
215.         printk("alinx_char node not find\n");
216.         return -EINVAL;
217.     }
218.     else
219.     {
220.         printk("alinx_char node find\n");
221.     }
222.
223.     /* 获取节点中 gpio 标号 */
224.     alinx_char.alinx_led_gpio = of_get_named_gpio(alinx_char.nd, "alinxled-gpios", 0);
225.     if(alinx_char.alinx_led_gpio < 0)
226.     {
227.         printk("can not get alinxled-gpios");
228.         return -EINVAL;
229.     }
230.     printk("alinxled-gpio num = %d\n", alinx_char.alinx_led_gpio);
231.
232.     /* 申请 gpio 标号对应的引脚 */
233.     ret = gpio_request(alinx_char.alinx_led_gpio, "alinxled");
234.     if(ret != 0)
235.     {
236.         printk("can not request gpio\n");
237.     }
238.
239.     /* 把这个 io 设置为输出 */
240.     ret = gpio_direction_output(alinx_char.alinx_led_gpio, 1);
241.     if(ret < 0)
242.     {
243.         printk("can not set gpio\n");
244.     }
245. }
```

```
246.  /* 注册设备号 */
247.  alloc_chrdev_region(&alinx_char.devid, MINOR1, DEVID_COUNT, DEVICE_NAME);
248.
249.  /* 初始化字符设备结构体 */
250.  cdev_init(&alinx_char.cdev, &ax_char_fops);
251.
252.  /* 注册字符设备 */
253.  cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
254.
255.  /* 创建类 */
256.  alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
257.  if(IS_ERR(alinx_char.class))
258.  {
259.      return PTR_ERR(alinx_char.class);
260.  }
261.
262.  /* 创建设备节点 */
263.  alinx_char.device = device_create(alinx_char.class, NULL,
264.                                   alinx_char.devid, NULL,
265.                                   DEVICE_NAME);
266.  if (IS_ERR(alinx_char.device))
267.  {
268.      return PTR_ERR(alinx_char.device);
269.  }
270.
271.  return 0;
272. }
273.
274. /* 卸载模块 */
275. static void __exit gpio_led_exit(void)
276. {
277.     /* 释放 gpio */
278.     gpio_free(alinx_char.alinx_led_gpio);
279.
280.     /* 注销字符设备 */
281.     cdev_del(&alinx_char.cdev);
282.
283.     /* 注销设备号 */
284.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
285.
286.     /* 删除设备节点 */
287.     device_destroy(alinx_char.class, alinx_char.devid);
288.
289.     /* 删除类 */
290.     class_destroy(alinx_char.class);
291.
292.     printk("gpio_led_dev_exit_ok\n");
293. }
294.
295. /* 标记加载、卸载函数 */
296. module_init(gpio_led_init);
297. module_exit(gpio_led_exit);
298.
299. /* 驱动描述信息 */
300. MODULE_AUTHOR("Alinx");
301. MODULE_ALIAS("gpio_led");
302. MODULE_DESCRIPTION("CONCURRENT driver");
303. MODULE_VERSION("v1.0");
304. MODULE_LICENSE("GPL");
```

与上一章有差异的部分加粗了。改动主要集中在：

- 1) 结构体定义中增加成员；
- 2) 驱动入口函数中，添加初始化操作；
- 3) open 函数中申请；
- 4) release 函数中释放。

通过同定义开关分成了三部分，分别对应原子操作、自旋锁、信号量，他们的用法都比较相似。按顺序来分析。

原子操作：

看宏定义 ATOMIC_T_ON 相关的部分，思路是：

- 1) 52 行定义一个原子变量，并把它作为共享资源使用状态的标志，0 为未被占用，其他值则已被占用。
- 2) 197 行在驱动入口函数中初始化原子变量为 0，即共享资源未被占用。
- 3) 77 行在 open 函数中，先判断资源占用状态，如果原子变量不为 0，则已被占用返回 busy。否则资源是空闲的，则使原子变量不为 0，然后 open 函数返回 0，表示调用共享资源设备节点成功。
- 4) 160 行在 release 函数中，应用程序释放了共享资源，此时把原子变量置为 0，共享资源恢复为空闲的状态。

实际上这种流程下，共享资源还没有做到完全的安全，因为 77 行的 if 语句不是原子操作，这个语句执行的过程中，还是有可能被抢占或被中断打断导致判断的结果与实际有差别。后面的自旋锁和信号量就没有这种问题。

自旋锁：

再看宏定义 SPINLOCK_T_ON 相关的部分，如果想编译出自旋锁的版本，别忘了把 38 行的注释解开，并把 36 行和 40 行注释。

前面介绍过自旋锁的临界区要尽量短，正常情况下，从应用程序调用 open 到应用程序调用 close 之间都是临界区。这中间的时间不由驱动来决定，有可能会拉的很长，所有在 open 函数中上锁，在 release 函数中解锁就有临界区很长的风险。可以借助上面原子变量实验中用到的标志 flag 的思想来实现：

- 1) 56 行定义一个自旋锁，同时 57 定义一个标志 source_status 来记录资源的使用状态，0 为未被占用，其他值则已被占用。
- 2) 202 行在驱动入口函数中初始化自旋锁，并把资源状态变量初始化成空闲状态 0。
- 3) 92 行是上锁，临界区仅做一个判断资源状态值，如果未被占用则标记为占用然后解锁，已被占用则直接解锁并返回 busy。这样一来，及时资源被占用了，线程也不会一直等待了。
- 4) 165 行 release 函数中，获取锁后，设置资源占用状态为 0，然后就解锁。

信号量：

信号量的用法就很简单了，因为信号量的临界区要长，所以我们放心的在 open 中上锁，在 release 中解锁，就不需要用标志了。当然临界区到底有多长，还是由应用程决定的。

5.3.4 测试代码

测试代码在上一章的基础上稍作修改。新建 QT 工程名为“axledlong_test”，新建 main.c，输入下列代码：

```
1. #include <stdio.h>
2. #include <string.h>
3. #include <unistd.h>
4. #include <fcntl.h>
5.
6. int main(int argc, char **argv)
7. {
8.     int fd;
9.     char buf;
10.    int count;
11.
```



```
12.     if(3 != argc)
13.     {
14.         printf("none para\n");
15.         return -1;
16.     }
17.
18.     fd = open(argv[1], O_RDWR);
19.     if(fd < 0)
20.     {
21.         printf("Can't open file %s\r\n", argv[1]);
22.         return -1;
23.     }
24.
25.     if(!strcmp("on",argv[2]))
26.     {
27.         printf("ps_led1 on\n");
28.         buf = 1;
29.         write(fd, &buf, 1);
30.     }
31.     else if(!strcmp("off",argv[2]))
32.     {
33.         printf("ps_led1 off\n");
34.         buf = 0;
35.         write(fd, &buf, 1);
36.     }
37.     else
38.     {
39.         printf("wrong para\n");
40.         return -2;
41.     }
42.
43.     count = 20;
44.     while(count --)
45.     {
46.         sleep(1);
47.     }
48.
49.     close(fd);
50.     return 0;
51. }
```

在 43 到 47 行添加代码，用 20 秒的 sleep 模拟以下对资源 20 秒的占用。其他和之前一样。

5.4.5 运行测试

加载驱动的步骤就略过了。如下图：

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ./ax-concled-dev.ko
ax_concled_dev: loading out-of-tree module taints kernel.
alinx_char node find
alinxled-gpio num = 899
root@ax_peta:/mnt#
```

用下列命令来打开测试程序：

```
./axledlong_test /dev/gpio_leds on&
./axledlong_test /dev/gpio_leds off
```

“&”表示在后台运行 app，可以使用 top 命令查看后台正在运行的程序。

```
root@ax_peta:/mnt/build-axledlong_test-ZYNQ-Debug# ./axledlong_test /dev/gpio_leds on
[1] 1328
gpio_test module open
ps_led1 on
root@ax_peta:/mnt/build-axledlong_test-ZYNQ-Debug# ./axledlong_test /dev/gpio_leds off
Can't open file /dev/gpio_leds
root@ax_peta:/mnt/build-axledlong_test-ZYNQ-Debug# gpio_test module release

[1]+  Done                  ./axledlong_test /dev/gpio_leds on
root@ax_peta:/mnt/build-axledlong_test-ZYNQ-Debug# ./axledlong_test /dev/gpio_leds off
gpio_test module open
ps_led1 off
```

可以看出, 先执行 app 打开 led, 这个线程就占用了设备节点。再执行 off, app 返回了“Can't open file /dev/gpio_leds”, 而不是直接关闭 led。等输出“gpio_test module release”时, 说明一开始的 app 线程已经释放了资源, 再执行 off, led 就被关闭了。

这是原子变量时的测试状态, 其他两个实验的步骤也是一样的, 自旋锁的现象与原子变量的实验现象是一致的。信号量则会略有区别, 因为信号量的实现中, 会休眠去等待资源, 因此 off 时不会直接返回错误, 而是会等输出“gpio_test module release”后, 再自动取熄灭 led。

第六章 gpio 输入

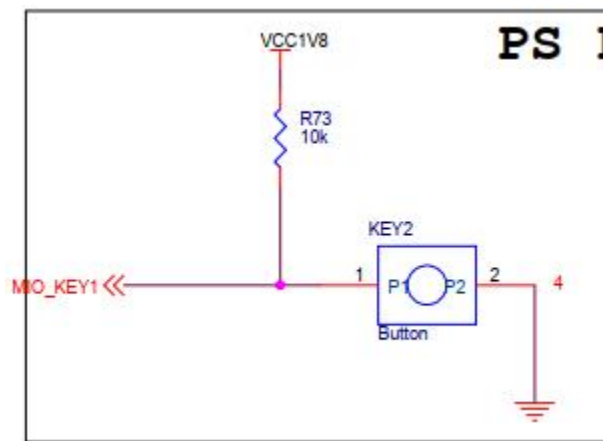
本章我们做一个 gpio 输出的实验，巩固一下 pinctrl 和 gpio 子系统，同时也为后面的学习做铺垫。

gpio 输出最经典的例子就是按键，我们就用按键来做个简单的 gpio 输入实验。实验的目标是应用程序通过 read 函数读取按键状态，如果按键被按下，就反转一次 led 的电平。用按键控制 led 的亮灭。

6.1 原理图

led 部分和章节 1.3.1 相同。

key 部分，使用板子上的 ps_key1，查看原理图，对应到原理图上的 KEY2，key 的另一端是接地的。按下按键 MIO_KEY1 就会被拉低。



再找到 MIO_KEY1 连接的 IO，为 MIO50。

PS_MIO48_501	C12	UART_RX	UART_TX	10
PS_MIO49_501	B13	MIO_KEY1	UART_RX	10
PS_MIO50_501	B9	MIO_KEY2	MIO_KEY1	10
PS_MIO51_501	C10	ETH_MDC	MIO_KEY2	10
PS_MIO52_501	C11	ETH_MDIO	ETH_MDC	10
PS_MIO53_501			ETH_MDIO	10
R10		DS SPST R		

6.2 设备树

打开 system-user.dtsi 文件，在根节点中添加下面的节点：

```

1.  amba {
2.      slcr@f8000000 {
3.          pinctrl@700 {
4.              pinctrl_led_default: led-default {
5.                  mux {
6.                      groups = "gpio0_0_grp";
7.                      function = "gpio0";
8.                  };
9.              };
10.             conf {
11.                 pins = "MIO0";

```

```
12.         io-standard = <1>;
13.         bias-disable;
14.         slew-rate = <0>;
15.     };
16. };
17. pinctrl_key_default: key-default {
18.     mux {
19.         groups = "gpio0_50_grp";
20.         function = "gpio0";
21.     };
22.
23.     conf {
24.         pins = "MIO50";
25.         io-standard = <1>;
26.         bias-high-impedance;
27.         slew-rate = <0>;
28.     };
29. };
30. };
31. };
32. };
33.
34. alinxled {
35.     compatible = "alinx-led";
36.     pinctrl-names = "default";
37.     pinctrl-0 = <&pinctrl_led_default>;
38.     alinxled-gpios = <&gpio0 0 0>;
39. };
40.
41. alinxkey {
42.     compatible = "alinx-key";
43.     pinctrl-names = "default";
44.     pinctrl-0 = <&pinctrl_key_default>;
45.     alinxkey-gpios = <&gpio0 50 0>;
46. };
```

其中 led 相关的部分和之前是一样的。

key 使用的 IO 是 MIO50, 所以 groups = "gpio0_50_grp"、pins = "MIO50"、alinxkey-gpio = <&gpio0 50 0>。输入引脚电气特性配置成高阻 bias-high-impedance。

6.3 驱动代码

使用 petalinux 新建驱动名为"ax-key-dev", 在 ax-key-dev 中输入以下代码:

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/fs.h>
4. #include <linux/init.h>
5. #include <linux/types.h>
6. #include <linux/errno.h>
7. #include <linux/cdev.h>
8. #include <linux/of.h>
9. #include <linux/of_address.h>
10. #include <linux/of_gpio.h>
11. #include <linux/device.h>
12. #include <linux/delay.h>
13. #include <linux/init.h>
14. #include <linux/gpio.h>
15. #include <linux/delay.h>
16. #include <asm/uaccess.h>
17. #include <asm/mach/map.h>
18. #include <asm/io.h>
19.
20. /* 设备节点名称 */
21. #define DEVICE_NAME      "gpio_key"
22. /* 设备号个数 */
23. #define DEVID_COUNT      1
24. /* 驱动个数 */
25. #define DRIVE_COUNT      1
26. /* 主设备号 */
```

```
27. #define MAJOR1
28. /* 次设备号 */
29. #define MINOR1 0
30.
31. /* 把驱动代码中会用到的数据打包进设备结构体 */
32. struct alinx_char_dev{
33.     dev_t      devid;          //设备号
34.     struct cdev cdev;          //字符设备
35.     struct class *class;       //类
36.     struct device *device;      //设备
37.     struct device_node *nd;     //设备树的设备节点
38.     int         alinx_key_gpio; //gpio 号
39. };
40. /* 声明设备结构体 */
41. static struct alinx_char_dev alinx_char = {
42.     .cdev = {
43.         .owner = THIS_MODULE,
44.     },
45. };
46.
47. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
48. static int gpio_key_open(struct inode *inode_p, struct file *file_p)
49. {
50.     /* 设置私有数据 */
51.     file_p->private_data = &alinx_char;
52.     printk("gpio_test module open\n");
53.     return 0;
54. }
55.
56.
57. /* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
58. static ssize_t gpio_key_read(struct file *file_p, char __user *buf, size_t len, loff_t *loff_t_p)
59. {
60.     int ret = 0;
61.     /* 返回按键的值 */
62.     unsigned int key_value = 0;
63.     /* 获取私有数据 */
64.     struct alinx_char_dev *dev = file_p->private_data;
65.
66.     /* 检查按键是否被按下 */
67.     if(0 == gpio_get_value(dev->alinx_key_gpio))
68.     {
69.         /* 按键被按下 */
70.         /* 防抖 */
71.         mdelay(50);
72.         /* 等待按键抬起 */
73.         while(!gpio_get_value(dev->alinx_key_gpio));
74.         key_value = 1;
75.     }
76.     else
77.     {
78.         /* 按键未被按下 */
79.     }
80.     /* 返回按键状态 */
81.     ret = copy_to_user(buf, &key_value, sizeof(key_value));
82.
83.     return ret;
84. }
85.
86. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
87. static int gpio_key_release(struct inode *inode_p, struct file *file_p)
88. {
89.     printk("gpio_test module release\n");
90.     return 0;
91. }
92.
93. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
94. static struct file_operations ax_char_fops = {
95.     .owner    = THIS_MODULE,
96.     .open     = gpio_key_open,
97.     .read     = gpio_key_read,
98.     .release  = gpio_key_release,
99. };
100.
101. /* 模块加载时会调用的函数 */
```

```
102. static int __init gpio_key_init(void)
103. {
104.     /* 用于接受返回值 */
105.     u32 ret = 0;
106.
107.     /* 获取设备节点 */
108.     alinx_char.nd = of_find_node_by_path("/alinxkey");
109.     if(alinx_char.nd == NULL)
110.     {
111.         printk("alinx_char node not find\r\n");
112.         return -EINVAL;
113.     }
114.     else
115.     {
116.         printk("alinx_char node find\r\n");
117.     }
118.
119.     /* 获取节点中 gpio 标号 */
120.     alinx_char.alinx_key_gpio = of_get_named_gpio(alinx_char.nd, "alinxkey-gpios", 0);
121.     if(alinx_char.alinx_key_gpio < 0)
122.     {
123.         printk("can not get alinxkey-gpios");
124.         return -EINVAL;
125.     }
126.     printk("alinxkey-gpio num = %d\r\n", alinx_char.alinx_key_gpio);
127.
128.     /* 申请 gpio 标号对应的引脚 */
129.     ret = gpio_request(alinx_char.alinx_key_gpio, "alinxkey");
130.     if(ret != 0)
131.     {
132.         printk("can not request gpio\r\n");
133.         return -EINVAL;
134.     }
135.
136.     /* 把这个 io 设置为输入 */
137.     ret = gpio_direction_input(alinx_char.alinx_key_gpio);
138.     if(ret < 0)
139.     {
140.         printk("can not set gpio\r\n");
141.         return -EINVAL;
142.     }
143.
144.     /* 注册设备号 */
145.     alloc_chrdev_region(&alinx_char.devid, MINOR1, DEVID_COUNT, DEVICE_NAME);
146.
147.     /* 初始化字符设备结构体 */
148.     cdev_init(&alinx_char.cdev, &ax_char_fops);
149.
150.     /* 注册字符设备 */
151.     cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
152.
153.     /* 创建类 */
154.     alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
155.     if(IS_ERR(alinx_char.class))
156.     {
157.         return PTR_ERR(alinx_char.class);
158.     }
159.
160.     /* 创建设备节点 */
161.     alinx_char.device = device_create(alinx_char.class, NULL,
162.                                     alinx_char.devid, NULL,
163.                                     DEVICE_NAME);
164.     if (IS_ERR(alinx_char.device))
165.     {
166.         return PTR_ERR(alinx_char.device);
167.     }
168.
169.     return 0;
170. }
171.
172. /* 卸载模块 */
173. static void __exit gpio_key_exit(void)
174. {
175.     /* 释放 gpio */
176.     gpio_free(alinx_char.alinx_key_gpio);
177.
178.     /* 注销字符设备 */
179.     cdev_del(&alinx_char.cdev);
```

```
180.
181.  /* 注销设备号 */
182.  unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
183.
184.  /* 删除设备节点 */
185.  device_destroy(alinx_char.class, alinx_char.devid);
186.
187.  /* 删除类 */
188.  class_destroy(alinx_char.class);
189.
190.  printk("gpio_key_dev_exit_ok\n");
191. }
192.
193. /* 标记加载、卸载函数 */
194. module_init(gpio_key_init);
195. module_exit(gpio_key_exit);
196.
197. /* 驱动描述信息 */
198. MODULE_AUTHOR("Alinx");
199. MODULE_ALIAS("gpio_key");
200. MODULE_DESCRIPTION("GPIO OUT driver");
201. MODULE_VERSION("v1.0");
202. MODULE_LICENSE("GPL");
```

和第 4 章的很相似，改动的部分已加粗。

138 原先是设置成输出，改成了输入。

59~82 行原先的 write 函数变成了 read。每次 read 就用 gpio_get_value 函数读取一下 IO 的电平，如果检测到低电平，则按键被按下，按键按下后则是常规的延时去抖，等待抬起。

read 函数最后要把读到的电平返回给用户 buf。

6.4 测试代码

新建 QT 工程名为“ax-key-test”，新建 main.c，输入下列代码：

```
1.  #include <stdio.h>
2.  #include <string.h>
3.  #include <unistd.h>
4.  #include <fcntl.h>
5.
6.  int main(int argc, char *argv[])
7.  {
8.      int fd, fd_l, ret;
9.      char *filename, led_value = 0;
10.     unsigned int key_value;
11.
12.     if(argc != 2)
13.     {
14.         printf("Error Usage\r\n");
15.         return -1;
16.     }
17.
18.     filename = argv[1];
19.     fd = open(filename, O_RDWR);
20.     if(fd < 0)
21.     {
22.         printf("file %s open failed\r\n", argv[1]);
23.         return -1;
24.     }
25.
26.     while(1)
27.     {
28.         ret = read(fd, &key_value, sizeof(key_value));
29.         if(ret < 0)
30.         {
31.             printf("read failed\r\n");
32.             break;
33.         }
34.         if(1 == key_value)
```



```
35.     {
36.         printf("ps_key1 press\r\n");
37.         led_value = !led_value;
38.
39.         fd_1 = open("/dev/gpio_leds", O_RDWR);
40.         if(fd_1 < 0)
41.         {
42.             printf("file /dev/gpio_leds open failed\r\n");
43.             break;
44.         }
45.
46.         ret = write(fd_1, &led_value, sizeof(led_value));
47.         if(ret < 0)
48.         {
49.             printf("write failed\r\n");
50.             break;
51.         }
52.
53.         ret = close(fd_1);
54.         if(ret < 0)
55.         {
56.             printf("file /dev/gpio_leds close failed\r\n");
57.             break;
58.         }
59.     }
60. }
61.
62. ret = close(fd);
63. if(ret < 0)
64. {
65.     printf("file %s close failed\r\n", argv[1]);
66.     return -1;
67. }
68.
69. return 0;
70. }
```

因为要点灯，所以还要用到前面 led 的设备节点，这边就直接用设备节点/dev/gpio_leds 了，在测试时记得加载 led 驱动就行了。

在 26 行的 while 循环里，不停的去调用 read 函数读取按键状态，一旦读取到按键被按下，就去对 led 的 io 进行写操作，每次写入相反的值以达到每一次按键 led 状态反转的效果。

6.5 运行测试

因为修改了设备树，所以要跟新 sd 卡中 image.ub 文件。因为同时用到了 led 和 key，所以要加载两个驱动，led 驱动用上面任意一章的都行。步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ./ax-concled-drv.ko
insmod ./ax-key-drv.ko
cd ./build-ax-key-test-ZYNQ-Debug/
./ax-key-test /dev/gpio_key
```

IP 和路径根据实际情况调整。

操作结果如下图：

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
random: fast init done
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ./ax-concled-dev.ko
ax_concled_dev: loading out-of-tree module taints kernel.
alinx_char node find
alinxled-gpio num = 899
root@ax_peta:/mnt# insmod ./ax-key-
ax-key-dev.ko ax-key-test/
root@ax_peta:/mnt# insmod ./ax-key-dev.ko
alinx_char node find
alinxkey-gpio num = 949
root@ax_peta:/mnt# cd ./build-ax-key-test-ZYNQ-Debug/
root@ax_peta:/mnt/build-ax-key-test-ZYNQ-Debug# ./ax-key-test /dev/gpio_key
gpio_test module open
ps_key1 press
gpio_test module open
gpio_test module release
```

执行 app 后，打印出的第一句“ gpio_test module open”是 open 函数打开/dev/gpio_key 设备时打印出来的，之后按下按键，led 的状态就反转一次并打印 ps_key1 press、gpio_test module open、gpio_test module release 三行信息。

第七章 定时器

本章简单介绍 Linux 内核提供的定时器及其用法。

7.1 Linux 内核中的定时器

内核中定时器的实现依赖于硬件定时器。硬件定时器提供固定频率的时钟源并产生中断，系统使用这个中断来计时。而内核中的定时器功能，则是在这个计时的基础上实现的。

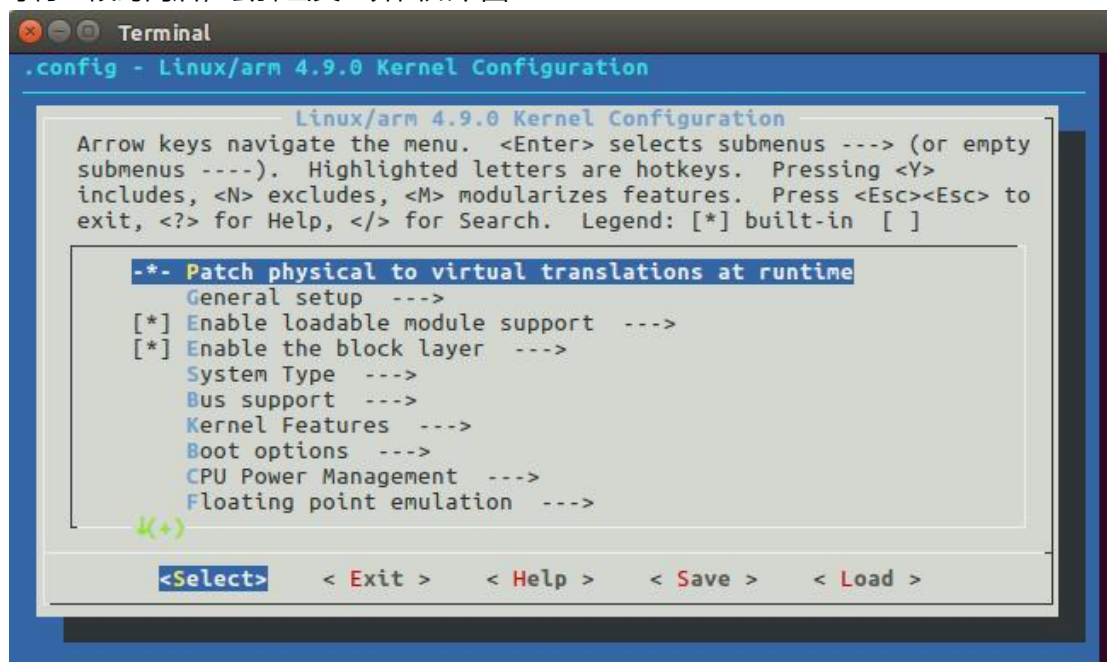
7.1.1 系统频率

产生系统中断的频率就是系统频率，也叫系统节拍即每秒的节拍数。系统节拍是可设置的，在 petalinux 中设置方法为：

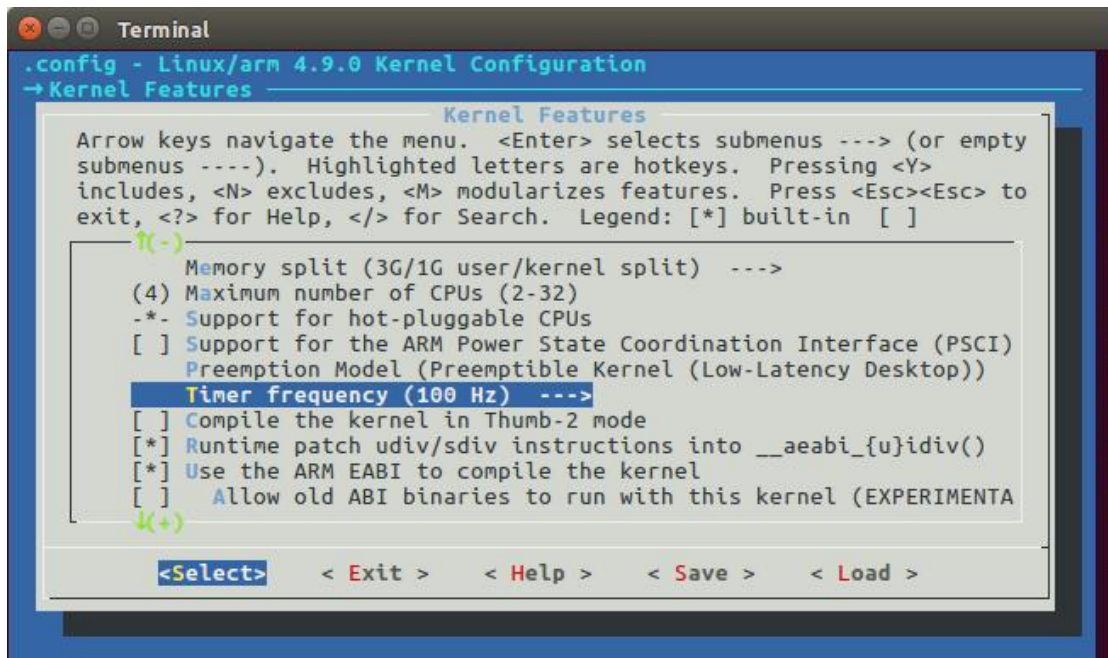
在 petalinux 工程目录中打开终端输入命令：

```
petalinux-config -c kernel
```

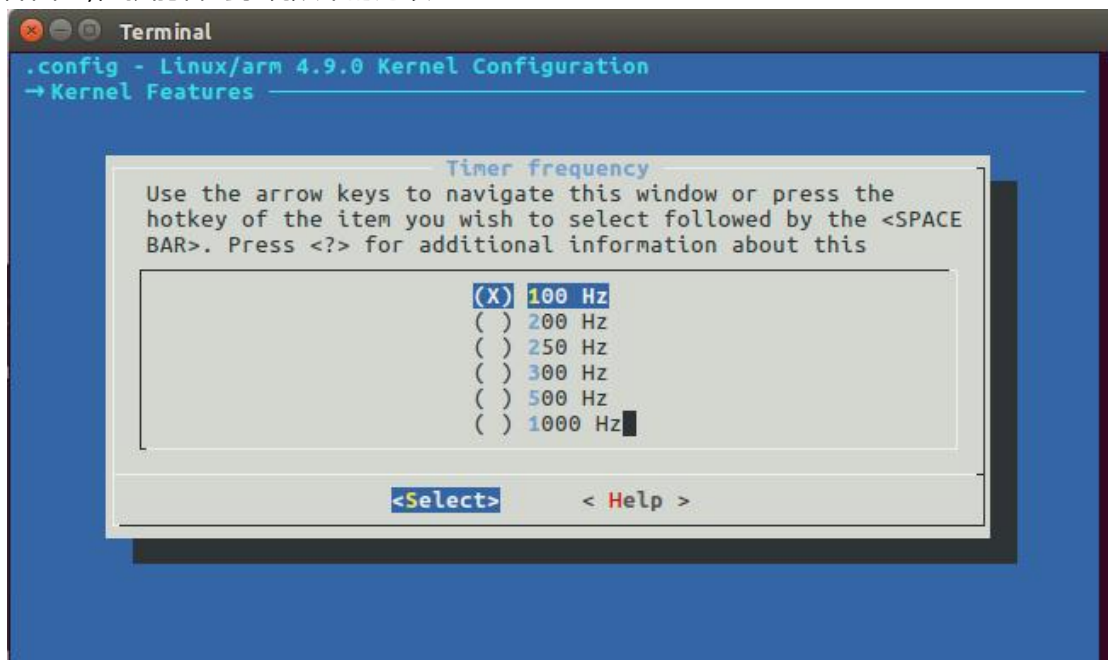
等待一段时间后，会弹出交互界面如下图：



用上下方向键移动选项置“Kernel Features”，按回车进入，如下图：



再按下方向键选择选项“Timer frequency(100 Hz)”按回车(这个选项被藏在很下面，不在一开始的界面里)，就能看到系统频率的选项：



这里默认选择的是 100Hz，每秒 100 个节拍。

系统频率越高，时钟精度也越高，100Hz 的时间进度能达到 10ms，而 1000Hz 能达到 1ms。同时中断产生的频率增加，cpu 的负担也就越大这需要根据实际情况去取舍，我们的实验使用 100Hz 就足够了。

系统时钟定义在 include/asm-generic/param.h 中，为宏定义 HZ，他最终会对应到我们设置的系统时钟值。

7.1.2 节拍数

系统计时的方式就是记录节拍数。系统运行的时间=节拍数/系统频率，单位为秒。

内核中用全局变量 jiffies 来记录节拍书，定义在文件 include/linux/jiffies.h 中：

```
extern u64 __jiffy_data jiffies_64;
extern unsigned long volatile __jiffy_data jiffies;
```

jiffies_64 用于 64 位的系统，jiffies 用于 32 位系统。既然是用来计数的变量，就会有溢出的风险。64 位且不提，32 位的 jiffies 在 100Hz 的系统频率下，500 天左右就会溢出回绕，在 1000Hz 的频率下 50 天左右就会回绕。回绕会带来什么问题，举个例子：

```
1. unsigned long time_mark = jiffies + HZ * 5;
2.
3. /* 业务处理 */
4.
5. if(time_mark > jiffies)
6. {
7.     /* 未超时 */
8. }
9. else
10. {
11.     /* 超时 */
12. }
```

这个例子中，先设定一个时间 time_mark 为当前节拍数再加上 5 秒的节拍，也就是 5 秒后系统节拍 jiffies 会到达的值。这个示例代码的目的是如果业务处理的时间超过 5 秒就执行超时的处理。极端情况下，比如此时 jiffies 为 0xFFFFFDA7，time_mark 则等于 0xFFFFF9B，如果业务处理时间用了 7 秒，jiffies 会因为超过最大值 0xFFFFFFFF 发生反转，从 0 开始重新计数变成 0x64，这个值远远小于 time_mark，明明超时了，却进入了未超时的条件。

对此，内核的通过把数据强专无符号数强转成有符号数巧妙的规避了回绕的问题，不用传统的比较运算符去比较，而使用下面几个宏来代替：

```
#define time_after(unknown,known) ((long)(known) - (long)(unknown)<0)
#define time_before(unknown,known) ((long)(unknown) - (long)(known)<0)
#define time_after_eq(unknown,known) ((long)(unknown) - (long)(known)>=0)
#define time_before_eq(unknown,known) ((long)(known) - (long)(unknown)>=0)
```

以 time_after_eq 为例，参数 unknown 一般指 jiffies，know 通常是我们设定的时间，如上面例子中的 time_mark。after_eq 即大于等于，如果 unknown 大于等于 know，即 jiffies 大于 time_mark，则返回真，对于上面的例程来说就是超时了。

7.1.3 内核定时器及相关函数

Linux 内核中定时器用数据结构 timer_list 来表示。定义在 include/linux/timer.h 中：

```
1. struct timer_list {
2.     /*
3.      * All fields that change during normal runtime grouped to the
4.      * same cacheline
5.      */
6.     struct hlist_node entry;
7.     unsigned long expires;
8.     void (*function)(unsigned long);
9.     unsigned long data;
10.    u32 flags;
11.
12.    #ifdef CONFIG_TIMER_STATS
13.        int start_pid;
14.        void *start_site;
15.        char start_comm[16];
16.    #endif
17.    #ifdef CONFIG_LOCKDEP
18.        struct lockdep_map lockdep_map;
19.    #endif
20. }
```

重要的成员介绍:

expires: 设置定时器超时时间点的节拍数, 如果我们需要定义一个 10s 的定时器, 那 expires 就应该等于 jiffies+10*HZ。为了方便, 内核还提供了毫秒、微妙、纳秒和 jiffies 互换的系列函数:

```
/* 毫秒互换 */
int jiffies_to_msecs(const unsigned long j)
long msecs_to_jiffies(const unsigned int m)
/* 微妙互换 */
int jiffies_to_usecs(const unsigned long j)
long usecs_to_jiffies(const unsigned int u)
/* 纳秒互换 */
u64 jiffies_to_nsecs(const unsigned long j)
unsigned long nsecs_to_jiffies(u64 n)
```

function: 定时器超回调函数指针。超时后的需要做的操作就在这个函数中实现。

data: 用于设置 function 的输入参数。

定时器相关的函数有:

1. void init_timer(struct timer_list *timer)

初始化定时器, 输入参数 timer 即需要初始化的定时器。

2. void add_timer(struct timer_list *timer)

向内核注册定时器, 注册后定时器就开始运行, 输入参数 timer 即需要注册的定时器。

3. int del_timer(struct timer_list * timer)

删除定时器, 不管定时器是否正在运行, 正在运行的 timer 会直接停止计时。输入参数 timer 即需要删除的定时器。删除的定时器若正在运行, 则返回 1, 否则返回 0。

4. int del_timer_sync(struct timer_list *timer)

删除定时器, 正在运行的 timer 会等到使用完成后才删除。输入参数 timer 即需要删除的定时器。删除的定时器若正在运行, 则返回 1, 否则返回 0。

5. int mod_timer(struct timer_list *timer, unsigned long expires)

修改定时器的 expires 即超时时间, 如果定时器不在运行, 则激活该定时器。输入参数 timer 为需要修改的定时器。expires 即为修改后的超时时间。

7.2 实验

实验目标是, 使用定时器定让 led 闪烁, 应用程序可以通过输入参数设置 led 闪烁的周期。

7.2.1 原理图

和章节 1.3.1 的内容相同。

7.2.2 设备树

和章节 4.3.2 相同。

7.2.3 驱动程序

使用 petalinux 新建驱动名为“ax-timer-dev”, 在 ax-timer-dev 中输入以下代码:

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/fs.h>
4. #include <linux/init.h>
5. #include <linux/types.h>
```



```
6. #include <linux/errno.h>
7. #include <linux/cdev.h>
8. #include <linux/of.h>
9. #include <linux/of_address.h>
10. #include <linux/of_gpio.h>
11. #include <linux/device.h>
12. #include <linux/delay.h>
13. #include <linux/init.h>
14. #include <linux/gpio.h>
15. #include <linux/semaphore.h>
16. #include <linux/timer.h>
17. #include <asm/uaccess.h>
18. #include <asm/mach/map.h>
19. #include <asm/io.h>
20.
21. /* 设备节点名称 */
22. #define DEVICE_NAME      "timer_led"
23. /* 设备号个数 */
24. #define DEVID_COUNT      1
25. /* 驱动个数 */
26. #define DRIVE_COUNT      1
27. /* 主设备号 */
28. #define MAJOR_U
29. /* 次设备号 */
30. #define MINOR_U          0
31.
32.
33. /* 把驱动代码中会用到的数据打包进设备结构体 */
34. struct alinx_char_dev{
35.     dev_t      devid;           //设备号
36.     struct cdev cdev;           //字符设备
37.     struct class *class;        //类
38.     struct device *device;      //设备
39.     struct device_node *nd;      //设备树的设备节点
40.     int         alinx_led_gpio;  //gpio 号
41.     char        led_status;      //gpio 状态
42.     unsigned int time_count;     //定时器时间
43.     struct timer_list timer;     //定时器
44. };
45. /* 声明设备结构体 */
46. static struct alinx_char_dev alinx_char = {
47.     .cdev = {
48.         .owner = THIS_MODULE,
49.     },
50. };
51.
52. void timer_function(struct timer_list *timer)
53. {
54.     /* 反转 led 状态 */
55.     alinx_char.led_status = !alinx_char.led_status;
56.     /* 设置 led */
57.     gpio_set_value(alinx_char.alinx_led_gpio, alinx_char.led_status);
58.     /* 重新开始计时 */
59.     mod_timer(&alinx_char.timer, jiffies + msecs_to_jiffies(alinx_char.time_count));
60. }
61.
62. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
63. static int timer_led_open(struct inode *inode_p, struct file *file_p)
64. {
65.     printk("gpio_test module open\n");
66.     return 0;
67. }
68.
69.
70. /* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
71. static ssize_t timer_led_write(struct file *file_p, const char __user *buf, size_t len, loff_t *
loff_t_p)
72. {
73.     int retvalue;
74.     /* 获取用户数据 */
75.     retvalue = copy_from_user(&alinx_char.time_count, buf, len);
76.     /* 设置好 timer 后先点亮 led */
77.     alinx_char.led_status = 1;
78.     gpio_set_value(alinx_char.alinx_led_gpio, alinx_char.led_status);
79.     /* 开启 timer */
80.     mod_timer(&alinx_char.timer, jiffies + msecs_to_jiffies(alinx_char.time_count));
```



```
81.
82.     return 0;
83. }
84.
85. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
86. static int timer_led_release(struct inode *inode_p, struct file *file_p)
87. {
88.     printk("gpio_test module release\n");
89.     /* 删除定时器 */
90.     del_timer_sync(&alinx_char.timer);
91.     return 0;
92. }
93.
94. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
95. static struct file_operations ax_char_fops = {
96.     .owner    = THIS_MODULE,
97.     .open     = timer_led_open,
98.     .write    = timer_led_write,
99.     .release  = timer_led_release,
100. };
101.
102. /* 模块加载时会调用的函数 */
103. static int __init timer_led_init(void)
104. {
105.     /* 用于接受返回值 */
106.     u32 ret = 0;
107.
108.     /* 获取 led 设备节点 */
109.     alinx_char.nd = of_find_node_by_path("/alinxled");
110.     if(alinx_char.nd == NULL)
111.     {
112.         printk("alinx_char node not find\n");
113.         return -EINVAL;
114.     }
115.     else
116.     {
117.         printk("alinx_char node find\n");
118.     }
119.
120.     /* 获取节点中 gpio 标号 */
121.     alinx_char.alinx_led_gpio = of_get_named_gpio(alinx_char.nd, "alinxled-gpios", 0);
122.     if(alinx_char.alinx_led_gpio < 0)
123.     {
124.         printk("can not get alinxled-gpios");
125.         return -EINVAL;
126.     }
127.     printk("alinxled-gpio num = %d\n", alinx_char.alinx_led_gpio);
128.
129.     /* 申请 gpio 标号对应的引脚 */
130.     ret = gpio_request(alinx_char.alinx_led_gpio, "alinxled");
131.     if(ret != 0)
132.     {
133.         printk("can not request gpio\n");
134.     }
135.
136.     /* 把这个 io 设置为输出 */
137.     ret = gpio_direction_output(alinx_char.alinx_led_gpio, 1);
138.     if(ret < 0)
139.     {
140.         printk("can not set gpio\n");
141.     }
142.
143.     /* 注册设备号 */
144.     alloc_chrdev_region(&alinx_char.devid, MINOR_U, DEVID_COUNT, DEVICE_NAME);
145.
146.     /* 初始化字符设备结构体 */
147.     cdev_init(&alinx_char.cdev, &ax_char_fops);
148.
149.     /* 注册字符设备 */
150.     cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
151.
152.     /* 创建类 */
153.     alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
154.     if(IS_ERR(alinx_char.class))
155.     {
156.         return PTR_ERR(alinx_char.class);
157.     }
158. }
```

```
159.  /* 创建设备节点 */
160.  alinx_char.device = device_create(alinx_char.class, NULL,
161.                                  alinx_char.devid, NULL,
162.                                  DEVICE_NAME);
163.  if (IS_ERR(alinx_char.device))
164.  {
165.      return PTR_ERR(alinx_char.device);
166.  }
167.
168.  /* 初始化定时器 */
169.  __init_timer(&alinx_char.timer, timer_function, 0);
170.  return 0;
171. }
172.
173. /* 卸载模块 */
174. static void __exit timer_led_exit(void)
175. {
176.     /* 释放 gpio */
177.     gpio_free(alinx_char.alinx_led_gpio);
178.
179.     /* 注销字符设备 */
180.     cdev_del(&alinx_char.cdev);
181.
182.     /* 注销设备号 */
183.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
184.
185.     /* 删除设备节点 */
186.     device_destroy(alinx_char.class, alinx_char.devid);
187.
188.     /* 删除类 */
189.     class_destroy(alinx_char.class);
190.
191.     printk("timer_led_dev_exit_ok\n");
192. }
193.
194. /* 标记加载、卸载函数 */
195. module_init(timer_led_init);
196. module_exit(timer_led_exit);
197.
198. /* 驱动描述信息 */
199. MODULE_AUTHOR("Alinx");
200. MODULE_ALIAS("gpio_led");
201. MODULE_DESCRIPTION("TIMER LED driver");
202. MODULE_VERSION("v1.0");
203. MODULE_LICENSE("GPL");
```

同样关注加粗的部分，其他都是常规的字符设备注册框架。

42 行定义一个 char 型数来记录 led 的状态。

43 行定义一个 unsigned int 来记录定时器的时间。

44 行定义一个定时器。

53~61 行是定时器的回掉函数，定时器计时结束后就会执行这个函数。在这个函数中进行一次 led 的状态反转，并重新开始计时。

72 行的 open 函数中，根据应用程序输入的值来设置 timer 的计时时间，并开启 timer，使用 mod_timer 函数。

91 行的 release 函数中删除 timer。

170 行在入口函数中，设置定时器的回掉函数为 timer_function()，并初始化定时器。

7.2.4 测试程序

新建 QT 工程名为“timer-test”，新建 main.c，输入下列代码：

```
1.  #include "stdio.h"
2.  #include "unistd.h"
3.  #include "sys/types.h"
4.  #include "sys/stat.h"
5.  #include "fcntl.h"
```

```
6. #include "stdlib.h"
7. #include "string.h"
8. #include "linux/ioctl.h"
9.
10. int main(int argc, char *argv[])
11. {
12.     int fd, ret;
13.     char *filename;
14.     unsigned int interval_new, interval_old = 0;
15.
16.     if(argc != 2)
17.     {
18.         printf("Error Usage!\r\n");
19.         return -1;
20.     }
21.
22.     filename = argv[1];
23.
24.     fd = open(filename, O_RDWR);
25.     if(fd < 0)
26.     {
27.         printf("can not open file %s\r\n", filename);
28.         return -1;
29.     }
30.
31.     while(1)
32.     {
33.         printf("Input interval:");
34.         scanf("%d", &interval_new);
35.
36.         if(interval_new != interval_old)
37.         {
38.             interval_old = interval_new;
39.             ret = write(fd, &interval_new, sizeof(interval_new));
40.             if(ret < 0)
41.             {
42.                 printf("write failed\r\n");
43.             }
44.             else
45.             {
46.                 printf("interval refreshed!\r\n");
47.             }
48.         }
49.         else
50.         {
51.             printf("same interval!");
52.         }
53.     }
54.     close(fd);
55. }
```

内容很简单。

34 行等待用户输入时间间隔。

36 行判断如果时间间隔没有变化就忽略。如果有变化则调用 write 函数用新的间隔重新开启 timer。

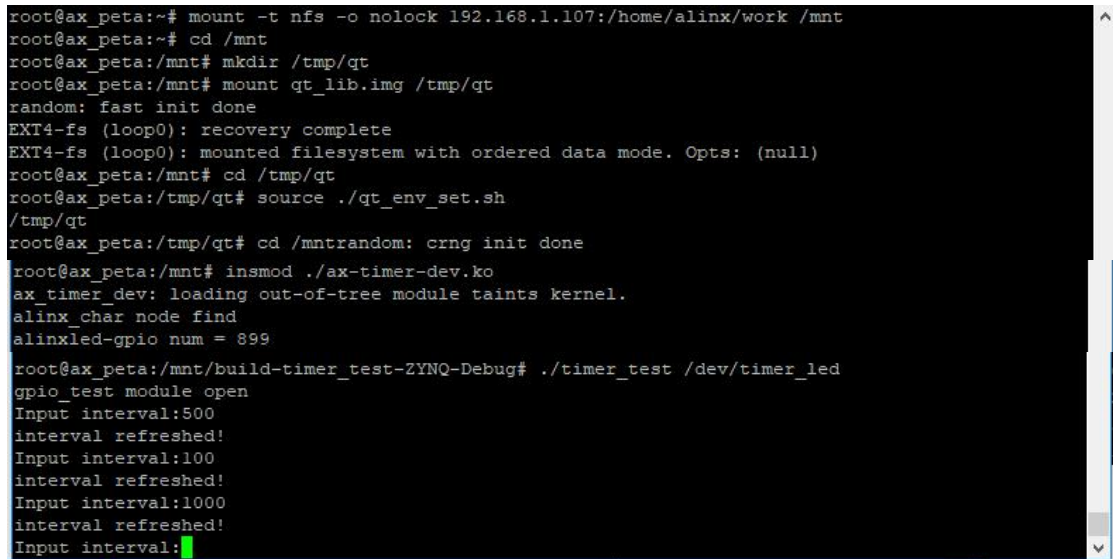
7.3.5 运行测试

测试步骤如下：

```
/* IP 和路径根据实际情况调整 */
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
./ax-timer-drv.ko
/* app 目录可能不一样，根据实际情况调整 */
```

```
cd ./build-timer_test-ZYNQ-Debug/  
./timer_test /dev/timer_led
```

结果如下图:



```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt  
root@ax_peta:~# cd /mnt  
root@ax_peta:/mnt# mkdir /tmp/qt  
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt  
random: fast init done  
EXT4-fs (loop0): recovery complete  
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)  
root@ax_peta:/mnt# cd /tmp/qt  
root@ax_peta:/tmp/qt# source ./qt_env_set.sh  
/tmp/qt  
root@ax_peta:/tmp/qt# cd /mnt  
random: crng init done  
root@ax_peta:/mnt# insmod ./ax-timer-dev.ko  
ax_timer_dev: loading out-of-tree module taints kernel.  
alinx_char node find  
alinxled-gpio num = 899  
root@ax_peta:/mnt/build-timer_test-ZYNQ-Debug# ./timer_test /dev/timer_led  
gpio_test module open  
Input interval:500  
interval refreshed!  
Input interval:100  
interval refreshed!  
Input interval:1000  
interval refreshed!  
Input interval:
```

运行 app 后，输入不同的时间间隔，观察 led 的变化。

第八章 中断

中断是很常用的功能，Linux 内核中也实现了完善的中断框架，这一章来学习一下 Linux 内核中中断的简单用法。

8.1 Linux 中断框架简介

8.1.1 接口函数

Linux 内核中的中断框架使用已经相当便捷，一般需要做三件事申请中断、实现中断服务函数、使能中断。对应的接口函数如下：

(一) 中断申请和释放函数

- 1) 中断申请函数 `request_irq`，该函数可能会导致睡眠，在申请成功后会使能中断，函数原型：

```
int request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags, const char *name, void *dev);
```

参数说明：

irq：申请的中断号，中断号也可以叫中断线，是中断的唯一标识，也是内核找到对应中断服务函数的依据。

handler：中断服务函数，中断触发后会执行的函数。

flags：中断标志，用于设置中断的触发方式和其他特性，常用的标识有：

```
/* 无触发 */
IRQF_TRIGGER_NONE
/* 上升沿触发 */
IRQF_TRIGGER_RISING
/* 下降沿触发 */
IRQF_TRIGGER_FALLING
/* 高电平触发 */
IRQF_TRIGGER_HIGH
/* 低电平触发 */
IRQF_TRIGGER_LOW
/* 单次中断 */
IRQF_ONESHOT
/* 作为定时器中断 */
IRQF_TIMER
/* 共享中断，多个设备共享一个中断号时需要此标志 */
IRQF_SHARED
```

可在 `/include/linux/interrupt.h` 文件中可以查看全部的 flag 和英文释义。中断标志可以使用 | 号来组合，如 `IRQF_TRIGGER_RISING | IRQF_ONESHOT` 意为上升沿触发的单次中断。

name：中断名称，申请中断成功后，在 `/proc/interrupts` 文件中可以找到这个名字。

dev：flag 设置 `IRQF_SHARED` 时，使用 dev 来区分不同的设备，dev 的值会传递给中断服务函数的第二个参数。

返回值：0-申请成功，-EBUSY-中断已被占用，其他值都表示申请失败。

- 2) 和中断申请相对的中断释放函数 `free_irq`，如果目标中断不是共享中断，那么 `free_irq` 函数在释放中断后，会禁止中断并删除中断服务函数，原型如下：

```
void free_irq(unsigned int irq, void *dev);
```

参数说明：

irq：需要释放的中断。

dev：释放的中断如果是共享中断，用这个参数来区分具体的中断，只有共享中断下所有的 dev 都被释放时，`free_irq` 函数才会禁止中断并删除中断服务函数。

(二) 实现服务申请函数

中断服务函数的格式为：

```
irqreturn_t (*irq_handler_t) (int, void *)
```

第一个参数 `int` 型时中断服务函数对应的中断号。

第二个参数是通用指针，需要和 `request_irq` 函数的参数 `dev` 一致。

返回值 `irqreturn_t` 为美制类型，一般在服务函数中用下面的方式返回值：

```
return IRQ_RETVAL(IRQ_HANDLED);
```

(三) 中断使能和禁止函数

- 1) `enable_irq(unsigned int irq)`、`disable_irq(unsigned int irq)`和 `disable_irq_nosync(unsigned int irq)`

`enable_irq` 和 `disable_irq` 分别是中断使能和禁止函数，`irq` 是目标中断号。`disable_irq` 会等待目标中断的中断服务函数执行结束才会禁止中断，如果想要立即禁止中断则可以使用 `disable_irq_nosync()`函数。

- 2) `local_irq_enable()`和 `local_irq_disable()`

`local_irq_enable()`函数用于使能当前处理器的中断系统。

`local_irq_disable()`函数用于禁止当前处理器的中断系统。

- 3) `local_irq_save(flags)`和 `local_irq_restore(flags)`

`local_irq_save(flags)`函数也是用于禁止当前处理器中断，但是会把进之前的中断状态保存在输入参数 `flags` 中。而 `local_irq_restore(flags)`函数则是把中断恢复到 `flags` 中记录的状态。

8.1.2 Linux 的下半部机制

上半部下半部是为了尽量缩短中断服务函数的处理过程而引入的机制，上半部就是中断触发后立即执行的中断服务函数，而下半部就是对中断服务函数的延时处理。因为中断的优先级较高，如果处理内容过多，长时间占用处理器会影响其他代码的运行，所以上半部要尽量短。裸机程序里，在中断处理函数中树 `flag`，然后到主程序大循环中去轮询判断这个 `flag` 再做相应的操作，就是一种上半部下半部的思想。Linux 中的上半部就是指中断服务函数 `irq_handler_t`。至于那些任务放在上半部哪些放在下半部，没有明确的界限，一般我们把对时间敏感、不能被打断的任务放到上半部中，其他的都可以考虑放到下半部。

Linux 针对下半部也提供了一些完善的机制：

- 1) 软中断

软中断结构体定义在 `include/linux/interrupt.h` 文件中，具体如下：

```
1. struct softirq_action
```

```
2. {
3.     void (*action)(struct softirq_action *);
4. };
```

内核在 kernel/softirq.c 文件中定义了全局的软中断向量表：

```
static struct softirq_action softirq_vec[NR_SOFTIRQS];
```

NR_SOFTIRQS 为枚举类型的最大值，该枚举类型定义在 include/linux/interrupt.h 中：

```
1. enum
2. {
3.     HI_SOFTIRQ=0,
4.     TIMER_SOFTIRQ,
5.     NET_TX_SOFTIRQ,
6.     NET_RX_SOFTIRQ,
7.     BLOCK_SOFTIRQ,
8.     IRQ_POLL_SOFTIRQ,
9.     TASKLET_SOFTIRQ,
10.    SCHED_SOFTIRQ,
11.    HRTIMER_SOFTIRQ,
12.    RCU_SOFTIRQ,
13.
14.    NR_SOFTIRQS
15.};
```

代表了十个软中断，要使用软中断只能向内核定义的软中断向量表注册，注册软中断需要使用函数：

```
void (int nr, void (*action)(struct softirq_action *));
```

参数说明：

nr：小于 NR_SOFTIRQS 的枚举值。

action：对应的软中断服务函数。

软中断必须在编译时静态注册，注册完成就需要使用 raise_softirq(unsigned int nr)触发，nr 即为需要触发的软中断。

但下半部机制通常不用软中断，而是使用下面要讲的 tasklets 机制。

2) tasklets 机制

Linux 内核在 softirq_int 函数中初始化软中断，其中 HI_SOFTIRQ 和 TASKLET_SOFTIRQ 是默认打开的。tasklets 机制就是在这两个软中断基础上实现的。

tasklets 的结构体也定义在头文件 include/linux/interrupt.h 中，定义如下：

```
1. struct tasklet_struct
2. {
3.     struct tasklet_struct *next;
4.     unsigned long state;
5.     atomic_t count;
6.     void (*func)(unsigned long);
7.     unsigned long data;
8.};
```

其中 func 就是相当于是 tasklet 的中断服务函数，tasklet 的定义和初始化可以直接用下面的宏定义来完成：

```
DECLARE_TASKLET(name, func, data)
```

name：tasklet 的名字。

func：tasklet 触发时的处理函数。

data：传递给 func 的输入参数。

初始化完成后，调用以下函数即可激活 tasklet：

```
tasklet_schedule(struct tasklet_struct *t)
```

激活后 tasklet 的服务函数就会在合适的时间运行。用作中断的下半段时，就在上半段中调用该函数。如果要用优先级较高的 tasklet，就使用 tasklet_hi_schedule(struct tasklet_struct *t)函数激活。

tasklet 的下半段机制使用示例：

```
1.  /* 定义 tasklet */
2.  struct tasklet_struct example;
3.  /* tasklet 处理函数 */
4.  void testtasklet_func(unsigned long data)
5.  {
6.      /* tasklet 具体处理内容 */
7.  }
8.  /* 中断处理函数 */
9.  irqreturn_t test_handler(int irq, void *dev_id)
10. {
11.     /* 调度 tasklet */
12.     tasklet_schedule(&example);
13. }
14. /* 驱动入口函数 */
15. static int __init xxxx_init(void)
16. {
17.     /* 初始化 tasklet */
18.     tasklet_init(&example, testtasklet_func, data);
19.     /* 注册中断处理函数 */
20.     request_irq(irq, test_handler, 0, "name", &dev);
21. }
```

3) 工作队列

工作队列也是下半部的实现方案。与 tasklet 相对的，工作队列是可阻塞的，因此不能在中断上下文中运行。工作队列的队列实现我们可以不用去管，要使用工作队列，只要定一个工作即可。

工作结构体为 work_struct，定义在/include/linux/workqueue.h 文件中：

```
1.  struct work_struct {
2.      atomic_long_t data;
3.      struct list_head entry;
4.      work_func_t func;
5.      #ifdef CONFIG_LOCKDEP
6.      struct lockdep_map lockdep_map;
7.      #endif
8.  };
```

func 即为需要处理的函数。可使用以下宏定义来创建并初始化工作：

```
DECLARE_WORK(n, f)
```

n：需要创建并初始化的工作结构体 work_struct 的名称。

f：工作队列需要处理的函数。

初始化完成后，使用下面的函数来调用工作队列：

```
bool schedule_work(struct work_struct *work)
```

work：需要调用的工作。

返回值：0 成功，1 失败。

workqueue 的下半段机制使用示例：

```
1.  /* 定义工作(work) */
```

```

2.  struct work_struct example;
3.  /* work 处理函数 */
4.  void work_func_t(struct work_struct *work);
5.  {
6.      /* work 具体处理内容 */
7.  }
8.  /* 中断处理函数 */
9.  irqreturn_t test_handler(int irq, void *dev_id)
10. {
11.     /* 调度 work */
12.     schedule_work(&example);
13. }
14. /* 驱动入口函数 */
15. static int __init xxxx_init(void)
16. {
17.     .....
18.     /* 初始化 work */
19.     INIT_WORK(&example, work_func_t);
20.     /* 注册中断处理函数 */
21.     request_irq(irq, test_handler, 0, "name", &dev);
22. }

```

8.1.3 设备树中的中断

设备树中，通用的中断设置方法可参考文档 `Documentation/devicetree/bindings/arm/arm,gic.txt`。Xilinx 的设备树中断控制器的设置与 Linux 内核的通用设置稍有区别，可以查看文档 `Documentation/devicetree/bindings/arm/xilinx,inct.txt` 了解详情。看这个文件最后的一段例子：

```

1. axi_intc_0: interrupt-controller@41800000 {
2.     #interrupt-cells = <2>;
3.     compatible = "xlnx,xps-intc-1.00.a";
4.     interrupt-controller;
5.     interrupt-parent = <&ps7_scugic_0>;
6.     interrupts = <0 29 4>;
7.     reg = <0x41800000 0x10000>;
8.     xlnx,kind-of-intr = <0x1>;
9.     xlnx,num-intr-inputs = <0x1>;
10. };

```

回头看一下 gpio 子系统的章节，那时候讲 gpio 的设备树时，就已经出现了这几个中断相关的属性。

第 2 行的"#interrupt-cells"是中断控制器节点的属性，用来描述子节点中"interrupts"属性值的数量。一般父节点的"#interrupt-cells"值为 3，则子节点的"interrupts"一个 cell 的三个 32 位整数的值为<中断域 中断号 触发方式>，如果父节点的该属性是 2，则是<中断号 触发方式>。

第 4 行的属性"interrupt-controller"代表这个节点是一个中断控制器。

第 5 行的"interrupt-parent"属性表明这个设备属于哪个中断控制器，如果没有这个属性会自动依附于父节点的"interrupt-parent"。

第 6 行的"interrupts"，第一个值为 0 表示 SPI 中断，1 表示 PPI 中断。在 zynq 中，第一个值如果是 0，则中断号等于第二个值加 32。

第 8 行的"xlnx,kind-of-intr"表示为每个可能的中断指定中断类型，1 表示 edge，0 表示 level。

第 9 行"xlnx,num-intr-inputs"属性指定控制器的特定实现支持的中断数，范围是 1~32。

我们需要从设备树中获取设备号信息，以向内核注册中断，of 函数中有对应的函数：

```
unsigned int irq_of_parse_and_map(struct device_node *dev, int index);
```

dev 是设备节点

index 是对属性"interrupts"元素的索引，因为中断号的位置有可能不同。

返回值就是中断号。

要使用这个函数的话，需要我们在对应的设备中设置好"interrupts"属性。

对于 gpio，内核提供了更方便的函数获取中断号：

```
int gpio_to_irq(unsigned int gpio);
```

gpio 为需要申请中断号的 gpio 编号。

返回值就是中断号。

zynq 下 gpio 是共享的一个中断，针对单个 io 去设置"interrupts"属性比较麻烦，gpio_to_irq 函数帮我们做了很多事，后面的 gpio 中断实验，我们就直接使用这个函数。

8.2 实验

本章写一个通过按键中断驱动，按下按键触发中断，触发中断后，在中断服务函数中开启一个 50ms 的定时器来实现按键去抖。

8.2.1 原理图

led 部分和章节 1.3.1 相同。

key 部分和章节 6.1 相同。

8.2.2 设备树

和章节 6.2 相同。

8.2.3 驱动程序

使用 petalinux 新建名为"ax-irq-drv"的驱动程序,并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-irq-drv.c 文件中输入下面的代码：

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/init.h>
4. #include <linux/types.h>
5. #include <linux/errno.h>
6. #include <linux/cdev.h>
7. #include <linux/of.h>
8. #include <linux/of_address.h>
9. #include <linux/of_gpio.h>
10. #include <linux/device.h>
11. #include <linux/delay.h>
12. #include <linux/init.h>
13. #include <linux/gpio.h>
14. #include <linux/semaphore.h>
15. #include <linux/timer.h>
16. #include <linux/of_irq.h>
17. #include <linux/irq.h>
18. #include <asm/uaccess.h>
19. #include <asm/mach/map.h>
20. #include <asm/io.h>
21. #include <linux/interrupt.h>
22. /* 设备节点名称 */
23. #define DEVICE_NAME "interrupt_led"
24. /* 设备号个数 */
25. #define DEVID_COUNT 1
26. /* 驱动个数 */
27. #define DRIVE_COUNT 1
28. /* 主设备号 */
29. #define MAJOR_U
30. /* 次设备号 */
31. #define MINOR_U 0
32.
```

```

33. /* 把驱动代码中会用到的数据打包进设备结构体 */
34. struct alinx_char_dev {
35.     /** 字符设备框架 **/
36.     dev_t      devid;           //设备号
37.     struct cdev cdev;           //字符设备
38.     struct class *class;        //类
39.     struct device *device;       //设备
40.     struct device_node *nd;      //设备树的设备节点
41.     /** 并发处理 **/
42.     spinlock_t lock;            //自旋锁变量
43.     /** gpio **/
44.     int         alinx_key_gpio;  //gpio 号
45.     int         key_sts;         //记录按键状态, 为 1 时被按下
46.     /** 中断 **/
47.     unsigned int irq;           //中断号
48.     /** 定时器 **/
49.     struct timer_list timer;     //定时器
50. };
51. /* 声明设备结构体 */
52. static struct alinx_char_dev alinx_char = {
53.     .cdev = {
54.         .owner = THIS_MODULE,
55.     },
56. };
57.
58. /** 回掉 **/
59. /* 中断服务函数 */
60. static irqreturn_t key_handler(int irq, void *dev)
61. {
62.     /* 按键按下或抬起时会进入中断 */
63.     /* 开启 50 毫秒的定时器用作防抖动 */
64.     mod_timer(&alinx_char.timer, jiffies + msecs_to_jiffies(50));
65.     return IRQ_RETVAL(IRQ_HANDLED);
66. }
67.
68. /* 定时器服务函数 */
69. void timer_function(struct timer_list *timer)
70. {
71.     unsigned long flags;
72.     /* 获取锁 */
73.     spin_lock_irqsave(&alinx_char.lock, flags);
74.
75.     /* value 用于获取按键值 */
76.     unsigned char value;
77.     /* 获取按键值 */
78.     value = gpio_get_value(alinx_char.alinx_key_gpio);
79.     if(value == 0)
80.     {
81.         /* 按键按下, 状态置 1 */
82.         alinx_char.key_sts = 1;
83.     }
84.     else
85.     {
86.         /* 按键抬起 */
87.     }
88.
89.     /* 释放锁 */
90.     spin_unlock_irqrestore(&alinx_char.lock, flags);
91. }
92.
93. /** 系统调用实现 **/
94. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
95. static int char_drv_open(struct inode *inode_p, struct file *file_p)
96. {
97.     printk("gpio_test module open\n");
98.     return 0;
99. }
100.
101.
102. /* read 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
103. static ssize_t char_drv_read(struct file *file_p, char __user *buf, size_t len, loff_t *loff_t_p)
104. {
105.     unsigned long flags;
106.     int ret;
107.     /* 获取锁 */

```

```
108. spin_lock_irqsave(&alinx_char.lock, flags);
109.
110. /* keysts 用于读取按键状态 */
111. /* 返回按键状态值 */
112. ret = copy_to_user(buf, &alinx_char.key_sts, sizeof(alinx_char.key_sts));
113. /* 清除按键状态 */
114. alinx_char.key_sts = 0;
115.
116. /* 释放锁 */
117. spin_unlock_irqrestore(&alinx_char.lock, flags);
118. return 0;
119. }
120.
121. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
122. static int char_drv_release(struct inode *inode_p, struct file *file_p)
123. {
124.     printk("gpio_test module release\n");
125.     return 0;
126. }
127.
128. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
129. static struct file_operations ax_char_fops = {
130.     .owner    = THIS_MODULE,
131.     .open     = char_drv_open,
132.     .read     = char_drv_read,
133.     .release  = char_drv_release,
134. };
135.
136. /* 模块加载时会调用的函数 */
137. static int __init char_drv_init(void)
138. {
139.     /* 用于接受返回值 */
140.     u32 ret = 0;
141.
142.     /** 并发处理 **/
143.     /* 初始化自旋锁 */
144.     spin_lock_init(&alinx_char.lock);
145.
146.     /** gpio 框架 **/
147.     /* 获取设备节点 */
148.     alinx_char.nd = of_find_node_by_path("/alinxkey");
149.     if(alinx_char.nd == NULL)
150.     {
151.         printk("alinx_char node not find\n");
152.         return -EINVAL;
153.     }
154.     else
155.     {
156.         printk("alinx_char node find\n");
157.     }
158.
159.     /* 获取节点中 gpio 标号 */
160.     alinx_char.alinx_key_gpio = of_get_named_gpio(alinx_char.nd, "alinxkey-gpios", 0);
161.     if(alinx_char.alinx_key_gpio < 0)
162.     {
163.         printk("can not get alinxkey-gpios");
164.         return -EINVAL;
165.     }
166.     printk("alinxkey-gpio num = %d\n", alinx_char.alinx_key_gpio);
167.
168.     /* 申请 gpio 标号对应的引脚 */
169.     ret = gpio_request(alinx_char.alinx_key_gpio, "alinxkey");
170.     if(ret != 0)
171.     {
172.         printk("can not request gpio\n");
173.         return -EINVAL;
174.     }
175.
176.     /* 把这个 io 设置为输入 */
177.     ret = gpio_direction_input(alinx_char.alinx_key_gpio);
178.     if(ret < 0)
179.     {
180.         printk("can not set gpio\n");
181.         return -EINVAL;
182.     }
183.
184.     /** 中断 **/
```

```
185.  /* 获取中断号 */
186.  alinx_char.irq = gpio_to_irq(alinx_char.alinx_key_gpio);
187.  /* 申请中断 */
188.  ret = request_irq(alinx_char.irq,
189.                  key_handler,
190.                  IRQF_TRIGGER_FALLING | IRQF_TRIGGER_RISING,
191.                  "alinxkey",
192.                  NULL);
193.  if(ret < 0)
194.  {
195.      printk("irq %d request failed\r\n", alinx_char.irq);
196.      return -EFAULT;
197.  }
198.
199.  /** 定时器 **/
200.  __init_timer(&alinx_char.timer, timer_function, 0);
201.
202.
203.  /** 字符设备框架 **/
204.  /* 注册设备号 */
205.  alloc_chrdev_region(&alinx_char.devid, MINOR_U, DEVID_COUNT, DEVICE_NAME);
206.
207.  /* 初始化字符设备结构体 */
208.  cdev_init(&alinx_char.cdev, &ax_char_fops);
209.
210.  /* 注册字符设备 */
211.  cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
212.
213.  /* 创建类 */
214.  alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
215.  if(IS_ERR(alinx_char.class))
216.  {
217.      return PTR_ERR(alinx_char.class);
218.  }
219.
220.  /* 创建设备节点 */
221.  alinx_char.device = device_create(alinx_char.class, NULL,
222.                                   alinx_char.devid, NULL,
223.                                   DEVICE_NAME);
224.  if (IS_ERR(alinx_char.device))
225.  {
226.      return PTR_ERR(alinx_char.device);
227.  }
228.
229.  return 0;
230. }
231.
232. /* 卸载模块 */
233. static void __exit char_drv_exit(void)
234. {
235.  /** gpio **/
236.  /* 释放 gpio */
237.  gpio_free(alinx_char.alinx_key_gpio);
238.
239.  /** 中断 **/
240.  /* 释放中断 */
241.  free_irq(alinx_char.irq, NULL);
242.
243.  /** 定时器 **/
244.  /* 删除定时器 */
245.  del_timer_sync(&alinx_char.timer);
246.
247.  /** 字符设备框架 **/
248.  /* 注销字符设备 */
249.  cdev_del(&alinx_char.cdev);
250.
251.  /* 注销设备号 */
252.  unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
253.
254.  /* 删除设备节点 */
255.  device_destroy(alinx_char.class, alinx_char.devid);
256.
257.  /* 删除类 */
258.  class_destroy(alinx_char.class);
259.
260.  printk("timer_led_dev_exit_ok\n");
261. }
```

```
262.  
263. /* 标记加载、卸载函数 */  
264. module_init(char_drv_init);  
265. module_exit(char_drv_exit);  
266.  
267. /* 驱动描述信息 */  
268. MODULE_AUTHOR("Alinx");  
269. MODULE_ALIAS("alinx_char");  
270. MODULE_DESCRIPTION("INTERRUPT LED driver");  
271. MODULE_VERSION("v1.0");  
272. MODULE_LICENSE("GPL");
```

187 行在驱动入口函数中，初始化 gpio 之后，使用 gpio_to_irq 函数通过 gpio 端口号来获取中断号。

189 行通过中断号向内核申请中断。上升沿或下降沿触发，命名为“alinxkey”，中断服务函数为 key_handler。

对照前面说的中断步骤，现在我们只要实现 key_handler 这个函数就可以了。

61 行实现了 key_handler，内容很简单先是开启了一个 50ms 的 timer，之后返回 IRQ_RETVAL(IRQ_HANDLED)就行了。

242 行驱动出口函数中把注册的中断号释放。

关于自旋锁保护的对象，实际上就是 alinx_char.key_sts 这个值，因为这个值在读函数中操作了，在中断开启定时器回掉函数中也操作了，这两个操作是有可能同时发生的，因此需要保护。

8.2.4 测试程序

和第六章的测试程序相同。

8.2.5 运行测试

测试目标是用板子上的 ps key1 去控制 ps led1，测试步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt  
cd /mnt  
mkdir /tmp/qt  
mount qt_lib.img /tmp/qt  
cd /tmp/qt  
source ./qt_env_set.sh  
cd /mnt  
insmod ./ax-concled-drv.ko  
insmod ./ax_irq_drv.ko  
cd ./build-ax-key-test-ZYNQ-Debug  
./ax-key-test /dev/interrupt_led
```

IP 和路径根据实际情况调整。测试现象也与第 6 章相同。

第九章 阻塞 IO

IO 是 Input stream/Output stream 的缩写，即输入输出。对于驱动程序来说的 IO 就是用户程序对设备资源的访问和操作。接下来简单的说说几种 IO 模型以及 Linux 对他们的支持。

9.1 阻塞和非阻塞、同步和异步与 IO 操作

阻塞和非阻塞、同步和异步是在 IO 操作中几种不可避免的状态。

先分开来看，通俗的讲：

- 1) 阻塞就是指某个操作如果不满足执行的条件，他会一直处于等待状态直至条件满足。
- 2) 非阻塞是指某个操作如果不满足执行的条件，他不会等待并且会返回未执行的结果。
- 3) 同步指多个操作同时发生时，这些操作需要排队逐个执行。
- 4) 异步指多个操作同时发生时，这些操作可以一起执行。

对于驱动来说 IO 操作一般可以理解为对外设的读写。一个完整的 IO 操作有两个阶段：

第一阶段：查看外设数据是否就绪；

第二阶段：数据就绪，读写外设数据。

再结合起来看。

阻塞 IO、非阻塞 IO：

当应用程序发出了 IO 请求。如果目标外设或数据没有准备好，对于阻塞 IO 来说，就会在 read 方法一直等待，直到数据准备好才会返回。而非阻塞 IO 会直接返回数据未准备好，应用程序再去处理 NG 的情况，重新读取或是其他。可见阻塞 IO 和非阻塞 IO 体现在 IO 操作的第一阶段。

同步 IO、异步 IO：

同步 IO 和异步 IO 实际上是针对应用程序和内核的交互来说的，应用程序发出 IO 请求后，如果数据没有就绪，需要应用程序不断的去轮询，直到准备就绪再执行第二阶段。对于异步 IO，应用程序发出 IO 请求之后，第一阶段和第二阶段全都交友内核完成，当然驱动程序也属于内核的一部分。

9.2 阻塞 IO

这里说的阻塞 IO 实际上是同步阻塞 IO。Linux 的阻塞式访问中，应用程序调用 read() 函数从设备中读取数据时，如果设备或者数据没有准备好，就会进入休眠让出 CPU 资源，准备好时就会唤醒并返回数据给应用程序。内核提供了等待队列机制来实现这里的休眠唤醒工作。

9.2.1 等待队列

等待队列也就是进程组成的队列，Linux 在系统执行会根据不同的状态把进程分成不同的队列，等待队列就是其中之一。

在驱动中使用等待队列步骤如下：

- 1) 创建并初始化等待队列

创建等待队列的方式为创建一个等待队列头，往队列头下添加项即为队列。队列头定义再 include/linux/wait.h 中，详情如下：

```
1. struct __wait_queue_head {
2.     spinlock_t lock;
3.     struct list_head task_list;
4. };
5. typedef struct __wait_queue_head wait_queue_head_t;
```

定义好队列头之后，使用下面的函数来初始化队列头：

```
void init_waitqueue_head(wait_queue_head_t *q)
```

也可以使用宏定义

```
DECLARE_WAIT_QUEUE_HEAD_ONSTACK(name)
```

一次性完成队列头的创建和初始化，name 为队列头的名字。

2) 创建代表进程的等待队列项

等待队列项也定义在 include/linux/wait.h 头文件中，可以用宏定义：

```
DECLARE_WAITQUEUE(name, tsk)
```

一次性完成队列项的定义和初始化，name 为队列项的名字，tsk 为队列项指代的进程，一般设置为 current。current 是内核中的一个全局变量，表示当前进程。

3) 添加或移除等待队列项到等待队列中并进入休眠

设备或数据不可访问时，就把进程添加进队列，使用接口函数：

```
void add_wait_queue(wait_queue_head_t *q, wait_queue_t *wait)
```

q 为需要加入的队列头，wait 就是需要加入的队列项。

添加完成后使用函数

```
__set_current_state(state_value);
```

来设置进程状态，state_value 可以为：

TASK_UNINTERRUPTIBLE 休眠不可被信号打断；

TASK_INTERRUPTIBLE 休眠可被信号打断。

之后调用任务切换函数

```
schedule();
```

使当前进程进入休眠。如果被唤醒就会接着这个函数的位置往下运行。

紧接着，如果进程被设置成了 TASK_INTERRUPTIBLE 状态，有必要的话，还需要判断进程是不是被信号唤醒，如果是的话那就是误唤醒，需要让进程重新休眠。

使用函数

```
signal_pending(current)
```

来判断当前进程是否为信号唤醒，current 就是当前进程，如果是则返回真。

进程被唤醒后，使用

```
set_current_state(TASK_RUNNING)
```

设置当前进程为运行状态。

如果设备可访问了，队列项从队列头中移除，使用函数：

```
void remove_wait_queue(wait_queue_head_t *q, wait_queue_t *wait)
```

4) 主动唤醒或等待事件

进程休眠后使用下面两个函数来主动唤醒整个队列：

```
void wake_up(wait_queue_head_t *q)
```

```
void wake_up_interruptible(wait_queue_head_t *q)
```

wake_up 函数可以唤醒处于 TASK_INTERRUPTIBLE 和 TASK_UNINTERRUPTIBLE 状态的进程。

wake_up_interruptible 函数只能唤醒处于 TASK_INTERRUPTIBLE 状态的进程。

除了主动唤醒之外，还可以设置成等待某个条件满足后自动唤醒，Linux 提供了这些宏：

```
/* 此函数会把进程设置为 TASK_UNINTERRUPTIBLE，condition 为真(条件)时会唤醒
队列 wq，会一直阻塞等待 condition 为真 */
wait_event(wq, condition)
/* 与 wait_event 类似，不过加了超时机制，timeout 为超时时间单位为 jiffies，时间
到了之后即使条件不满足也会唤醒队列 wq */
wait_event_timeout(wq, condition, timeout)
/*与 wait_event 类似，但是会把进程设置为 TASK_INTERRUPTIBLE */
wait_event_interruptible(wq, condition)
/*与 wait_event_timeout 类似，但是会把进程设置为 TASK_INTERRUPTIBLE */
wait_event_interruptible_timeout(wq, condition, timeout)
```

9.3 实验

我们前面做的按键实验中，测试程序中读取 key 状态的方式都是在 while 循环中不断的去调用 read 方法。而我们在驱动程序中实现的 read 方法也只是简单的返回按键当前的值。这样做就导致测试程序和驱动程序都一直处于活跃状态，导致 cpu 占用率很高。以上一章的例程为例，使用 ./ax-key-test /dev/interrupt_led 命令让 ax-key-test 程序在后台运行。再使用 top 命令来查看 cpu 的占用情况，如下图：

PID	PPID	USER	STAT	VSZ	%VSZ	CPU	%CPU	COMMAND
1274	1254	root	R	13164	1.2	0	49.9	./ax-key-test /dev/interrupt_led
1275	1254	root	R	3024	0.2	1	0.1	top
269	2	root	SW	0	0.0	1	0.1	[kworker/1:1]
1247	1	root	S	14352	1.3	1	0.0	/usr/sbin/tcf-agent -d -L- -10
761	1	root	S	3052	0.3	1	0.0	/sbin/udev -d

双 cpu 的 soc 光是一个按键程序就占用了 49.9% 几乎是一个 cpu 的资源，显然是不可取的。

分析一下，应用程序轮询 read 函数读取按键状态，大部分时候读到的都是未被按下的状态，而我们需要捕捉到的仅是按键被按下的状态，那是不是可以理解为，按键未按下就等同于我们需要的数据还没有准备好呢？在此基础上，我们就可以使用等待队列来是驱动程序中的 read 进程在按键没有按下时进入休眠，应用程序的 read 函数就得不到返回值，就不会一直轮询，从而降低 cpu 占用率。然后在按键按下时，唤醒进程，又能达到驱动程序捕捉按键被按下的动作的要求。

9.3.1 原理图

led 部分和章节 1.3.1 相同。

key 部分和章节 6.1 相同。

9.3.2 设备树

和章节 6.2 相同。

9.3.3 驱动程序

使用 petalinux 新建名为“ax-bio-drv”的驱动程序，并执行 petalinux-config -c rootfs 命令选

上新增的驱动程序。

在 ax-bio-drv.c 文件中输入下面的代码：

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/init.h>
4. #include <linux/types.h>
5. #include <linux/errno.h>
6. #include <linux/cdev.h>
7. #include <linux/of.h>
8. #include <linux/of_address.h>
9. #include <linux/of_gpio.h>
10. #include <linux/device.h>
11. #include <linux/delay.h>
12. #include <linux/init.h>
13. #include <linux/gpio.h>
14. #include <linux/semaphore.h>
15. #include <linux/timer.h>
16. #include <linux/of_irq.h>
17. #include <linux/irq.h>
18. #include <asm/uaccess.h>
19. #include <asm/mach/map.h>
20. #include <asm/io.h>
21. #include <linux/interrupt.h>
22. /* 设备节点名称 */
23. #define DEVICE_NAME "bio_led"
24. /* 设备号个数 */
25. #define DEVID_COUNT 1
26. /* 驱动个数 */
27. #define DRIVE_COUNT 1
28. /* 主设备号 */
29. #define MAJOR_U
30. /* 次设备号 */
31. #define MINOR_U 0
32.
33. /* 把驱动代码中会用的数据打包进设备结构体 */
34. struct alinx_char_dev {
35. /** 字符设备框架 */
36.     dev_t          devid;           //设备号
37.     struct cdev      cdev;           //字符设备
38.     struct class     *class;         //类
39.     struct device     *device;       //设备
40.     struct device_node *nd;          //设备树的设备节点
41. /** gpio */
42.     int              alinx_key_gpio; //gpio 号
43. /** 并发处理 */
44.     atomic_t         key_sts;        //记录按键状态，为 1 时被按下
45. /** 中断 */
46.     unsigned int      irq;           //中断号
47. /** 定时器 */
48.     struct timer_list timer;         //定时器
49. /** 等待队列 */
50.     wait_queue_head_t wait_q_h;      //等待队列头
51. };
52. /* 声明设备结构体 */
53. static struct alinx_char_dev alinx_char = {
54.     .cdev = {
55.         .owner = THIS_MODULE,
56.     },
57. };
58.
59. /** 回掉 */
60. /* 中断服务函数 */
61. static irqreturn_t key_handler(int irq, void *dev)
62. {
63.     /* 按键按下或抬起时会进入中断 */
64.     /* 开启 50 毫秒的定时器用作防抖动 */
65.     mod_timer(&alinx_char.timer, jiffies + msecs_to_jiffies(50));
66.     return IRQ_RETVAL(IRQ_HANDLED);
67. }
68.
69. /* 定时器服务函数 */
70. void timer_function(struct timer_list *timer)
```

```
71. {
72.     /* value 用于获取按键值 */
73.     unsigned char value;
74.     /* 获取按键值 */
75.     value = gpio_get_value(alinx_char.alinx_key_gpio);
76.     if(value == 0)
77.     {
78.         /* 按键按下, 状态置1 */
79.         atomic_set(&alinx_char.key_sts, 1);
80.         /** 等待队列 **/
81.         /* 唤醒进程 */
82.         wake_up_interruptible(&alinx_char.wait_q_h);
83.     }
84.     else
85.     {
86.         /* 按键抬起 */
87.     }
88. }
89.
90. /** 系统调用实现 **/
91. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
92. static int char_drv_open(struct inode *inode_p, struct file *file_p)
93. {
94.     printk("gpio_test module open\n");
95.     return 0;
96. }
97.
98.
99. /* read 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
100. static ssize_t char_drv_read(struct file *file_p, char __user *buf, size_t len, loff_t *loff_t_p)
101. {
102.     unsigned int keysts = 0;
103.     int ret;
104.
105.     /* 读取 key 的状态 */
106.     keysts = atomic_read(&alinx_char.key_sts);
107.     /* 判断当前按键状态 */
108.     if(!keysts)
109.     {
110.         /* 按键未被按下(数据未准备好) */
111.         /* 以当前进程创建并初始化为队列项 */
112.         DECLARE_WAITQUEUE(queue_mem, current);
113.         /* 把当前进程的队列项添加到队列头 */
114.         add_wait_queue(&alinx_char.wait_q_h, &queue_mem);
115.         /* 设置当前进程成为可被信号打断的状态 */
116.         __set_current_state(TASK_INTERRUPTIBLE);
117.         /* 切换进程, 是当前进程休眠 */
118.         schedule();
119.
120.         /* 被唤醒, 修改当前进程状态为 RUNNING */
121.         set_current_state(TASK_RUNNING);
122.         /* 把当前进程的队列项从队列头中删除 */
123.         remove_wait_queue(&alinx_char.wait_q_h, &queue_mem);
124.
125.         /* 判断是否是被信号唤醒 */
126.         if(signal_pending(current))
127.         {
128.             /* 如果是直接返回错误 */
129.             return -ERESTARTSYS;
130.         }
131.         else
132.         {
133.             /* 被按键唤醒 */
134.         }
135.     }
136.     else
137.     {
138.         /* 按键被按下(数据准备好了) */
139.     }
140.
141.     /* 读取 key 的状态 */
142.     keysts = atomic_read(&alinx_char.key_sts);
143.     /* 返回按键状态值 */
144.     ret = copy_to_user(buf, &keysts, sizeof(keysts));
145.     /* 清除按键状态 */
```

```
146.     atomic_set(&alinx_char.key_sts, 0);
147.     return 0;
148. }
149.
150. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
151. static int char_drv_release(struct inode *inode_p, struct file *file_p)
152. {
153.     printk("gpio_test module release\n");
154.     return 0;
155. }
156.
157. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
158. static struct file_operations ax_char_fops = {
159.     .owner    = THIS_MODULE,
160.     .open     = char_drv_open,
161.     .read     = char_drv_read,
162.     .release  = char_drv_release,
163. };
164.
165. /* 模块加载时会调用的函数 */
166. static int __init char_drv_init(void)
167. {
168.     /* 用于接受返回值 */
169.     u32 ret = 0;
170.
171.     /** 并发处理 **/
172.     /* 初始化原子变量 */
173.     atomic_set(&alinx_char.key_sts, 0);
174.
175.     /** gpio 框架 **/
176.     /* 获取设备节点 */
177.     alinx_char.nd = of_find_node_by_path("/alinxkey");
178.     if(alinx_char.nd == NULL)
179.     {
180.         printk("alinx_char node not find\n");
181.         return -EINVAL;
182.     }
183.     else
184.     {
185.         printk("alinx_char node find\n");
186.     }
187.
188.     /* 获取节点中 gpio 标号 */
189.     alinx_char.alinx_key_gpio = of_get_named_gpio(alinx_char.nd, "alinxkey-gpios", 0);
190.     if(alinx_char.alinx_key_gpio < 0)
191.     {
192.         printk("can not get alinxkey-gpios");
193.         return -EINVAL;
194.     }
195.     printk("alinxkey-gpio num = %d\n", alinx_char.alinx_key_gpio);
196.
197.     /* 申请 gpio 标号对应的引脚 */
198.     ret = gpio_request(alinx_char.alinx_key_gpio, "alinxkey");
199.     if(ret != 0)
200.     {
201.         printk("can not request gpio\n");
202.         return -EINVAL;
203.     }
204.
205.     /* 把这个 io 设置为输入 */
206.     ret = gpio_direction_input(alinx_char.alinx_key_gpio);
207.     if(ret < 0)
208.     {
209.         printk("can not set gpio\n");
210.         return -EINVAL;
211.     }
212.
213.     /** 中断 **/
214.     /* 获取中断号 */
215.     alinx_char.irq = gpio_to_irq(alinx_char.alinx_key_gpio);
216.     /* 申请中断 */
217.     ret = request_irq(alinx_char.irq,
218.                       key_handler,
219.                       IRQF_TRIGGER_FALLING | IRQF_TRIGGER_RISING,
220.                       "alinxkey",
221.                       NULL);
222.     if(ret < 0)
```

```
223.     {
224.         printk("irq %d request failed\r\n", alinx_char.irq);
225.         return -EFAULT;
226.     }
227.
228. /** 定时器 **/
229.     __init_timer(&alinx_char.timer, timer_function, 0);
230.
231.
232. /** 等待队列 **/
233.     init_waitqueue_head(&alinx_char.wait_q_h);
234.
235. /** 字符设备框架 **/
236.     /* 注册设备号 */
237.     alloc_chrdev_region(&alinx_char.devid, MINOR_U, DEVID_COUNT, DEVICE_NAME);
238.
239.     /* 初始化字符设备结构体 */
240.     cdev_init(&alinx_char.cdev, &ax_char_fops);
241.
242.     /* 注册字符设备 */
243.     cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
244.
245.     /* 创建类 */
246.     alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
247.     if(IS_ERR(alinx_char.class))
248.     {
249.         return PTR_ERR(alinx_char.class);
250.     }
251.
252.     /* 创建设备节点 */
253.     alinx_char.device = device_create(alinx_char.class, NULL,
254.                                     alinx_char.devid, NULL,
255.                                     DEVICE_NAME);
256.     if (IS_ERR(alinx_char.device))
257.     {
258.         return PTR_ERR(alinx_char.device);
259.     }
260.
261.     return 0;
262. }
263.
264. /* 卸载模块 */
265. static void __exit char_drv_exit(void)
266. {
267.     /** gpio **/
268.     /* 释放 gpio */
269.     gpio_free(alinx_char.alinx_key_gpio);
270.
271.     /** 中断 **/
272.     /* 释放中断 */
273.     free_irq(alinx_char.irq, NULL);
274.
275.     /** 定时器 **/
276.     /* 删除定时器 */
277.     del_timer_sync(&alinx_char.timer);
278.
279.     /** 字符设备框架 **/
280.     /* 注销字符设备 */
281.     cdev_del(&alinx_char.cdev);
282.
283.     /* 注销设备号 */
284.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
285.
286.     /* 删除设备节点 */
287.     device_destroy(alinx_char.class, alinx_char.devid);
288.
289.     /* 删除类 */
290.     class_destroy(alinx_char.class);
291.
292.     printk("timer_led_dev_exit_ok\n");
293. }
294.
295. /* 标记加载、卸载函数 */
296. module_init(char_drv_init);
297. module_exit(char_drv_exit);
298.
299. /* 驱动描述信息 */
```



```
300. MODULE_AUTHOR("Alinx");  
301. MODULE_ALIAS("alinx_char");  
302. MODULE_DESCRIPTION("BIO LED driver");  
303. MODULE_VERSION("v1.0");  
304. MODULE_LICENSE("GPL");
```

这次的程序在上一章中断的驱动程序基础上修改，只要修改集中在 read 函数中。

这次我们把自旋锁换成了原子变量，仅对 key_sts 这个状态值的读写做保护。

代码的 50 行先定义了一个等待队列头。

在入口函数的 234 行吧队列头进行了初始化。

应用程序通过 read 方法来读取 key 的状态，所以，先到 read 函数中做一些改动。前面提到过，key 被按下才认为是数据准备好了。

进入 read 函数后 107~109 行我们先判断 key 的状态，如果 key 没有被按下，就使用等待队列，等待按键被按下。

113 行以当前进程创建并初始化名为 queue_mem 的队列项。

115 行把队列项加入队列头。

117 行设置进程为可被信号打断的状态，然后 119 行调用 schedule 切换进程，使当前进程休眠。

休眠了就需要相应的唤醒契机，我们是在等待按键被按下，所以，唤醒就可以放在按键的中断中去执行，在 93 行定时器的回掉中，最终确定案件被按下的同时，调用 wake_up_interruptible(&alinx_char.wait_q_h);唤醒等待队列。

唤醒之后我们就又回到了 122 行，接着刚才休眠的位置继续运行，先调用 set_current_state(TASK_RUNNING);把当前进程的状态设为 RUNNING。

124 行再把队列项从队列头中删除。

由于进程是可以被信号唤醒的，所以还需要判断进程是否是被信号还信，如果是则直接返回错误。

如果不是，就把按键的值返回给用户。

9.3.4 测试程序

和第六章的测试程序相同。

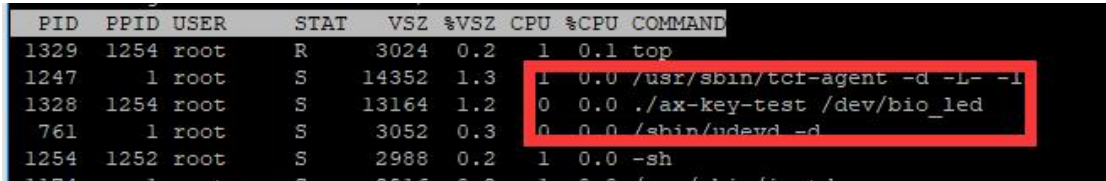
9.3.5 运行测试

测试步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt  
cd /mnt  
mkdir /tmp/qt  
mount qt_lib.img /tmp/qt  
cd /tmp/qt  
source ./qt_env_set.sh  
cd /mnt  
insmod ./ax-concled-drv.ko  
insmod ./ax-bio-drv.ko  
cd ./build-ax-key-test-ZYNQ-Debug  
./ax-key-test /dev/bio_led&  
top
```

IP 和路径根据实际情况调整。按键的现象与上一章相同。

此外，我们再看一下测试程序的 cpu 占用量。



PID	PPID	USER	STAT	VSZ	%VSZ	CPU	%CPU	COMMAND
1329	1254	root	R	3024	0.2	1	0.1	top
1247	1	root	S	14352	1.3	1	0.0	/usr/sbin/tcr-agent -d -L- -l
1328	1254	root	S	13164	1.2	0	0.0	./ax-key-test /dev/bio_led
761	1	root	S	3052	0.3	0	0.0	/sbin/udevd -d
1254	1252	root	S	2988	0.2	1	0.0	-sh

几乎可以忽略了，因为测试程序没有改动，所以看来是等待队列起作用了。

如果想要关闭后台运行的程序，可以使用 kill 命令加上 top 命令中对应的 PID，比如我们这里想要关闭 ax-key-test 程序，就是用命令 kill 1328 即可

第十章 非阻塞 IO

这章来讲另一种 IO 模型非阻塞 IO(NIO)，也就是同步非阻塞 IO。上一章说过，IO 操作的两个阶段先查询再读写，而非阻塞 IO 在查询阶段的处理和阻塞 IO 不同。应用程序需要进行 IO 操作前，先发起查询，驱动程序根据数据情况返回查询结果，如果返回查询结果 NG，应用程序就不执行读写操作了。如果应用程序非要读写的话，就继续去查询，直到驱动程序返回数据准备完成，才会做下一步的读写操作。

10.1 Linux 中的 NIO

非阻塞 IO 的处理方式是轮询。Linux 中提供了应用程序的轮询机制和相应的驱动程序系统调用。

10.1.1 应用程序中的轮询方法

应用程序中提供了三种轮询的方法：select、poll、epoll。实际上他们也是多路复用 IO 的解决方法，这里就不展开说了。

1) select

select 有良好的跨平台支持性，但是他的单个进程能够监视的文件描述符的数量存在最大限制(Linux 中一般为 1024)。

函数原型：

```
int select(int maxfdp, fd_set *readfds, fd_set *writefds, fd_set *errorfds, struct timeval *timeout);
```

参数说明：

maxfdp：是集合中所有文件描述符的范围，即等于所有文件描述符的最大值加 1。

readfds：struct fd_set 结构体可以理解为是文件描述符(file descriptor)的集合，也就是文件句柄，他的每一位就代表一个描述符，readfds 用于监视指定描述符集的读变化，只要其中有一位可读，select 就会返回一个大于 0 的值。可以已使用这些宏来操作 fd_set 变量：

```
/* 把 fdset 所有位置 0, 清楚 fdset 和所有文件句柄的关系 */
FD_ZERO(fd_set *fdset)
/* 把 fdset 某个位置 1, 把 fdset 和文件句柄 fd 关联 */
FD_SET(int fd, fd_set *fdset)
/* 把 fdset 某个位置 0, 取消 fdset 和文件句柄的关联 */
FD_CLR(int fd, fd_set *fdset)
/* 判断某个文件句柄是否为 1, 即是否可操作 */
FD_ISSET(int fd, fdset *fdset)
```

writefds：用于见识文件是否可写。

errorfds：用于监视文件异常。

timeout：struct timeval 用来代表时间值，有两个成员，一个是秒数，另一个是毫秒数。定义如下：

```
1. struct timeval{
2.     long tv_sec;    /*秒 */
3.     long tv_usec;  /*微秒 */
```

```
4. }
```

这个参数的值有三种情况

如果传入 NULL，即不传入时间结构，就是一个阻塞函数，直到某个文件描述符发生变化才会返回。

如果把秒和微妙都设为 0，就是一个非阻塞函数，会立刻返，文件无变化返回 0，有变化返回正值。

如果值大于 0，则意为超时时间，select 若在 timeout 时间内没有检测到文件描述符变化，则会直接返回 0，有变化则返回正值。

使用示例：

```
1. void main(void)
2. {
3.     /* ret 获取返回值, fd 获取文件句柄 */
4.     int ret, fd;
5.     /* 定义一个监视文件读变化的描述符合集 */
6.     fd_set readfds;
7.     /* 定义一个超时时间结构体 */
8.     struct timeval timeout;
9.
10.    /* 获取文件句柄, O_NONBLOCK 表示非阻塞访问 */
11.    fd = open("dev_xxx", O_RDWR | O_NONBLOCK);
12.
13.    /* 初始化描述符合集 */
14.    FD_ZERO(&readfds);
15.    /* 把文件句柄 fd 指向的文件添加到描述符 */
16.    FD_SET(fd, &readfds);
17.
18.    /* 超时时间初始化为 1.5 秒 */
19.    timeout.tv_sec = 1;
20.    timeout.tv_usec = 500000;
21.
22.    /* 调用 select, 注意第一个参数为 fd+1 */
23.    ret = select(fd + 1, &readfds, NULL, NULL, &timeout);
24.
25.    switch (ret)
26.    {
27.        case 0:
28.        {
29.            /* 超时 */
30.            break;
31.        }
32.        case -1:
33.        {
34.            /* 出错 */
35.            break;
36.        }
37.        default:
38.        {
39.            /* 监视的文件可操作 */
40.            /* 判断可操作的文件是不是文件句柄 fd 指向的文件 */
41.            if(FD_ISSET(fd, &readfds))
42.            {
43.                /* 操作文件 */
44.            }
45.            break;
46.        }
47.    }
48. }
```

在 23 行调用 select 函数之前，做了很多准备工作，主要是 select 函数输入参数的初始化。

注意 11 行 open 函数输入参数中的 O_NONBLOCK 属性，如果需要非阻塞的访问文件，则需要添加这个属性。

41 行, 在 ret 返回大于 0 时, 使用宏定义 FD_ISSET 判断可操作的句柄是不是我们需要的句柄, 在只等待一个文件的情况下, 可以不做这个判断。

2) poll

poll 本质上和 select 没有区别, 但是他的最大连接数没有限制。

函数原型:

```
int poll (struct pollfd *fds, unsigned int nfds, int timeout);
```

参数说明:

fds: struct pollfd 结构体是文件句柄和事件的组合, 定义如下:

```
1. struct pollfd {
2.     int fd;
3.     short events;
4.     short revents;
5. };
```

fd 是文件句柄, **events** 是对于这个文件需要监视的事件类型, **revents** 是内核返回的事件类型。事件类型有:

```
POLLIN    //有数据可读
POLLPRI   //有紧急数据可读
POLLOUT   //数据可写
POLLERR   //指定文件描述符发生错误
POLLHUP   //指定文件描述符挂起
POLLNVAL  //无效请求
POLLRDNORM //有数据可读
```

nfds: poll 监视的文件句柄数量, 也就是 fds 的数组长度。

timeout: 超时时间, 单位为毫秒。

使用示例:

```
1. void main(void)
2. {
3.     /* ret 获取返回值, fd 获取文件句柄 */
4.     int ret, fd;
5.     /* 定义 struct pollfd 结构体变量 */
6.     struct pollfd fds[1];
7.
8.     /* 非阻塞访问文件 */
9.     fd = open(filename, O_RDWR | O_NONBLOCK);
10.
11.    /* 初始化 struct pollfd 结构体变量 */
12.    fds[0].fd = fd;
13.    fds[0].events = POLLIN;
14.
15.    /* 调用 poll */
16.    ret = poll(fds, sizeof(fds), 1500);
17.    if(ret == 0)
18.    {
19.        /* 超时 */
20.    }
21.    else if (ret < 0)
22.    {
23.        /* 错误 */
24.    }
25.    else
26.    {
27.        /* 操作数据 */
28.    }
```

```
29. }
```

3) epoll

可以理解为 event poll，设计用于大并发时的 IO 查询，常用于网络编程，暂不介绍。

10.1.2 驱动程序中的 poll 函数

应用程序中调用 select、poll、epoll 时，系统调用就会执行驱动程序中 file_operations 的 poll 函数。也就是我们需要实现的函数。原型如下：

```
unsigned int (*poll) (struct file *filp, struct poll_table_struct *wait)
```

参数说明：

filp：应用程序传递过来的值，应用程序 open 之后获得的目标文件句柄。

wait：应用程序传递过来的值，代表应用程序线程。我们需要在 poll 函数中调用 poll_wait 将应用程序线程 wait 添加到 poll_table 等待队列中，poll_wait 函数原型如下：

```
void poll_wait(struct file * filp, wait_queue_head_t * wait_address, poll_table *p)
```

wait 作为参数 p 传递给 poll_wait 函数。

返回值：返回值和 struct pollfd 结构体中的事件类型相同。

10.2 实验

这章的实验目标和上一张相同，使用 ps_key1 控制 ps_led1，并使 cpu 占用率相较于第八章降低。

10.2.1 原理图

led 部分和章节 1.3.1 相同。

key 部分和章节 6.1 相同。

10.2.2 设备树

和章节 6.2 相同。

10.2.3 驱动代码

使用 petalinux 新建名为“ax-nio-driv”的驱动程序，并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-nio-driv.c 文件中输入下面的代码：

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/init.h>
4. #include <linux/types.h>
5. #include <linux/errno.h>
6. #include <linux/cdev.h>
7. #include <linux/of.h>
8. #include <linux/of_address.h>
9. #include <linux/of_gpio.h>
10. #include <linux/device.h>
11. #include <linux/delay.h>
12. #include <linux/init.h>
13. #include <linux/gpio.h>
14. #include <linux/semaphore.h>
15. #include <linux/timer.h>
16. #include <linux/of_irq.h>
17. #include <linux/irq.h>
18. #include <linux/wait.h>
19. #include <linux/poll.h>
20. #include <asm/uaccess.h>
```

```
21. #include <asm/mach/map.h>
22. #include <asm/io.h>
23. #include <linux/interrupt.h>
24. /* 设备节点名称 */
25. #define DEVICE_NAME "nio_led"
26. /* 设备号个数 */
27. #define DEVID_COUNT 1
28. /* 驱动个数 */
29. #define DRIVE_COUNT 1
30. /* 主设备号 */
31. #define MAJOR_U
32. /* 次设备号 */
33. #define MINOR_U 0
34.
35. /* 把驱动代码中会用到的数据打包进设备结构体 */
36. struct alinx_char_dev {
37. /** 字符设备框架 */
38.     dev_t          devid;           //设备号
39.     struct cdev     cdev;           //字符设备
40.     struct class    *class;         //类
41.     struct device    *device;        //设备
42.     struct device_node *nd;          //设备树的设备节点
43. /** gpio */
44.     int             alinx_key_gpio;  //gpio 号
45. /** 并发处理 */
46.     atomic_t        key_sts;         //记录按键状态, 为 1 时被按下
47. /** 中断 */
48.     unsigned int     irq;            //中断号
49. /** 定时器 */
50.     struct timer_list timer;         //定时器
51. /** 等待队列 */
52.     wait_queue_head_t wait_q_h;      //等待队列头
53. };
54. /* 声明设备结构体 */
55. static struct alinx_char_dev alinx_char = {
56.     .cdev = {
57.         .owner = THIS_MODULE,
58.     },
59. };
60.
61. /** 回掉 */
62. /* 中断服务函数 */
63. static irqreturn_t key_handler(int irq, void *dev)
64. {
65.     /* 按键按下或抬起时会进入中断 */
66.     /* 开启 50 毫秒的定时器用作防抖动 */
67.     mod_timer(&alinx_char.timer, jiffies + msecs_to_jiffies(50));
68.     return IRQ_RETVAL(IRQ_HANDLED);
69. }
70.
71. /* 定时器服务函数 */
72. void timer_function(struct timer_list *timer)
73. {
74.     /* value 用于获取按键值 */
75.     unsigned char value;
76.     /* 获取按键值 */
77.     value = gpio_get_value(alinx_char.alinx_key_gpio);
78.     if(value == 0)
79.     {
80.         /* 按键按下, 状态置 1 */
81.         atomic_set(&alinx_char.key_sts, 1);
82. /** 等待队列 */
83.         /* 唤醒进程 */
84.         wake_up_interruptible(&alinx_char.wait_q_h);
85.     }
86.     else
87.     {
88.         /* 按键抬起 */
89.     }
90. }
91.
92. /** 系统调用实现 */
93. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
94. static int char_drv_open(struct inode *inode_p, struct file *file_p)
95. {
```



```
96.     printk("gpio_test module open\n");
97.     return 0;
98. }
99.
100. /* read 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
101. static ssize_t char_drv_read(struct file *file_p, char __user *buf, size_t len, loff_t *loff_t_p)
102. {
103.     unsigned int keysts = 0;
104.     int ret;
105.
106.     /* 读取 key 的状态 */
107.     keysts = atomic_read(&alinx_char.key_sts);
108.     /* 判断文件打开方式 */
109.     if(file_p->f_flags & O_NONBLOCK)
110.     {
111.         /* 如果是非阻塞访问, 说明以满足读取条件 */
112.     }
113.     /* 判断当前按键状态 */
114.     else if(!keysts)
115.     {
116.         /* 按键未被按下(数据未准备好) */
117.         /* 以当前进程创建并初始化为队列项 */
118.         DECLARE_WAITQUEUE(queue_mem, current);
119.         /* 把当前进程的队列项添加到队列头 */
120.         add_wait_queue(&alinx_char.wait_q_h, &queue_mem);
121.         /* 设置当前进程成为可被信号打断的状态 */
122.         __set_current_state(TASK_INTERRUPTIBLE);
123.         /* 切换进程, 是当前进程休眠 */
124.         schedule();
125.
126.         /* 被唤醒, 修改当前进程状态为 RUNNING */
127.         set_current_state(TASK_RUNNING);
128.         /* 把当前进程的队列项从队列头中删除 */
129.         remove_wait_queue(&alinx_char.wait_q_h, &queue_mem);
130.
131.         /* 判断是否是被信号唤醒 */
132.         if(signal_pending(current))
133.         {
134.             /* 如果是直接返回错误 */
135.             return -ERESTARTSYS;
136.         }
137.         else
138.         {
139.             /* 被按键唤醒 */
140.         }
141.     }
142.     else
143.     {
144.         /* 按键被按下(数据准备好了) */
145.     }
146.
147.     /* 读取 key 的状态 */
148.     keysts = atomic_read(&alinx_char.key_sts);
149.     /* 返回按键状态值 */
150.     ret = copy_to_user(buf, &keysts, sizeof(keysts));
151.     /* 清除按键状态 */
152.     atomic_set(&alinx_char.key_sts, 0);
153.     return 0;
154. }
155.
156. /* poll 函数实现 */
157. unsigned int char_drv_poll(struct file *filp, struct poll_table_struct *wait)
158. {
159.     unsigned int ret = 0;
160.
161.     /* 将应用程序添加到等待队列中 */
162.     poll_wait(filp, &alinx_char.wait_q_h, wait);
163.
164.     /* 判断 key 的状态 */
165.     if(atomic_read(&alinx_char.key_sts))
166.     {
167.         /* key 准备好了, 返回数据可读 */
168.         ret = POLLIN;
169.     }
170.     else
```

```
171. {
172.
173. }
174.
175. return ret;
176. }
177.
178. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
179. static int char_drv_release(struct inode *inode_p, struct file *file_p)
180. {
181.     printk("gpio_test module release\n");
182.     return 0;
183. }
184.
185. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
186. static struct file_operations ax_char_fops = {
187.     .owner    = THIS_MODULE,
188.     .open     = char_drv_open,
189.     .read     = char_drv_read,
190.     .poll     = char_drv_poll,
191.     .release  = char_drv_release,
192. };
193.
194. /* 模块加载时会调用的函数 */
195. static int __init char_drv_init(void)
196. {
197.     /* 用于接受返回值 */
198.     u32 ret = 0;
199.
200.     /** 并发处理 **/
201.     /* 初始化原子变量 */
202.     atomic_set(&alinx_char.key_sts, 0);
203.
204.     /** gpio 框架 **/
205.     /* 获取设备节点 */
206.     alinx_char.nd = of_find_node_by_path("/alinxkey");
207.     if(alinx_char.nd == NULL)
208.     {
209.         printk("alinx_char node not find\n");
210.         return -EINVAL;
211.     }
212.     else
213.     {
214.         printk("alinx_char node find\n");
215.     }
216.
217.     /* 获取节点中 gpio 标号 */
218.     alinx_char.alinx_key_gpio = of_get_named_gpio(alinx_char.nd, "alinxkey-gpios", 0);
219.     if(alinx_char.alinx_key_gpio < 0)
220.     {
221.         printk("can not get alinxkey-gpios");
222.         return -EINVAL;
223.     }
224.     printk("alinxkey-gpio num = %d\n", alinx_char.alinx_key_gpio);
225.
226.     /* 申请 gpio 标号对应的引脚 */
227.     ret = gpio_request(alinx_char.alinx_key_gpio, "alinxkey");
228.     if(ret != 0)
229.     {
230.         printk("can not request gpio\n");
231.         return -EINVAL;
232.     }
233.
234.     /* 把这个 io 设置为输入 */
235.     ret = gpio_direction_input(alinx_char.alinx_key_gpio);
236.     if(ret < 0)
237.     {
238.         printk("can not set gpio\n");
239.         return -EINVAL;
240.     }
241.
242.     /** 中断 **/
243.     /* 获取中断号 */
244.     alinx_char.irq = gpio_to_irq(alinx_char.alinx_key_gpio);
245.     /* 申请中断 */
246.     ret = request_irq(alinx_char.irq,
247.                       key_handler,
```

```
248.             IRQF_TRIGGER_FALLING | IRQF_TRIGGER_RISING,
249.             "alinxkey",
250.             NULL);
251.     if(ret < 0)
252.     {
253.         printk("irq %d request failed\r\n", alinx_char.irq);
254.         return -EFAULT;
255.     }
256.
257. /** 定时器 **/
258.     __init_timer(&alinx_char.timer, timer_function, 0);
259.
260.
261. /** 等待队列 **/
262.     init_waitqueue_head(&alinx_char.wait_q_h);
263.
264. /** 字符设备框架 **/
265.     /* 注册设备号 */
266.     alloc_chrdev_region(&alinx_char.devid, MINOR_U, DEVID_COUNT, DEVICE_NAME);
267.
268.     /* 初始化字符设备结构体 */
269.     cdev_init(&alinx_char.cdev, &ax_char_fops);
270.
271.     /* 注册字符设备 */
272.     cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
273.
274.     /* 创建类 */
275.     alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
276.     if(IS_ERR(alinx_char.class))
277.     {
278.         return PTR_ERR(alinx_char.class);
279.     }
280.
281.     /* 创建设备节点 */
282.     alinx_char.device = device_create(alinx_char.class, NULL,
283.                                     alinx_char.devid, NULL,
284.                                     DEVICE_NAME);
285.     if (IS_ERR(alinx_char.device))
286.     {
287.         return PTR_ERR(alinx_char.device);
288.     }
289.
290.     return 0;
291. }
292.
293. /** 卸载模块 */
294. static void __exit char_drv_exit(void)
295. {
296. /** gpio **/
297.     /* 释放 gpio */
298.     gpio_free(alinx_char.alinx_key_gpio);
299.
300. /** 中断 **/
301.     /* 释放中断 */
302.     free_irq(alinx_char.irq, NULL);
303.
304. /** 定时器 **/
305.     /* 删除定时器 */
306.     del_timer_sync(&alinx_char.timer);
307.
308. /** 字符设备框架 **/
309.     /* 注销字符设备 */
310.     cdev_del(&alinx_char.cdev);
311.
312.     /* 注销设备号 */
313.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
314.
315.     /* 删除设备节点 */
316.     device_destroy(alinx_char.class, alinx_char.devid);
317.
318.     /* 删除类 */
319.     class_destroy(alinx_char.class);
320.
321.     printk("timer_led_dev_exit_ok\r\n");
322. }
323.
324. /** 标记加载、卸载函数 */
```

```
325. module_init(char_drv_init);
326. module_exit(char_drv_exit);
327.
328. /* 驱动描述信息 */
329. MODULE_AUTHOR("Alinx");
330. MODULE_ALIAS("alinx char");
331. MODULE_DESCRIPTION("NIO LED driver");
332. MODULE_VERSION("v1.0");
333. MODULE_LICENSE("GPL");
```

驱动代码在上一章的代码基础上，增加了 poll 实现，并稍微修改了 read 函数。

191 行在 file_operations 结构体中添加 poll 函数实现。

158 行实现 poll 函数，调用一下 poll_wait 函数，之后哦按段数据状态，如果数据准备好，就返回 POLLIN 状态标识。

110 行在 read 函数中稍作修改，先判断文件打开方式，如果是非阻塞的方式访问，就不去做队列相关的操作了，直接返回数据给用户，否则按照阻塞访问处理。

10.2.4 测试代码

测试代码在 6.4 节的基础上修改，新建 QT 工程名为“ax_nioled_test”，新建 main.c，输入下列代码：

```
1. #include "stdio.h"
2. #include "unistd.h"
3. #include "sys/types.h"
4. #include "sys/stat.h"
5. #include "fcntl.h"
6. #include "stdlib.h"
7. #include "string.h"
8. #include "poll.h"
9. #include "sys/select.h"
10. #include "sys/time.h"
11. #include "linux/ioctl.h"
12.
13. int main(int argc, char *argv[])
14. {
15.
16.     /* ret 获取返回值, fd 获取文件句柄 */
17.     int ret, fd, fd_l;
18.     /* 定义一个监视文件读变化的描述符合集 */
19.     fd_set readfds;
20.     /* 定义一个超时时间结构体 */
21.     struct timeval timeout;
22.
23.     char *filename, led_value = 0;
24.     unsigned int key_value;
25.
26.     if(argc != 2)
27.     {
28.         printf("Error Usage\r\n");
29.         return -1;
30.     }
31.
32.     filename = argv[1];
33.     /* 获取文件句柄, O_NONBLOCK 表示非阻塞访问 */
34.     fd = open(filename, O_RDWR | O_NONBLOCK);
35.     if(fd < 0)
36.     {
37.         printf("can not open file %s\r\n", filename);
38.         return -1;
39.     }
40.
41.     while(1)
42.     {
43.         /* 初始化描述符合集 */
44.         FD_ZERO(&readfds);
45.         /* 把文件句柄 fd 指向的文件添加到描述符 */
46.         FD_SET(fd, &readfds);
47.
```

```
48.      /* 超时时间初始化为 1.5 秒 */
49.      timeout.tv_sec = 1;
50.      timeout.tv_usec = 500000;
51.
52.      /* 调用 select, 注意第一个参数为 fd+1 */
53.      ret = select(fd + 1, &readfds, NULL, NULL, &timeout);
54.      switch (ret)
55.      {
56.          case 0:
57.          {
58.              /* 超时 */
59.              break;
60.          }
61.          case -1:
62.          {
63.              /* 出错 */
64.              break;
65.          }
66.          default:
67.          {
68.              /* 监视的文件可操作 */
69.              /* 判断可操作的文件是不是文件句柄 fd 指向的文件 */
70.              if(FD_ISSET(fd, &readfds))
71.              {
72.                  /* 操作文件 */
73.                  ret = read(fd, &key_value, sizeof(key_value));
74.                  if(ret < 0)
75.                  {
76.                      printf("read failed\r\n");
77.                      break;
78.                  }
79.                  printf("key_value = %d\r\n", key_value);
80.                  if(1 == key_value)
81.                  {
82.                      printf("ps_key1 press\r\n");
83.                      led_value = !led_value;
84.
85.                      fd_1 = open("/dev/gpio_leds", O_RDWR);
86.                      if(fd_1 < 0)
87.                      {
88.                          printf("file /dev/gpio_leds open failed\r\n");
89.                          break;
90.                      }
91.
92.                      ret = write(fd_1, &led_value, sizeof(led_value));
93.                      if(ret < 0)
94.                      {
95.                          printf("write failed\r\n");
96.                          break;
97.                      }
98.
99.                      ret = close(fd_1);
100.                     if(ret < 0)
101.                     {
102.                         printf("file /dev/gpio_leds close failed\r\n");
103.                         break;
104.                     }
105.                 }
106.             }
107.             break;
108.         }
109.     }
110.     close(fd);
111.     return ret;
112. }
113. }
```

73 行的 read 函数开始, 之后的代码与 6.4 节是一样的, 通过判断 key 的状态, 来改变 led 的状态。

在调用 read 之前, 先调用 select 函数来检测数据状态, 用法和 10.1.1 节中的用法相同, 就不重复说明了。

10.2.5 运行测试

测试方式和现象和上一章一样，步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ./ax-concled-drv.ko
insmod ./ax-nio-drv.ko
cd ./build-ax_nioled_test-ZYNQ-Debug
./ax-nioled-test /dev/nio_led&
top
```

此外，可以尝试一下，把测试程中的超时时间改成 0 或者 NULL 来贯彻现象。

第十一章 异步 IO

这里要说的异步 IO 准确的说应该叫“信号驱动的异步 I/O”，也可以成为异步通知。前面两章说的阻塞和非阻塞 IO，他们都是同步 IO，需要应用程序不断的轮询设备是否可以被访问。而异步 IO 模型下，设备可被访问时，可以由驱动程序主动通知应用程序进行访问。他的形式类似于硬件层面的中断，可以理解为由软件实现的模拟中断机制。

11.1 Linux 中的异步 IO

11.1.1 信号

Linux 系统中，使用信号来实现异步 IO 机制。在头文件 `arch/xtensa/include/uapi/asm/signal.h` 的 34~72 行定义了所有 Linux 系统支持的信号，相当于是中断系统中的中断号。信号的使用和中断也很想色，一个信号对应一个回掉函数，收到信号时，就会执行对应的回掉。

对于应用程序来说，信号是一种模拟中断方法，调用接口使用即可，而驱动程序则需要实现相应的方法，提供底层的支持。分别来看应用程序和驱动程序中信号的用法和实现方法。

11.1.2 应用程序中信号的使用

应用程序中使用信号的步骤如下：

- 1) 指定信号并指定对应的信号处理函数

使用下面的函数来选择一个信号并指定对应的信号处理函数：

```
sighandler_t signal(int signum, sighandler_t handler)
```

参数说明：

signum：需要选择的信号，从文件 `arch/xtensa/include/uapi/asm/signal.h` 的 34~72 行宏定义中选一个。

handler：对应的信号处理函数。

返回值：成功返回信号的前一个处理函数，失败返回 `SGI_ERR`。

信号处理函数 `sighandler_t` 的原型如下：

```
typedef void (*sighandler_t)(int)
```

- 2) 设置即将接收信号的进程 ID

```
fcntl(fd, F_SETOWN, getpid());
```

`fd` 是设备文件句柄，`F_SETOWN` 命令表示设置将接收 `SIGIO` 或 `SIGURG` 信号的线程 ID，`getpid()` 是获取当前线程 ID。

- 3) 获取当前线程状态，并使它进入 `FASYNC` 状态

使用下面的方法获取当前进程状态：

```
flag = fcntl(fd, F_GETFL);
```

`flag` 是整型，`fd` 是设备文件句柄，`F_GETFL` 命令表示获取当前线程状态。

获取到当前线程状态后，在当前状态的基础上，设置进程为 `FASYNC` 状态，即开启当前线程的异步通知功能，使用下面的命令：

```
fcntl(fd, F_SETFL, flags | FASYNC);
```


fd 是设备文件句柄，F_SETFL 命令表示设置当前进程状态，flag | FASYNC 中 flag 是获取到的当前线状态，或上 FASYNC 即在当前的状态上增加 FASYNC 状态。当线程的 FASYNC 状态被设置时，对应驱动程序 file_operations 操作函数中的 fasync 就会被调用。

11.1.3 驱动程序中信号的实现

在驱动程序中支持信号，需要以下步骤：

- 1) 在设备结构体中声明 fasync_struct 结构体指针：

```
struct xxx_dev {
    .....
    struct fasync_struct *fasync;
}
```

- 2) 实现 fasync 函数

fasync()函数的实现，一般只需将该函数的 3 个参数以及 fasync_struct 结构体指针传入 fasync_helper()函数就可以了，如下：

```
static int xxx_fasync(int fd, struct file *filp, int mode)
{
    struct xxx_dev *dev = filp->private_data;
    return fasync_helper(fd, filp, mode, &dev->fasync);
}

static struct file_operations xxx_ops = {
    .....
    .fasync = xxx_fasync,
};
```

然后在 release 函数中调用 fasync 使用释放 fasync_struct 结构体，如下：

```
static int xxx_release(struct inode *inode, struct file *filp)
{
    .....
    return xxx_fasync(-1, filp, 0);
}
```

- 3) 设备可访问时，发出信号

应用程序开启异步通知之后，就是在等待设备可操作的信号，驱动程序需要在设备可操作时发出信号，使用函数：

```
void kill_fasync(struct fasync_struct **fp, int sig, int band)
```

参数说明：

fp：目标 fasync_struct 结构体变量指针的地址。

sig：发送的信号类型，范围是文件 arch/xtensa/include/uapi/asm/signal.h 的 34~72 行宏定义，需要和应用程序的需求一致。

band：设备可写时设为 POLL_IN，可读时为 POLL_OUT。

11.2 实验

驱动程序在上一章的基础上，增加异步 IO 的实现。然后再完成对应的测试程序。

11.2.1 原理图

led 部分和章节 1.3.1 相同。

key 部分和章节 6.1 相同。

11.2.2 设备树

和章节 6.2 相同。

11.2.3 驱动代码

使用 petalinux 新建名为“ax-fasync-drv”的驱动程序，并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-fasync-drv.c 文件中输入下面的代码：

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/init.h>
4. #include <linux/types.h>
5. #include <linux/errno.h>
6. #include <linux/cdev.h>
7. #include <linux/of.h>
8. #include <linux/of_address.h>
9. #include <linux/of_gpio.h>
10. #include <linux/device.h>
11. #include <linux/delay.h>
12. #include <linux/init.h>
13. #include <linux/gpio.h>
14. #include <linux/semaphore.h>
15. #include <linux/timer.h>
16. #include <linux/of_irq.h>
17. #include <linux/irq.h>
18. #include <linux/wait.h>
19. #include <linux/poll.h>
20. #include <linux/fcntl.h>
21. #include <asm/uaccess.h>
22. #include <asm/mach/map.h>
23. #include <asm/io.h>
24. #include <linux/interrupt.h>
25. /* 设备节点名称 */
26. #define DEVICE_NAME "fasync_led"
27. /* 设备号个数 */
28. #define DEVID_COUNT 1
29. /* 驱动个数 */
30. #define DRIVE_COUNT 1
31. /* 主设备号 */
32. #define MAJOR_U
33. /* 次设备号 */
34. #define MINOR_U 0
35.
36. /* 把驱动代码中会用到的数据打包进设备结构体 */
37. struct alinx_char_dev {
38.     dev_t devid; //设备号
39.     struct cdev cdev; //字符设备
40.     struct class *class; //类
41.     struct device *device; //设备
42.     struct device_node *nd; //设备树的设备节点
43.     int alinx_key_gpio; //gpio 号
44.     atomic_t key_sts; //记录按键状态，为 1 时被按下
45.     unsigned int irq; //中断号
46.     struct timer_list timer; //定时器
47.     wait_queue_head_t wait_q_h; //等待队列头
48.     struct fasync_struct *fasync; //异步信号
49. };
50. /* 声明设备结构体 */
51. static struct alinx_char_dev alinx_char = {
52.     .cdev = {
53.         .owner = THIS_MODULE,
54.     },
55. };
56.
57. /* 中断服务函数 */
58. static irqreturn_t key_handler(int irq, void *dev_in)
59. {
60.     /* 按键按下或抬起时会进入中断 */
```

```
61. struct alinx_char_dev *dev = (struct alinx_char_dev *)dev_in;
62. /* 开启 50 毫秒的定时器用作防抖动 */
63. dev->timer.data = (volatile long)dev_in;
64. mod_timer(&dev->timer, jiffies + msecs_to_jiffies(50));
65. return IRQ_RETVAL(IRQ_HANDLED);
66. }
67.
68. /* 定时器服务函数 */
69. void timer_function(struct timer_list *timer)
70. {
71.     struct alinx_char_dev *dev = &alinx_char;
72.     /* value 用于获取按键值 */
73.     unsigned char value;
74.     /* 获取按键值 */
75.     value = gpio_get_value(dev->alinx_key_gpio);
76.     if(value == 0)
77.     {
78.         /* 按键按下, 状态置 1 */
79.         atomic_set(&dev->key_sts, 1);
80.         /* fasync 有没有初始化过 */
81.         if(dev->fasync)
82.         {
83.             /* 初始化过说明应用程序调用过 */
84.             kill_fasync(&dev->fasync, SIGIO, POLL_OUT);
85.         }
86.         else if((current->state & TASK_INTERRUPTIBLE) != 0)
87.         {
88.             /* 是等待队列, 需要唤醒进程 */
89.             wake_up_interruptible(&dev->wait_q_h);
90.         }
91.         else
92.         {
93.             /* do nothing */
94.         }
95.     }
96.     else
97.     {
98.         /* 按键抬起 */
99.     }
100. }
101.
102. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
103. static int char_drv_open(struct inode *inode_p, struct file *file_p)
104. {
105.     printk("gpio_test module open\n");
106.     file_p->private_data = &alinx_char;
107.     return 0;
108. }
109.
110. /* read 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
111. static ssize_t char_drv_read(struct file *file_p, char __user *buf, size_t len, loff_t *loff_t_p)
112. {
113.     unsigned int keysts = 0;
114.     int ret;
115.
116.     struct alinx_char_dev *dev = (struct alinx_char_dev *)file_p->private_data;
117.
118.     /* 读取 key 的状态 */
119.     keysts = atomic_read(&dev->key_sts);
120.     /* 判断文件打开方式 */
121.     if(file_p->f_flags & O_NONBLOCK)
122.     {
123.         /* 如果是非阻塞访问, 说明已满足读取条件 */
124.     }
125.     /* 判断当前按键状态 */
126.     else if(!keysts)
127.     {
128.         /* 按键未被按下(数据未准备好) */
129.         /* 以当前进程创建并初始化为队列项 */
130.         DECLARE_WAITQUEUE(queue_mem, current);
131.         /* 把当前进程的队列项添加到队列头 */
132.         add_wait_queue(&dev->wait_q_h, &queue_mem);
133.         /* 设置当前进程为可被信号打断的状态 */
134.         __set_current_state(TASK_INTERRUPTIBLE);
135.         /* 切换进程, 是当前进程休眠 */
136.         schedule();
137.     }
```

```
137.
138.     /* 被唤醒, 修改当前进程状态为 RUNNING */
139.     set_current_state(TASK_RUNNING);
140.     /* 把当前进程的队列项从队列头中删除 */
141.     remove_wait_queue(&dev->wait_q_h, &queue_mem);
142.
143.     /* 判断是否是被信号唤醒 */
144.     if(signal_pending(current))
145.     {
146.         /* 如果是直接返回错误 */
147.         return -ERESTARTSYS;
148.     }
149.     else
150.     {
151.         /* 被按键唤醒 */
152.     }
153. }
154. else
155. {
156.     /* 按键被按下(数据准备好了) */
157. }
158.
159. /* 读取 key 的状态 */
160. keysts = atomic_read(&dev->key_sts);
161. /* 返回按键状态值 */
162. ret = copy_to_user(buf, &keysts, sizeof(keysts));
163. /* 清除按键状态 */
164. atomic_set(&dev->key_sts, 0);
165. return 0;
166. }
167.
168. /* poll 函数实现 */
169. unsigned int char_drv_poll(struct file *file_p, struct poll_table_struct *wait)
170. {
171.     unsigned int ret = 0;
172.
173.     struct alinx_char_dev *dev = (struct alinx_char_dev *)file_p->private_data;
174.
175.     /* 将应用程序添加到等待队列中 */
176.     poll_wait(file_p, &dev->wait_q_h, wait);
177.
178.     /* 判断 key 的状态 */
179.     if(atomic_read(&dev->key_sts))
180.     {
181.         /* key 准备好了, 返回数据可读 */
182.         ret = POLLIN;
183.     }
184.     else
185.     {
186.     }
187. }
188.
189. return ret;
190. }
191.
192. /* fasync 函数实现 */
193. static int char_drv_fasync(int fd, struct file *file_p, int mode)
194. {
195.     struct alinx_char_dev *dev = (struct alinx_char_dev *)file_p->private_data;
196.     return fasync_helper(fd, file_p, mode, &dev->fasync);
197. }
198.
199. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
200. static int char_drv_release(struct inode *inode_p, struct file *file_p)
201. {
202.     printk("gpio_test module release\n");
203.     return char_drv_fasync(-1, file_p, 0);
204. }
205.
206. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
207. static struct file_operations ax_char_fops = {
208.     .owner    = THIS_MODULE,
209.     .open     = char_drv_open,
210.     .read     = char_drv_read,
211.     .poll     = char_drv_poll,
212.     .fasync   = char_drv_fasync,
213.     .release  = char_drv_release,
```

```
214. };
215.
216. /* 模块加载时会调用的函数 */
217. static int __init char_drv_init(void)
218. {
219.     /* 用于接受返回值 */
220.     u32 ret = 0;
221.
222.     /* 初始化原子变量 */
223.     atomic_set(&alinx_char.key_sts, 0);
224.
225.     /* 获取设备节点 */
226.     alinx_char.nd = of_find_node_by_path("/alinxkey");
227.     if(alinx_char.nd == NULL)
228.     {
229.         printk("alinx_char node not find\r\n");
230.         return -EINVAL;
231.     }
232.     else
233.     {
234.         printk("alinx_char node find\r\n");
235.     }
236.
237.     /* 获取节点中 gpio 标号 */
238.     alinx_char.alinx_key_gpio = of_get_named_gpio(alinx_char.nd, "alinxkey-gpios", 0);
239.     if(alinx_char.alinx_key_gpio < 0)
240.     {
241.         printk("can not get alinxkey-gpios");
242.         return -EINVAL;
243.     }
244.     printk("alinxkey-gpio num = %d\r\n", alinx_char.alinx_key_gpio);
245.
246.     /* 申请 gpio 标号对应的引脚 */
247.     ret = gpio_request(alinx_char.alinx_key_gpio, "alinxkey");
248.     if(ret != 0)
249.     {
250.         printk("can not request gpio\r\n");
251.         return -EINVAL;
252.     }
253.
254.     /* 把这个 io 设置为输入 */
255.     ret = gpio_direction_input(alinx_char.alinx_key_gpio);
256.     if(ret < 0)
257.     {
258.         printk("can not set gpio\r\n");
259.         return -EINVAL;
260.     }
261.
262.     /* 获取中断号 */
263.     alinx_char.irq = gpio_to_irq(alinx_char.alinx_key_gpio);
264.     /* 申请中断 */
265.     ret = request_irq(alinx_char.irq,
266.                       key_handler,
267.                       IRQF_TRIGGER_FALLING | IRQF_TRIGGER_RISING,
268.                       "alinxkey",
269.                       &alinx_char);
270.     if(ret < 0)
271.     {
272.         printk("irq %d request failed\r\n", alinx_char.irq);
273.         return -EFAULT;
274.     }
275.
276.     __init_timer(&alinx_char.timer, timer_function, 0);
277.
278.
279.     init_waitqueue_head(&alinx_char.wait_q_h);
280.
281.     /* 注册设备号 */
282.     alloc_chrdev_region(&alinx_char.devid, MINOR_U, DEVID_COUNT, DEVICE_NAME);
283.
284.     /* 初始化字符设备结构体 */
285.     cdev_init(&alinx_char.cdev, &ax_char_fops);
286.
287.     /* 注册字符设备 */
288.     cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
289.
290.     /* 创建类 */
291.     alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
```

```
292.     if(IS_ERR(alinx_char.class))
293.     {
294.         return PTR_ERR(alinx_char.class);
295.     }
296.
297.     /* 创建设备节点 */
298.     alinx_char.device = device_create(alinx_char.class, NULL,
299.                                     alinx_char.devid, NULL,
300.                                     DEVICE_NAME);
301.     if (IS_ERR(alinx_char.device))
302.     {
303.         return PTR_ERR(alinx_char.device);
304.     }
305.
306.     return 0;
307. }
308.
309. /* 卸载模块 */
310. static void __exit char_drv_exit(void)
311. {
312.     /* 释放 gpio */
313.     gpio_free(alinx_char.alinx_key_gpio);
314.
315.     /* 释放中断 */
316.     free_irq(alinx_char.irq, NULL);
317.
318.     /* 删除定时器 */
319.     del_timer_sync(&alinx_char.timer);
320.
321.     /* 注销字符设备 */
322.     cdev_del(&alinx_char.cdev);
323.
324.     /* 注销设备号 */
325.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
326.
327.     /* 删除设备节点 */
328.     device_destroy(alinx_char.class, alinx_char.devid);
329.
330.     /* 删除类 */
331.     class_destroy(alinx_char.class);
332.
333.     printk("timer_led_dev_exit_ok\n");
334. }
335.
336. /* 标记加载、卸载函数 */
337. module_init(char_drv_init);
338. module_exit(char_drv_exit);
339.
340. /* 驱动描述信息 */
341. MODULE_AUTHOR("Alinx");
342. MODULE_ALIAS("alinx char");
343. MODULE_DESCRIPTION("FASync LED driver");
344. MODULE_VERSION("v1.0");
345. MODULE_LICENSE("GPL");
```

21 行添加头文件 linux/fcntl.h。

49 行添加 fasync_struct 结构体到设备结构体中。

82 行在定时器的处理函数中，确认按键按下后，先判断 fasync_struct 结构体有没有初始化过，初始化过说明调用过 fasync 函数，也就是应用程序是能了异步通知，所以就发送对应的信号。

204~208 行实现 fasync 函数，里面就是简单的调用了 fasync_helper 来初始化 fasync_struct 结构体。

204 行在 release 函数中调用 fasync 函数，释放 fasync_struct 结构体。

213 行把我们实现的 fasync 函数添加到 file_operations 结构体中。

另外，相比较上一章，这章里面使用了私有数据来代替设备结构体变量的全局变量，具体的用法说明，可以回顾一下第一章的实验代码里的解释。

注意 270 行中断的服务函数输入参数设置和 64 行定时器的服务函数输入参数设置。

11.2.4 测试代码

新建 QT 工程名为“ax_fasync_test”，新建 main.c，输入下列代码：

```
1. #include "stdio.h"
2. #include "unistd.h"
3. #include "sys/types.h"
4. #include "sys/stat.h"
5. #include "fcntl.h"
6. #include "stdlib.h"
7. #include "string.h"
8. #include "poll.h"
9. #include "sys/select.h"
10. #include "sys/time.h"
11. #include "linux/ioctl.h"
12. #include "signal.h"
13.
14. static int fd = 0, fd_l = 0;
15.
16. static void sigio_signal_func()
17. {
18.     int ret = 0;
19.     static char led_value = 0;
20.     unsigned int key_value;
21.
22.     /* 获取按键状态 */
23.     ret = read(fd, &key_value, sizeof(key_value));
24.     if(ret < 0)
25.     {
26.         printf("read failed\r\n");
27.     }
28.
29.     /* 判断按键状态 */
30.     if(1 == key_value)
31.     {
32.         /* 按键被按下，改变 led 状态 */
33.         printf("ps_key1 press\r\n");
34.         led_value = !led_value;
35.
36.         fd_l = open("/dev/gpio_leds", O_RDWR);
37.         if(fd_l < 0)
38.         {
39.             printf("file /dev/gpio_leds open failed\r\n");
40.         }
41.
42.         ret = write(fd_l, &led_value, sizeof(led_value));
43.         if(ret < 0)
44.         {
45.             printf("write failed\r\n");
46.         }
47.
48.         ret = close(fd_l);
49.         if(ret < 0)
50.         {
51.             printf("file /dev/gpio_leds close failed\r\n");
52.         }
53.     }
54. }
55.
56. int main(int argc, char *argv[])
57. {
58.     int flags = 0;
59.     char *filename;
60.
61.     if(argc != 2)
62.     {
63.         printf("wrong para\r\n");
64.         return -1;
65.     }
66.
67.     filename = argv[1];
68.     fd = open(filename, O_RDWR);
69.     if(fd < 0)
70.     {
71.         printf("can not open file %s\r\n", filename);
```

```

72.         return -1;
73.     }
74.
75.     /* 指定信号 SIGIO, 并绑定处理函数 */
76.     signal(SIGIO, sigio_signal_func);
77.     /* 把当前线程指定为将接收信号的进程 */
78.     fcntl(fd, F_SETOWN, getpid());
79.     /* 获取当前线程状态 */
80.     flags = fcntl(fd, F_GETFD);
81.     /* 设置当前线程为 FASYNC 状态 */
82.     fcntl(fd, F_SETFL, flags | FASYNC);
83.
84.     while(1)
85.     {
86.         sleep(2);
87.     }
88.
89.     close(fd);
90.     return 0;
91. }

```

注意 19 行给 led_value 加上 static。

75~82 行就是按照 11.1.2 节中说的步骤操作即可，不重复解释了。

11.2.5 运行测试

测试方式和现象和上一章一样，步骤如下：

```

mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ./ax-concled-drv.ko
insmod ./ax-fasync-drv.ko
cd ./build-ax_fasync_test-ZYNQ-Debug
./ax_fasync_test /dev/fasync_led&
top

```

结果如下图：

```

root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
random: fast init done
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ./ax-concled-drv.ko
ax_concled_dev: loading out-of-tree module taints kernel.
alinx_char node find
alinxled-gpio num = 899
root@ax_peta:/mnt# insmod ./ax-fasync-drv.ko
alinx_char node find
alinxkey-gpio num = 949
root@ax_peta:/mnt# cd ./build-ax_fasync_test-ZYNQ-Debug
root@ax_peta:/mnt/build-ax_fasync_test-ZYNQ-Debug# ./ax_fasync_test /dev/fasync_led&
[1] 1266
root@ax_peta:/mnt/build-ax_fasync_test-ZYNQ-Debug# gpio_test module open
ps_key1 press
gpio_test module open
gpio_test module release
ps_key1 press
gpio_test module open
gpio_test module release

```



```

Mem: 57072K used, 973180K free, 13408K shrd, 1196K buff, 43064K cached
CPU:  0.0% usr  0.0% sys  0.0% nic 100% idle  0.0% io  0.0% irq  0.0% sirq
Load average: 0.00 0.00 0.00 1/87 1267

```

PID	PPID	USER	STAT	VSZ	%VSZ	CPU	%CPU	COMMAND
1242	1	root	S	14352	1.3	1	0.0	/usr/sbin/tcf-agent -d -L- -10
1266	1249	root	S	13164	1.2	1	0.0	./ax_fasync_test /dev/fasync_led
1249	1247	root	S	2988	0.2	1	0.0	-sh
756	1	root	S	2972	0.2	1	0.0	/sbin/udevd -d
1169	1	root	S	2916	0.2	0	0.0	/usr/sbin/inetd
1228	1	root	S	2788	0.2	0	0.0	/sbin/syslogd -n -O /var/log/messages
1231	1	root	S	2788	0.2	1	0.0	/sbin/klogd -n
1267	1249	root	R	2788	0.2	1	0.0	top
1213	1	root	S	2788	0.2	0	0.0	udhcpc -R -b -p /var/run/udhcpc.eth0.pid -i eth0
1248	1	root	S	2788	0.2	1	0.0	/sbin/getty 38400 tty1
1247	1	root	S	2756	0.2	0	0.0	{start_getty} /bin/sh /bin/start_getty 115200 ttyP
1220	1	root	S	2364	0.2	0	0.0	/usr/sbin/dropbear -r /etc/dropbear/dropbear_rsa_h
1	0	root	S	1700	0.1	1	0.0	init
1254	2	root	SW<	0	0.0	0	0.0	[kworker/0:1H]

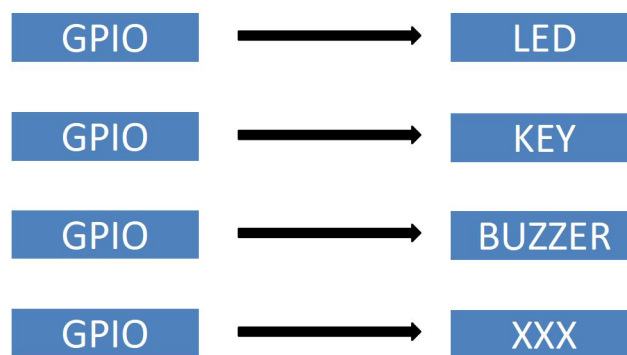
第十二章 platform 平台设备

前面一直都是使用字符设备框架来编写简单的 IO 读写驱动,但是遇到复杂的总线和设备时,仅有字符设备框架是没法应对的。比如,当 SPI 总线挂载了多个设备时,这些设备的驱动该如何实现, SPI 的驱动如何实现,他们之间又如何关联?强行去写也许可以实现功能,但最终的代码会很很环保,复用性很难保证,不符合 Linux 的设计思想。这时就需要用到驱动分离的思想。对此, Linux 内核提供了 platform 设备框架。

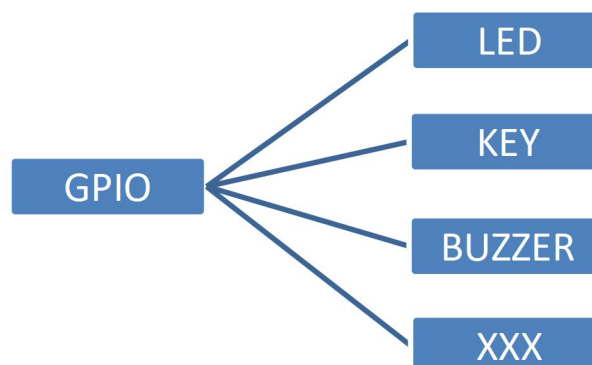
12.1 驱动分离

在说 platform 之前,先简单说说驱动分离的思想。

有没有想过这个问题,我们前面写的点亮 LED 实验中的驱动,到底是 IO 的驱动还是 LED 的驱动呢?暂且结合起来看作是 IO_LED 驱动,那么后来我们又用到按键,就有了 IO_KEY 驱动,如果现在要使用蜂鸣器了,就会出现 IO_BUZZER 驱动,每多一个设备,就会多一个 IO_XXX 驱动。

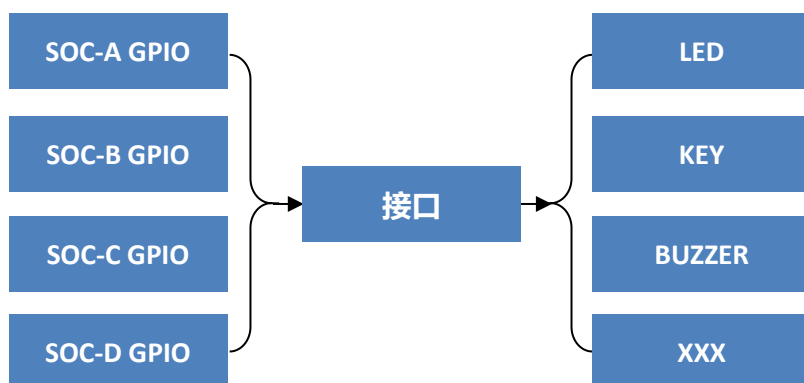


但是这些 IO_XXX 驱动中, IO 的部分几乎是一样的,为了保证代码的简介,把 IO 的部分单独拎出来,写成一个 IO 驱动,然后设备的驱动就只剩下设备自身的内容,只要去调用 IO 驱动中的接口即可。

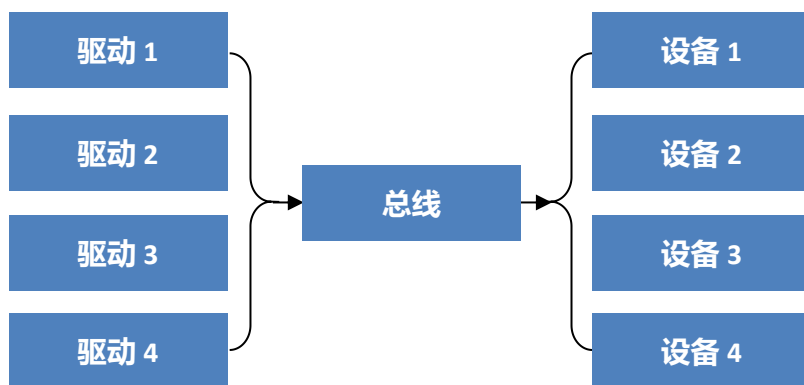


问题又来了, Linux 的一大卖点是易与移植,现在需要在其他的硬件平台上运行使用这些设备驱动又会变成什么样子呢。不同厂家 SOC 上的 GPIO 底层的实现都有所差别,那每出现一个

新的 SOC，就需要重写一个 GPIO 驱动。当然这是不可避免的，但是这些 GPIO 驱动中，还是有共同的部分，即 IO 和设备之间的接口，把这些 GPIO 中的接口部分再拎出来，单独写成一个驱动。



这样就形成了驱动的分隔，一边是 SOC 的硬件资源，另一边是用户设备，他们通过统一的接口来连接。驱动分离的思想带来了很多好处，Linux 长久的发展中，省去了很多冗余的代码，SOC 厂家根据统一的接口提供 SOC 片上外设的驱动，设备厂家也根据结构统一的接口提供设备的驱动，用户只需要选择好 SOC 和外设，就能很方便的关联到一起。这样的思想也延续到了单个 SOC 中，片上驱动(之后称为驱动)和设备驱动(之后称为设备)通过总线协议来关联。



当我们往内核中添加驱动时，总线就会查找对应的设备，添加设备时就去查找对应的驱动。这就是 Linux 中的 bus、driver、device 模型，platform 就是这种模型的实现。

12.2 platform 模型

SPI 等总线设备很容易去对应 bus、driver、device 模型，但是 SOC 上并不是所有资源都有总线的概念，比如 GPIO。为了对应 bus、driver、device 模型，platform 模型定义了 platform_bus 虚拟总线、platform_driver 和 platform_device 分别来对应模型总线、驱动和设备。有一点要说明的是，platform 平台设备模型虽然名字里有“平台设备”，但他并不是独立于 Linux 三种设备之外的新的设备类型，他强调的模型二字，只是一种框架，在使用中，是无法和字符设备、块设备、网络设备脱离关系的。

12.2.1 platform_bus

内核中使用 bus_type 结构体表示总线，定义在文件 include/linux/device.h 中：

```
1. struct bus_type {
2.     .....
3.     int (*match)(struct device *dev, struct device_driver *drv);
4.     .....
5. };
```

成员中的 match 函数就是驱动和设备匹配的关键，他的两个输入参数一个是 drv 一个是 dev 也就是驱动和设备，每个总线类型都必须实现这个函数。

platform 虚拟总线也是 bus_type 类型的变量，在文件 drivers/base/platform.c 中定义如下：

```
1. struct bus_type platform_bus_type = {
2.     .name = "platform",
3.     .dev_groups = platform_dev_groups,
4.     .match = platform_match,
5.     .uevent = platform_uevent,
6.     .pm = &platform_dev_pm_ops,
7. };
```

看一下其中 match 函数的实现 platform_match 函数，也在这个文件中：

```
1. static int platform_match(struct device *dev, struct device_driver *drv)
2. {
3.     struct platform_device *pdev = to_platform_device(dev);
4.     struct platform_driver *pdrv = to_platform_driver(drv);
5.
6.     /* When driver_override is set, only bind to the matching driver */
7.     if (pdev->driver_override)
8.         return !strcmp(pdev->driver_override, drv->name);
9.
10.    /* Attempt an OF style match first */
11.    if (of_driver_match_device(dev, drv))
12.        return 1;
13.
14.    /* Then try ACPI style match */
15.    if (acpi_driver_match_device(dev, drv))
16.        return 1;
17.
18.    /* Then try to match against the id table */
19.    if (pdrv->id_table)
20.        return platform_match_id(pdrv->id_table, pdev) != NULL;
21.
22.    /* fall-back to driver name match */
23.    return (strcmp(pdev->name, drv->name) == 0);
24. }
```

platform_match 函数提供了 4 种匹配方式。

11~12 行，设备树下会使用的 OF 匹配表匹配方式，match 的输入参数之一 drv 的数据类型中有一个成员变量 of_match_table，of_match_table 又有一个成员为 compatible，如果这个 compatible 能和设备树种的 compatible 属性相匹配，驱动代码中的 probe 就会被调用。

如果没有使用设备树，一般会使用 23 行的第四种匹配方式，直接比较驱动和设备中 name 成员。

platform 总线是不需要驱动层面去管理的，也就是说对于驱动开发开闭，了解其中原理即可。

12.2.2 platform_driver

1) 定义并初始化 platform_driver

platform 驱动用 platform_driver 结构体来表示，在头文件 include/linux/platform_device.h 中：

```
1. struct platform_driver {
2.     int (*probe)(struct platform_device *);
```

```

3.     int (*remove)(struct platform_device *);
4.     void (*shutdown)(struct platform_device *);
5.     int (*suspend)(struct platform_device *, pm_message_t state);
6.     int (*resume)(struct platform_device *);
7.     struct device_driver driver;
8.     const struct platform_device_id *id_table;
9.     bool prevent_deferred_probe;
10. };

```

a) 成员 probe

probe 函数前面提到过，当设备和驱动匹配成功时，就会执行这个函数。在这个函数中调用原先在驱动入口函数中调用的内容，如字符设备中调用的 cdev_init 函数。

b) 成员 remove

remove 函数在驱动或对应设备注销时会执行，与 probe 相对的，这个函数中调用原先在驱动出口函数中调用的内容，如字符设备中的 cdev_del 函数以及其他初始化内容。

c) 成员 driver

driver 是 struct device_driver 类型，device_driver 是基本的设备驱动类型，platform_driver 是在 device_driver 基础上扩展的，所以需要包含这个类型的元素，以使用他的成员。

device_driver 结构体中有个成员变量

```
const struct of_device_id *of_match_table;
```

of_match_table 也就是上面提到的 OF 匹配表匹配方式用到的成员，of_device_id 结构体中有名为 compatible 的成员，设备树中的 compatible 就是和这个成员来比较的。

of_device_id 结构体中还有名为 name 的成员，需要与 platform 设备中的 name 字段相同。

d) 成员 id_table

id_table 是用于匹配设备的，是上面介绍的 platform_match 函数提供的第三种匹配方式需要用到的成员。

2) platform_driver 注册和注销

定义好 struct platform_driver 后，需要在驱动入口函数中调用下面的函数来注册 platform 驱动，取代原先的初始化内容：

```
int platform_driver_register(struct platform_driver *driver);
```

注册成功返回 0，失败返回负。

在出口函数中做相应的注销操作取代原先的注销内容：

```
void platform_driver_unregister(struct platform_driver *drv);
```

12.2.3 platform_device

platform 设备用结构体 platform_device 来表示。在支持设备树的内核中，可以使用设备树代替 platform_device，但是 platform_device 仍然保留使用，我们先完整的了解一下 platform 的完整流程，之后再去结合设备树。

1) platform_device 结构体

platform_device 结构体定义在 include/linux/platform_device.h 中，内容如下：

```

1. struct platform_device {
2.     const char *name;
3.     int id;
4.     bool id_auto;
5.     struct device dev;
6.     u32 num_resources;
7.     struct resource *resource;
8.
9.     const struct platform_device_id *id_entry;

```

```
10.     char *driver_override; /* Driver name to force a match */
11.
12.     /* MFD cell pointer */
13.     struct mfd_cell *mfd_cell;
14.
15.     /* arch specific additions */
16.     struct pdev_archdata archdata;
17. };
```

成员 name 用于和驱动匹配，需要和 platform_driver 中的 name 相同。

成员 id 表示当前设备在这类设备中的编号，只有一个这种类型的设备是 id 赋值-1。

成员 num_resources 表示资源数量。

成员 resource 是资源的数组，struct resource 定义如下：

```
1. struct resource {
2.     resource_size_t start;
3.     resource_size_t end;
4.     const char *name;
5.     unsigned long flags;
6.     struct resource *parent, *sibling, *child;
7. };
```

start 表示资源的起始地址，end 表示资源的结束地址。name 是资源名称，flags 表示资源类型，资源类型的宏定义在头文件 include/linux/ioport.h 的 29~105 行。

2) platform_device 注册和注销

在声明初始化 platform_device 结构体后，使用下面的方法注册 platform_device：

```
int platform_device_register(struct platform_device *pdev);
```

注销设备时，使用下面的方法注销：

```
void platform_device_unregister(struct platform_device *pdev);
```

3) 获取资源的方法

当 platform 设备设置好资源后，platform 驱动就可以通过下面的函数来获取资源信息：

```
struct resource *platform_get_resource(struct platform_device *dev, unsigned int
type, unsigned int num)
```

参数说明：

dev：目标 platform 设备。

type：也就是上面说的 platform_device 结构体成员 resource 结构体的 flags 成员。

num：指定 type 的资源的下标。

返回值：资源的信息，成功时返回 resource 结构体类型指针，失败时返回 NULL。

12.3 实验

这一章我们用 platform 架构来实现简单的点亮 led 实验。

12.3.1 原理图

和第一章 1.3.1 节的内容相同。

12.3.2 设备树

这一章的实验使用 platform_device 来表示设备，不用设备树。

12.3.3 驱动程序

驱动程序分为驱动和设备两个部分。

- 1) 先完成驱动的代码，使用 petalinux 新建名为“ax-platform-driv”的驱动程序，并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-platform-driv.c 文件中输入下面的代码：

```
1. #include <linux/types.h>
2. #include <linux/kernel.h>
3. #include <linux/delay.h>
4. #include <linux/init.h>
5. #include <linux/module.h>
6. #include <linux/errno.h>
7. #include <linux/gpio.h>
8. #include <linux/cdev.h>
9. #include <linux/device.h>
10. #include <linux/of_gpio.h>
11. #include <linux/semaphore.h>
12. #include <linux/timer.h>
13. #include <linux/irq.h>
14. #include <linux/wait.h>
15. #include <linux/poll.h>
16. #include <linux/fs.h>
17. #include <linux/fcntl.h>
18. #include <linux/platform_device.h>
19. #include <asm/mach/map.h>
20. #include <asm/uaccess.h>
21. #include <asm/io.h>
22.
23. /* 设备节点名称 */
24. #define DEVICE_NAME      "gpio_leds"
25. /* 设备号个数 */
26. #define DEVID_COUNT      1
27. /* 驱动个数 */
28. #define DRIVE_COUNT      1
29. /* 主设备号 */
30. #define MAJOR
31. /* 次设备号 */
32. #define MINOR            0
33.
34. /* gpio 寄存器虚拟地址 */
35. static u32 *GPIO_DIRM_0;
36. /* gpio 使能寄存器 */
37. static u32 *GPIO_OEN_0;
38. /* gpio 控制寄存器 */
39. static u32 *GPIO_DATA_0;
40. /* AMBA 外设时钟使能寄存器 */
41. static u32 *APER_CLK_CTRL;
42.
43. /* 把驱动代码中会用到的数据打包进设备结构体 */
44. struct alinx_char_dev{
45.     dev_t      devid;      //设备号
46.     struct cdev cdev;      //字符设备
47.     struct class *class;   //类
48.     struct device *device; //设备
49. };
50. /* 声明设备结构体 */
51. static struct alinx_char_dev alinx_char = {
52.     .cdev = {
53.         .owner = THIS_MODULE,
54.     },
55. };
56.
57. /* open 函数实现，对应到 Linux 系统调用函数的 open 函数 */
58. static int gpio_leds_open(struct inode *inode_p, struct file *file_p)
59. {
60.     /* 设置私有数据 */
61.     file_p->private_data = &alinx_char;
62.
63.     return 0;
64. }
65.
66. /* write 函数实现，对应到 Linux 系统调用函数的 write 函数 */
67. static ssize_t gpio_leds_write(struct file *file_p, const char __user *buf, size_t len, loff_t *loff_t
    _p)
68. {
69.     int rst;
70.     char writeBuf[5] = {0};
71.
72.     rst = copy_from_user(writeBuf, buf, len);
73.     if(0 != rst)
```



```
74.     {
75.         return -1;
76.     }
77.
78.     if(1 != len)
79.     {
80.         printk("gpio_test len err\n");
81.         return -2;
82.     }
83.     if(1 == writeBuf[0])
84.     {
85.         *GPIO_DATA_0 &= 0xFFFFFFFF;
86.     }
87.     else if(0 == writeBuf[0])
88.     {
89.         *GPIO_DATA_0 |= 0x00000001;
90.     }
91.     else
92.     {
93.         printk("gpio_test para err\n");
94.         return -3;
95.     }
96.
97.     return 0;
98. }
99.
100. /* release 函数实现, 对应该 Linux 系统调用函数的 close 函数 */
101. static int gpio_leds_release(struct inode *inode_p, struct file *file_p)
102. {
103.     return 0;
104. }
105.
106. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
107. static struct file_operations ax_char_fops = {
108.     .owner    = THIS_MODULE,
109.     .open     = gpio_leds_open,
110.     .write    = gpio_leds_write,
111.     .release  = gpio_leds_release,
112. };
113.
114. /* probe 函数实现, 驱动和设备匹配时会被调用 */
115. static int gpio_leds_probe(struct platform_device *dev)
116. {
117.     /* 资源大小 */
118.     int regsize[4];
119.     /* 资源信息 */
120.     struct resource *led_source[4];
121.
122.     int i;
123.     for(i = 0; i < 4; i++)
124.     {
125.         /* 获取 dev 中的 IORESOURCE_MEM 资源 */
126.         led_source[i] = platform_get_resource(dev, IORESOURCE_MEM, i);
127.         /* 返回 NULL 获取资源失败 */
128.         if(!led_source[i])
129.         {
130.             dev_err(&dev->dev, "get resource failed\n");
131.             return -ENXIO;
132.         }
133.         /* 获取当前资源大小 */
134.         regsize[i] = resource_size(led_source[i]);
135.     }
136.
137.     /* 把需要修改的物理地址映射到虚拟地址 */
138.     GPIO_DIRM_0 = ioremap(led_source[0]->start, regsize[0]);
139.     GPIO_OEN_0  = ioremap(led_source[1]->start, regsize[1]);
140.     GPIO_DATA_0 = ioremap(led_source[2]->start, regsize[2]);
141.     APER_CLK_CTRL = ioremap(led_source[3]->start, regsize[3]);
142.
143.     /* MIO_0 时钟使能 */
144.     *APER_CLK_CTRL |= 0x00400000;
145.     /* MIO_0 设置成输出 */
146.     *GPIO_DIRM_0 |= 0x00000001;
147.     /* MIO_0 使能 */
148.     *GPIO_OEN_0 |= 0x00000001;
149.
150.     /* 注册设备号 */
151.     alloc_chrdev_region(&alinx_char.devid, MINOR, DEVID_COUNT, DEVICE_NAME);
152.
153.     /* 初始化字符设备结构体 */
154.     cdev_init(&alinx_char.cdev, &ax_char_fops);
155.
156.     /* 注册字符设备 */
```



```
157.     cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
158.
159.     /* 创建类 */
160.     alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
161.     if(IS_ERR(alinx_char.class))
162.     {
163.         return PTR_ERR(alinx_char.class);
164.     }
165.
166.     /* 创建设备节点 */
167.     alinx_char.device = device_create(alinx_char.class, NULL,
168.                                     alinx_char.devid, NULL,
169.                                     DEVICE_NAME);
170.     if (IS_ERR(alinx_char.device))
171.     {
172.         return PTR_ERR(alinx_char.device);
173.     }
174.
175.     return 0;
176. }
177.
178. static int gpio_leds_remove(struct platform_device *dev)
179. {
180.     /* 注销字符设备 */
181.     cdev_del(&alinx_char.cdev);
182.
183.     /* 注销设备号 */
184.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
185.
186.     /* 删除设备节点 */
187.     device_destroy(alinx_char.class, alinx_char.devid);
188.
189.     /* 删除类 */
190.     class_destroy(alinx_char.class);
191.
192.     /* 释放对虚拟地址的占用 */
193.     iounmap(GPIO_DIRM_0);
194.     iounmap(GPIO_OEN_0);
195.     iounmap(GPIO_DATA_0);
196.     return 0;
197. }
198.
199. /* 声明并初始化 platform 驱动 */
200. static struct platform_driver led_driver = {
201.     .driver = {
202.         /* 将会用 name 字段和设备匹配, 这里 name 命名为 alinx-led */
203.         .name = "alinx-led",
204.     },
205.     .probe = gpio_leds_probe,
206.     .remove = gpio_leds_remove,
207. };
208.
209. /* 驱动入口函数 */
210. static int __init gpio_led_drv_init(void)
211. {
212.     /* 在入口函数中调用 platform_driver_register, 注册 platform 驱动 */
213.     return platform_driver_register(&led_driver);
214. }
215.
216. /* 驱动出口函数 */
217. static void __exit gpio_led_dev_exit(void)
218. {
219.     /* 在出口函数中调用 platform_driver_unregister, 卸载 platform 驱动 */
220.     platform_driver_unregister(&led_driver);
221. }
222.
223. /* 标记加载、卸载函数 */
224. module_init(gpio_led_drv_init);
225. module_exit(gpio_led_dev_exit);
226.
227. /* 驱动描述信息 */
228. MODULE_AUTHOR("Alinx");
229. MODULE_ALIAS("gpio_led");
230. MODULE_DESCRIPTION("PLATFORM LED driver");
231. MODULE_VERSION("v1.0");
232. MODULE_LICENSE("GPL");
```

可以和第二章设备树下的驱动代码做比较, 字符设备的部分几乎是一样的。open 函数、write 函数、release 函数都是熟悉的字符设备驱动写法。

116 行实现 probe 函数，把第二章驱动代码驱动入口函数中的内容复制过来，修改资源的获取方式。第二张是从设备树获取资源信息的，修改成从 platform 设备中获取。使用 platform_get_resource 函数，可以通过 resource_size 函数获取资源大小。

179 行实现 remove 函数，把第二章驱动代码驱动出口函数中的内容复制过来即可，甚至不需要修改。

201 行定义 platform_driver 并初始化。

204 行的 name 命名为 alinx-led，之后在实现 platform_device 时，要保持一致。

214 行在驱动入口函数中注册 platform_driver。

221 行在出口函数中注销 platform_driver。

2) 再完成设备的部分，使用 petalinux 新建名为“ax-platform-dev”的驱动程序，并执行 petalinux-config -c rootfs 命令选上新增的程序。

在 ax-platform-dev.c 文件中输入下面的代码：

```
1. #include <linux/init.h>
2. #include <linux/module.h>
3. #include <linux/errno.h>
4. #include <linux/gpio.h>
5. #include <linux/cdev.h>
6. #include <linux/device.h>
7. #include <linux/of_gpio.h>
8. #include <linux/semaphore.h>
9. #include <linux/timer.h>
10. #include <linux/irq.h>
11. #include <linux/wait.h>
12. #include <linux/poll.h>
13. #include <linux/fs.h>
14. #include <linux/fcntl.h>
15. #include <linux/platform_device.h>
16. #include <asm/mach/map.h>
17. #include <asm/uaccess.h>
18. #include <asm/io.h>
19.
20. /* 寄存器首地址 */
21. /* gpio 方向寄存器 */
22. #define GPIO_DIRM_0      0xE000A204
23. /* gpio 使能寄存器 */
24. #define GPIO_OEN_0      0xE000A208
25. /* gpio 控制寄存器 */
26. #define GPIO_DATA_0      0xE000A040
27. /* AMBA 外设时钟使能寄存器 */
28. #define APER_CLK_CTRL    0xF800012C
29. /* 寄存器大小 */
30. #define REGISTER_LENGTH  4
31.
32. /* 删除设备时会执行此函数 */
33. static void led_release(struct device *dev)
34. {
35.     printk("led device released\n");
36. }
37.
38. /* 初始化 LED 的设备信息，即寄存器信息 */
39. static struct resource led_resources[] =
40. {
41.     {
42.         .start = GPIO_DIRM_0,
43.         .end   = GPIO_DIRM_0 + REGISTER_LENGTH - 1,
44.         /* 寄存器当作内存处理 */
45.         .flags = IORESOURCE_MEM,
46.     },
47.     {
48.         .start = GPIO_OEN_0,
49.         .end   = GPIO_OEN_0 + REGISTER_LENGTH - 1,
50.         .flags = IORESOURCE_MEM,
51.     },
52.     {
53.         .start = GPIO_DATA_0,
54.         .end   = GPIO_DATA_0 + REGISTER_LENGTH - 1,
55.         .flags = IORESOURCE_MEM,
56.     },
57. }
```

```

57.     {
58.         .start = APER_CLK_CTRL,
59.         .end   = APER_CLK_CTRL + REGISTER_LENGTH - 1,
60.         .flags = IORESOURCE_MEM,
61.     },
62. };
63.
64. /* 声明并初始化 platform_device */
65. static struct platform_device led_device =
66. {
67.     /* 名字和 driver 中的 name 一致 */
68.     .name = "alinx-led",
69.     /* 只有一个设备 */
70.     .id = -1,
71.     .dev = {
72.         /* 设置 release 函数 */
73.         .release = &led_release,
74.     },
75.     /* 设置资源个数 */
76.     .num_resources = ARRAY_SIZE(led_resources),
77.     /* 设置资源信息 */
78.     .resource = led_resources,
79. };
80.
81. /* 入口函数 */
82. static int __init led_device_init(void)
83. {
84.     /* 在入口函数中调用 platform_driver_register, 注册 platform 驱动 */
85.     return platform_device_register(&led_device);
86. }
87.
88. /* 出口函数 */
89. static void __exit led_device_exit(void)
90. {
91.     /* 在出口函数中调用 platform_driver_unregister, 卸载 platform 驱动 */
92.     platform_device_unregister(&led_device);
93. }
94.
95. /* 标记加载、卸载函数 */
96. module_init(led_device_init);
97. module_exit(led_device_exit);
98.
99. /* 驱动描述信息 */
100. MODULE_AUTHOR("Alinx");
101. MODULE_ALIAS("gpio_led");
102. MODULE_DESCRIPTION("PLATFORM LED device");
103. MODULE_VERSION("v1.0");
104. MODULE_LICENSE("GPL");

```

platform_device 也是使用驱动入口出口的方式，当时候加载设备的方式也和驱动一样使用 insmod 命令。

platform_device 两个关键的地方有两点。

一是 39 行开始的 struct resource 结构体类型的数组，这里面是我们需要的设备信息，每个元素都需要初始化的三个成员变量时 start、end、flags，当我们在 drv 中调用 resource_size 函数时，会根据 start 和 end 返回资源大小，flags 是 platform_get_resource 函数获取资源信息的依据之一。

二是 65 行的 platform_device 结构体的实现，关键就是 name 字段要和 drv 中的一致。num_resource 成员的值可以通过宏 ARRAY_SIZE 来获取资源的个数。

33 行实现 release 函数，删除设备时会执行这个函数。

85 行在入口函数中注册 platform_device 设备。

92 行在出口函数中注销 platform_device 设备。

12.3.4 测试程序

测试 APP 和第一章 1.3.4 节内容一致，可以使用第一章的测试程序。

12.3.5 运行测试

测试步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ax-platform-dev.ko
insmod ax-platform-driv.ko
cd ./build-axleddev_test-ZYNQ-Debug/
./axleddev_test /dev/gpio_leds on
```

IP 和路径根据实际情况调整。

现象如下，板子上的 ps_led1 会被点亮火熄灭：

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ax-platform-dev.ko
root@ax_peta:/mnt# insmod ax-platform-driv.ko
[drm] load() is deferred & will be called again
root@ax_peta:/mnt# cd ./build-axleddev_test-ZYNQ-Debug/
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# ./axleddev_test /dev/gpio_leds on
ps_led1 on
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug#
```

如果我们把设备删除，设备文件也就不见了：

```
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# rmmod ax_platform_dev
led device released
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# ./axleddev_test /dev/gpio_leds on
Can't open file /dev/gpio_leds
```

再重新加载，又存在了，说明 probe 函数就是在 dev 和 drv 相匹配的时候执行的

```
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# insmod ./ax-platform-dev.ko
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# [drm] load() is deferred & will be called again

root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# ./axleddev_test /dev/gpio_leds on
ps_led1 on
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug#
```

第十三章 platform 与设备树

13.1 设备树下的 platform

在设备树引入之后，platform 结构中的 platform_device 就可以用设备树去代替了。设备树下的 platform 驱动相较于原始的 platform 驱动，只需要把 platform_device 中描述的设备信息放到设备树中，同时修改 platform_driver 中对资源的读取方法即可。

1) 在设备树中描述设备信息

使用 platform_device 时，我们可以通过 name 字段或者 id_table 来匹配设备和驱动，当 platform_device 变成设备树时，根据上一章在 platform_bus 中的介绍，则使用 of_match_table 方法来匹配。of_match_table 对于设备树来说，要做的事，就是保证设备节点的 compatible 属性和 platform_driver 中的 compatible 保持一致。

```
1. alinxled {
2.     compatible = "alinx-led";
3.     pinctrl-names = "default";
4.     pinctrl-0 = <&pinctrl_led_default>;
5.     alinxled-gpios = <&gpio0 0 0>;
6. };
```

除了这点要注意之外，设备树的写法就没有什么特别的了。

2) of_match_table

platform_driver 中的 compatible 属性设置在上一章中已经介绍过了，compatible 位于 platform_driver->device_driver->of_device_id->compatible，保证和设备树中的 compatible 字段一致即可。of_device_id 结构体在 platform_driver 结构体中的成员名为 of_match_table，of 匹配表。初始化示例如下：

```
1. static const struct of_device_id led_of_match[] = {
2.     /* compatible 字段和设备树中保持一致 */
3.     { .compatible = "alinx-led" },
4.     { /* Sentinel */ }
5. };
```

需要注意 of_device_id 的最后一个成员必须为空。

还有一点，即使我们不是用 name 字段做匹配工作 device_driver 结构体中的 name 字段还是需要保留，否则在加载驱动时可能会出错。

3) 修改 platform_driver 读取设备信息的方法

既然 platform_device 换成了设备树，那 platform_driver 读取设备信息的方式肯定得换成 of 函数了。回顾一下第四章中的 of 函数内容，就可以动手来实现设备树下的 platform 驱动代码了。

13.2 实验

这一章的实验在上一章的基础上，用设备树代替 platform_device 的情况下，使用 platform 架构来实现简单的点亮 led 实验。

13.2.1 原理图

和第一章 1.3.1 节的内容相同。

13.2.2 设备树

这里我们使用第四章，pinctrl 子系统和 gpio 子系统下的设备树，写法都是一样的，直接用就行了。只要注意设备节点中的 compatible 属性，要和 platform_driver 中的 compatible 保持一致。

```

1. ....
2. amba {
3.     ....
4.
5.     slcr@f8000000 {
6.         pinctrl@700 {
7.             pinctrl_led_default: led-default {
8.                 mux {
9.                     groups = "gpio0_0_grp";
10.                    function = "gpio0";
11.                };
12.
13.                conf {
14.                    pins = "MIO0";
15.                    io-standard = <1>;
16.                    bias-disable;
17.                    slew-rate = <0>;
18.                };
19.            };
20.        };
21.    };
22. };
23.
24. alinxled {
25.     compatible = "alinx-led";
26.     pinctrl-names = "default";
27.     pinctrl-0 = <&pinctrl_led_default>;
28.     alinxled-gpios = <&gpio0 0 0>;
29. };
30. ....

```

13.2.3 驱动程序

使用 petalinux 新建名为“ax-platformdt-drv”的驱动程序，并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-platformdt-drv.c 文件中输入下面的代码：

```

1. #include <linux/types.h>
2. #include <linux/kernel.h>
3. #include <linux/delay.h>
4. #include <linux/init.h>
5. #include <linux/module.h>
6. #include <linux/errno.h>
7. #include <linux/gpio.h>
8. #include <linux/cdev.h>
9. #include <linux/device.h>
10. #include <linux/of_gpio.h>
11. #include <linux/semaphore.h>
12. #include <linux/timer.h>
13. #include <linux/irq.h>
14. #include <linux/wait.h>

```

```
15. #include <linux/poll.h>
16. #include <linux/fs.h>
17. #include <linux/fcntl.h>
18. #include <linux/platform_device.h>
19. #include <asm/mach/map.h>
20. #include <asm/uaccess.h>
21. #include <asm/io.h>
22.
23. /* 设备节点名称 */
24. #define DEVICE_NAME      "gpio_leds"
25. /* 设备号个数 */
26. #define DEVID_COUNT      1
27. /* 驱动个数 */
28. #define DRIVE_COUNT      1
29. /* 主设备号 */
30. #define MAJOR_AX
31. /* 次设备号 */
32. #define MINOR_AX         0
33. /* LED 点亮时输入的值 */
34. #define ALINX_LED_ON     1
35. /* LED 熄灭时输入的值 */
36. #define ALINX_LED_OFF    0
37.
38. /* 把驱动代码中会用到的数据打包进设备结构体 */
39. struct alinx_char_dev{
40.     dev_t          devid;        //设备号
41.     struct cdev      cdev;        //字符设备
42.     struct class      *class;      //类
43.     struct device     *device;     //设备
44.     struct device_node *nd;        //设备树的设备节点
45.     int              ax_led_gpio; //gpio 号
46. };
47. /* 声明设备结构体 */
48. static struct alinx_char_dev alinx_char = {
49.     .cdev = {
50.         .owner = THIS_MODULE,
51.     },
52. };
53.
54. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
55. static int gpio_leds_open(struct inode *inode_p, struct file *file_p)
56. {
57.     /* 设置私有数据 */
58.     file_p->private_data = &alinx_char;
59.
60.     return 0;
61. }
62.
63. /* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
64. static ssize_t gpio_leds_write(struct file *file_p, const char __user *buf, size
_t len, loff_t *loff_t_p)
65. {
66.     int retvalue;
67.     unsigned char databuf[1];
68.     /* 获取私有数据 */
69.     struct alinx_char_dev *dev = file_p->private_data;
70.
71.     /* 获取用户数据 */
72.     retvalue = copy_from_user(databuf, buf, len);
73.     if(retvalue < 0)
74.     {
75.         printk("alinx led write failed\r\n");
76.         return -EFAULT;
77.     }
```



```
78.
79.     if(databuf[0] == ALINX_LED_ON)
80.     {
81.         /* gpio_set_value 方法设置 GPIO 的值, 使用!!对 0 或者 1 二值化 */
82.         gpio_set_value(dev->ax_led_gpio, !!0);
83.     }
84.     else if(databuf[0] == ALINX_LED_OFF)
85.     {
86.         gpio_set_value(dev->ax_led_gpio, !!1);
87.     }
88.     else
89.     {
90.         printk("gpio_test para err\n");
91.     }
92.
93.     return 0;
94. }
95.
96. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
97. static int gpio_leds_release(struct inode *inode_p, struct file *file_p)
98. {
99.     return 0;
100. }
101.
102. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关键 */
103. static struct file_operations ax_char_fops = {
104.     .owner    = THIS_MODULE,
105.     .open     = gpio_leds_open,
106.     .write    = gpio_leds_write,
107.     .release  = gpio_leds_release,
108. };
109.
110. /* probe 函数实现, 驱动和设备匹配时会被调用 */
111. static int gpio_leds_probe(struct platform_device *dev)
112. {
113.     /* 用于接受返回值 */
114.     u32 ret = 0;
115.
116.     /* 获取设备节点 */
117.     alinx_char.nd = of_find_node_by_path("/alinxled");
118.     if(alinx_char.nd == NULL)
119.     {
120.         printk("gpioled node not find\n");
121.         return -EINVAL;
122.     }
123.
124.     /* 获取节点中 gpio 标号 */
125.     alinx_char.ax_led_gpio = of_get_named_gpio(alinx_char.nd, "alinxled-gpios",
0);
126.     if(alinx_char.ax_led_gpio < 0)
127.     {
128.         printk("can not get alinxled-gpios\n");
129.         return -EINVAL;
130.     }
131.
132.     /* 申请 gpio 标号对应的引脚 */
133.     ret = gpio_request(alinx_char.ax_led_gpio, "alinxled");
134.     if(ret != 0)
135.     {
136.         printk("can not request gpio\n");
137.     }
138.     /* 把这个 io 设置为输出 */
139.     ret = gpio_direction_output(alinx_char.ax_led_gpio, 1);
140.     if(ret < 0)
```



```
141.     {
142.         printk("can not set gpio\r\n");
143.     }
144.
145.     /* 注册设备号 */
146.     alloc_chrdev_region(&alinx_char.devid, MINOR_AX, DEVID_COUNT, DEVICE_NAME);
147.
148.     /* 初始化字符设备结构体 */
149.     cdev_init(&alinx_char.cdev, &ax_char_fops);
150.
151.     /* 注册字符设备 */
152.     cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
153.
154.     /* 创建类 */
155.     alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
156.     if(IS_ERR(alinx_char.class))
157.     {
158.         return PTR_ERR(alinx_char.class);
159.     }
160.
161.     /* 创建设备节点 */
162.     alinx_char.device = device_create(alinx_char.class, NULL,
163.                                     alinx_char.devid, NULL,
164.                                     DEVICE_NAME);
165.     if (IS_ERR(alinx_char.device))
166.     {
167.         return PTR_ERR(alinx_char.device);
168.     }
169.
170.     return 0;
171. }
172.
173. static int gpio_leds_remove(struct platform_device *dev)
174. {
175.     /* 注销字符设备 */
176.     cdev_del(&alinx_char.cdev);
177.
178.     /* 注销设备号 */
179.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
180.
181.     /* 删除设备节点 */
182.     device_destroy(alinx_char.class, alinx_char.devid);
183.
184.     /* 删除类 */
185.     class_destroy(alinx_char.class);
186.     return 0;
187. }
188.
189. /* 初始化 of_match_table */
190. static const struct of_device_id led_of_match[] = {
191.     /* compatible 字段和设备树中保持一致 */
192.     { .compatible = "alinx-led" },
193.     /* Sentinel */
194. };
195.
196.
197. /* 声明并初始化 platform 驱动 */
198. static struct platform_driver led_driver = {
199.     .driver = {
200.         /* name 字段需要保留 */
201.         .name = "alinx-led",
202.         /* 用 of_match_table 代替 name 匹配 */
203.         .of_match_table = led_of_match,
```

```
204.     },
205.     .probe = gpio_leds_probe,
206.     .remove = gpio_leds_remove,
207. };
208.
209. /* 驱动入口函数 */
210. static int __init gpio_led_drv_init(void)
211. {
212.     /* 在入口函数中调用 platform_driver_register, 注册 platform 驱动 */
213.     return platform_driver_register(&led_driver);
214. }
215.
216. /* 驱动出口函数 */
217. static void __exit gpio_led_dev_exit(void)
218. {
219.     /* 在出口函数中调用 platform_driver_unregister, 卸载 platform 驱动 */
220.     platform_driver_unregister(&led_driver);
221. }
222.
223. /* 标记加载、卸载函数 */
224. module_init(gpio_led_drv_init);
225. module_exit(gpio_led_dev_exit);
226.
227. /* 驱动描述信息 */
228. MODULE_AUTHOR("Alinx");
229. MODULE_ALIAS("gpio_led");
230. MODULE_DESCRIPTION("PLATFORM DT LED driver");
231. MODULE_VERSION("v1.0");
232. MODULE_LICENSE("GPL");
```

把第四章的驱动代码和上一章的代码结合起来看，这章的代码其实就是这两张的驱动代码的组合。

platform 结构套用上一章的结构，设备树资源获取的方法套用第四章的方法。read、write、release 方法野河第四章相同。

为一要注意的就是 200 行的 driver 字段，name 要保留。of_match_table 中的 compatible 字段要和设备树中的一致。

13.2.4 测试程序

测试 APP 和第一章 1.3.4 节内容一致，可以使用第一章的测试程序。

13.2.5 运行测试

测试方法和上一章基本一致，省去了加载 device 的命令，步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ax-platformdt-drv.ko
cd ./build-axleddev_test-ZYNQ-Debug/
./axleddev_test /dev/gpio_leds on
```

IP 和路径根据实际情况调整。

除了 led 的现象之外，串口工具中的调试结果如下：

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
random: fast init done
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ax-platformdt-drv.ko
ax_platformdt_drv: loading out-of-tree module taints kernel.
[drm] load() is deferred & will be called again
root@ax_peta:/mnt# cd ./build-axleddev_test-ZYNQ-Debug/
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# ./axleddev_test /dev/gpio_leds on
ps_led1 on
```

第十四章 MISC 设备驱动

做了这么多实验之后，一直没有离开过字符设备。比较这些章节的驱动代码不难发现，字符设备框架中，有很大一部分的代码是相同的。对此，Linux 内核提出了 MISC 驱动模型，把这些共性的代码封装起来。

14.1 MISC 设备简介

MISC 意为杂散设备，诸如蜂鸣器、adc、led 这些难以分类的设备，都可以用 MISC 框架来实现驱动。

MISC 驱动模型实际上是对字符设备驱动模型的一次封装，他的本质是还是字符设备。MISC 的源码目录为/drivers/char/misc.c。相比一般的字符设备驱动框架，MISC 这层封装带来的好处有以下几点：

1) 节省主设备号

在文件/drivers/char/misc.c 的 214 行，在 misc_register 函数中，使用宏 MISC_MAJOR 表示主设备号来注册次设备号，MISC_MAJOR 宏定义在文件 include/uapi/linux/major.h 的第 25 行，值为 10，也就是说 MISC 设备的主设备号固定为 10。使用 MISC 框架注册的字符设备，都会在主设备号 10 下分配次设备号(部分次设备号已被内核占用)。而普通字符设备框架注册时必须分配一个主设备号，因此相对于一般的字符设备框架，使用 MISC 框架能节省主设备号。

2) 简洁使用方便

一般的字符设备驱动框架需要用 alloc_chrdev_region()、cdev_init()、cdev_add()、class_create()、device_create()一串函数来完成设备的创建，但是 MISC 框架使用 misc_register()函数对设备的创建过程做了很好的优化和封装，我们只需调用这一个函数就可以完成设备的创建，方便很多。

3) 驱动分层

MISC 也是对 Linux 驱动分层思想的一种体现，给难以分类的设备提供了归属类型，也方便了他们的使用。

14.2 MISC 框架的使用

MISC 框架的使用对于一般字符设备框架来说只是做一些替换。方法如下。

1) 创建并初始化 miscdevice 结构体

miscdevice 表示 misc 设备，定义在文件 include/linux/miscdevice.h 中，如下：

```
1. struct miscdevice {
2.     int minor;
3.     const char *name;
4.     const struct file_operations *fops;
5.     struct list_head list;
6.     struct device *parent;
7.     struct device *this_device;
8.     const struct attribute_group **groups;
9.     const char *nodename;
10.    umode_t mode;
11. };
```

结构体成员中没有主设备号，因为 misc 设备的主设备号都固定为 10。声明了 miscdevice 结构体变量之后需要初始化三个成员变量：

- a) 第 2 行的 minor，子设备号。使用 misc 框架注册的设备以子设备号来区分。内核中已经占用了部分 misc 的子设备号，在文件 include/linux/miscdevice.h 的 14~54 行中可查看已被占用的子设备号(不同的内核源码行号可能不一样)。如果需要在系统中查看已被占用的子设备号，可以 ls 一下/sys/dev/char 这个路径如下图。ls 结果中的如第一项的 10:1，冒号前的 10 为主设备，冒号后的 1 为此次设备号，也就是说主设备号 10 下的次设备号 1 已被占用。

```
root@ax_peta:/sys/dev/char# ls
10:1 10:61 1:1 1:8 254:0 4:11 4:17 4:22 4:28 4:33 4:39 4:44 4:5 4:55 4:60 4:9 7:128
90:2
10:130 10:62 1:11 1:9 254:1 4:12 4:18 4:23 4:29 4:34 4:4 4:45 4:50 4:56 4:61 5:0 7:129
90:3
10:20 10:63 1:3 246:0 254:2 4:13 4:19 4:24 4:3 4:35 4:40 4:46 4:51 4:57 4:62 5:1 89:0
90:4
10:237 116:33 1:4 247:0 4:0 4:14 4:2 4:25 4:30 4:36 4:41 4:47 4:52 4:58 4:63 5:2 89:1
90:5
10:59 13:63 1:5 248:0 4:1 4:15 4:20 4:26 4:31 4:37 4:42 4:48 4:53 4:59 4:7 7:0 90:0
90:6
10:60 189:0 1:7 253:0 4:10 4:16 4:21 4:27 4:32 4:38 4:43 4:49 4:54 4:6 4:8 7:1 90:1
90:7
```

- b) 第 3 行的 name，设备名称。将在/dev 目录中生成的节点名。
- c) 第 4 行的 fops，设备操作函数集。和字符设备中实现的 fops 相同。

2) 使用 misc_register 函数注册设备代替原先的注册过程

声明并初始化完成 miscdevice 后，使用 misc_register 函数来注册 misc 设备，函数定义在文件/drivers/char/misc.c 中，原型如下：

```
int misc_register(struct miscdevice * misc)
```

misc 即为 miscdevice 结构体变量。

注册成功返回 0，失败返回负数。

3) 使用 misc_deregister 函数注销设备代替原先的注销过程

与注册相对的就有注销函数，代替一般字符设备在驱动出口函数中一系列的注销操作，函数原型：

```
void misc_deregister(struct miscdevice * misc)
```

misc 即为 miscdevice 结构体变量。

14.3 实验

本章的实验在上一章的基础上修改，在 platform 框架下结合 misc 框架和把一般的字符设备框架修改成 misc 框架的方法几乎没有差别。

14.3.1 原理图

和第一章 1.3.1 节的内容相同。

14.3.2 设备树

和上一章相同。

14.3.3 驱动代码

使用 petalinux 新建名为“ax-misc-driv”的驱动程序，并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-misc-driv.c 文件中输入下面的代码：

```
1. #include <linux/types.h>
2. #include <linux/kernel.h>
3. #include <linux/delay.h>
4. #include <linux/init.h>
5. #include <linux/module.h>
6. #include <linux/errno.h>
7. #include <linux/gpio.h>
8. #include <linux/cdev.h>
9. #include <linux/device.h>
10. #include <linux/of_gpio.h>
11. #include <linux/semaphore.h>
12. #include <linux/timer.h>
13. #include <linux/irq.h>
14. #include <linux/wait.h>
15. #include <linux/poll.h>
16. #include <linux/fs.h>
17. #include <linux/fcntl.h>
18. #include <linux/platform_device.h>
19. #include <linux/miscdevice.h>
20. #include <asm/mach/map.h>
21. #include <asm/uaccess.h>
22. #include <asm/io.h>
23.
24. /* 设备节点名称 */
25. #define DEVICE_NAME      "gpio_leds"
26. /* 设备号个数 */
27. #define DEVID_COUNT      1
28. /* 驱动个数 */
29. #define DRIVE_COUNT      1
30. /* 主设备号 */
31. #define MAJOR_AX
32. /* 次设备号 */
33. #define MINOR_AX          20
34. /* LED 点亮时输入的值 */
35. #define ALINX_LED_ON      1
36. /* LED 熄灭时输入的值 */
37. #define ALINX_LED_OFF    0
38.
39. /* 把驱动代码中会用到的数据打包进设备结构体 */
40. struct alinx_char_dev{
41.     dev_t          devid;          //设备号
42.     struct cdev     cdev;          //字符设备
43.     struct class     *class;       //类
44.     struct device    *device;      //设备
45.     struct device_node *nd;        //设备树的设备节点
46.     int             ax_led_gpio;   //gpio 号
47. };
48. /* 声明设备结构体 */
49. static struct alinx_char_dev alinx_char = {
50.     .cdev = {
51.         .owner = THIS_MODULE,
52.     },
53. };
54.
55. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
56. static int gpio_leds_open(struct inode *inode_p, struct file *file_p)
57. {
58.     /* 设置私有数据 */
59.     file_p->private_data = &alinx_char;
60.
61.     return 0;
62. }
63.
64. /* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
```

```
65. static ssize_t gpio_leds_write(struct file *file_p, const char __user *buf, size
   _t len, loff_t *loff_t_p)
66. {
67.     int retvalue;
68.     unsigned char databuf[1];
69.     /* 获取私有数据 */
70.     struct alinx_char_dev *dev = file_p->private_data;
71.
72.     /* 获取用户数据 */
73.     retvalue = copy_from_user(databuf, buf, len);
74.     if(retvalue < 0)
75.     {
76.         printk("alinx led write failed\r\n");
77.         return -EFAULT;
78.     }
79.
80.     if(databuf[0] == ALINX_LED_ON)
81.     {
82.         /* gpio_set_value 方法设置 GPIO 的值, 使用!!对 0 或者 1 二值化 */
83.         gpio_set_value(dev->ax_led_gpio, !!0);
84.     }
85.     else if(databuf[0] == ALINX_LED_OFF)
86.     {
87.         gpio_set_value(dev->ax_led_gpio, !!1);
88.     }
89.     else
90.     {
91.         printk("gpio_test para err\n");
92.     }
93.
94.     return 0;
95. }
96.
97. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
98. static int gpio_leds_release(struct inode *inode_p, struct file *file_p)
99. {
100.     return 0;
101. }
102.
103. /* file_operations 结构体声明, 是上面 open、write 实现函数与系统调用函数对应的关
   键 */
104. static struct file_operations ax_char_fops = {
105.     .owner    = THIS_MODULE,
106.     .open     = gpio_leds_open,
107.     .write    = gpio_leds_write,
108.     .release  = gpio_leds_release,
109. };
110.
111. /* MISC 设备结构体 */
112. static struct miscdevice led_miscdev = {
113.     .minor = MINOR_AX,
114.     .name  = DEVICE_NAME,
115.     /* file_operations 结构体 */
116.     .fops  = &ax_char_fops,
117. };
118.
119. /* probe 函数实现, 驱动和设备匹配时会被调用 */
120. static int gpio_leds_probe(struct platform_device *dev)
121. {
122.     /* 用于接受返回值 */
123.     u32 ret = 0;
124.
125.     /* 获取设备节点 */
126.     alinx_char.nd = of_find_node_by_path("/alinxled");
127.     if(alinx_char.nd == NULL)
```

```
128.     {
129.         printk("gpioled node not found\n");
130.         return -EINVAL;
131.     }
132.
133.     /* 获取节点中 gpio 标号 */
134.     alinx_char.ax_led_gpio = of_get_named_gpio(alinx_char.nd, "alinxled-gpios",
135.     0);
136.     if(alinx_char.ax_led_gpio < 0)
137.     {
138.         printk("can not get alinxled-gpios\n");
139.         return -EINVAL;
140.     }
141.     /* 申请 gpio 标号对应的引脚 */
142.     ret = gpio_request(alinx_char.ax_led_gpio, "alinxled");
143.     if(ret != 0)
144.     {
145.         printk("can not request gpio\n");
146.     }
147.     /* 把这个 io 设置为输出 */
148.     ret = gpio_direction_output(alinx_char.ax_led_gpio, 1);
149.     if(ret < 0)
150.     {
151.         printk("can not set gpio\n");
152.     }
153.
154.     /*
155.     alloc_chrdev_region(&alinx_char.devid, MINOR_AX, DEVID_COUNT, DEVICE_NAME);
156.
157.     cdev_init(&alinx_char.cdev, &ax_char_fops);
158.
159.     cdev_add(&alinx_char.cdev, alinx_char.devid, DRIVE_COUNT);
160.
161.     alinx_char.class = class_create(THIS_MODULE, DEVICE_NAME);
162.     if(IS_ERR(alinx_char.class))
163.     {
164.         return PTR_ERR(alinx_char.class);
165.     }
166.
167.     alinx_char.device = device_create(alinx_char.class, NULL,
168.                                     alinx_char.devid, NULL,
169.                                     DEVICE_NAME);
170.     if (IS_ERR(alinx_char.device))
171.     {
172.         return PTR_ERR(alinx_char.device);
173.     }
174.     */
175.
176.     /* 注册 misc 设备 */
177.     ret = misc_register(&led_miscdev);
178.     if(ret < 0) {
179.         printk("misc device register failed\n");
180.         return -EFAULT;
181.     }
182.
183.     return 0;
184. }
185.
186. static int gpio_leds_remove(struct platform_device *dev)
187. {
188.     /*
189.     cdev_del(&alinx_char.cdev);
190.
```



```

191.     unregister_chrdev_region(alinx_char.devid, DEVID_COUNT);
192.
193.     device_destroy(alinx_char.class, alinx_char.devid);
194.
195.     class_destroy(alinx_char.class);
196.     */
197.
198.     /* 注销 misc 设备 */
199.     misc_deregister(&led_miscdev);
200.     return 0;
201. }
202.
203. /* 初始化 of_match_table */
204. static const struct of_device_id led_of_match[] = {
205.     /* compatible 字段和设备树中保持一致 */
206.     { .compatible = "alinx-led" },
207.     { /* Sentinel */ }
208. };
209.
210.
211. /* 声明并初始化 platform 驱动 */
212. static struct platform_driver led_driver = {
213.     .driver = {
214.         /* name 字段需要保留 */
215.         .name = "alinx-led",
216.         /* 用 of_match_table 代替 name 匹配 */
217.         .of_match_table = led_of_match,
218.     },
219.     .probe = gpio_leds_probe,
220.     .remove = gpio_leds_remove,
221. };
222.
223. /* 驱动入口函数 */
224. static int __init gpio_led_drv_init(void)
225. {
226.     /* 在入口函数中调用 platform_driver_register, 注册 platform 驱动 */
227.     return platform_driver_register(&led_driver);
228. }
229.
230. /* 驱动出口函数 */
231. static void __exit gpio_led_dev_exit(void)
232. {
233.     /* 在出口函数中调用 platform_driver_unregister, 卸载 platform 驱动 */
234.     platform_driver_unregister(&led_driver);
235. }
236.
237. /* 标记加载、卸载函数 */
238. module_init(gpio_led_drv_init);
239. module_exit(gpio_led_dev_exit);
240.
241. /* 驱动描述信息 */
242. MODULE_AUTHOR("Alinx");
243. MODULE_ALIAS("gpio_led");
244. MODULE_DESCRIPTION("MISC LED driver");
245. MODULE_VERSION("v1.0");
246. MODULE_LICENSE("GPL");

```

与上一章相比，改动很少，看加粗的部分。

34 行设置次设备号为 20。首先查看已被内核占用的次设备号，这里我们用的 20 没有被占用，其他没有被占用的次设备号都可以。

113~118 行定义并初始化 miscdevice 结构体。

155~175 行驱动入口函数中被注释掉的部分替换为 117~182 行的 misc_register 函数，达到注册设备的目的。明显可以看到，代码被简化了很多。

189~197 行驱动出口函数中的注销函数替换为 200 行的 misc_deregister。同样的简化了很多。

14.3.4 测试代码

测试 APP 和第一章 1.3.4 节内容一致，可以使用第一章的测试程序。

14.3.5 运行测试

测试方法和上一章基本一致，省去了加载 device 的命令，步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ax-misc-drv.ko
cd ./build-axleddev_test-ZYNQ-Debug/
./axleddev_test /dev/gpio_leds on
```

IP 和路径根据实际情况调整。

串口工具中的调试结果如下：

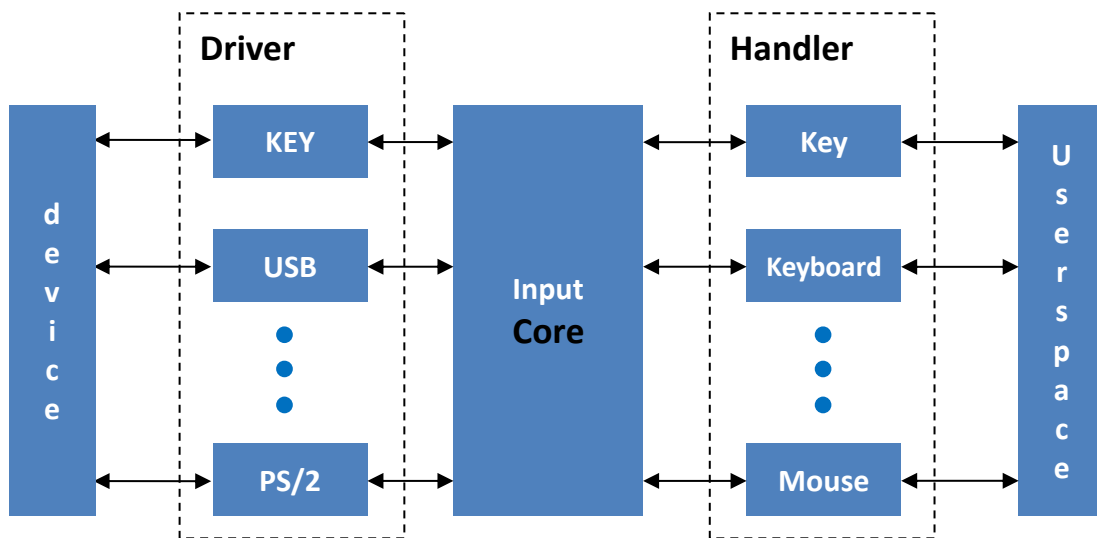
```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ./ax-misc-drv.ko
ax_misc_drv: loading out-of-tree module taints kernel.
[drm] load() is deferred & will be called again
root@ax_peta:/mnt# cd ./build-axleddev_test-ZYNQ-Debug/
root@ax_peta:/mnt/build-axleddev_test-ZYNQ-Debug# ./axleddev_test /dev/gpio_leds on
ps_led1 on
```

第十五章 input 子系统

Linux 支持的输入设备很多，如键盘、鼠标、触摸屏或简单的按键等等。这些输入设备的原理不尽相同，为了管理这些多样的输入设备，Linux 内核提供了 input 子系统框架。

15.1 input 子系统简介

相较于前面学到的 platform 框架 misc 框架，input 子系统框架的复杂度稍高。它分为事件层、核心层、设备驱动层，上中下三层，如下：



可以看出，input 子系统中可以注册多个不同的输入设备。

- 1) Driver 对应下层驱动，和各种输入设备对接。
- 2) Handler 对应上层事件，通过设备节点文件和用户空间相对接。不同的 handler 设备节点文件名称也不同。
- 3) Core 核心层负责上层和下层的协调，实现上下层的数据交互。下层的事件会激活整个系统，核心会上传事件到上层，最终传递到用户空间。

input 子系统解决了不同输入设备与应用层数据交互的问题，为输入设备驱动开发提供了很多方便。

15.1.1 input 子系统驱动框架的使用

input 子系统的核心是在文件 `drivers/input/input.c` 中实现的。我们找到其中 2416 行的 `input_init()` 函数。

```
.....
2416. static int __init input_init(void)
2417. {
2418.     int err;
2419.
```

```
2420.     err = class_register(&input_class);
2421.     if (err) {
2422.         pr_err("unable to register input_dev class\n");
2423.         return err;
2424.     }
2425.
2426.     err = input_proc_init();
2427.     if (err)
2428.         goto fail1;
2429.
2430.     err = register_chrdev_region(MKDEV(INPUT_MAJOR, 0),
2431.                                  INPUT_MAX_CHAR_DEVICES, "input");
2432.     if (err) {
2433.         pr_err("unable to register char major %d", INPUT_MAJOR);
2434.         goto fail2;
2435.     }
2436.
2437.     return 0;
2438.
2439.     fail2: input_proc_exit();
2440.     fail1: class_unregister(&input_class);
2441.     return err;
2442. }
.....
2452.     subsys_initcall(input_init);
.....
```

2452 行用 `subsys_initcall(input_init)` 标记了 `input_init` 函数, `subsys_initcall` 作用于 `module_init` 类似用于标记驱动入口函数, 区别在于 `subsys_initcall` 标记的是静态加载的驱动。也就是说 `input_init` 函数是系统启动后就会执行的函数, 那我们看看 `input_init` 函数中做了什么工作。

2420 行先是注册了 `input_class`。

2430 行使用主设备号 `INPUT_MAJOR` 注册了 `input` 设备, `INPUT_MAJOR` 定义在文件 `include/uapi/linux/major.h` 中, 值为 13。

也就是说我们使用 `input` 子系统来加载输入设备, 不需要再注册字符设备, 只需要向 `input` 子系统注册 `input_dev` 就可以了。

接下来我们介绍一下什么是 `input_dev`, `input` 子系统具体如何使用。

1) 申请 `input_dev` 结构体变量

使用 `input` 子系统需要注册一个 `input` 设备, `input_dev` 结构体就是用于表示 `input` 设备, 定义在文件 `include/linux/input.h` 中, 如下:

```
121. struct input_dev {
122.     const char *name;
123.     const char *phys;
124.     const char *uniq;
125.     struct input_id id;
126.
127.     unsigned long proppbit[BITS_TO_LONGS(INPUT_PROP_CNT)];
128.
129.     unsigned long evbit[BITS_TO_LONGS(EV_CNT)];
130.     unsigned long keybit[BITS_TO_LONGS(KEY_CNT)];
131.     unsigned long relbit[BITS_TO_LONGS(REL_CNT)];
132.     unsigned long absbit[BITS_TO_LONGS(ABS_CNT)];
```

```

133.     unsigned long mscbit[BITS_TO_LONGS(MSC_CNT)];
134.     unsigned long ledbit[BITS_TO_LONGS(LED_CNT)];
135.     unsigned long sndbit[BITS_TO_LONGS(SND_CNT)];
136.     unsigned long ffbitt[BITS_TO_LONGS(FF_CNT)];
137.     unsigned long swbit[BITS_TO_LONGS(SW_CNT)];
138.
.....
189.
190.     bool devres_managed;
191. };

```

129~137 行是一些事件的位图，用于传递事件。可选的事件类型定义在文件 `include/uapi/linux/input-event-codes.h` 中。第一个位图 `evbit` 用于设置启用下面的哪种事件，比如我们需要使用 `keybit`，就需要把 `evbit` 中对应的位置起来对应的宏定义为 `EV_KEY`。

申请 `input_dev` 结构体变量使用下面的函数：

```
struct input_dev *input_allocate_device(void)
```

释放 `input_dev` 结构体变量使用下面的函数：

```
void input_free_device(struct input_dev *dev)
```

2) 初始化 `input_dev` 结构体变量

申请 `input_dev` 成功后，首先要初始化一些成员变量，如 `name` 字段。主要需要初始化的是事件位图。设置事件位图的方法针对我们的实验介绍以下几种：

- a) 直接赋值，如 `input_dev->evbit[0] = BIT_MASK(EV_KEY);`
- b) 使用 `__set_bit()` 方法，如 `__set_bit(EV_KEY, input_dev->evbit);`
清除事件使用 `__clear_bit(EV_KEY, input_dev->evbit);`
- c) 使用 `input_set_capability` 方法，如 `input_set_capability(input_dev, EV_KEY, KEY_Q)`。这个函数实际上还是调用的 `__set_bit` 方法，这一句实际上等价于 `__set_bit(KEY_Q, input_dev->keybit);`。 `input_set_capability` 相当于是对 `evbit` 的进一步设置，因此这个函数不可以设置 `evbit`。

3) 注册 `input_dev` 结构体变量

初始化完成后，就可以向系统注册 `input_dev` 结构体变量了，使用函数：

```
int input_register_device(struct input_dev *dev)
```

返回 0 注册成功，返回负值注册失败。

相对的注销使用下面的函数：

```
void input_unregister_device(struct input_dev *dev)
```

注意，`input_register_device` 函数注册成功后，如果需要释放 `input_dev` 结构体，需要先用 `input_unregister_device` 再调用 `input_free_device`。

4) 输入事件上报

设置好 `input_dev` 结构体变量之后，就可以通过这个变量来上报捕捉到的输入事件给内核，使用函数：

```
void input_event(struct input_dev *dev, unsigned int type, unsigned int code, int value)
```

参数说明：

dev: input_dev 结构体指针;
type: 上报的事件类型, 可选值为 Event types, 定义在文件 include/uapi/linux/input-event-codes.h 中的 37~48 行。
code: 事件码, 定义在文件 include/uapi/linux/input-event-codes.h 中, 根据 type 的至来选择, 比如 type 为 EV_KEY 时, code 可选 KEY_0、KEY_1 等等。
value: 事件对应的值, 比如按键事件中, 可以规定一个值表示案件被按下或松开。
input_event()函数可以上报所有的事件类型, Linux 内核还提供了一些针对性的事件上报函数, 实际上都是对 input_event()函数的封装。

上报之后, 还需要使用下面的函数来提交到 input 子系统:

```
void input_sync(struct input_dev *dev);
```

dev 为设置好的 input_dev 结构体变量。

15.1.2 input 子系统应用程序的使用

驱动上报了输入事件后, 应用程序就可以获取到输入事件。获取输入时间使用 read()方法。但是输入参数较为特殊, 需要使用结构体变量 input_event。input_event 定义在文件 include/uapi/linux/input.h 中, 如下:

```
struct input_event {  
    struct timeval time;  
    __u16 type;  
    __u16 code;  
    __s32 value;  
};
```

time 表示时间, timeval 结构体有两个成员, 分别表示秒和微妙。

type、code、value 和前面讲的 input_event()函数的输入参数相对应。

15.3 实验

本章的实验在第八章中断按键实验驱动代码基础上修改。

15.3.1 原理图

和章节 6.1 相同。

15.3.2 设备树

和章节 6.2 相同。

15.3.3 驱动程序

使用 petalinux 新建名为“ax-input-drv”的驱动程序, 并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-input-drv.c 文件中输入下面的代码:

```
1. #include <linux/kernel.h>  
2. #include <linux/module.h>  
3. #include <linux/init.h>  
4. #include <linux/types.h>  
5. #include <linux/errno.h>  
6. #include <linux/cdev.h>  
7. #include <linux/of.h>  
8. #include <linux/of_address.h>
```

```
9. #include <linux/of_gpio.h>
10. #include <linux/device.h>
11. #include <linux/delay.h>
12. #include <linux/init.h>
13. #include <linux/gpio.h>
14. #include <linux/semaphore.h>
15. #include <linux/timer.h>
16. #include <linux/of_irq.h>
17. #include <linux/irq.h>
18. #include <linux/input.h>
19. #include <asm/uaccess.h>
20. #include <asm/mach/map.h>
21. #include <asm/io.h>
22.
23. /* 设备节点名称 */
24. #define INPUT_DEV_NAME "input_key"
25.
26. /* 把驱动代码中会用到的数据打包进设备结构体 */
27. struct alinx_char_dev {
28.     dev_t          devid;           //设备号
29.     struct cdev     cdev;           //字符设备
30.     struct class    *class;         //类
31.     struct device   *device;        //设备
32.     struct device_node *nd;          //设备树的设备节点
33.     spinlock_t      lock;           //自旋锁变量
34.     int              alinx_key_gpio; //gpio 号
35.     unsigned int     irq;           //中断号
36.     struct timer_list timer;        //定时器
37.     struct input_dev *inputdev;     //input_dev 结构体
38.     unsigned char    code;          //input 事件码
39. };
40. /* 声明设备结构体 */
41. static struct alinx_char_dev alinx_char = {
42.     .cdev = {
43.         .owner = THIS_MODULE,
44.     },
45. };
46.
47. /* 中断服务函数 */
48. static irqreturn_t key_handler(int irq, void *dev)
49. {
50.     /* 按键按下或抬起时会进入中断 */
51.     struct alinx_char_dev *cdev = (struct alinx_char_dev *)dev;
52.     cdev->timer.data = (volatile long)dev;
53.     /* 开启 50 毫秒的定时器用作防抖动 */
54.     mod_timer(&cdev->timer, jiffies + msecs_to_jiffies(50));
55.     return IRQ_RETVAL(IRQ_HANDLED);
56. }
57.
58. /* 定时器服务函数 */
59. void timer_function(struct timer_list *timer)
60. {
61.     unsigned long flags;
62.     struct alinx_char_dev *dev = (struct alinx_char_dev *)arg;
63.     /* value 用于获取按键值 */
64.     unsigned char value;
65.
66.     /* 获取锁 */
67.     spin_lock_irqsave(&dev->lock, flags);
68.
69.     /* 获取按键值 */
70.     value = gpio_get_value(dev->alinx_key_gpio);
71.
72.     if(value == 0)
73.     {
74.         /* 按键按下, 状态置 1 */
75.         input_report_key(dev->inputdev, dev->code, 1);
76.         input_sync(dev->inputdev);
77.     }
78.     else
79.     {
```

```
80.      /* 按键抬起 */
81.      input_report_key(dev->inputdev, dev->code, 0);
82.      input_sync(dev->inputdev);
83.  }
84.
85.      /* 释放锁 */
86.      spin_unlock_irqrestore(&dev->lock, flags);
87.  }
88.
89.  /* 模块加载时会调用的函数 */
90.  static int __init char_drv_init(void)
91.  {
92.      /* 用于接受返回值 */
93.      u32 ret = 0;
94.
95.      /* 初始化自旋锁 */
96.      spin_lock_init(&alinx_char.lock);
97.
98.      /* 获取设备节点 */
99.      alinx_char.nd = of_find_node_by_path("/alinxkey");
100.     if(alinx_char.nd == NULL)
101.     {
102.         printk("alinx_char node not find\r\n");
103.         return -EINVAL;
104.     }
105.     else
106.     {
107.         printk("alinx_char node find\r\n");
108.     }
109.
110.     /* 获取节点中 gpio 标号 */
111.     alinx_char.alinx_key_gpio = of_get_named_gpio(alinx_char.nd, "alinxkey-gpios", 0);
112.     if(alinx_char.alinx_key_gpio < 0)
113.     {
114.         printk("can not get alinxkey-gpios");
115.         return -EINVAL;
116.     }
117.     printk("alinxkey-gpio num = %d\r\n", alinx_char.alinx_key_gpio);
118.
119.     /* 申请 gpio 标号对应的引脚 */
120.     ret = gpio_request(alinx_char.alinx_key_gpio, "alinxkey");
121.     if(ret != 0)
122.     {
123.         printk("can not request gpio\r\n");
124.         return -EINVAL;
125.     }
126.
127.     /* 把这个 io 设置为输入 */
128.     ret = gpio_direction_input(alinx_char.alinx_key_gpio);
129.     if(ret < 0)
130.     {
131.         printk("can not set gpio\r\n");
132.         return -EINVAL;
133.     }
134.
135.     /* 获取中断号 */
136.     alinx_char.irq = gpio_to_irq(alinx_char.alinx_key_gpio);
137.     /* 申请中断 */
138.     ret = request_irq(alinx_char.irq,
139.                      key_handler,
140.                      IRQF_TRIGGER_FALLING | IRQF_TRIGGER_RISING,
141.                      "alinxkey",
142.                      &alinx_char);
143.     if(ret < 0)
144.     {
145.         printk("irq %d request failed\r\n", alinx_char.irq);
146.         return -EFAULT;
147.     }
148.
149.     __init_timer(&alinx_char.timer, timer_function, 0);
150.
151. }
```



```
152.    /* 设置事件码为 KEY_0 */
153.    alinx_char.code = KEY_0;
154.
155.    /* 申请 input_dev 结构体变量 */
156.    alinx_char.inputdev = input_allocate_device();
157.
158.    alinx_char.inputdev->name = INPUT_DEV_NAME;
159.    /* 设置按键事件 */
160.    __set_bit(EV_KEY, alinx_char.inputdev->evbit);
161.    /* 设置按键重复事件 */
162.    __set_bit(EV_REP, alinx_char.inputdev->evbit);
163.    /* 设置按键事件码 */
164.    __set_bit(KEY_0, alinx_char.inputdev->keybit);
165.
166.    /* 注册 input_dev 结构体变量 */
167.    ret = input_register_device(alinx_char.inputdev);
168.    if(ret) {
169.        printk("register input device failed\r\n");
170.        return ret;
171.    }
172.
173.    return 0;
174.}
175.
176./* 卸载模块 */
177.static void __exit char_drv_exit(void)
178.{
179.    /* 删除定时器 */
180.    del_timer_sync(&alinx_char.timer);
181.    /* 释放中断号 */
182.    free_irq(alinx_char.irq, &alinx_char);
183.    /* 注销 input_dev 结构体变量 */
184.    input_unregister_device(alinx_char.inputdev);
185.    /* 释放 input_dev 结构体变量 */
186.    input_free_device(alinx_char.inputdev);
187.}
188.
189./* 标记加载、卸载函数 */
190.module_init(char_drv_init);
191.module_exit(char_drv_exit);
192.
193./* 驱动描述信息 */
194.MODULE_AUTHOR("Alinx");
195.MODULE_ALIAS("alinx char");
196.MODULE_DESCRIPTION("INPUT LED driver");
197.MODULE_VERSION("v1.0");
198.MODULE_LICENSE("GPL");
```

重点关注加粗的部分。

19 行包含 input.h 头文件。

38 行在设备结构体中增加 struct input_dev 型指针成员变量。

39 行添加一个事件码 code。

76 行在 timer 去抖的回掉函数中，如果案件被按下则使用 input_report_key() 函数上报事件。input_report_key() 函数定义在文件 include/linux/input.h 中，内容很简单，就时调用了一下 input_event(dev, EV_KEY, code, !!value); 函数专门用于按键的时间上报。input_report_key() 的第一个参数为 input_dev 结构体类型的指针，也就是我们驱动中申请的输入设备，第二个参数是事件码，第三个参数为事件对应的值。

77 行调用 input_sync(dev->inputdev) 提交上报。

82~83 行同理上报按键抬起的事件，我们这里用第三个参数 value 来区分按下和抬起，1 表示按下，0 表示抬起。

在驱动入口函数中，153 行之前，就是和第八章一样的获取设备树中信息，设置中断和定时器。

154 行个事件码赋值为 KEY_0，之后我们在写应用程序时，需要对应到 KEY_0 这个值。

157 行使用 input_allocate_device()函数申请结构体变量。

159~165 行初始化 input_dev 结构体变量。先给 name 赋值。

161 行调用__set_bit 把 evbit 中的按键事件置 1。163 行同理设置重复按键。

165 行设置按键事件的事件码为 KEY_0。

在驱动出口函数中，185 行先调用 input_unregister_device 注销 input_dev，187 行再调用 input_free_device 释放 input_dev 结构体变量的空间。

全程我们没有再去实现 file_operations 设备操作函数集，都是使用 input 子系统中的操作函数。也没有再去注册字符设备，所以使用 input 子系统，相当于是向 input 子系统注册设备，而不是在向内核注册。我们直接交流的对象是 input 子系统而不是内核，和内核的交互，全权交给 input 子系统了，这点和前面说的 misc 设备框架是有所区别的。

15.3.4 测试程序

新建 QT 工程名为“ax_inputkey_test”，新建 main.c，输入下面的代码：

```
1. #include "stdio.h"
2. #include "unistd.h"
3. #include <fcntl.h>
4. #include <linux/input.h>
5. /* 定义一个 input_event 结构体变量获取事件 */
6. static struct input_event inputevent;
7.
8. /* 点亮火熄灭 led */
9. int led_change_sts()
10. {
11.     int fd, ret;
12.     static char led_value = 0;
13.
14.     fd = open("/dev/gpio_leds", O_RDWR);
15.     if(fd < 0)
16.     {
17.         printf("file /dev/gpio_leds open failed\n");
18.     }
19.
20.     led_value = !led_value;
21.     ret = write(fd, &led_value, sizeof(led_value));
22.
23.     if(ret < 0)
24.     {
25.         printf("write failed\n");
26.     }
27.
28.     ret = close(fd);
29.     if(ret < 0)
30.     {
31.         printf("file /dev/gpio_leds close failed\n");
32.     }
33.
34.     return ret;
35. }
36.
37. int main(int argc, char *argv[])
38. {
39.     int fd;
40.     int err = 0;
41.     char *filename;
42.
43.     filename = argv[1];
```

```
44.
45.  /* 验证输入参数个数 */
46.  if(argc != 2) {
47.      printf("Error Usage\r\n");
48.      return -1;
49.  }
50.
51.  /* 打开输入的设备文件, 获取文件句柄 */
52.  fd = open(filename, O_RDWR);
53.  if(fd < 0) {
54.      /* 打开文件失败 */
55.      printf("can not open file %s\r\n", filename);
56.      return -1;
57.  }
58.
59.  while(1)
60.  {
61.      err = read(fd, &inputevent, sizeof(inputevent));
62.      if(err > 0)
63.      {
64.          switch(inputevent.type)
65.          {
66.              case EV_KEY:
67.                  if(KEY_0 == inputevent.code)
68.                  {
69.                      if(0 == inputevent.value)
70.                      {
71.                          /* 按键抬起 */
72.                          err = led_change_sts();
73.                      }
74.                      else
75.                      {
76.                          /* 按键按下 */
77.                      }
78.                  }
79.                  else
80.                  {
81.                      /* ignore */
82.                  }
83.                  break;
84.
85.                  default :
86.                      /* ignore */
87.                      break;
88.          }
89.
90.          if(err < 0)
91.          {
92.              printf("led open filed");
93.          }
94.      }
95.      else
96.      {
97.          printf("get data failed\r\n");
98.      }
99.  }
100.
101.  return 0;
102. }
```

第 4 行包含头文件 input.h。

第 6 行定义 input_event 结构体变量以获取事件。

8~35 行把点亮或熄灭 led 的代码拎出来, 这样看 main 函数能清楚些。

main 函数中 61 行, 调用 read 函数读取事件, 这里第二个参数传递的就是 input_event 结构体变量。

64 行先判断事件类型, 如果是 EV_KEY 类型, 则进入 66 行的 case, 其他类型忽略。

67 行再判断事件码是不是 KEY_0，不是则忽略，是就继续往下判断。

69 行判断事件对应的值，驱动代码中我们说过 0 代表按键抬起，1 表示按下。按键抬起是一次按键的结束，所以在按键抬起后，我们切换一次 led 的状态。

15.3.5 运行测试

input 子系统框架下生成的设备节点文件在路径/dev/input 下，在使用 insmod 加载 input 设备时，系统会提示“input: input_key as /devices/virtual/input/input0”，一般设备文件会对应这里的 input0，也就是/dev/input/event0。

测试方法步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ax-input-drv.ko
insmod ax-concled-drv.ko
cd ./build-ax_inputkey_test-ZYNQ-Debug
./ax_inputkey_test /dev/input/event0
```

IP 和路径根据实际情况调整。

串口工具中的调试结果如下：

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
random: fast init done
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ax-input-drv.ko
ax_input_drv: loading out-of-tree module taints kernel.
alinx_char node find
alinxkey-gpio num = 949
input: input_key as /devices/virtual/input/input0
root@ax_peta:/mnt# insmod ax-concled-drv.ko
alinx_char node find
alinxled-gpio num = 899
root@ax_peta:/mnt# cd ./build-ax_inputkey_test-ZYNQ-Debug
root@ax_peta:/mnt/build-ax_inputkey_test-ZYNQ-Debug# ./ax_inputkey_test /dev/input/event0
gpio_test module open
gpio_test module release
gpio_test module open
gpio_test module release
```

可以看一下 input 子系统下，按键的 cpu 占用量，也是几乎为零。

```
root@ax_peta:/mnt/build-ax_inputkey_test-ZYNQ-Debug# ./ax_inputkey_test /dev/input/event0
[1] 1273
root@ax_peta:/mnt/build-ax_inputkey_test-ZYNQ-Debug# top
Mem: 56700K used, 973552K free, 13496K shrd, 1196K buff, 42908K cached
CPU:  0.0% usr  0.0% sys  0.0% nic 100% idle  0.0% io  0.0% irq  0.0% sirq
Load average: 0.05 0.02 0.00 1/88 1274
  PID PPID USER   STAT  VSZ %VSZ CPU %CPU COMMAND
 1247   1 root    S     14352  1.3   1  0.0 /usr/sbin/tcf-agent -d -L- -10
 1273  1254 root    S     13164  1.2   0  0.0 ./ax_inputkey_test /dev/input/event0
 1254  1252 root    S     2988  0.2   1  0.0 -sh
 1174   1 root    S     2916  0.2   1  0.0 /usr/sbin/inetd
   761   1 root    S     2868  0.2   0  0.0 /sbin/udevd -d
  1233   1 root    S     2788  0.2   1  0.0 /sbin/syslogd -n -O /var/log/messages
  1225   1 root    S     2788  0.2   0  0.0 /sbin/klogd -n
```

第十六章 pwm 驱动

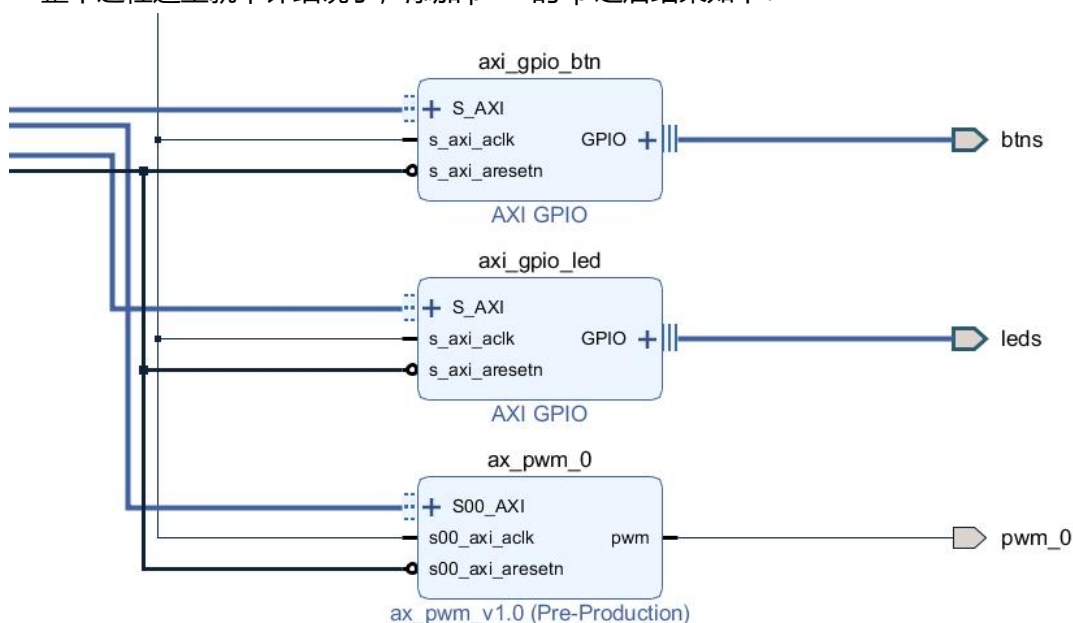
pwm 在嵌入式设备中使用很多，常用于控制电机、控制振动器件、调节背光、呼吸灯等等。这一章就来学习 zynq 平台上 Linux 系统中的 pwm 驱动的实现方法。

16.1 zynq 上的 pwm 实现

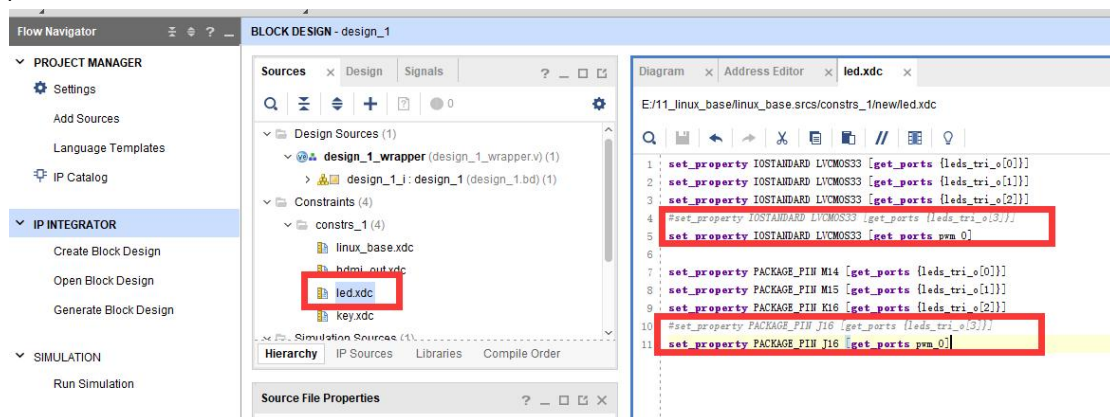
16.1.1 修改 vivado 工程

zynq 平台上输出 pwm 需要借助 pl 端(fpga)的资源。参考文档 course_s2 的第七章，自定义 IP。

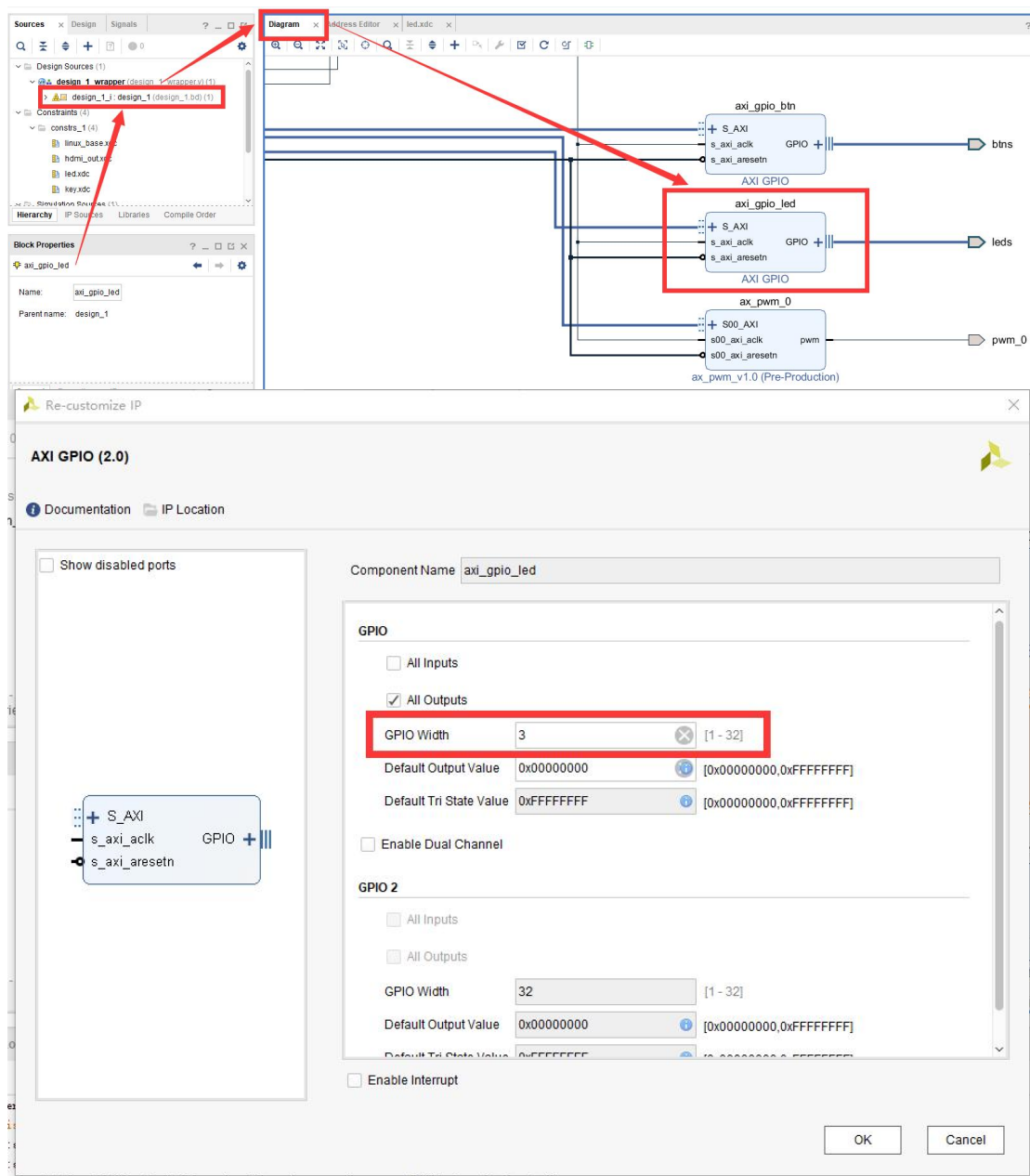
整个过程这里就不详细说了，添加 pwm 的 ip 之后结果如下：



ip 添加完成后，需要修改 xdc 更改引脚约束，这里我们准备使用 pwm 来控制开发板上 PL_LED4 这个 led。打开 led.xdc 文件，原先的 J16 脚是配置成 led 的，现在修改成 pwm_0，如下图。



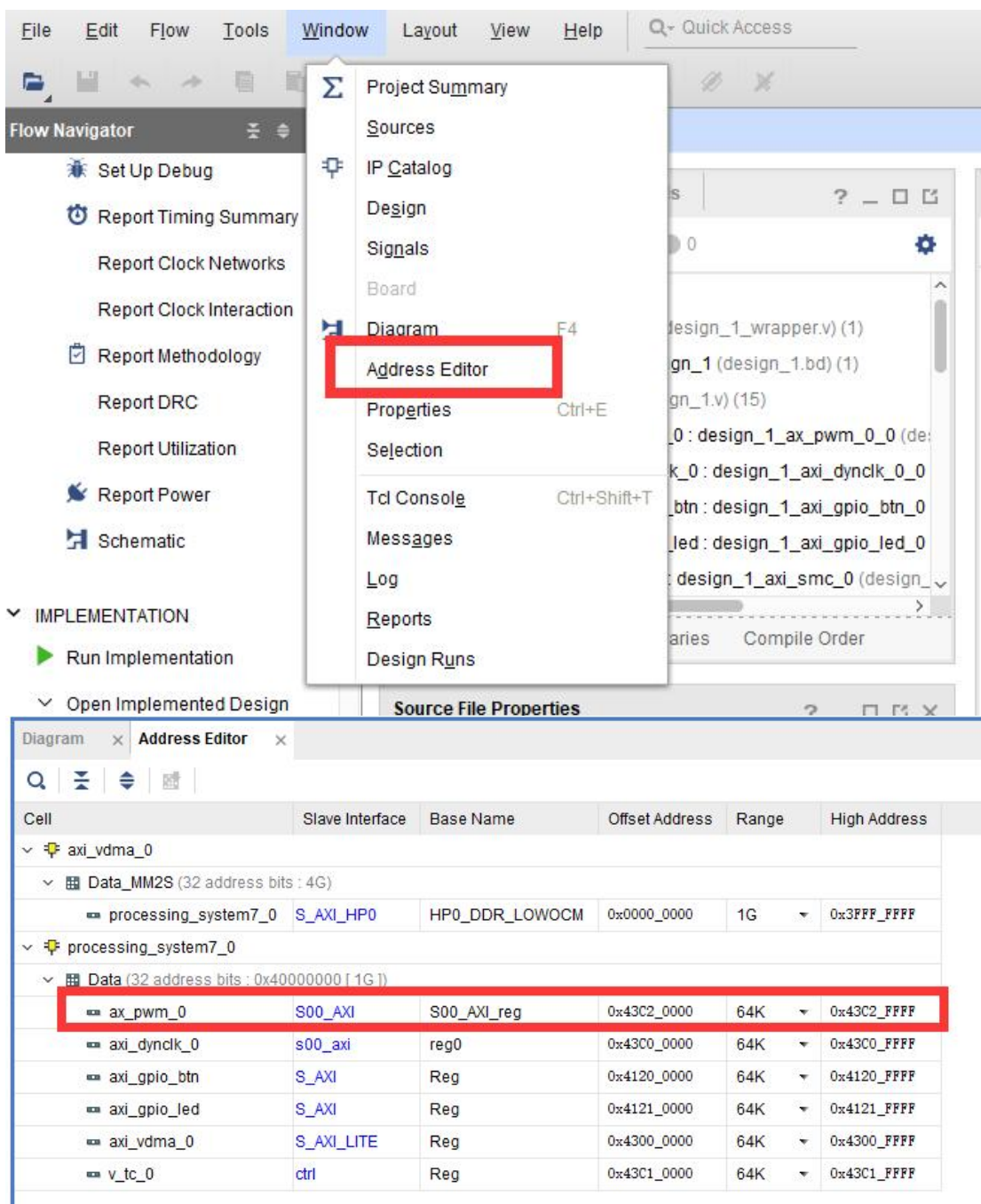
这里修改完后，原先 gpio 输出的 ip 就少了一个引脚的使用，我们对应修改一下 ip，双击 axi_gpio_led 这个 ip，打开配置界面，如下图：



把 GPIO Width 字段修改成 3。这样就修改完成了，再按照第十八章中的步骤到处硬件信息并重新定制一个 petalinux 系统。

16.1.2 控制 pwm 输出的方法

控制 pwm 输出有两个关键参数，一是频率，二是占空比。在上面我们配置好硬件信后，该如何去操作 pwm 的这两个参数呢。自定义 IP 可以理解为配置 fpga 的寄存器资源变成我们想要的设备，添加 IP 后，我们就可以像操作 ARM 资源一样，通过操作寄存器来操作 fpga 的资源。那首先我们就要找到 pwm 设备的寄存器首地址。在菜单栏 Window 选项中打开 Address Editor 窗口。如图下：



就能找到 ip 对应的寄存器范围，这个寄存器首地址是 vivado 工程自动分配的，所以每个人的可能会不一样，当然也可以手动修改。

光是找到了寄存器首地址，还是不知道寄存器的具体设置方法，这里我们参考一下文档 course_s2 的第七章中的 SDK 软件编写调试试验。

实验工程中的主函数如下图：

```
unsigned int duty;
int main()
{
    init_platform();
    print("Hello World\n\r");
    //pwm out period = frequency(pwm_out) * (2 ** N) / frequency(clk);
    AX_PWM_mWriteReg(XPAR_AX_PWM_0_S00_AXI_BASEADDR, AX_PWM_S00_AXI_SLV_REG0_OFFSET, 17179); //200hz
    while (1) {
        for (duty = 0x8ffffff; duty < 0xffffffff; duty = duty + 100000) {
            AX_PWM_mWriteReg(XPAR_AX_PWM_0_S00_AXI_BASEADDR, AX_PWM_S00_AXI_SLV_REG1_OFFSET, duty);
            usleep(100);
        }
    }
    cleanup_platform();
    return 0;
}
```

AX_PWM_mWriteReg 函数用于设置寄存器的值，第一个参数是首地址，第二个参数是偏移量，第三个参数是目标值。

第一次调用 AX_PWM_mWriteReg 函数是设置频率。XPAR_AX_PWM_0_S00_AXI_BASEADDR 的值就是我们上面在 Address Editor 界面中查到的首地址，看一下偏移量 AX_PWM_S00_AXI_SLV_REG0_OFFSET 的值为 0。也就是说设置频率的寄存器就是首地址了。

第二次调用是设置占空比，首地址没变，偏移量 AX_PWM_S00_AXI_SLV_REG1_OFFSET 的值为 4，也就是首地址加 4。

16.2 Linux 中的 pwm 框架

在 Linux 内核提供了一套完整的 pwm 框架。但是在 zynq 上使用这套框架需要注意，zynq 上的 pwm 是借助 fpga 资源来实现的，自定义 ip 的寄存器设置可能会因人而异，因此要使用这套框架，需要保证自定义 ip 的一致。再者如果只是简单的 pwm 应用，如本章实验的 led 呼吸灯，使用这个框架就显得有些大材小用了。虽然在本章的实验中不会这个框架，但我们还是简单的介绍一下，了解一下其中的结构与思路。

pwm 框架还是融入了驱动的分层思想，把 pwm 驱动分成了 pwm_device(设备)和 pwm_chip(芯片)两个部分，pwm_chip 意为 pwm 的芯片，它可以输出单路或者多路 pwm 波形，而 pwm_device 即为使用 pwm_chip 其中一路或多路的设备，以本章例程 led 呼吸灯为例，led 就是 pwm_device，我们自定义的 pwmip 就是 pwm_chip。pwm 的框架一般会 and platform 框架结合使用。接下来我们分别介绍他们的实现方法。

16.2.1 pwm_device

一个使用 pwm 的设备，需要关注四个参数：使能、极性、频率、占空比，对此 pwm 框架提供了以下方法来与之对应：

- 1) int pwm_enable(struct pwm_device *pwm);和 void pwm_disable(struct pwm_device *pwm);
pwm_enable 函数用使能 pwm 输出，pwm_disable 用于失能 pwm 输出。
- 2) int pwm_set_polarity(struct pwm_device *pwm, enum pwm_polarity polarity);
pwm_set_polarity 用于设置极性，可选参数有 PWM_POLARITY_NORMAL(普通)和 PWM_POLARITY_INVERSED(反转)两种。
- 3) int pwm_config(struct pwm_device *pwm, int duty_ns, int period_ns);
pwm_config 函数用于设置占空比和频率，参数 duty_ns 为占空比，period_ns 为频

率。

这些接口函数定义在文件 include/linux/pwm.h 中, 他们都需要一个 struct pwm_device 类型的指针作为操作句柄。

pwm_device 结构体则是指代使用 pwm 的设备, 我们要使用这些接口, 需要先获取一个 pwm_device 结构体。

获取 pwm_device 结构体的方法也定义在文件 include/linux/pwm.h 中。如下:

- 1) struct pwm_device *pwm_get(struct device *dev, const char *con_id);和
struct pwm_device *of_pwm_get(struct device_node *np, const char *con_id);
这两个方法将从指定设备(参数 dev)的设备树节点或者直接从设备树节点(参数 np)中去获取 pwm_device。对设备树的格式要求如下:

```
bl: backlight {
    pwms = <&pwm 0 5000000 PWM_POLARITY_INVERTED>;
    pwm-names = "backlight";
}
```

bl: backlight 即为设备节点名称。节点中的两个属性都是必须的

pwms 指代这个设备用到的 pwm 列表, 第一个参数&pwm 实际是引用了 pwm_chip 节点, 具体在讲 pwm_chip 的时候再说。第二个参数指代 platform_device 的设备号, 具体根据实际情况来定, 不一定会用到。第三个参数 500000 是 pwm 的默认周期, 单位是纳秒。第四个参数是可选字段, 表示极性。

pwm-names 是与 pwms 相对应的 pwm 设备名称。"backlight"就是与&pwm 0 5000000 PWM_POLARITY_INVERTED 相对应。

函数参数 con_id 用于指定相匹配的 pwm 设备, 如果 NULL, 就会返回 pwms 中的第一个设备, 如果指定名称如"backlight", 就会返回对应的设备。

- 2) void pwm_put(struct pwm_device *pwm);
与获取 get 函数相对的 put, 用于释放 pwm_device。

pwm_device 需要关注的事情就这些, pwm 设备调用了接口函数后, 最终就会调用 pwm_chip 中的操作函数, 接下来我们再看 pwm_chip。

16.2.1 pwm_chip

pwm_chip 关在作用在于, 它提供了 pwm_device 接口函数对应的操作函数集, pwm_chip 结构体定义在 include/linux/pwm.h 中, 如下:

```
struct pwm_chip {
    struct device          *dev;
    struct list_head       list;
    const struct pwm_ops   *ops;
    int                    base;
    unsigned int            npwm;
    struct pwm_device       *pwms;
    struct pwm_device *    (*of_xlate)(struct pwm_chip *pc, const struct
of_phandle_args *args);
    unsigned int            of_pwm_n_cells;
    bool                    can_sleep;
};
```

dev 为 pwm chip 对应的设备, 一般由 pwm driver 对应的 platform 驱动指定且必须提供。

ops 为 pwm_device 中接口函数对应的操作函数集, 必须提供。

npwm 为 pwm chip 可以支持的 pwm_device 个数，必须提供。

pwm_s 为该 pwm chip 中 pwm_device 的数组，kernel 会自行分配，不需要手动设置。

重点介绍一下 struct pwm_ops 结构体指定 ops。struct pwm_ops 定义如下：

```
struct pwm_ops {
    int (*request)(struct pwm_chip *chip, struct pwm_device *pwm);
    void (*free)(struct pwm_chip *chip, struct pwm_device *pwm);
    int (*config)(struct pwm_chip *chip, struct pwm_device *pwm, int duty_ns, int
period_ns);
    int (*set_polarity)(struct pwm_chip *chip, struct pwm_device *pwm, enum
pwm_polarity polarity);
    int (*enable)(struct pwm_chip *chip, struct pwm_device *pwm);
    void (*disable)(struct pwm_chip *chip, struct pwm_device *pwm);
#ifdef CONFIG_DEBUG_FS
    void (*dbg_show)(struct pwm_chip *chip, struct seq_file *s);
#endif
    struct module *owner;
};
```

request 和 free 函数已经弃用了。

函数与 pwm_device 接口函数中的 pwm_config 函数相对应，配置 pwm_device 的频率、占空比。必须提供。

enable、disable 函数与 pwm_device 接口函数中的 pwm_enable、pwm_disable 函数相对应，用于使能/禁止 pwm 信号输出。必须提供。

set_polarity 函数与 pwm_device 接口函数中的 pwm_set_polarity 函数对应，设置 pwm 信号的极性。可选。

定义一个 pwm_chip 之后，需要实现 ops 中至少 config、enable、disable 三个函数。初始化完成后，使用下面的函数向内核注册 pwm_chip：

```
int pwmchip_add(struct pwm_chip *chip);
```

相对的注销使用函数：

```
int pwmchip_remove(struct pwm_chip *chip);
```

在实现 config 等函数时，需要通过设备树获取设备信息，pwm_chip 在设备树中的节点格式并不固定，添加相关信息即可。如下：

```
pwm: pwm@43C20000 {
    compatible = "test-pwm";
    reg = <0x43C20000 0x100>;
};
```

上面再 pwm_device 的节点中有对 pwm_chip 的引用 "pwm_s = <&pwm 0 5000000 PWM_POLARITY_INVERTED>;"。这里的 &pwm，实际上就是引用如 "pwm: pwm@43C20000" 这样的 pwm_chip 节点。引用的目的也就是把 device 和 chip 相关联。

专用的 pwm 芯片厂家会提供 pwm_chip 的驱动，但是 xilinx 没有提供，也许是考虑到自定义 ip 的多样性，因此在 zynq 平台上使用 pwm 框架，还需要驱动开发人员去实现 pwm_chip，比较麻烦。

16.2.1 示例

设备树：

```

1.  pwm: pwm@43C20000 {
2.      compatible = "alinx-pwm";
3.      reg = <0x43C20000 1>;
4.      #pwm-cells = <2>;
5.  };
6.
7.  pwm-led {
8.      compatible = "pwm-led";
9.      pwms = <&pwm 0 5000000>;
10. };

```

pwm_chip:

结合 platform 框架，从设备树中获取信息，注册 pwm_chip。

```

1.  struct ax_pwm_chip {
2.      struct pwm_chip chip;
3.  };
4.
5.  struct ax_pwm_chip ax_pwm;
6.
7.  static int ax_pwm_config(struct pwm_chip *chip, struct pwm_device *pwm, int duty_ns, int period_ns)
8.  {
9.      return 0;
10. }
11.
12. static int ax_pwm_enable(struct pwm_chip *chip, struct pwm_device *pwm)
13. {
14.     return 0;
15. }
16.
17. static void ax_pwm_disable(struct pwm_chip *chip, struct pwm_device *pwm)
18. {
19.
20. }
21.
22. static const struct pwm_ops ax_pwm_ops = {
23.     .owner = THIS_MODULE,
24.     .config = ax_pwm_config,
25.     .enable = ax_pwm_enable,
26.     .disable = ax_pwm_disable,
27. };
28.
29. static int ax_pwm_probe(struct platform_device *pdev)
30. {
31.     int err;
32.
33.     ax_pwm.chip.dev = &pdev->dev;
34.     ax_pwm.chip.ops = &ax_pwm_ops;
35.     ax_pwm.chip.npwm = 1;
36.
37.     err = pwmchip_add(&ax_pwm.chip);
38.     if (err < 0)
39.     {
40.         return err;
41.     }
42.
43.     return 0;
44. }
45.
46. static int ax_pwm_remove(struct platform_device *pdev)
47. {
48.     int err;
49.
50.     err = pwmchip_remove(&ax_pwm.chip);
51.     if (err < 0)
52.     {
53.         return err;
54.     }
55.
56.     return 0;
57. }
58.
59. static const struct of_device_id of_ax_pwm_match[] = {
60.     { .compatible = "alinx-pwm", },
61.     { /* Sentinel */ },
62. };
63.
64. static struct platform_driver ax_pwm_driver = {
65.     .driver = {
66.         .name = "alinx-pwm",
67.         .of_match_table = of_ax_pwm_match,

```

```
68.     },
69.     .probe = ax_pwm_probe,
70.     .remove = ax_pwm_remove,
71. };
72.
73. module_platform_driver(ax_pwm_driver);
```

pwm_device:

结合 misc 设备框架，注册 pwm_device。

```
1.  #define PWM_ON   0x100001
2.  #define PWM_OFF  0x100002
3.
4.  struct pwm_device *ax_pwm_dev;
5.
6.  static long ax_pwm_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
7.  {
8.      int ret;
9.      switch(cmd) {
10.         case PWM_ON:
11.             ret = pwm_config(ax_pwm_dev, 200000, 500000);
12.             pwm_enable(ax_pwm_dev);
13.             break;
14.
15.         case PWM_OFF:
16.             ret = pwm_config(ax_pwm_dev, 0, 500000);
17.             pwm_disable(ax_pwm_dev);
18.             break;
19.     }
20.     return 0;
21. }
22.
23. static struct file_operations ax_pwm_fops = {
24.     .owner = THIS_MODULE,
25.     .unlocked_ioctl = ax_pwm_ioctl,
26. };
27.
28. static struct miscdevice pwm_misc = {
29.     .minor = MISC_DYNAMIC_MINOR,
30.     .name = "ax-pwm",
31.     .fops = &ax_pwm_fops
32. };
33. static int ax_pwm_init(void)
34. {
35.     int ret;
36.     struct device_node *nd;
37.
38.     nd = of_find_node_by_path("/pwm-led");
39.
40.     ax_pwm_dev = of_pwm_get(nd, "pwm-led");
41.
42.     misc_register(&pwm_misc);
43.     return 0;
44. }
45.
46. static void ax_pwm_exit(void)
47. {
48.     pwm_put(ax_pwm_dev);
49. }
50. module_init(ax_pwm_init);
51. module_exit(ax_pwm_exit);
```

16.3 实验

本章实验结合 platform 和 misc 框架，实现简单的 led 呼吸灯。

16.3.1 原理图

和第一章的原理图相同。

16.3.2 设备树

打开 system-user.dtsi 文件，在根节点中添加下面的节点：

```
1. alinxpwm {
2.     compatible = "alinx-pwm";
3.     reg-freq = <0x43C20000 1>;
4.     reg-duty = <0x43C20004 1>;
5. };
```

compatible 节点兼容性为“alinx-pwm”，之后的驱动代码中 platform 的 of 匹配表需要和这个属性一致。

reg-freq 和 reg-duty 分别的频率和占空比的物理地址，从 vivado 中获得。

16.3.3 驱动程序

使用 petalinux 新建名为“ax-pwm”的驱动程序，并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-pwm.c 文件中输入下面的代码：

```
1. #include <linux/types.h>
2. #include <linux/kernel.h>
3. #include <linux/delay.h>
4. #include <linux/init.h>
5. #include <linux/module.h>
6. #include <linux/errno.h>
7. #include <linux/of.h>
8. #include <linux/cdev.h>
9. #include <linux/device.h>
10. #include <linux/semaphore.h>
11. #include <linux/timer.h>
12. #include <linux/irq.h>
13. #include <linux/wait.h>
14. #include <linux/poll.h>
15. #include <linux/fs.h>
16. #include <linux/fcntl.h>
17. #include <linux/platform_device.h>
18. #include <linux/miscdevice.h>
19. #include <asm/mach/map.h>
20. #include <asm/uaccess.h>
21.
22. /* 设备节点名称 */
23. #define DEVICE_NAME      "ax_pwm"
24. /* 设备号个数 */
25. #define DEVID_COUNT      1
26. /* 驱动个数 */
27. #define DRIVE_COUNT      1
28. /* 主设备号 */
29. #define MAJOR_AX
30. /* 次设备号 */
31. #define MINOR_AX
32. /* 设置频率 */
33. #define PWM_FREQ         0x100001
34. /* 设置占空比 */
35. #define PWM_DUTY         0x100002
36.
37. /* 把驱动代码中会用的数据打包进设备结构体 */
38. struct alinx_char_dev{
39.     dev_t      devid;      //设备号
40.     struct cdev cdev;      //字符设备
41.     struct device_node *nd; //设备树的设备节点
42.     unsigned int *freq;    //频率的寄存器虚拟地址
43.     unsigned int *duty;    //占空比的寄存器虚拟地址
44. };
45. /* 声明设备结构体 */
46. static struct alinx_char_dev alinx_char = {
47.     .cdev = {
48.         .owner = THIS_MODULE,
49.     },
50. };
51.
52. /* open 函数实现，对应到 Linux 系统调用函数的 open 函数 */
53. static int ax_pwm_open(struct inode *inode_p, struct file *file_p)
54. {
55.     /* 设置私有数据 */
56.     file_p->private_data = &alinx_char;
57.     return 0;
58. }
```

```
60.
61. /* ioctl 函数实现, 对应到 ioctl 系统调用函数的 write 函数 */
62. static long ax_pwm_ioctl(struct file *file_p, unsigned int cmd, unsigned long arg)
63. {
64.     /* 获取私有数据 */
65.     struct alinx_char_dev *dev = file_p->private_data;
66.
67.     switch(cmd)
68.     {
69.         case PWM_FREQ:
70.             {
71.                 *(dev->freq) = (unsigned int)arg;
72.                 break;
73.             }
74.
75.         case PWM_DUTY:
76.             {
77.                 *(dev->duty) = (unsigned int)arg;
78.                 break;
79.             }
80.
81.         default :
82.             {
83.                 break;
84.             }
85.     }
86.
87.     return 0;
88. }
89.
90. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
91. static int ax_pwm_release(struct inode *inode_p, struct file *file_p)
92. {
93.     return 0;
94. }
95.
96. /* file_operations 结构体声明 */
97. static struct file_operations ax_char_fops = {
98.     .owner          = THIS_MODULE,
99.     .open           = ax_pwm_open,
100.    .unlocked_ioctl = ax_pwm_ioctl,
101.    .release         = ax_pwm_release,
102. };
103.
104. /* MISC 设备结构体 */
105. static struct miscdevice led_miscdev = {
106.     /* 自动分配设备号 */
107.     .minor = MISC_DYNAMIC_MINOR,
108.     .name = DEVICE_NAME,
109.     /* file_operations 结构体 */
110.     .fops = &ax_char_fops,
111. };
112.
113. /* probe 函数实现, 驱动和设备匹配时会被调用 */
114. static int ax_pwm_probe(struct platform_device *dev)
115. {
116.     /* 用于接受返回值 */
117.     u32 ret = 0;
118.     /* 频率的寄存器物理地址 */
119.     int freq_addr;
120.     /* 占空比的寄存器物理地址 */
121.     int duty_addr;
122.
123.     /* 获取设备节点 */
124.     alinx_char.nd = of_find_node_by_path("/alinx_pwm");
125.     if(alinx_char.nd == NULL)
126.     {
127.         printk("gpioled node not found\n");
128.         return -EINVAL;
129.     }
130.
131.     /* 获取寄存器中 freq 的地址 */
132.     of_property_read_u32(alinx_char.nd, "reg-freq", &freq_addr);
133.     if(!freq_addr) {
134.         printk("can not get reg-freq\n");
135.         return -EINVAL;
136.     }
137.     else
138.     {
139.         /* 映射地址 */
140.         alinx_char.freq = ioremap(freq_addr, 4);
141.     }
142. }
```

```

143.  /* 获取寄存器中 duty 的地址 */
144.  of_property_read_u32(alinx_char.nd, "reg-duty", &duty_addr);
145.  if(!duty_addr) {
146.      printk("can not get reg-duty\r\n");
147.      iounmap((unsigned int *)alinx_char.freq);
148.      return -EINVAL;
149.  }
150.  else
151.  {
152.      /* 映射地址 */
153.      alinx_char.duty = ioremap(duty_addr, 4);
154.  }
155.
156.  /* 注册 misc 设备 */
157.  ret = misc_register(&led_miscdev);
158.  if(ret < 0) {
159.      printk("misc device register failed\r\n");
160.      return -EFAULT;
161.  }
162.
163.  return 0;
164. }
165.
166. static int ax_pwm_remove(struct platform_device *dev)
167. {
168.     /* 释放虚拟地址 */
169.     iounmap((unsigned int *)alinx_char.freq);
170.     iounmap((unsigned int *)alinx_char.duty);
171.     /* 注销 misc 设备 */
172.     misc_deregister(&led_miscdev);
173.     return 0;
174. }
175.
176. /* 初始化 of_match_table */
177. static const struct of_device_id pwm_of_match[] = {
178.     /* compatible 字段和设备树中保持一致 */
179.     { .compatible = "alinx-pwm" },
180.     /* Sentinel */
181. };
182.
183.
184. /* 声明并初始化 platform 驱动 */
185. static struct platform_driver pwm_driver = {
186.     .driver = {
187.         /* name 字段需要保留 */
188.         .name = "alinx-pwm",
189.         /* 用 of_match_table 代替 name 匹配 */
190.         .of_match_table = pwm_of_match,
191.     },
192.     .probe = ax_pwm_probe,
193.     .remove = ax_pwm_remove,
194. };
195.
196. /* 驱动入口函数 */
197. static int __init pwm_drv_init(void)
198. {
199.     /* 在入口函数中调用 platform_driver_register, 注册 platform 驱动 */
200.     return platform_driver_register(&pwm_driver);
201. }
202.
203. /* 驱动出口函数 */
204. static void __exit pwm_drv_exit(void)
205. {
206.     /* 在出口函数中调用 platform_driver_unregister, 卸载 platform 驱动 */
207.     platform_driver_unregister(&pwm_driver);
208. }
209.
210. /* 标记加载、卸载函数 */
211. module_init(pwm_drv_init);
212. module_exit(pwm_drv_exit);
213.
214. /* 驱动描述信息 */
215. MODULE_AUTHOR("Alinx");
216. MODULE_ALIAS("pwm_led");
217. MODULE_DESCRIPTION("PWM LED driver");
218. MODULE_VERSION("v1.0");
219. MODULE_LICENSE("GPL");

```

platform 和 misc 框架的部分可以参考第十四章，几乎没变。

pwm 的处理也很简单，从设备树中获取地址，然后再操作函数中操作地址。

33~36 行定义了两个宏，用于 ioctl 的 cmd，分别表示设置频率和设置占空比。

43 行的 freq 用于从设备树中获取设置频率寄存器的地址。

44 行的 freq 用于从设备树中获取设置占空比寄存器的地址。

16.3.4 测试程序

新建 QT 工程名为“ax-pwm-test”，新建 main.c，输入下面的代码：

```
1. #include "stdio.h"
2. #include "unistd.h"
3. #include "sys/types.h"
4. #include "sys/stat.h"
5. #include "fcntl.h"
6. #include "stdlib.h"
7. #include "string.h"
8. #include "sys/ioctl.h"
9.
10. #define FREQ_DEFAULT 1717900
11. /* 设置频率 */
12. #define PWM_FREQ 0x100001
13. /* 设置占空比 */
14. #define PWM_DUTY 0x100002
15.
16. int main(int argc, char *argv[])
17. {
18.     int fd, retvalue, flag = 0;
19.     char *filename;
20.     unsigned int duty = 0xffffffff;
21.
22.     if(argc != 2)
23.     {
24.         printf("Error Usage\r\n");
25.         return -1;
26.     }
27.
28.     filename = argv[1];
29.     fd = open(filename, O_RDWR);
30.     if(fd < 0)
31.     {
32.         printf("file %s open failed\r\n", argv[1]);
33.         return -1;
34.     }
35.
36.     /* 设置频率 */
37.     retvalue = ioctl(fd, PWM_FREQ, FREQ_DEFAULT);
38.     if(retvalue < 0)
39.     {
40.         printf("pwm Failed\r\n");
41.         close(fd);
42.         return -1;
43.     }
44.
45.     while(1)
46.     {
47.         if(duty <= 0xffffffff && 0 == flag)
48.         {
49.             duty += 2000000;
50.         }
51.         else if(duty >= 0xffffffff)
52.         {
53.             duty -= 500000;
54.             flag = 1;
55.         }
56.         else
57.         {
58.             flag = 0;
59.         }
60.
61.         /* 设置占空比 */
62.         retvalue = ioctl(fd, PWM_DUTY, duty);
63.         if(retvalue < 0)
64.         {
65.             printf("pwm Failed\r\n");
66.             close(fd);
67.             return -1;
68.         }
69.         usleep(5);
70.     }
```

```
71.  
72.  
73.     return 0;  
74. }
```

16.3.5 运行测试

测试方法步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt  
cd /mnt  
mkdir /tmp/qt  
mount qt_lib.img /tmp/qt  
cd /tmp/qt  
source ./qt_env_set.sh  
cd /mnt  
insmod ax-pwm.ko  
cd ./build-ax-pwm-test-ZYNQ-Debug  
./ax-pwm-test /dev/ax_pwm
```

IP 和路径根据实际情况调整。

串口工具中的调试结果如下：

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt  
root@ax_peta:~# cd /mnt  
root@ax_peta:/mnt# mkdir /tmp/qt  
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt  
random: fast init done  
EXT4-fs (loop0): recovery complete  
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)  
root@ax_peta:/mnt# cd /tmp/qt  
root@ax_peta:/tmp/qt# source ./qt_env_set.sh  
/tmp/qt  
root@ax_peta:/tmp/qt# cd /mnt  
root@ax_peta:/mnt# insmod ax-pwm.ko  
ax_pwm: loading out-of-tree module taints kernel.  
[drm] load() is deferred & will be called again  
root@ax_peta:/mnt# cd ./build-ax-pwm-test-ZYNQ-Debug  
root@ax_peta:/mnt/build-ax-pwm-test-ZYNQ-Debug# ./ax-pwm-test /dev/ax_pwm
```

开发板上的 PL_LED4 开始呼吸灯式的和闪烁。

第十七章 I2C 驱动

I2C 是常用的串行接口总线，Linux 内核中也提供了完善的 I2C 驱动框架。本章就来了解一下 Linux 内核中提供的 I2C 驱动框架。

17.1 I2C 驱动框架

在 platform 驱动框架和 pwm 驱动框架中，都提到过驱动的分离，也就是控制器或总线和设备的分离。I2C 也是相似的结构，分为 I2C 总线和 I2C 设备。总线是芯片本身的 I2C 资源，而设备则是 I2C 外接的用户设备如 RTC、EEPROM 等。我们分别来看他们的实现方法。

17.1.1 I2C 控制器驱动

内核中使用结构体 `i2c_adapter` 来表示 I2C 控制器，`i2c_adapter` 结构体定义在文件 `include/linux/i2c.h` 中。如下：

```
546. struct i2c_adapter {
547.     struct module *owner;
548.     unsigned int class; /* classes to allow probing for */
549.     const struct i2c_algorithm *algo; /* the algorithm to access the bus */
550.     void *algo_data;
551.
552.     /* data fields that are valid for all devices */
553.     const struct i2c_lock_operations *lock_ops;
554.     struct rt_mutex bus_lock;
555.     struct rt_mutex mux_lock;
556.
557.     int timeout; /* in jiffies */
558.     int retries;
559.     struct device dev; /* the adapter device */
560.
561.     int nr;
562.     char name[48];
563.     struct completion dev_released;
564.
565.     struct mutex userspace_clients_lock;
566.     struct list_head userspace_clients;
567.
568.     struct i2c_bus_recovery_info *bus_recovery_info;
569.     const struct i2c_adapter_quirks *quirks;
570.};
```

549 行的 `const struct i2c_algorithm` 型指针成员变量 `algo`，是 I2C 设备访问总线的接口函数的合集，是 I2C 设备和 I2C 控制器通讯的方法。

`i2c_algorithm` 结构体定义如下：

```
407. struct i2c_algorithm {
408.     /* If an adapter algorithm can't do I2C-level access, set master_xfer
409.     to NULL. If an adapter algorithm can do SMBus access, set
410.     smbus_xfer. If set to NULL, the SMBus protocol is simulated
411.     using common I2C messages */
412.     /* master_xfer should return the number of messages successfully
413.     processed, or a negative value on error */
414.     int (*master_xfer)(struct i2c_adapter *adap, struct i2c_msg *msgs,
415.     int num);
416.     int (*smbus_xfer)(struct i2c_adapter *adap, u16 addr,
417.     unsigned short flags, char read_write,
418.     u8 command, int size, union i2c_smbus_data *data);
419.};
```

```

420.  /* To determine what the adapter supports */
421.  u32 (*functionality) (struct i2c_adapter *);
422.
423. #if IS_ENABLED(CONFIG_I2C_SLAVE)
424.  int (*reg_slave)(struct i2c_client *client);
425.  int (*unreg_slave)(struct i2c_client *client);
426. #endif

```

414 行的 master_xfer 函数就是用于与 I2C 设备通讯的函数。

416 行的 smbus_xfer 函数是 smbus 的传输函数。

I2C 总线驱动的实现首先要定义并初始化一个 i2c_adapter 结构体型的变量，初始化包括实现 i2c_algorithm 中的函数，至少要实现 master_xfer 函数。

初始化完成后，使用下面的函数向内核注册 i2c_algorithm：

```

int i2c_add_adapter(struct i2c_adapter *adapter)
int i2c_add_numbered_adapter(struct i2c_adapter *adap)

```

这两个函数的却别的前者使用动态总线号，后者使用静态总线号。输入参数为初始化后的 i2c_add_adapter 结构体变量。返回 0 注册成功，返回负值注册失败。

相对的注销函数为：

```

void i2c_del_adapter(struct i2c_adapter * adap)

```

输入参数 adap 为需要删除的 I2C 控制器。

I2C 控制器的驱动一般由芯片厂家提供，从设备树中找到 I2C 控制器的节点，可以发现节点中的 compatible 属性为 cdns,i2c-r1p10，通过这个 compatible 可以追寻到，使用的 I2C 控制器驱动为 drivers/i2c/busses/i2c-cadence.c。I2C 控制器的驱动相关暂且了解这么多，把重点放在 I2C 的设备驱动中去。

17.1.2 I2C 设备驱动

设备驱动又分层为两个部分，设备 i2c_client 和驱动 i2c_driver。

1) i2c_client

i2c_client 用于描述设备信息，定义如下：

```

228. struct i2c_client {
229.     unsigned short flags;          /* div., see below */
230.     unsigned short addr;          /* chip address - NOTE: 7bit */
231.     /* addresses are stored in the */
232.     /* _LOWER_ 7 bits */
233.     char name[I2C_NAME_SIZE];
234.     struct i2c_adapter *adapter;    /* the adapter we sit on */
235.     struct device dev;             /* the device structure */
236.     int irq;                       /* irq issued by device */
237.     struct list_head detected;
238. #if IS_ENABLED(CONFIG_I2C_SLAVE)
239.     i2c_slave_cb_t slave_cb;      /* callback for slave mode */
240. #endif
241. };

```

230 行的 addr 表示芯片地址，储存在低 7 位。

233 行的 name 表示设备名称。

234 行的 adapter 位设备对应 I2C 控制器。

每一个 I2C 设备对应一个 i2c_client。

2) i2c_driver

i2c_driver 是 Linux 框架中 I2C 处理的重点，定义如下：

```

169. struct i2c_driver {
170.     unsigned int class;
171.
172.     /* Notifies the driver that a new bus has appeared. You should avoid
173.      * using this, it will be removed in a near future.
174.      */
175.     int (*attach_adapter)(struct i2c_adapter *) __deprecated;
176.
177.     /* Standard driver model interfaces */
178.     int (*probe)(struct i2c_client *, const struct i2c_device_id *);
179.     int (*remove)(struct i2c_client *);
180.
181.     /* driver model interfaces that don't relate to enumeration */
182.     void (*shutdown)(struct i2c_client *);
183.
184.     /* Alert callback, for example for the SMBus alert protocol.
185.      * The format and meaning of the data value depends on the protocol.
186.      * For the SMBus alert protocol, there is a single bit of data passed
187.      * as the alert response's low bit ("event flag").
188.      * For the SMBus Host Notify protocol, the data corresponds to the
189.      * 16-bit payload data reported by the slave device acting as master.
190.      */
191.     void (*alert)(struct i2c_client *, enum i2c_alert_protocol protocol,
192.                  unsigned int data);
193.
194.     /* a ioctl like command that can be used to perform specific functions
195.      * with the device.
196.      */
197.     int (*command)(struct i2c_client *client, unsigned int cmd, void *arg);
198.
199.     struct device_driver driver;
200.     const struct i2c_device_id *id_table;
201.
202.     /* Device detection callback for automatic device creation */
203.     int (*detect)(struct i2c_client *, struct i2c_board_info *);
204.     const unsigned short *address_list;
205.     struct list_head clients;
206. };

```

178 行的 probe 函数和 platform 框架中的类似，I2C 设备和驱动匹配成功后就会执行。

199 行的 device_driver 结构体变量 driver 就是用于和设备匹配，和 platform 框架类似，使用设备树的话，需要设置 driver.of_match_table 中的 compatible 属性。

200 行的 id_table 和 platform 框架类似，未使用设备树时使用这个匹配表。

定义并初始化完成 i2c_driver 之后使用下面的函数来向内核注册：

```
int i2c_register_driver(struct module *owner, struct i2c_driver *driver)
```

owner 一般位 THIS_MODULE。

driver 就是需要注册的 i2c_driver。

返回 0 成功，负值失败。

相对的注销函数为：

```
void i2c_del_driver(struct i2c_driver *driver)
```

I2C 设备驱动编写示例：

```

1. static int ax_probe(struct i2c_client *client, const struct i2c_device_id *id)
2. {
3.     return 0;
4. }
5.
6. static int ax_remove(struct i2c_client *client)
7. {
8.     return 0;
9. }
10.

```

```

11. static const struct of_device_id ax_of_match[] =
12. {
13.     { .compatible = "alinx-xxx"},
14.     { /* sentinel */}
15. };
16.
17. static struct i2c_driver ax_driver = {
18.     .driver = {
19.         .owner = THIS_MODULE,
20.         .name = "alinx-xxx",
21.         .of_match_table = ax_of_match,
22.     },
23.     .probe = ax_probe,
24.     .remove = ax_remove,
25. };
26.
27.
28. static int ax_init(void)
29. {
30.     i2c_add_driver(&ax_driver);
31.     return 0;
32. }
33.
34. static void ax_exit(void)
35. {
36.     i2c_del_driver(&ax_driver);
37. }
38.
39. module_init(ax_init);
40. module_exit(ax_exit);

```

17.1.3 I2C 设备驱动的实现流程

1) 设备树

首先要在对应的 I2C 节点中添加设备节点，如下：

```

1. i2c0: i2c@e0004000 {
2.     compatible = "cdns,i2c-r1p10";
3.     clocks = <&clk 38>;
4.     interrupt-parent = <&intc>;
5.     interrupts = <0 25 4>;
6.     reg = <0xe0004000 0x1000>;
7.     #address-cells = <1>;
8.     #size-cells = <0>;
9. };
10. ....
11. &i2c0 {
12.     axrtc@68 {
13.         compatible = "alinx-rtc";
14.         status = "okay";
15.         reg = <0x68>;
16.     };
17. };

```

第一行的 i2c0 是控制器的节点。第 11 行引用控制器节点&i2c, 并在里面添加设备节点 axrtc。

设备节点名称后面@接的是设备的地址 68。设备节点的关键属性有两个。compatible 兼容性用于和设备驱动相匹配。reg 和节点名@后面的值相同，都是设备地址。

设备树配置好后就可以仔 probe 函数中获取设备地址。

2) 数据收发

I2C 的手法通过内核中的 i2c_transfer 函数实现，这个函数最终会调用 i2c 控制器驱动中的 master_xfer 函数。

i2c_transfer 函数定义在 include/linux/i2c.h 中，原型如下：

```
int i2c_transfer(struct i2c_adapter *adap, struct i2c_msg *msgs, int num)
```

参数 adap 可以在 probe 函数中获取，probe 函数被调用时，第一个输入参数为设备树中对应节点的 struct i2c_client *client，client 中的 adap 也就是对应的控制器。

参数 msgs 是需要发送的数据。

参数 num 为需要发送 msgs 数量。

返回负值失败，返回非负值为 msgs 发送的数量。

msgs 的为 struct i2c_msg 类型的指针，struct i2c_msg 定义在 include/uapi/linux/i2c.h 中如下：

```
1. struct i2c_msg {
2.     __u16 addr; /* slave address */
3.     __u16 flags;
4.     #define I2C_M_RD      0x0001 /* read data, from slave to master */
5.     /* I2C_M_RD is guaranteed to be 0x0001! */
6.     #define I2C_M_TEN     0x0010 /* this is a ten bit chip address */
7.     #define I2C_M_RECV_LEN 0x0400 /* length will be first received byte */
8.     #define I2C_M_NO_RD_ACK 0x0800 /* if I2C_FUNC_PROTOCOL_MANGLING */
9.     #define I2C_M_IGNORE_NAK 0x1000 /* if I2C_FUNC_PROTOCOL_MANGLING */
10.    #define I2C_M_REV_DIR_ADDR 0x2000 /* if I2C_FUNC_PROTOCOL_MANGLING */
11.    #define I2C_M_NOSTART    0x4000 /* if I2C_FUNC_NOSTART */
12.    #define I2C_M_STOP      0x8000 /* if I2C_FUNC_PROTOCOL_MANGLING */
13.    __u16 len; /* msg length */
14.    __u8 *buf; /* pointer to msg data */
15. };
```

根据 flags 的值，i2c_transfer 函数执行不同的工作，包括读写。

调用 i2c_transfer 函数之前，需要先构建 struct i2c_msg 变量，如下：

```
1. static int ax_read_regs(struct axrtc_dev *dev, u8 reg, void *val, int len)
2. {
3.     int ret;
4.     struct i2c_msg msg[2];
5.     struct i2c_client *client = (struct i2c_client *)dev->private_data;
6.
7.     msg[0].addr = client->addr;
8.     msg[0].flags = 0;
9.     msg[0].buf = &reg;
10.    msg[0].len = 1;
11.
12.    msg[1].addr = client->addr;
13.    msg[1].flags = I2C_M_RD;
14.    msg[1].buf = val;
15.    msg[1].len = len;
16.
17.    ret = i2c_transfer(client->adapter, msg, 2);
18.    if(2 == ret)
19.    {
20.        ret = 0;
21.    }
22.    else
23.    {
24.        printk("i2c read failed %d", ret);
25.        ret = -EREMOTEIO;
26.    }
27.
28.    return ret;
29. }
```

第 4 行定义了 struct i2c_msg 类型的数组 msg。

这里的 client 获取了 dev->private_data 中的值，dev->private_data 是仔 probe 中设置的私有

变量，具体到后面的实验中再去分析，先看 msg 的构造。

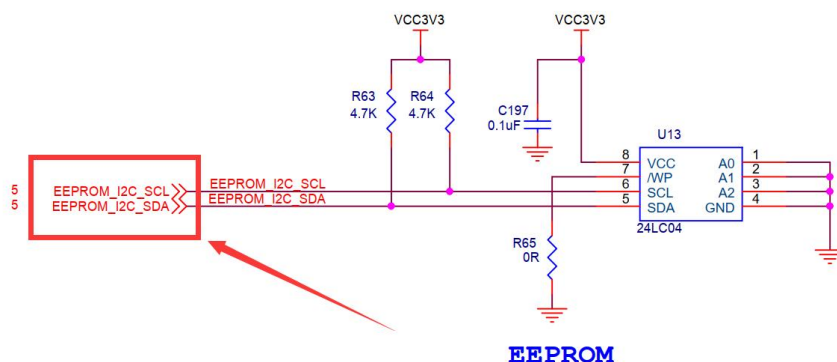
7~10 行构建 msg[0]，addr 为设备地址值，使用 client 中的 addr 即可。flags 赋为 0 时为写。flags 为写时 buf 的值为写入的数据的首地址。len 为写入数据的长度。

12~15 行构建 msg[1]，flag 等于 I2C_M_RD 为读数据，此时 buf 为储存读出数据 buffer 的首地址。len 为读出数据的长度。

17.3 实验

本章的实验使用 I2C 来做一个开发板上的 eeprom 数据读写实验。主要目标是验证 i2c 框架的使用是否正确，先往 eeprom 中写入数据再读出，两次数据一致则成功。

17.3.1 原理图

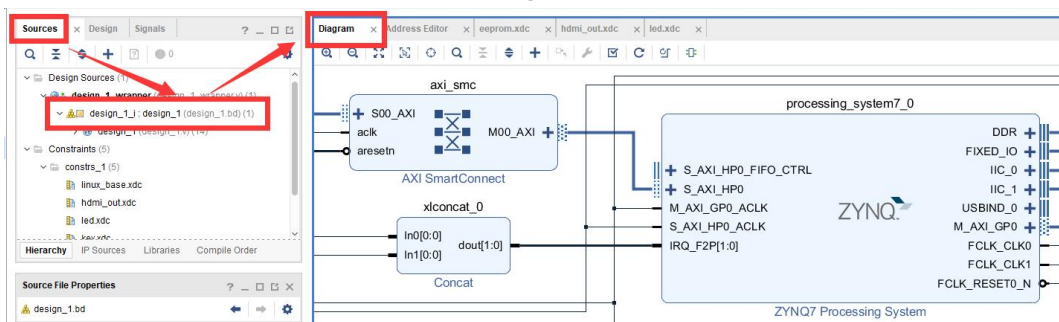


找到原理图中 eeprom 的位置，看看 scl 和 sda 两根线接在哪里。

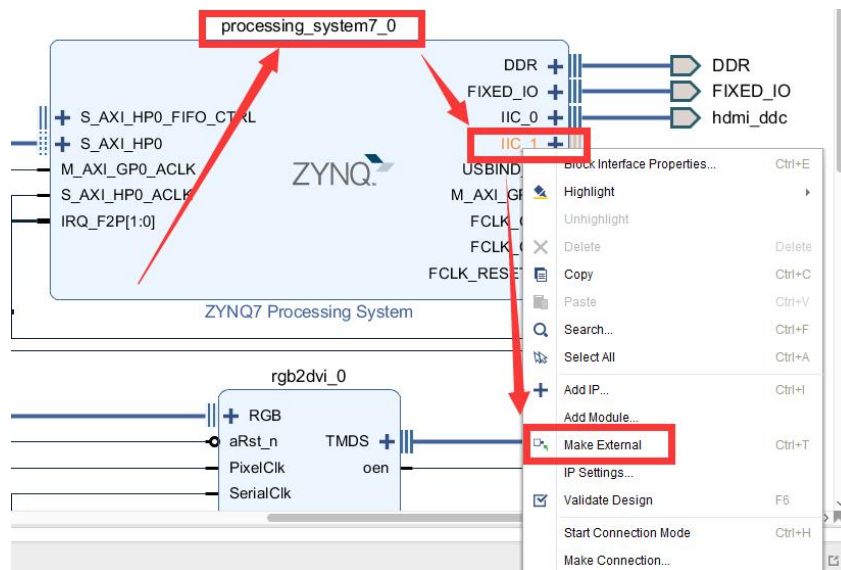


分别接在 pl 端的 T19 引脚和 U19 引脚上。使用 pl 端的资源，需要借助 vivado 工具来配置。

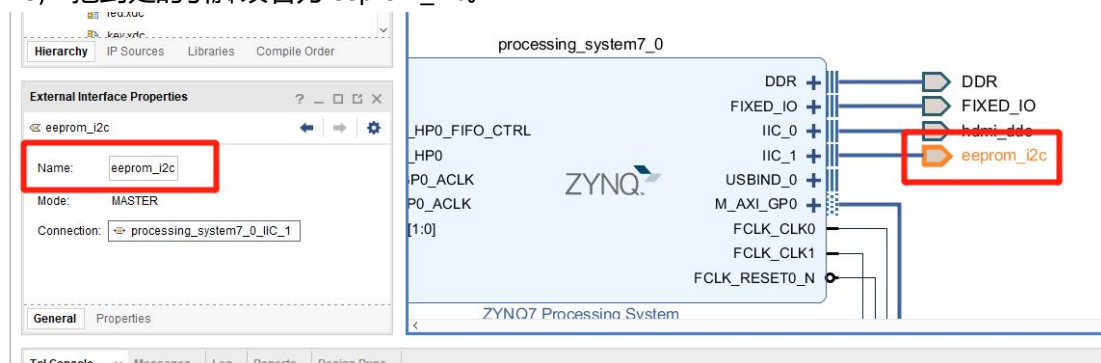
1) 打开之前使用的 vivado 工程。打开 Diagram 界面。



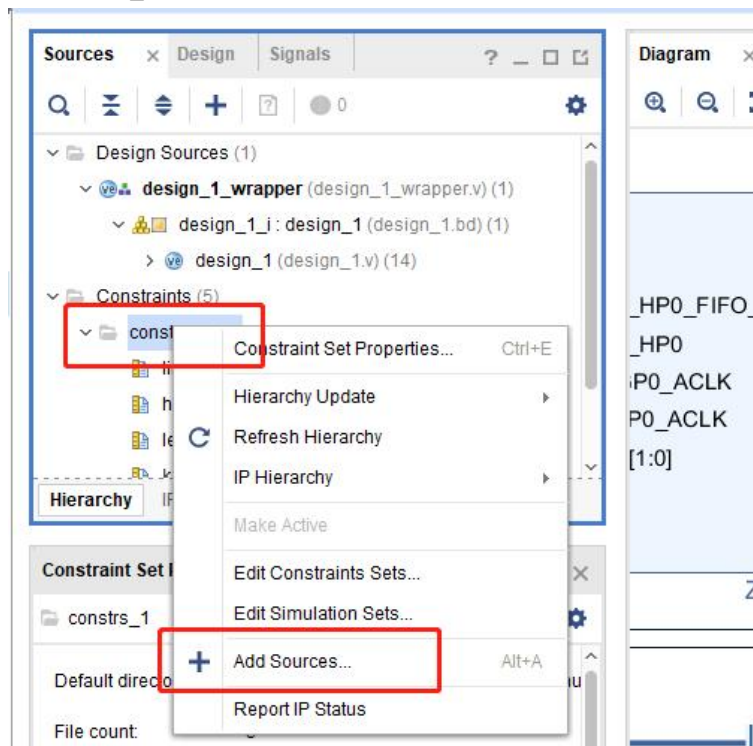
2) 在 processing_system7_0 的 IIC1 引脚上右击，选择 Make External 导出引脚。



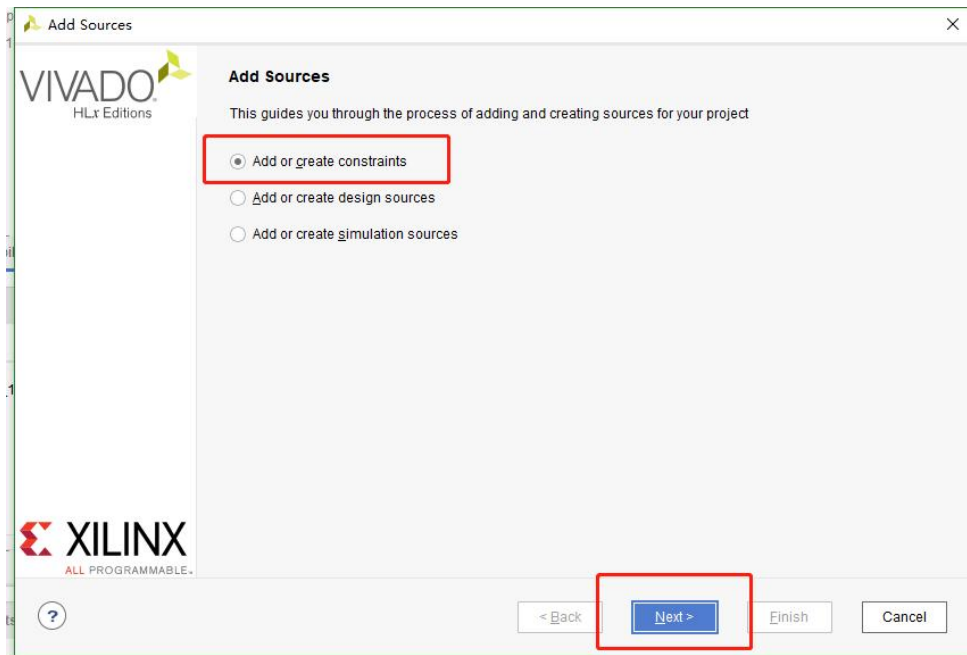
3) 把到处的引脚改名为 eeprom_i2c。



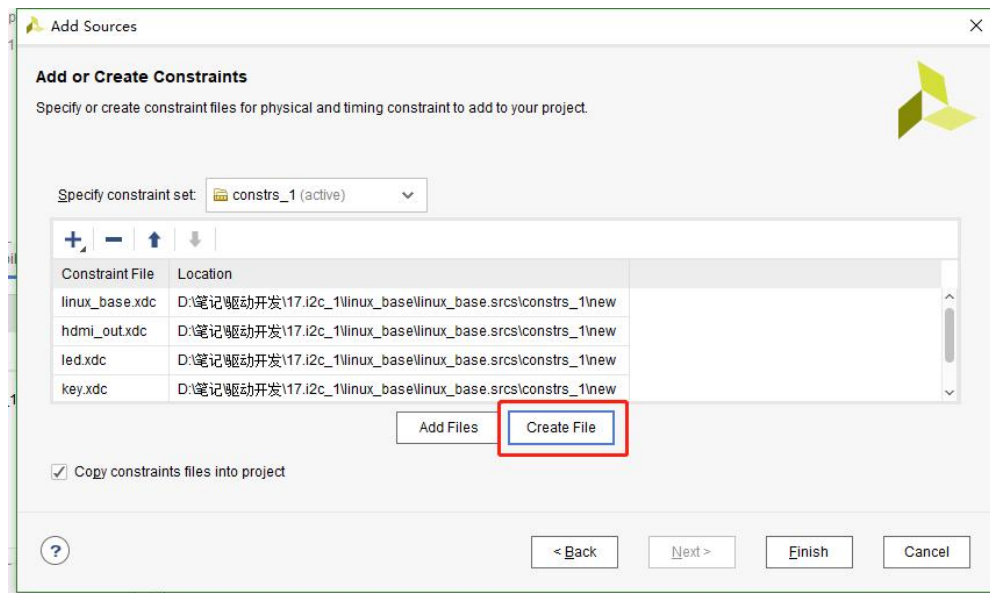
4) 关键的一步，添加新的 xdc 约束管脚，使 IIC_1 导出的引脚与原图上连接 eeprom 的引脚对应。在 constrs_1 上右击，选择 Add Sources。



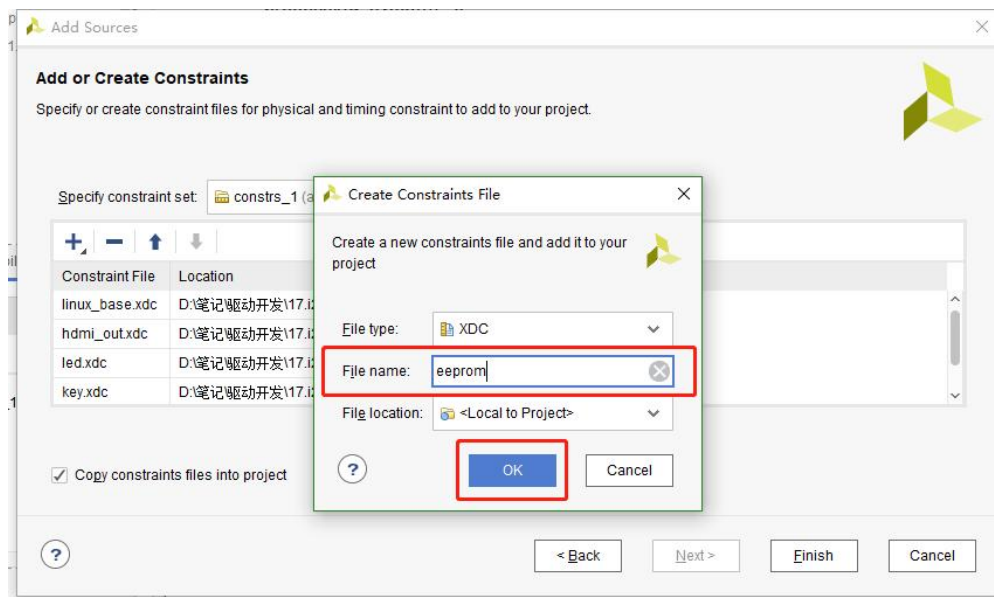
弹出的对话框中直接点击 Next。



下一个界面点击 Create File。



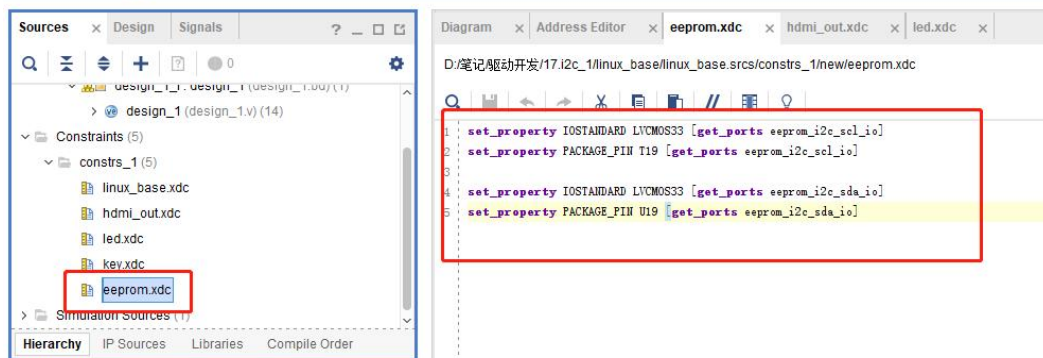
命名为 eeprom，点击 OK。



然后点击 finish，之后在 constrs_1 中就有了刚才新建的文件。打开这个文件添加下面的代码：

```
set_property IOSTANDARD LVCMOS33 [get_ports eeeprom_i2c_scl_io]
set_property PACKAGE_PIN T19 [get_ports eeeprom_i2c_scl_io]

set_property IOSTANDARD LVCMOS33 [get_ports eeeprom_i2c_sda_io]
set_property PACKAGE_PIN U19 [get_ports eeeprom_i2c_sda_io]
```



这样 pl 端就配置好了，重新编译导出硬件信息，再使用 petalinux 重新制作 linux 系统，这个过程就不再重复说了。

eeeprom 的使用可以参考 24AA04/24LC04B。

17.3.2 设备树

打开 system-user.dtsi 文件，在根目录外添加下面的节点：

```
1. &i2c1 {
2.     clock-frequency = <100000>;
3.
4.     ax-e2p1@50 {
5.         compatible = "ax-e2p1";
6.         reg = <0x50>;
7.     };
8. };
```

第 1 行应用 i2c1 节点，因为上面我们 vivado 中配置引脚时，把 eeeprom 的引脚约束到 i2c1

上了，所以设备树中需要对应的添加在 i2c_1 节点中。

第 2 行设置 i2c_1 的时钟为 100hz。

第 4 行添加 eeprom 的节点 ax-e2p1@50，设备地址为 50。添加 compatible 属性等于“x-e2p1”，用于和驱动匹配。添加 reg 属性等于设备地址 0x50。

17.3.3 驱动程序

使用 petalinux 新建名为“ax-i2c”驱动程序，并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-i2c.c 文件中输入下面的代码：

```
1. #include <linux/types.h>
2. #include <linux/kernel.h>
3. #include <linux/delay.h>
4. #include <linux/init.h>
5. #include <linux/module.h>
6. #include <linux/errno.h>
7. #include <linux/cdev.h>
8. #include <linux/device.h>
9. #include <linux/string.h>
10. #include <linux/timer.h>
11. #include <linux/i2c.h>
12. #include <linux/irq.h>
13. #include <linux/wait.h>
14. #include <linux/poll.h>
15. #include <linux/fs.h>
16. #include <linux/fcntl.h>
17. #include <linux/platform_device.h>
18. #include <asm/mach/map.h>
19. #include <asm/uaccess.h>
20. #include <asm/io.h>
21.
22. /* 驱动个数 */
23. #define AX_I2C_CNT 1
24. /* 设备节点名称 */
25. #define AX_I2C_NAME "ax_i2c_e2p"
26.
27. struct ax_i2c_dev {
28.     dev_t devid;           //设备号
29.     struct cdev cdev;      //字符设备
30.     struct class *class;    //类
31.     struct device *device;  //设备
32.     int major;             //主设备号
33.     void *private_data;     //用于在 probe 函数中获取 client
34. };
35. /* 声明设备结构体变量 */
36. struct ax_i2c_dev axi2cdev;
37.
38. /* i2c 数据读取
39.  * struct ax_i2c_dev *dev : 设备结构体
40.  * u8 reg : 数据在目标设备中的地址
41.  * void *val : 数据 buffer 首地址
42.  * int len : 数据长度
43.  */
44. static int ax_i2c_read_regs(struct ax_i2c_dev *dev, u8 reg, void *val, int len)
45. {
46.     int ret;
47.     /* 构建 msg，读取时一般使用一个至少两个元素的 msg 数组
48.      * 第一个元素用于发送目标数据地址(写)，第二个元素发送 buffer 地址(读) */
49.     struct i2c_msg msg[2];
50.     /* 从设备结构体变量中获取 client 数据 */
51.     struct i2c_client *client = (struct i2c_client *)dev->private_data;
52.
53.     /* 构造 msg */
54.     msg[0].addr = client->addr;    //设置设备地址
```

```
55.     msg[0].flags = 0;                //标记为写，先给 eeprom 发送读取数据的所在地址
56.     msg[0].buf = 0;                 //读取数据的所在地址
57.     msg[0].len = 1;                 //地址数据长度，只发送首地址的话长度就为 1
58.
59.     msg[1].addr = client->addr;      //设置设备地址
60.     msg[1].flags = I2C_M_RD;        //标记为读
61.     msg[1].buf = val;               //数据读出的 buffer 地址
62.     msg[1].len = len;               //读取数据长度
63.
64.     /* 调用 i2c_transfer 发送 msg */
65.     ret = i2c_transfer(client->adapter, msg, 2);
66.     if(2 == ret)
67.     {
68.         ret = 0;
69.     }
70.     else
71.     {
72.         printk("i2c read failed %d\r\n", ret);
73.         ret = -EREMOTEIO;
74.     }
75.
76.     return ret;
77. }
78.
79. /* i2c 数据写入
80.  * struct ax_i2c_dev *dev : 设备结构体
81.  * u8 reg                 : 数据在目标设备中的地址
82.  * void *val              : 数据 buffer 首地址
83.  * int len                : 数据长度
84.  */
85. static s32 ax_i2c_write_regs(struct ax_i2c_dev *dev, u8 reg, u8 *buf, int len)
86. {
87.     int ret;
88.     /* 数据 buffer */
89.     u8 b[100] = {0};
90.     /* 构建 msg */
91.     struct i2c_msg msg;
92.     /* 从设备结构体变量中获取 client 数据 */
93.     struct i2c_client *client = (struct i2c_client *)dev->private_data;
94.
95.     /* 把写入目标地址放在 buffer 的第一个元素中首先发送 */
96.     b[0] = reg;
97.     /* 把需要发送的数据拷贝到随后的地址中 */
98.     memcpy(&b[1], buf, 100 > len ? len : 100);
99.
100.    /* 构建 msg */
101.    msg.addr = client->addr;          //设置设备地址
102.    msg.flags = 0;                   //标记为写
103.    msg.buf = b;                     //数据写入的 buffer 地址
104.    msg.len = len + 1;               //写入的数据长度，因为除了用户数据外，
105.                                     //还需要发送数据地址所以要+1
106.
107.    /* 调用 i2c_transfer 发送 msg */
108.    ret = i2c_transfer(client->adapter, &msg, 1);
109.
110.    if(1 == ret)
111.    {
112.        ret = 0;
113.    }
114.    else
115.    {
116.        printk("i2c write failed %d\r\n", ret);
117.        ret = -EREMOTEIO;
118.    }
119.    return ret;
120. }
121.
122. /* open 函数实现，对应到 Linux 系统调用函数的 open 函数 */
123. static int ax_i2c_open(struct inode *inode, struct file *filp)
124. {
```



```
125.    /* 设置私有数据 */
126.    filp->private_data = &axi2cdev;
127.    return 0;
128.}
129.
130./* read 函数实现, 对应到 Linux 系统调用函数的 read 函数 */
131.static ssize_t ax_i2c_read(struct file *file, char __user *buf, size_t size, loff_t *offset)
132.{
133.    /* 获取私有数据 */
134.    struct ax_i2c_dev *dev = (struct ax_i2c_dev *)file->private_data;
135.    /* 读取数据 buffer */
136.    char b[100] = {0};
137.    int ret = 0;
138.    /* 从 0 地址开始读, 这里只是为了实验方便使用了 read 并且把地址写死了,
139.       实际的应用中不应该在驱动中把地址写死, 可以尝试使用 iotctl 去实现灵活的方法 */
140.    ax_i2c_read_regs(dev, 0x00, b, 100 > size ? size : 100);
141.
142.    /* 把读取到的数据拷贝到用户读取的地址 */
143.    ret = copy_to_user(buf, b, 100 > size ? size : 100);
144.    return 0;
145.}
146.
147./* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
148.static ssize_t ax_i2c_write(struct file *file, const char __user *buf, size_t size, loff_t *offset)
149.{
150.    /* 获取私有数据 */
151.    struct ax_i2c_dev *dev = (struct ax_i2c_dev *)file->private_data;
152.    /* 写入数据的 buffer */
153.    static char user_data[100] = {0};
154.    int ret = 0;
155.    /* 获取用户需要发送的数据 */
156.    ret = copy_from_user(user_data, buf, 100 > size ? size : 100);
157.    if(ret < 0)
158.    {
159.        printk("copy user data failed\r\n");
160.        return ret;
161.    }
162.    /* 和读对应的从 0 开始写 */
163.    ax_i2c_write_regs(dev, 0x00, user_data, size);
164.
165.    return 0;
166.}
167.
168./* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
169.static int ax_i2c_release(struct inode *inode, struct file *filp)
170.{
171.    return 0;
172.}
173.
174./* file_operations 结构体声明 */
175.static const struct file_operations ax_i2c_ops = {
176.    .owner = THIS_MODULE,
177.    .open = ax_i2c_open,
178.    .read = ax_i2c_read,
179.    .write = ax_i2c_write,
180.    .release = ax_i2c_release,
181.};
182.
183./* probe 函数实现, 驱动和设备匹配时会被调用 */
184.static int axi2c_probe(struct i2c_client *client, const struct i2c_device_id *id)
185.{
186.    printk("eeprom probe\r\n");
187.    /* 构建设备号 */
188.    alloc_chrdev_region(&axi2cdev.devid, 0, AX_I2C_CNT, AX_I2C_NAME);
189.
190.    /* 注册设备 */
191.    cdev_init(&axi2cdev.cdev, &ax_i2c_ops);
192.    cdev_add(&axi2cdev.cdev, axi2cdev.devid, AX_I2C_CNT);
193.}
```



```
194.    /* 创建类 */
195.    axi2cdev.class = class_create(THIS_MODULE, AX_I2C_NAME);
196.    if(IS_ERR(axi2cdev.class))
197.    {
198.        return PTR_ERR(axi2cdev.class);
199.    }
200.
201.    /* 创建设备 */
202.    axi2cdev.device = device_create(axi2cdev.class, NULL, axi2cdev.devid, NULL, AX_I2C_NAME);
203.    if(IS_ERR(axi2cdev.device))
204.    {
205.        return PTR_ERR(axi2cdev.device);
206.    }
207.
208.    axi2cdev.private_data = client;
209.
210.    return 0;
211.}
212.
213./* remove 函数实现，驱动卸载时会被调用 */
214.static int axi2c_remove(struct i2c_client *client)
215.{
216.    /* 删除设备 */
217.    cdev_del(&axi2cdev.cdev);
218.    unregister_chrdev_region(axi2cdev.major, AX_I2C_CNT);
219.    /* 注销类 */
220.    device_destroy(axi2cdev.class, axi2cdev.devid);
221.    class_destroy(axi2cdev.class);
222.    return 0;
223.}
224.
225./* of 匹配表，设备树下的匹配方式 */
226.static const struct of_device_id axi2c_of_match[] =
227.{
228.    { .compatible = "ax-e2p1"},
229.    { /* sentinel */}
230.};
231.
232./* 传统的 id_table 匹配方式 */
233.static const struct i2c_device_id axi2c_id[] = {
234.    {"ax-e2p1"},
235.    {}
236.};
237.
238./* 声明并初始化 i2c 驱动 */
239.static struct i2c_driver axi2c_driver = {
240.    .driver = {
241.        .owner = THIS_MODULE,
242.        .name = "ax-e2p1",
243.        /* 用 of_match_table 匹配 */
244.        .of_match_table = axi2c_of_match,
245.    },
246.    /* 使用传统的方式匹配 */
247.    .id_table = axi2c_id,
248.    .probe = axi2c_probe,
249.    .remove = axi2c_remove,
250.};
251.
252./* 驱动入口函数 */
253.static int __init ax_i2c_init(void)
254.{
255.    /* 在入口函数中调用 i2c_add_driver，注册驱动 */
256.    return i2c_add_driver(&axi2c_driver);
257.}
258.
259./* 驱动出口函数 */
260.static void __exit ax_i2c_exit(void)
261.{
262.    /* 在出口函数中调用 i2c_del_driver，卸载驱动 */
263.    i2c_del_driver(&axi2c_driver);
264.}
```

```
265.  
266./* 标记加载、卸载函数 */  
267.module_init(ax_i2c_init);  
268.module_exit(ax_i2c_exit);  
269.  
270./* 驱动描述信息 */  
271.MODULE_AUTHOR("Alinx");  
272.MODULE_ALIAS("pwm_led");  
273.MODULE_DESCRIPTION("I2C EEPROM driver");  
274.MODULE_VERSION("v1.0");  
275.MODULE_LICENSE("GPL");
```

34 行添加一个 void 指针用于获取 client。

45 行到 78 行实现一个通用的 i2c 读取的函数。i2c 读取中构建 msg 一般构建一个两个元素的数组，第一个 msg 标记为写用于发送目标数据地址，第二个 msg 标记为读。

86 行到 121 行实现通用的 i2c 写函数。

127 行在 open 函数中设置私有数据。

132 行实现 read 函数，调用前面实现的通用读函数。

149 行实现 write 函数，里面调用了前面实现的通用写函数。

185 行实现 probe 函数，内容就是字符设备的注册。

209 行实现把 probe 输入参数的 client 赋值给设备结构体变量中的私有数据。在 i2c 读写函数中会用到的 ax_i2c_dev 类型句柄。

227 行定义 of 匹配表，compatible 字段和设备树中的保持一致。

234 行定义一个 id_table 用于传统的方式匹配。

240 行声明并初始化 i2c_driver。

257 行在驱动入口函数中使用 i2c_add_driver 函数注册 i2c 驱动。i2c_add_driver 函数是对 i2c_register_driver 的封装，省去了输入 THIS_MODULE。

264 行在驱动出口函数中使用 i2c_del_driver 删除驱动。

17.3.4 测试程序

新建 QT 工程名为“ax-i2c-test”，新建 main.c，输入下面的代码：

```
1. #include "stdio.h"  
2. #include "unistd.h"  
3. #include "sys/types.h"  
4. #include "sys/stat.h"  
5. #include "fcntl.h"  
6. #include "stdlib.h"  
7. #include "string.h"  
8. #include "assert.h"  
9.  
10. int main(int argc, char *argv[])  
11. {  
12.     int fd, ret;  
13.     char *filename;  
14.     char buffer[3] = {0};  
15.  
16.     if(argc != 2)  
17.     {  
18.         printf("Error Usage\r\n");  
19.         return -1;  
20.     }  
21.  
22.     filename = argv[1];  
23.     fd = open(filename, O_RDWR);  
24.     if(fd < 0)
```

```
25.     {
26.         printf("file %s open failed\r\n", argv[1]);
27.         return -1;
28.     }
29.
30.     /* 随便写入一些数据 */
31.     buffer[0] = 0x5A;
32.     buffer[1] = 0x55;
33.     buffer[2] = 0xAA;
34.     ret = write(fd, buffer, sizeof(buffer));
35.     if(ret < 0)
36.     {
37.         printf("write failed\r\n");
38.     }
39.     /* 在控制台打印写入的数据 */
40.     printf("write data %X, %X, %X\r\n", buffer[0], buffer[1], buffer[2]);
41.
42.     /* 初始化 buffer, 再用来读取数据 */
43.     memset(buffer, 0, sizeof(buffer));
44.
45.     /* 读出数据 */
46.     ret = read(fd, buffer, sizeof(buffer));
47.     if(ret < 0)
48.     {
49.         printf("read failed\r\n");
50.     }
51.     /* 在控制台打印读出的数据 */
52.     printf("read data %X, %X, %X\r\n", buffer[0], buffer[1], buffer[2]);
53.
54.     close(fd);
55.
56.     return 0;
57. }
```

测试程序就是简单的读写并分别打印出来，比较读写结果。

17.3.5 运行测试

测试方法步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ./ax_i2c.ko
cd ./build-ax_i2c_test-ZYNQ-Debug/
./ax_i2c_test /dev/ax_i2c_e2p
```

IP 和路径根据实际情况调整。

串口工具中的调试结果如下：

```
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ./ax-i2c.ko
ax_i2c: loading out-of-tree module taints kernel.
eeprom probe
[drm] load() is deferred & will be called again
root@ax_peta:/mnt# cd ./build-ax_i2c_test-ZYNQ-Debug/
root@ax_peta:/mnt/build-ax_i2c_test-ZYNQ-Debug# ./ax_i2c_test /dev/ax_i2c_e2p
write data 5A, 55, AA
read data 5A, 55, AA
root@ax_peta:/mnt/build-ax_i2c_test-ZYNQ-Debug#
```

读写结果一致，试验成功。

第十八章 USB 驱动

usb 是通用串行总线的总称。和 windows 一样，Linux 也支持几乎所有的 usb 设备如鼠标、键盘、打印机等。这章来学习一下 Linux 中 usb 相关的内容。

18.1 usb 识别过程

usb 设备接入主机后，匹配过程如下：

1) 硬件检测

usb 接口有四条线分别为 5V、GND、D-、D+。主机端 usb 接口中的 D-或 D+有下拉电阻，空载时为低电平。设备端的 usb 接口内部 D-或 D+有上拉电阻，usb 设备接入主机后，会把主机 usb 接口的 D-或 D+拉高，从而主机从硬件的角度检测到 usb 设备的接入。

2) 握手匹配

主机检测到 usb 设备接入后，就会和设备进行交互，主机端主动发起获取设备信息的描述符，设备则需要按照固定格式返回描述符。如果实在 windows 桌面系统下，此时我们就能看到 xxx 设备接入的弹窗了。

3) 分配地址

一个主机上可以接入多个 usb 设备，为了区分这些设备，在握手成功后，主机会给设备分配设备地址，主机发出的命令中都会包办地址信息。在握手之前，主机会用编号 0 来和设备交互。

18.2 usb 的传输

usb 的传输时主从结构，主就是主机，从就是设备。所有的传输都是由主机发起的，设备没有主动通知主机的能力。我们常说的输入输出，都是在主机的角度去描述的，比如输入设备键盘、鼠标，输出设备显示器等等。

18.2.1 usb 的传输模式

在主从结构的基础上，usb 可以分为四种传输模式：

1) 控制传输

控制传输是一种特殊的传输方式，相对复杂一些。usb 设备连接主机时，主机使用控制传输方式发送控制命令对设备进行配置。同时，需要通过控制传输获取 usb 设备的描述符对设备进行识别，在设备的枚举过程中都是使用控制传输进行数据交换。可靠，且时效性可保证。

2) 批量传输

批量传输一般用于数据量大但对时间要求又不高的场合，如 u 盘、打印机等。可靠，不保证时效性。

3) 中断传输

中断传输一般用于数据量小，不连续且实时要求搞得场合，如鼠标、键盘等输入设备。

4) 等时传输

等时传输一般用于数据量大、连续且实时要求高的场合，但可靠性难以保证，一般用于摄像头、话筒等设备。

18.2.2 端点

端点是 usb 的传输对象。端点是 usb 协议栈层次中抽象出来的概念，可以说是为了软件工作者而存在的概念。usb 协议中从主机 host 开始 一个 host 可以连接一个或多个设备 device，这个层面是物理上的 usb 线连接。一个 device 可以有一个或者多个接口 interface，这层开始就是逻辑概念了，一个二合一的 usb 设备比如耳麦，就有话筒和耳机两接口。一个 interface 可以有一个或多个端点 endpoint，这也是逻辑概念，比如 u 盘需要有输入端点和输出端点。

除了功能性的端点，每个 usb 设备都必须要有端点 0，端点 0 一般用于设备的初始化和配置，支持控制传输，总是在设备接入主机时就进行配置。

18.2.3 管道

管道 pipe 是设备上的端点与主机上的软件之间的关联。管道表示通过内存缓冲区和设备上的端点在主机上的软件之间移动数据的能力。 有两种互斥的管道通信模式：

流：通过管道传输的数据没有 USB 定义的结构。

消息：通过管道移动的数据具有一些 USB 定义的结构。

18.3 usb 总线驱动

总线驱动已经不陌生了，前面的 palform 驱动和 i2c 驱动都是这种思想。同样 usb 也是分为总线和设备两块驱动，usb 驱动框架层次如下：



usb 总线驱动和硬件对接，起到一个承接作用。前面说的 usb 的初始化过程，就需要总线驱动来完成。总线驱动需要做的事有这些：

- 1) 识别 usb 设备
- 2) 查找并安装相应的设备驱动
- 3) 为设备驱动提供都谐函数

总线驱动一般会由芯片厂家提供，少做了解即可。打开开发板，接上鼠标后拔出，可以在控制台中看到如下信息：

```
root@ax_peta:~# usb 1-1: new low-speed USB device number 2 using ci_hsrc
input: Logitech USB Optical Mouse as /devices/soc0/amba/e0002000.usb/ci_hsrc.0/usb1/1-1/1-1:1.0/0003:046D:C077.0001/input/input0
hid-generic 0003:046D:C077.0001: input: USB HID v1.11 Mouse [Logitech USB Optical Mouse] on usb-ci_hsrc.0-1/input0
[drm] load() is deferred & will be called again
usb 1-1: USB disconnect, device number 2
```

18.4 usb 设备驱动

usb 设备驱动中，同样是分成了设备和驱动两个部分。与 i2c 等有区别的地方是，usb 只需要的去实现驱动部分，设备部分由总线驱动获取。

当接上一个 USB 设备时，就会产生中断 `hub_irq()`，在中断里会分配一个编号地址 `choose_address(udev)`。总线把这个地址告诉设备 `hub_set_address()`。接着发出命令获取设备描述符 `usb_get_device_descriptor()`。获取到设备信息后，会注册一个设备 `device_add()`。这个 device 会被放到 usb 总线 `usb_bus_type` 的设备链表中。usb 驱动程序注册后会从 usb 总线的驱动链表中取出设备驱动结构 `usb_driver`，把 `usb_interface` 和 `usb_driver` 的 `id_table` 比较，匹配成功就会调用 `usb_driver` 中的 `probe` 函数。

而我们要做的就是定义初始化并注册 `usb_driver`，并实现 `probe` 函数。

先来看看 `usb_driver` 结构体，定义在文件 `include/linux/usb.h` 中如下：

```
1. struct usb_driver {
2.     const char *name;
3.
4.     int (*probe) (struct usb_interface *intf,
5.                  const struct usb_device_id *id);
6.
7.     void (*disconnect) (struct usb_interface *intf);
8.
9.     int (*unlocked_ioctl) (struct usb_interface *intf, unsigned int code,
10.                          void *buf);
11.
12.     int (*suspend) (struct usb_interface *intf, pm_message_t message);
13.     int (*resume) (struct usb_interface *intf);
14.     int (*reset_resume)(struct usb_interface *intf);
15.
16.     int (*pre_reset)(struct usb_interface *intf);
17.     int (*post_reset)(struct usb_interface *intf);
18.
19.     const struct usb_device_id *id_table;
20.
21.     struct usb_dynids dynids;
22.     struct usbdrv_wrap drvwrap;
23.     unsigned int no_dynamic_id:1;
24.     unsigned int supports_autosuspend:1;
25.     unsigned int disable_hub_initiated_lpm:1;
26.     unsigned int soft_unbind:1;
27. };
```

`name` 为设备名。

`probe` 函数就是设备匹配成功后会执行的函数，必须实现。

`disconnect` 函数在设备不可用时会执行，如设备拔出。

`id_table` 用于匹配设备。`usb_device_id` 定义在 `include/linux/mod_devicetable.h` 中，何以使用下面的宏去初始化：

```
USB_INTERFACE_INFO(cl,sc,pr)
```


cl 之 class 类, sc 是 sub class 子类, pr 是指协议。这些设备描述符定义在文件 include/linux/usb/Ch9.h 中。总线驱动会根据这些描述符去匹配设备和驱动。

定义并初始化好 usb_driver 后, 使用下面的宏向内核注册:

```
usb_register(driver)
```

相对的使用下面的函数注销:

```
void usb_deregister(struct usb_driver *);
```

usb 设备驱动示例:

```
1. static struct usb_device_id usb_id_table [] =
2. {
3.     { USB_INTERFACE_INFO(XXX, XXX, XXX) },
4.     { }
5. };
6.
7.
8. static int usb_probe(struct usb_interface *intf, const struct usb_device_id *id)
9. {
10.     return 0;
11. }
12.
13. static void usb_disconnect(struct usb_interface *intf)
14. {
15.
16. }
17.
18. static struct usb_driver usb_driver = {
19.     .name      = "xxx",
20.     .probe     = usbmouse_as_key_probe,
21.     .disconnect = usbmouse_as_key_disconnect,
22.     .id_table  = usbmouse_as_key_id_table,
23. };
24.
25.
26. static int usb_init(void)
27. {
28.     usb_register(&usb_driver);
29.     return 0;
30. }
31.
32. static void usb_exit(void)
33. {
34.     usb_deregister(&usb_driver);
35. }
36.
37. module_init(usb_init);
38. module_exit(usb_exit);
39.
40. MODULE_LICENSE("GPL");
```

probe 函数和 disconnect 函数的输入参数 struct usb_interface *intf 可以使用和函数 interface_to_usbdev() 来获取 usb_device, 如

```
struct usb_device *dev = interface_to_usbdev(intf);
```

18.5 urb 请求块

urb 是设备驱动中用来描述 usb 设备通信的基本数据结构, 是端点处理的对象, 操作 usb 设备需要使用 urb 来进行。urb 定义在文件 include/linux/usb.h 中, 他的定义和注释如下:

```
1. struct urb {
2.     /* 私有的: 只能由 USB 核心和主机控制器访问的字段 */
```

```

3.     struct kref kref; /*urb 引用计数 */
4.     void *hcpriv; /* 主机控制器私有数据 */
5.     atomic_t use_count; /* 并发传输计数 */
6.     u8 reject; /* 传输将失败*/
7.     int unlink; /* unlink 错误码 */
8.     /* 公共的: 可以被驱动使用的字段 */
9.     struct list_head urb_list; /* 链表头*/
10.    struct usb_anchor *anchor;
11.    struct usb_device *dev; /* 关联的 USB 设备 */
12.    struct usb_host_endpoint *ep;
13.    unsigned int pipe; /* 管道信息 */
14.    int status; /* URB 的当前状态 */
15.    unsigned int transfer_flags; /* URB_SHORT_NOT_OK | ...*/
16.    void *transfer_buffer; /* 发送数据到设备或从设备接收数据的缓冲区 */
17.    dma_addr_t transfer_dma; /*用来以 DMA 方式向设备传输数据的缓冲区 */
18.    int transfer_buffer_length; /*transfer_buffer 或 transfer_dma 指向缓冲区的大小 */
19.
20.    int actual_length; /* URB 结束后, 发送或接收数据的实际长度 */
21.    unsigned char *setup_packet; /* 指向控制 URB 的设置数据包的指针*/
22.    dma_addr_t setup_dma; /*控制 URB 的设置数据包的 DMA 缓冲区*/
23.    int start_frame; /*等时传输中用于设置或返回初始帧*/
24.    int number_of_packets; /*等时传输中等时缓冲区数量 */
25.    int interval; /* URB 被轮询到的时间间隔 (对中断和等时 urb 有效) */
26.    int error_count; /* 等时传输错误数量 */
27.    void *context; /* completion 函数上下文 */
28.    usb_complete_t complete; /* 当 URB 被完全传输或发生错误时, 被调用 */
29.    /*单个 URB 一次可定义多个等时传输时, 描述各个等时传输 */
30.    struct usb_iso_packet_descriptor iso_frame_desc[0];
31. };

```

urb 是使用流程如下:

1) 创建 urb

使用下面的函数创建一个 urb:

```
struct urb *usb_alloc_urb(int iso_packets, int mem_flags);
```

相对的使用下面的函数释放 urb:

```
void usb_free_urb(struct urb *urb);
```

2) 填充 urb

1) 对于中断 urb, 使用下面的函数来初始化:

```
void usb_fill_int_urb(struct urb *urb, struct usb_device *dev, unsigned int pipe,
void *transfer_buffer, int buffer_length, usb_complete_t complete, void *context,
int interval);
```

参数说明:

urb: 要被初始化的 urb 的指针;

dev: 该 urb 要被发送到的 usb 设备;

pipe: 该 urb 要被发送到的 usb 设备的特定端点, 使用 `usb_sndctrlpipe()` 或 `usb_rcvctrlpipe()` 函数来创建;

transfer_buffer: 发送数据或接收数据的缓冲区的指针。它也不能是静态缓冲区, 必须使用 `kmalloc()` 来分配;

buffer_length: transfer_buffer 指针所指向缓冲区的大小;

complete: 该 urb 完成时的回调函数;

context: 完成处理函数的上下文;

interval: 该这个 urb 调度的间隔。

2) 对于批量 urb, 使用下面的函数来初始化:

```
void usb_fill_bulk_urb(struct urb *urb, struct usb_device *dev, unsigned int pipe,
void *transfer_buffer, int buffer_length, usb_complete_t complete, void *context);
```

这里的 pipe 需要使用 usb_sndbulkpipe() 或者 usb_rcvbulkpipe() 函数来创建。

其他参数与 usb_fill_int_urb() 相同。

3) 对于控制 urb, 使用下面的函数来初始化:

```
void usb_fill_control_urb(struct urb *urb, struct usb_device *dev, unsigned int pipe,
unsigned char *setup_packet, void *transfer_buffer, int
buffer_length, usb_complete_t complete, void *context);
```

setup_packet: 即将被发送到端点的设置数据包。

这里的 pipe 需要使用 usb_sndctrlpipe() 或 usb_rcvctrlpipe() 函数来创建。

其他参数与 usb_fill_int_urb() 相同。

3) 提交 urb

完成填充后, 使用下面的函数来提交 urb:

```
int usb_submit_urb(struct urb *urb, int mem_flags);
```

mem_flags 有以下定义:

GFP_ATOMIC: 在中断服务函数、底半部、tasklet、定时器处理函数以及 urb 完成回调函数中, 如果调用者持有自旋锁或者读写锁时以及当驱动将当前进程修改为非 TASK_RUNNING 时使用。

GFP_NOIO: 在存储设备的块 I/O 和错误处理路径中使用;

GFP_KERNEL: 其他情况都使用这个标志。

在 urb 提交后, 如果 urb 成功发送给设备、数据发送发生错误或驱动使用 usb_unlink_urb() 或 usb_kill_urb() 主动取消 urb 时, urb 会结束。urb 结束时可以通过成员变量 status 来查看结束的原因。

18.6 实验

本章写一个 usb 鼠标动点点击动作捕捉实验, 点击左键时在控制台输出 1, 右键在控制台输出 0。

18.6.1 原理图

前面说过硬件是与 usb 总线驱动相关的, usb 总线驱动由芯片厂家提供, 板子上的 usb 连接和 xilinx 样板是一样的, 所以 usb 总线驱动程序不需要修改。

18.6.2 设备树

设备树可以和前面任意章节的一致, 保持 usb 节点即可。

18.6.3 驱动程序

使用 petalinux 新建名为 "ax-usb-driv" 驱动程序, 并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-usb-driv.c 文件中输入下面的代码:

```
1. #include <linux/kernel.h>
2. #include <linux/slab.h>
```

```
3. #include <linux/module.h>
4. #include <linux/init.h>
5. #include <linux/usb/input.h>
6. #include <linux/hid.h>
7.
8. /* 定义一个输入事件，表示鼠标的点击事件 */
9. static struct input_dev *mouse_dev;
10. /* 定义缓冲区首地址 */
11. static char *usb_buf;
12. /* dma 缓冲区 */
13. static dma_addr_t usb_buf_dma;
14. /* 缓冲区长度 */
15. static int usb_buf_len;
16. /* 定义一个 urb */
17. static struct urb *mouse_urb;
18.
19. static void ax_usb_irq(struct urb *urb)
20. {
21.     static unsigned char pre_sts;
22.     int i;
23.
24.     /* 左键发生了变化 */
25.     if ((pre_sts & 0x01) != (usb_buf[0] & 0x01))
26.     {
27.         printk("lf click\n");
28.         input_event(mouse_dev, EV_KEY, KEY_L, (usb_buf[0] & 0x01) ? 1 : 0);
29.         input_sync(mouse_dev);
30.     }
31.
32.     /* 右键发生了变化 */
33.     if ((pre_sts & 0x02) != (usb_buf[0] & 0x02))
34.     {
35.         printk("rt click\n");
36.         input_event(mouse_dev, EV_KEY, KEY_S, (usb_buf[0] & 0x02) ? 1 : 0);
37.         input_sync(mouse_dev);
38.     }
39.
40.     /* 记录当前状态 */
41.     pre_sts = usb_buf[0];
42.
43.     /* 重新提交 urb */
44.     usb_submit_urb(mouse_urb, GFP_KERNEL);
45. }
46.
47. static int ax_usb_probe(struct usb_interface *intf, const struct usb_device_id *id)
48. {
49.     /* 获取 usb_device */
50.     struct usb_device *dev = interface_to_usbdev(intf);
51.     struct usb_host_interface *interface;
52.     struct usb_endpoint_descriptor *endpoint;
53.     int pipe;
54.
55.     /* 获取端点 */
56.     interface = intf->cur_altsetting;
57.     endpoint = &interface->endpoint[0].desc;
58.
59.     /* 分配 input_dev */
60.     mouse_dev = input_allocate_device();
61.     /* 设置 input_dev */
62.     set_bit(EV_KEY, mouse_dev->evbit);
63.     set_bit(EV_REP, mouse_dev->evbit);
64.     set_bit(KEY_L, mouse_dev->keybit);
65.     set_bit(KEY_S, mouse_dev->keybit);
66.     /* 注册 input_dev */
67.     input_register_device(mouse_dev);
68.
69.     /* 获取 USB 设备端点对应的管道 */
70.     pipe = usb_rcvintpipe(dev, endpoint->bEndpointAddress);
71.
72.     /* 获取端点最大长度作为缓冲区长度 */
73.     usb_buf_len = endpoint->wMaxPacketSize;
```

```

74.
75.     /* 分配缓冲区 */
76.     usb_buf = usb_alloc_coherent(dev, usb_buf_len, GFP_ATOMIC, &usb_buf_dma);
77.
78.     /* 创建 urb */
79.     mouse_urb = usb_alloc_urb(0, GFP_KERNEL);
80.
81.     /* 分配 urb */
82.     usb_fill_int_urb(mouse_urb, dev, pipe, usb_buf, usb_buf_len, ax_usb_irq, NULL, endpoint
t->bInterval);
83.     mouse_urb->transfer_dma = usb_buf_dma;
84.     mouse_urb->transfer_flags |= URB_NO_TRANSFER_DMA_MAP;
85.
86.     /* 提交 urb */
87.     usb_submit_urb(mouse_urb, GFP_KERNEL);
88.
89.     return 0;
90. }
91.
92. static void ax_usb_disconnect(struct usb_interface *intf)
93. {
94.     struct usb_device *dev = interface_to_usbdev(intf);
95.
96.     /* 主动结束 urb */
97.     usb_kill_urb(mouse_urb);
98.     /* 释放 urb */
99.     usb_free_urb(mouse_urb);
100.    /* 释放缓冲区 */
101.    usb_free_coherent(dev, usb_buf_len, usb_buf, &usb_buf_dma);
102.    /* 注销输入事件 */
103.    input_unregister_device(mouse_dev);
104.    /* 释放输入事件 */
105.    input_free_device(mouse_dev);
106.}
107.
108./* 定义初始化 id_table */
109.static struct usb_device_id ax_usb_id_table [] = {
110.    /* 鼠标 mouse 接口描述符里类是 HID 类, 子类 boot, 协议 mouse */
111.    {
112.        USB_INTERFACE_INFO(USB_INTERFACE_CLASS_HID,
113.                            USB_INTERFACE_SUBCLASS_BOOT,
114.                            USB_INTERFACE_PROTOCOL_MOUSE)
115.    }, { }
116.};
117.
118./* 定义并初始化 usb_driver */
119.static struct usb_driver ax_usb_driver = {
120.    .name      = "ax_usb_test",
121.    .probe     = ax_usb_probe,
122.    .disconnect = ax_usb_disconnect,
123.    .id_table  = ax_usb_id_table,
124.};
125.
126./* 驱动入口函数 */
127.static int ax_usb_init(void)
128.{
129.    /* 注册 usb_driver */
130.    return usb_register(&ax_usb_driver);
131.}
132.
133./* 驱动出口函数 */
134.static void ax_usb_exit(void)
135.{
136.    /* 注销 usb_driver */
137.    usb_deregister(&ax_usb_driver);
138.}
139.
140./* 标记加载、卸载函数 */
141.module_init(ax_usb_init);
142.module_exit(ax_usb_exit);
143.

```

```

144. /* 驱动描述信息 */
145. MODULE_AUTHOR("Alinx");
146. MODULE_ALIAS("pwm_led");
147. MODULE_DESCRIPTION("USB TEST driver");
148. MODULE_VERSION("v1.0");
149. MODULE_LICENSE("GPL");

```

usb_driver 的框架很简单，关键部分是 probe 函数和 urb 的回掉函数中的处理。

结合了 input 子系统，把 usb 鼠标模拟成按键。

56 行，probe 函数中从 intf 中获取接口，再从接口中获取端点。

70 行获取管道。

73~79 行分配缓冲区。

这些都是在位分配 urb 做准备。

79~87 行创建 urb，并用上面获取的参数来分配 urb，完成后提交。

usb_fill_int_urb 注册终端输入端点数据，当输入设备 usb 鼠标产生动作时，就会触发终端函数 ax_usb_irq。

25~38 行在终端函数中，通过 input 子系统判断输入类型，打印对应的信息。

18.6.4 运行测试

本次实验的程序加载了鼠标，和系统中原先的会冲突，所以需要先把系统自带的鼠标驱动去掉。方法如下：

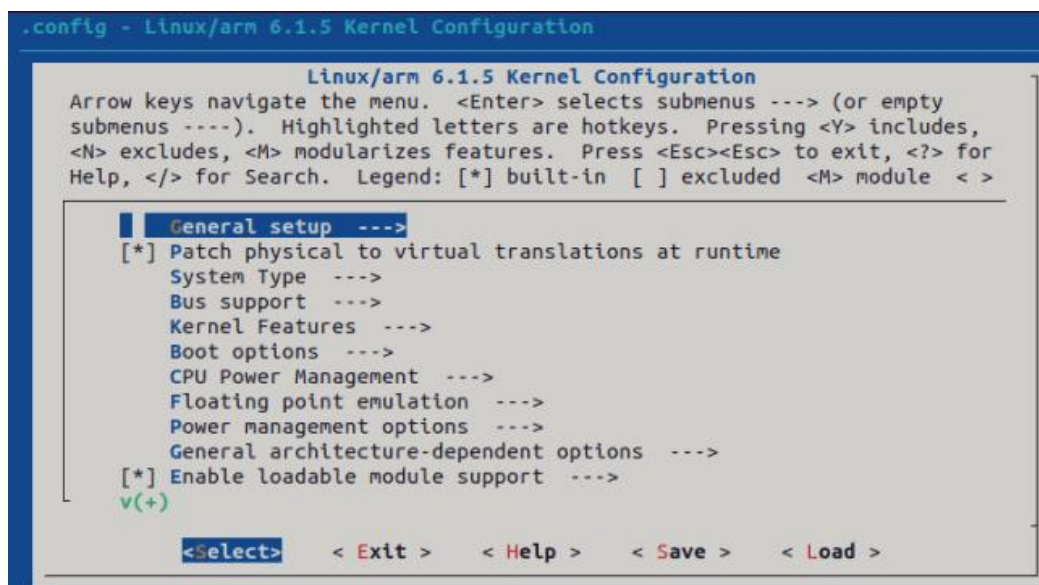
- 1) 在终端中输入命令配置内核 petalinux-config -c kernel，弹出配置界面如下：

```

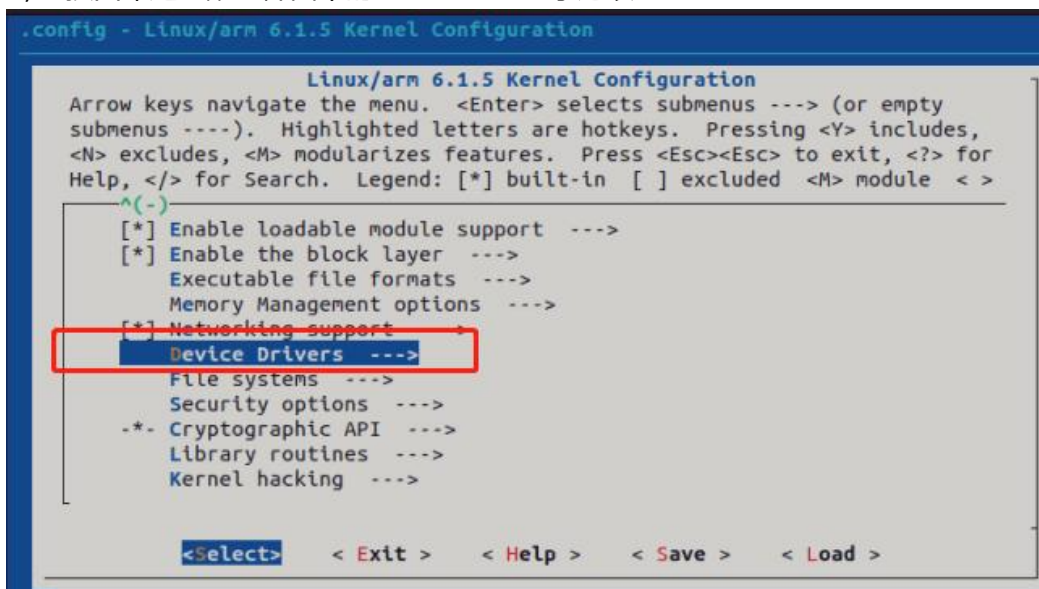
alinx@ubuntu:~/Downloads/peta_pri_pwm/ax_peta$ petalinux-config -c kernel
[INFO] generating Kconfig for project

[INFO] sourcing bitbake
[INFO] generating plnxtool conf
[INFO] generating meta-plnx-generated layer
~/Downloads/peta_pri_pwm/ax_peta/build/misc/plnx-generated ~/Downloads/peta_pri_pwm/ax_peta
~/Downloads/peta_pri_pwm/ax_peta
[INFO] generating machine configuration
[INFO] configuring: kernel
[INFO] generating kernel configuration files
[INFO] bitbake virtual/kernel -c menuconfig
Loading cache: 100% |#####| Time: 0:00:00
Loaded 3278 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:01
Parsing of 2487 .bb files complete (2455 cached, 32 parsed). 3280 targets, 226 s
kipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |#####| Time: 0:00:04
NOTE: Executing RunQueue Tasks
NOTE: Tasks Summary: Attempted 2 tasks of which 0 didn't need to be rerun and al
l succeeded.
Loading cache: 100% |#####| Time: 0:00:00
Loaded 3278 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:01
Parsing of 2487 .bb files complete (2455 cached, 32 parsed). 3280 targets, 226 s
kipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |#####| Time: 0:00:04
NOTE: Executing SetScene Tasks
NOTE: Executing RunQueue Tasks
Currently 1 running tasks (318 of 318) 99% |#####|
0: linux-xlnx-4.9-xilinx-v2017.4+gitAUTOINC+b450e900fd-r0 do_menuconfig - 24s (p
id 122206)

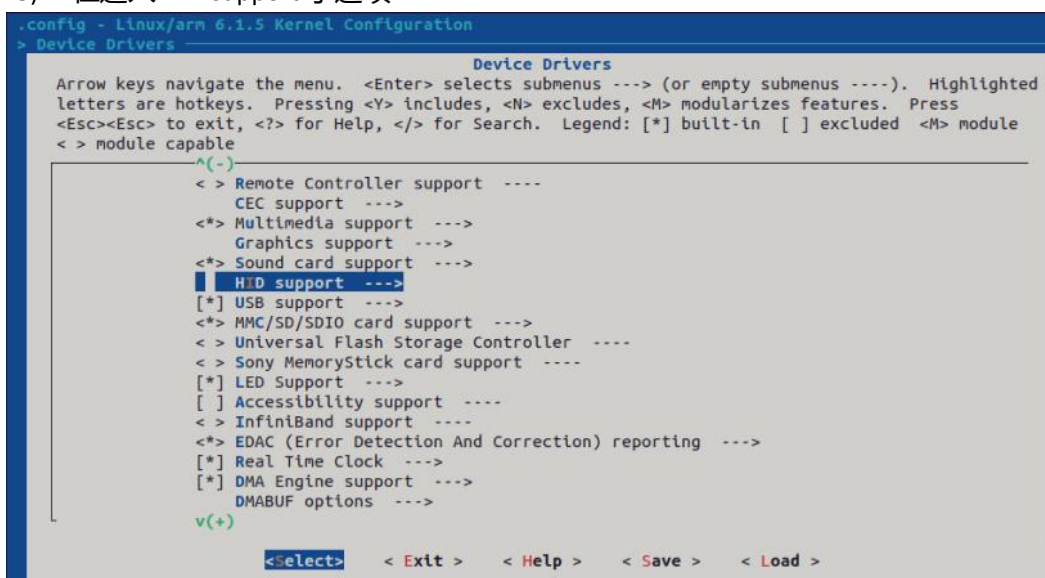
```

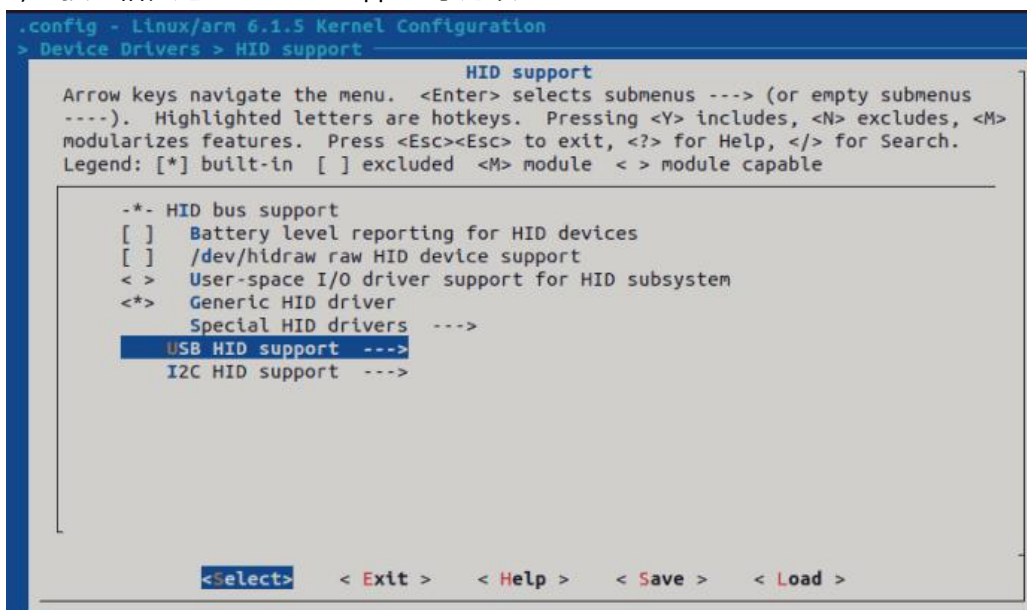
2) 按回车进入配置界面中的 Device Drivers 子选项



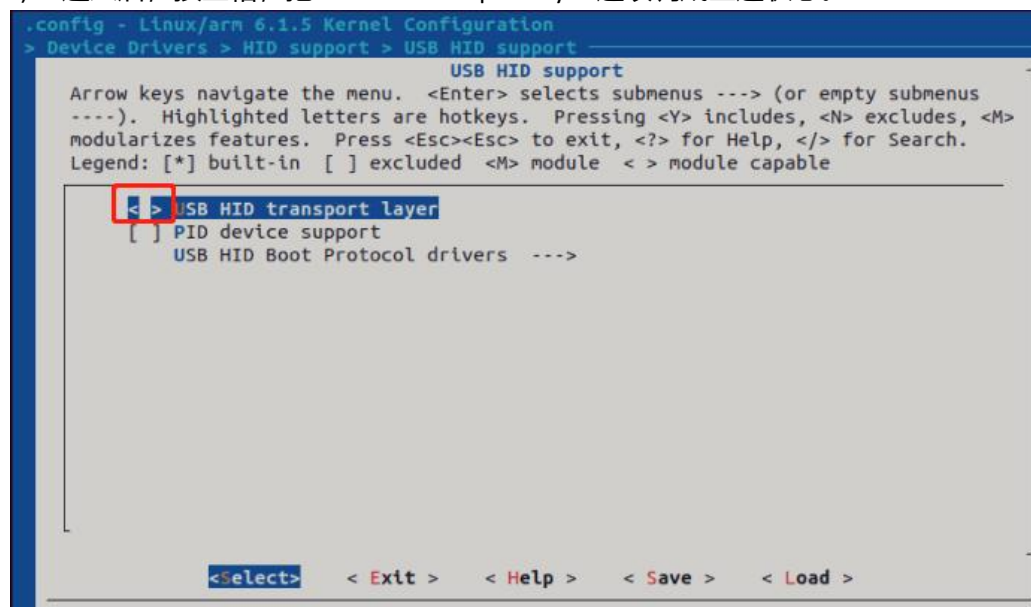
3) 在进入 HID support 子选项



4) 按空格, 进入 USB HID support 子选项



5) 进入后, 按空格, 把 USB HID transport layer 选项调成空选状态。



之后, 测试方法步骤如下:

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ./ax-usb-drv.ko
```

IP 和路径根据实际情况调整。

之后接上 usb 鼠标。点击左键右键查看控制台输出情况。如下:

```
insmod ./ax-usb-drv.ko
usbcore: registered new interface driver ax_usb_test
root@ax_peta:/mnt# usb 1-1: new low-speed USB device number 29 using ci_hdrc
input: Unspecified device as /devices/virtual/input/input27
[drm] load() is deferred & will be called again
lf click
lf click
lf click
lf click
lf click
lf click
lf click
rt click
rt click
rt click
rt click
rt click
rt click
```

拔出鼠标后，会打印 disconnect 信息。

```
rt click
rt click
usb 1-1: USB disconnect, device number 29
```

第十九章 SPI 驱动

和 usb、i2c 一样，SPI 驱动的框架也分为总线(控制器)驱动和设备驱动。控制器驱动由内核提供也是不需要我们操心，重点还是放在设备驱动的实现上。

19.1 SPI 控制器驱动

大概了解一下 SPI 控制器总线。内核中使用 spi_master 结构体来表示一个 SPI 主机控制器，定义在文件 include/linux/spi/spi.h 中的 402 行。用 i2c 总线框来类比，spi_master 中也有数据传输的函数 transfer、transfer_one_message 等。transfer_one_message 的区别在于会把数据包保成 spi_message，以队列形式发送。这些功能函数都由芯片厂家提供，我们稍做了解就行了。

1) spi_master 申请与释放

定义好 spi_master 结构体变量之后，使用下面的方法向内核申请：

```
struct spi_master *spi_alloc_master(struct device *dev, unsigned size)
```

dev 一般是 platform_device 中的 dev 成员变量。

size 是私有数据大小，可以通过函数 spi_master_get_devdata 获取私有数据。

申请成功返回 spi_master。

释放 spi_master 使用下面的函数：

```
void spi_master_put(struct spi_master *master)
```

2) spi_master 注册与注销

spi_master 初始化完成后使用下面的函数注册：

```
int spi_register_master(struct spi_master *master)
```

使用下面的函数注销：

```
void spi_unregister_master(struct spi_master *master)
```

19.2 SPI 设备驱动

1) spi_driver

Linux 内核中使用 spi_driver 结构体来表示 spi 设备驱动。spi_driver 结构体定义在文件 include/linux/spi/spi.h 中如下：

```
1. struct spi_driver {
2.     const struct spi_device_id *id_table;
3.     int (*probe)(struct spi_device *spi);
4.     int (*remove)(struct spi_device *spi);
5.     void (*shutdown)(struct spi_device *spi);
6.     struct device_driver driver;
7. };
```

id_table 用于匹配 spi 设备，匹配成功就会执行 probe 函数。设备移除时则执行 remove 函数。

spi_driver 初始化完成后，使用下面的函数向内和注册：

```
int spi_register_driver(struct spi_driver *sdrv)
```

使用下面的函数注销 spi_driver:

```
void spi_unregister_driver(struct spi_driver *sdrv)
```

spi_driver 框架示例如下:

```
1. static int ax_probe(struct spi_device *spi)
2. {
3.     return 0;
4. }
5.
6. static int ax_remove(struct spi_device *spi)
7. {
8.     return 0;
9. }
10.
11. static const struct spi_device_id ax_id[] = {
12.     {"xxx", 0},
13.     { }
14. };
15.
16. static const struct of_device_id ax_of_match[] = {
17.     { .compatible = "xxx" },
18.     { }
19. };
20.
21. static struct spi_driver ax_driver = {
22.     .probe = ax_probe,
23.     .remove = ax_remove,
24.     .driver = {
25.         .owner = THIS_MODULE,
26.         .name = "xxx",
27.         .of_match_table = ax_of_match,
28.     },
29.     .id_table = ax_id,
30. };
31.
32. static int __init ax_init(void)
33. {
34.     return spi_register_driver(&ax_driver);
35. }
36.
37. static void __exit ax_exit(void)
38. {
39.     spi_unregister_driver(&ax_driver);
40. }
41.
42. module_init(ax_init);
43. module_exit(ax_exit);
```

编写 spi 设备驱动, 至少需要我们去实现其中 probe 和 remove 两个函数。

2) spi 设备

内核中用 spi_device 结构体表示 spi 设备, 引入设备树后, spi_device 几乎很少用了, 重点看一下设备树中的描述 spi 设备方法。spi 设备的设备树节点格式可参考文件

Documentation\devicetree\bindings\mtd 中的说明。示例如下:

```
1. qspi: spi@e000d000 {
2.     clock-names = "ref_clk", "pclk";
3.     clocks = <&clkc 10>, <&clkc 43>;
4.     compatible = "xlnx,zynq-qspi-1.0";
5.     status = "disabled";
6.     interrupt-parent = <&intc>;
7.     interrupts = <0 19 4>;
8.     reg = <0xe000d000 0x1000>;
9.     #address-cells = <1>;
10.    #size-cells = <0>;
11. }
```

```
12.     flash: w25q256@0
13.     {
14.         #address-cells = <1>;
15.         #size-cells = <1>;
16.         compatible = "w25q256";
17.         reg = <0>;
18.         spi-max-frequency = <40000000>;
19.         m25p,fast-read;
20.     };
21. };
```

第 1 行 qspi 时 zynq 上的其中一路 spi 总线，也就是总线节点。

2~10 行是 spi 总线节点的属性，这部分是 xilinx 提供的，硬件相关的信息都可以和寄存器手册中的对应，没什么特别要注意的。

12 行是设备节点，他是挂在于 qspi 这路 spi 总线下的设备。flash 是别称，这个节点就是个 flash 芯片。w25q256 是节点名称，后面的@0 是指这个设备接在这个 spi 总线的通道 0 上。

14、15 行的属性是老朋友了。

16 行的 compatible 是驱动和设备匹配时会用到的重要属性，他的值需要和 spi_driver 中的 compatible 字段一致。这与前面的总线框架中遇到的也是一样的。

17 行的 reg 属性和@后一样，表示 spi 通道。

18 行的 spi-max-frequency 属性设置比 spi 的最高频率，这里的频率就是 20Mhz。

19 行的 fast-read 表示这个设备支持快速读取，根据实际情况，如果不支持就去掉。

3) 驱动和设备的匹配

这部分和 i2c 也很相似，驱动和设备的匹配在总线(控制器)驱动中完成。

spi 总线定义为结构体 spi_bus_type，在文件 drivers/spi/spi.c 文件中，如下：

```
1. struct bus_type spi_bus_type = {
2.     .name = "spi",
3.     .dev_groups = spi_dev_groups,
4.     .match = spi_match_device,
5.     .uevent = spi_uevent,
6. };
```

match 函数就是匹配函数，内核中 match 函数的实现为函数 spi_match_device()。如下：

```
1. static int spi_match_device(struct device *dev, struct device_driver *drv)
2. {
3.     const struct spi_device *spi = to_spi_device(dev);
4.     const struct spi_driver *sdrv = to_spi_driver(drv);
5.
6.     /* Attempt an OF style match */
7.     if (of_driver_match_device(dev, drv))
8.         return 1;
9.
10.    /* Then try ACPI */
11.    if (acpi_driver_match_device(dev, drv))
12.        return 1;
13.
14.    if (sdrv->id_table)
15.
16.        return !!spi_match_id(sdrv->id_table, spi);
17.
18.    return strcmp(spi->modalias, drv->name) == 0;
19. }
```

又设备树、ACPI、id_tabel 和 name 四种匹配方式，和前面总线驱动框架的类似。

19.3 SPI 设备驱动中数据收发处理

spi 设备驱动中收发数据需要用到一些结构和函数，我们——来看。

1) spi_transfer

spi_transfer 用于描述 spi 的传输信息，定义如下：

```
1. struct spi_transfer {
2.     /* it's ok if tx_buf == rx_buf (right?)
3.      * for MicroWire, one buffer must be null
4.      * buffers must work with dma_map_single() calls, unless
5.      * spi_message.is_dma_mapped reports a pre-existing mapping
6.      */
7.     const void *tx_buf;
8.     void *rx_buf;
9.     unsigned len;
10.
11.     dma_addr_t tx_dma;
12.     dma_addr_t rx_dma;
13.     struct sg_table tx_sg;
14.     struct sg_table rx_sg;
15.
16.     unsigned cs_change:1;
17.     unsigned tx_nbits:3;
18.     unsigned rx_nbits:3;
19. #define SPI_NBITS_SINGLE 0x01 /* 1bit transfer */
20. #define SPI_NBITS_DUAL 0x02 /* 2bits transfer */
21. #define SPI_NBITS_QUAD 0x04 /* 4bits transfer */
22.     u8 bits_per_word;
23.     u16 delay_usecs;
24.     u32 speed_hz;
25.     u32 dummy;
26.     struct list_head transfer_list;
27. };
```

tx_buf、rx_buf 分别是保存发送和接收数据。

len 是数据长度，spi 是全双工通讯，在单词通讯中收发数据长度是一样的，所以只要一个 len 就行了。

2) spi_message

spi_message 相当于是 spi_transfer 的发送队列，spi_transfer 需要添加到 spi_message 中去发送。

3) spi_message_init()

spi_message 需要使用函数 spi_message_init() 来初始化。

4) spi_message_add_tail()

spi_message 初始化完成后使用 spi_message_add_tail() 函数把 spi_transfer 添加到 spi_message 中，函数原型为：

```
void spi_message_add_tail(struct spi_transfer *t, struct spi_message *m)
```

5) spi_sync()

spi_sync() 函数使用同步阻塞的方式传输 spi_message，原型如下：

```
int spi_sync(struct spi_device *spi, struct spi_message *message)
```

6) spi_async()

spi_async() 函数使用异步非阻塞的方式传输 spi_message，原型如下：

```
int spi_async(struct spi_device *spi, struct spi_message *message)
```

整体步骤示例如下，具体的写法到实验中去看：

```

1. static int ax_spi_send(struct spi_device *spi, u8 *buf, int len)
2. {
3.     int ret;
4.     struct spi_message msg;
5.     struct spi_transfer trans =
6.     {
7.         .tx_buf = buf,
8.         .len = len,
9.     };
10.    spi_message_init(&msg);
11.    spi_message_add_tail(trans, &msg);
12.    ret = spi_sync(spi, &msg);
13.    return ret;
14. }

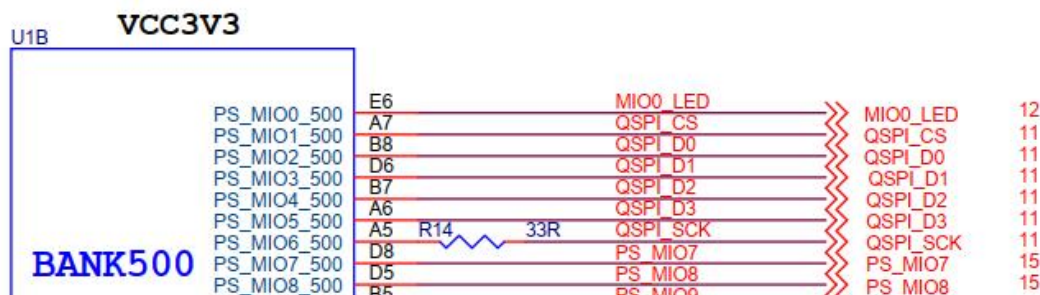
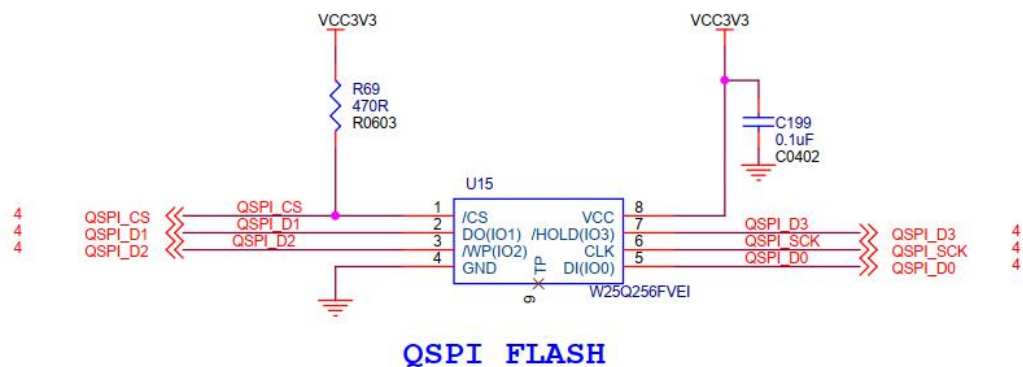
```

19.4 实验

本章使用 zynq 上的 QSPI 读写 qflash。思路和 17 张的 i2c 差不多。

19.4.1 原理图

原理图部分不需要关心，由总线驱动部分去完成。



19.4.2 设备树

打开 system-user.dtsi 文件，在根目录外添加下面的节点：

```

1. &flash0 {
2.     compatible = "w25q256";
3. };

```

qflash 节点在 zynq 的设备树中已经存在了，我们只要改一下其中 compatible 属性即可。

19.4.3 驱动程序

使用 petalinux 新建名为“ax-spi-driv”驱动程序，并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-spi-driv.c 文件中输入下面的代码：


```
1. #include <linux/err.h>
2. #include <linux/errno.h>
3. #include <linux/device.h>
4. #include <linux/mtd/mtd.h>
5. #include <linux/mtd/partitions.h>
6. #include <linux/spi/spi.h>
7. #include <linux/spi/flash.h>
8. #include <linux/types.h>
9. #include <linux/kernel.h>
10. #include <linux/init.h>
11. #include <linux/module.h>
12. #include <linux/cdev.h>
13. #include <linux/fs.h>
14. #include <linux/fcntl.h>
15. #include <linux/platform_device.h>
16. #include <asm/mach/map.h>
17. #include <asm/uaccess.h>
18. #include <asm/io.h>
19.
20. /* 驱动个数 */
21. #define AX_FLASH_CNT 1
22. /* 设备节点名称 */
23. #define AX_FLASH_NAME "ax_flash"
24.
25. /* Flash 操作命令 */
26. #define CMD_WRITE_ENABLE 0x06
27. #define CMD_BULK_ERASE 0xc7
28. #define CMD_READ_BYTES 0x03
29. #define CMD_PAGE_PROGRAM 0x02
30. #define CMD_MAX 5
31.
32. struct ax_qflash_dev {
33.     dev_t devid; //设备号
34.     struct cdev cdev; //字符设备
35.     struct class *class; //类
36.     struct device *device; //设备
37.     int major; //主设备号
38.     void *private_data; //私有数据, 获取 spi_device
39.     char cmd[CMD_MAX]; //SPI 命令和地址
40. };
41.
42. struct ax_qflash_dev ax_qflash;
43.
44. static int ax_spi_write(struct ax_qflash_dev *dev, loff_t addr, const char *buf,
45.     size_t len)
46. {
47.     int ret;
48.     char cmd_buf[1] = {0};
49.     struct spi_device *spi = (struct spi_device *)dev->private_data;
50.     struct spi_transfer trans[2] = {0};
51.     struct spi_message msg;
52.     spi_message_init(&msg);
53.
54.     /* 写使能 */
55.     cmd_buf[0] = CMD_WRITE_ENABLE;
56.     spi_write(spi, cmd_buf, 1);
57.
58.     dev->cmd[0] = CMD_PAGE_PROGRAM;
59.     dev->cmd[1] = addr >> 24;
60.     dev->cmd[2] = addr >> 16;
61.     dev->cmd[3] = addr >> 8;
62.     dev->cmd[4] = addr;
63.
64.     trans[0].tx_buf = dev->cmd;
65.     trans[0].len = CMD_MAX;
```

```
65.     spi_message_add_tail(&trans[0], &msg);
66.
67.     trans[1].tx_buf = buf;
68.     trans[1].len = len;
69.     spi_message_add_tail(&trans[1], &msg);
70.
71.     ret = spi_sync(spi, &msg);
72.
73.     return ret;
74. }
75.
76. static int ax_spi_read(struct ax_qflash_dev *dev, loff_t addr, const char *buf,
77.     size_t len)
78. {
79.     int ret;
80.     struct spi_device *spi = (struct spi_device *)dev->private_data;
81.     struct spi_transfer trans[2] = {0};
82.     struct spi_message msg;
83.     spi_message_init(&msg);
84.
85.     dev->cmd[0] = CMD_READ_BYTES;
86.     dev->cmd[1] = addr >> 24;
87.     dev->cmd[2] = addr >> 16;
88.     dev->cmd[3] = addr >> 8;
89.     dev->cmd[4] = addr;
90.
91.     trans[0].tx_buf = dev->cmd;
92.     trans[0].len = CMD_MAX;
93.     spi_message_add_tail(&trans[0], &msg);
94.
95.     trans[1].rx_buf = buf;
96.     trans[1].len = len;
97.     spi_message_add_tail(&trans[1], &msg);
98.
99.     ret = spi_sync(spi, &msg);
100.
101.     return ret;
102. }
103. /* open 函数实现, 对应到 Linux 系统调用函数的 open 函数 */
104. static int ax_flash_open(struct inode *inode, struct file *filp)
105. {
106.     /* 设置私有数据 */
107.     filp->private_data = &ax_qflash;
108.     return 0;
109. }
110.
111. /* read 函数实现, 对应到 Linux 系统调用函数的 read 函数 */
112. static ssize_t ax_flash_read(struct file *file, char __user *buf, size_t size,
113.     loff_t *offset)
114. {
115.     /* 获取私有数据 */
116.     struct ax_qflash_dev *dev = (struct ax_qflash_dev *)file->private_data;
117.     /* 读取数据 buffer */
118.     char b[100] = {0};
119.     int ret = 0;
120.
121.     /* 读取数据 */
122.     ax_spi_read(dev, 0, b, 100 > size ? size : 100);
123.
124.     /* 把读取到的数据拷贝到用户读取的地址 */
125.     ret = copy_to_user(buf, b, 100 > size ? size : 100);
126.     return 0;
127. }
```

```
128. /* write 函数实现, 对应到 Linux 系统调用函数的 write 函数 */
129. static ssize_t ax_flash_write(struct file *file, const char __user *buf, size_
    t size, loff_t *offset)
130. {
131.     /* 获取私有数据 */
132.     struct ax_qflash_dev *dev = (struct ax_qflash_dev *)file->private_data;
133.     /* 写入数据的 buffer */
134.     static char user_data[100] = {0};
135.     int ret = 0;
136.     /* 获取用户需要发送的数据 */
137.     ret = copy_from_user(user_data, buf, 100 > size ? size : 100);
138.     if(ret < 0)
139.     {
140.         printk("copy user data failed\r\n");
141.         return ret;
142.     }
143.
144.     /* 写入数据 */
145.     ax_spi_write(dev, 0, user_data, 100 > size ? size : 100);
146.
147.     return 0;
148. }
149.
150. /* release 函数实现, 对应到 Linux 系统调用函数的 close 函数 */
151. static int ax_flash_release(struct inode *inode, struct file *filp)
152. {
153.     return 0;
154. }
155.
156. /* file_operations 结构体声明 */
157. static const struct file_operations ax_flash_ops = {
158.     .owner = THIS_MODULE,
159.     .open = ax_flash_open,
160.     .read = ax_flash_read,
161.     .write = ax_flash_write,
162.     .release = ax_flash_release,
163. };
164.
165. static int ax_spi_probe(struct spi_device *spi)
166. {
167.     char cmd_buf[1] = {0};
168.     printk("flash probe\r\n");
169.
170.     /* 构建设备号 */
171.     alloc_chrdev_region(&ax_qflash.devid, 0, AX_FLASH_CNT, AX_FLASH_NAME);
172.
173.     /* 注册设备 */
174.     cdev_init(&ax_qflash.cdev, &ax_flash_ops);
175.     cdev_add(&ax_qflash.cdev, ax_qflash.devid, AX_FLASH_CNT);
176.
177.     /* 创建类 */
178.     ax_qflash.class = class_create(THIS_MODULE, AX_FLASH_NAME);
179.     if(IS_ERR(ax_qflash.class))
180.     {
181.         return PTR_ERR(ax_qflash.class);
182.     }
183.
184.     /* 创建设备 */
185.     ax_qflash.device = device_create(ax_qflash.class, NULL, ax_qflash.devid, N
        ULL, AX_FLASH_NAME);
186.     if(IS_ERR(ax_qflash.device))
187.     {
188.         return PTR_ERR(ax_qflash.device);
189.     }
190. }
```

```
191.     ax_qflash.private_data = spi;
192.     /* 擦除 */
193.     cmd_buf[0] = CMD_BULK_ERASE;
194.     spi_write(spi, cmd_buf, 1);
195.     return 0;
196. }
197.
198. static void ax_spi_remove(struct spi_device *spi)
199. {
200.     /* 删除设备 */
201.     cdev_del(&ax_qflash.cdev);
202.     unregister_chrdev_region(ax_qflash.major, AX_FLASH_CNT);
203.     /* 注销类 */
204.     device_destroy(ax_qflash.class, ax_qflash.devid);
205.     class_destroy(ax_qflash.class);
206.
207. }
208.
209. static const struct spi_device_id ax_id_table[] = {
210.     {"w25q256", 0},
211.     { }
212. };
213.
214. static const struct of_device_id ax_of_match[] = {
215.     { .compatible = "w25q256" },
216.     { }
217. };
218.
219. static struct spi_driver ax_spi_driver = {
220.     .probe = ax_spi_probe,
221.     .remove = ax_spi_remove,
222.     .driver = {
223.         .owner = THIS_MODULE,
224.         .name = "w25q256",
225.         .of_match_table = ax_of_match,
226.     },
227.     .id_table = ax_id_table,
228. };
229.
230. static int __init ax_init(void)
231. {
232.     return spi_register_driver(&ax_spi_driver);
233. }
234.
235. static void __exit ax_exit(void)
236. {
237.     spi_unregister_driver(&ax_spi_driver);
238. }
239.
240. module_init(ax_init);
241. module_exit(ax_exit);
242.
243. /* 驱动描述信息 */
244. MODULE_AUTHOR("Alinx");
245. MODULE_ALIAS("qspi flash");
246. MODULE_DESCRIPTION("I2C FLASH driver");
247. MODULE_VERSION("v1.0");
248. MODULE_LICENSE("GPL");
```

45~102 行是 spi-flash 的读写实现。读写操作都需要分两步走，第一步是发送命令和从机的目标地址，第二部是发送待发送数据的地址或者读出数据的 buffer。注意第一步中发送 buffer 都是 trans.tx_buf，第二步中，发送 tx_buf，读取为 re_buf。发送步骤就像之前说的，先打包

spi_transfer, 再添加到 spi_message 中, 使用 spi_sync 函数发送。

113~149 行是字符设备的 read、write 函数实现, 和第 17 章 i2c 中的实现基本一样。所以后面的测试也可以直接用第 17 章中的测试程序。

166~197 行是 probe 实现, 匹配成功后就会执行。probe 中 168~190 行首先注册字符设备。然后 192 行把 spi 赋值给设备结构体的私有变量。

之后就是匹配方式的实现, 这部分与至今为止的总线框架相比, 除了类型不同, 其他都是一样的。

19.4.4 测试程序

可以直接使用第 17 章的测试代码。

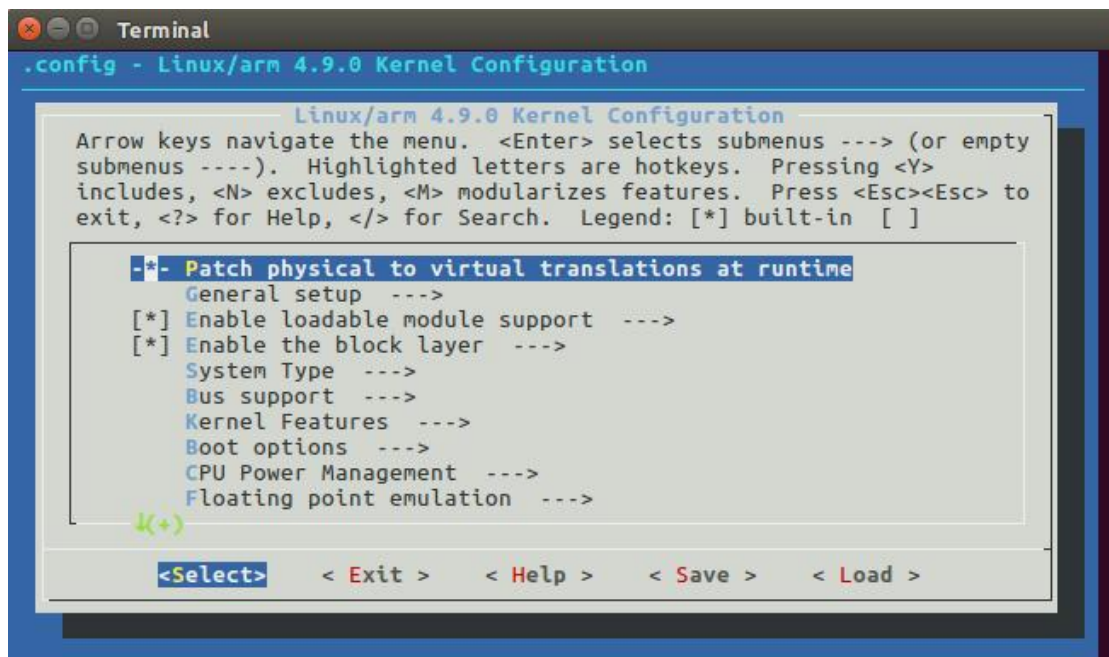
19.4.5 运行测试

petalinux 定制的系统中, 默认是不包含 spi 总线驱动的, 需要使用命令配置, 方法如下:

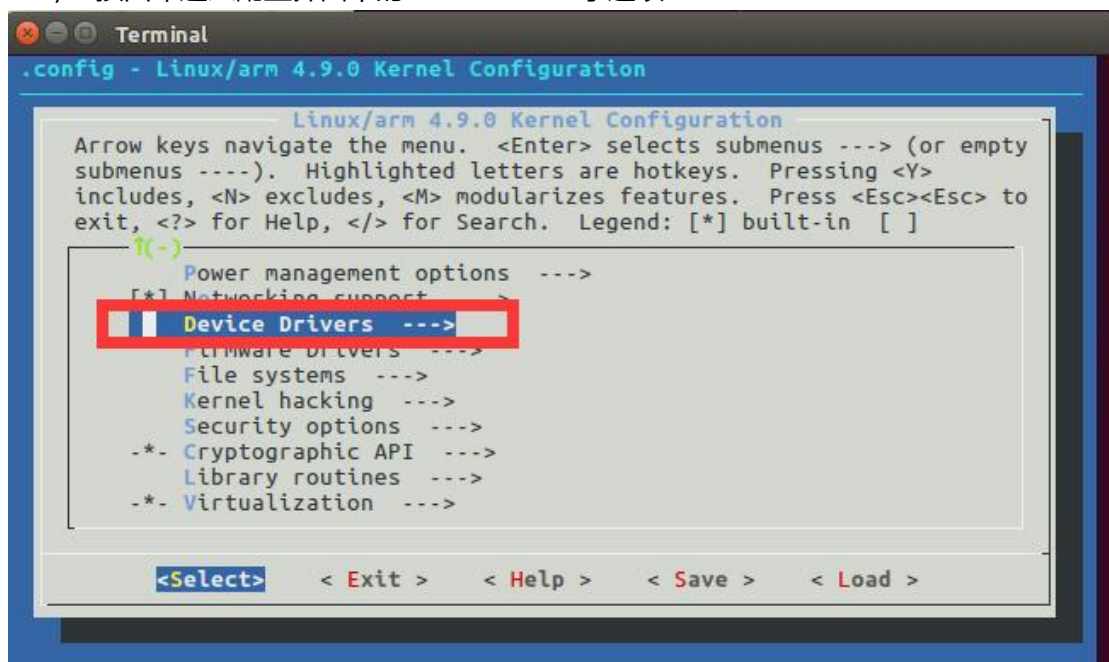
6) 在终端中输入命令配置内核 petalinux-config -c kernel, 弹出配置界面如下:

```
alinx@ubuntu:~/Downloads/peta_pri_pwm/ax_peta$ petalinux-config -c kernel
[INFO] generating Kconfig for project

[INFO] sourcing bitbake
[INFO] generating plnxtool conf
[INFO] generating meta-plnx-generated layer
~/Downloads/peta_pri_pwm/ax_peta/build/misc/plnx-generated ~/Downloads/peta_pri_pwm/ax_peta
~/Downloads/peta_pri_pwm/ax_peta
[INFO] generating machine configuration
[INFO] configuring: kernel
[INFO] generating kernel configuration files
[INFO] bitbake virtual/kernel -c menuconfig
Loading cache: 100% |#####| Time: 0:00:00
Loaded 3278 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:01
Parsing of 2487 .bb files complete (2455 cached, 32 parsed). 3280 targets, 226 skipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |#####| Time: 0:00:04
NOTE: Executing RunQueue Tasks
NOTE: Tasks Summary: Attempted 2 tasks of which 0 didn't need to be rerun and all succeeded.
Loading cache: 100% |#####| Time: 0:00:00
Loaded 3278 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:01
Parsing of 2487 .bb files complete (2455 cached, 32 parsed). 3280 targets, 226 skipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |#####| Time: 0:00:04
NOTE: Executing SetScene Tasks
NOTE: Executing RunQueue Tasks
Currently 1 running tasks (318 of 318) 99% |#####|
0: linux-xlnx-4.9-xilinx-v2017.4+gitAUTOINC+b450e900fd-r0 do_menuconfig - 24s (pid 122206)
```

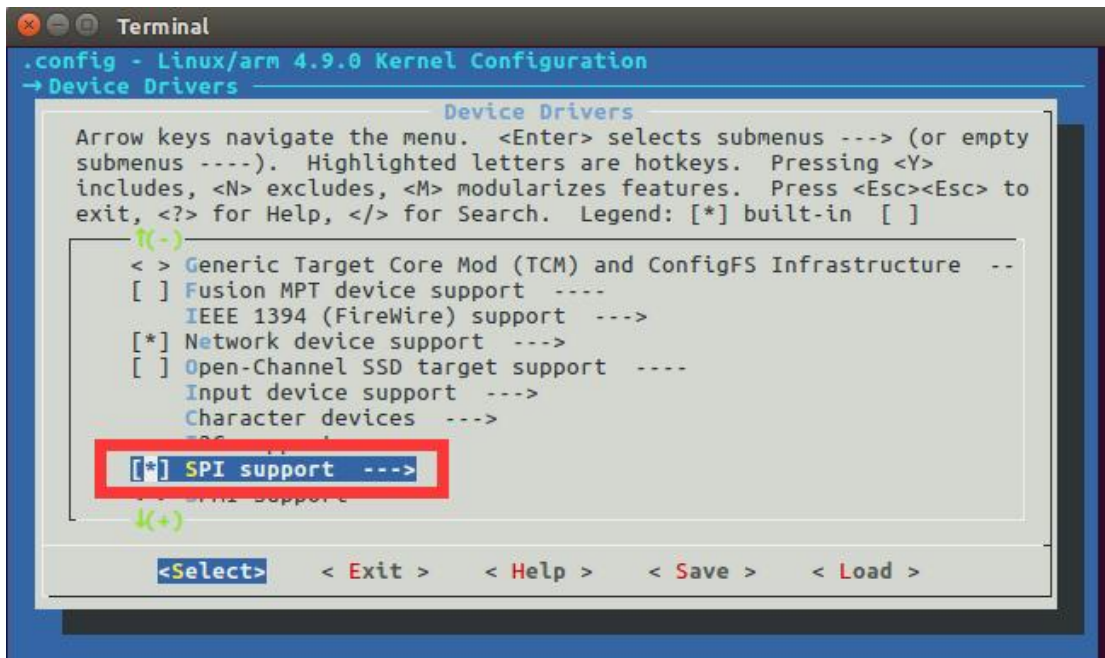



7) 按回车进入配置界面中的 Device Drivers 子选项

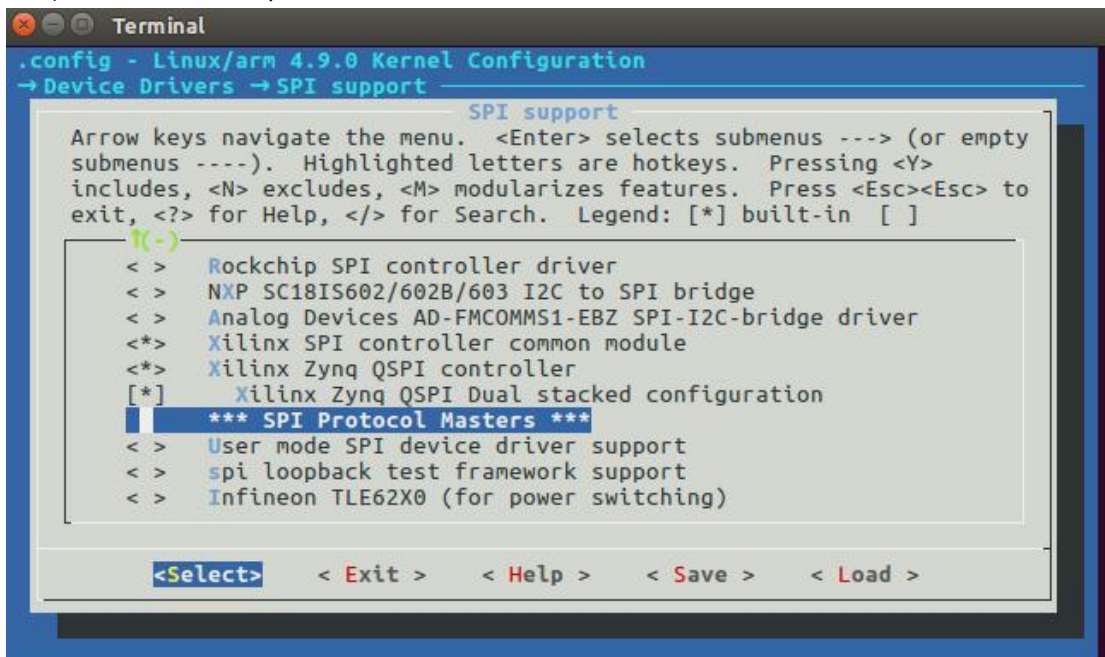


8) 选择 SPI support

先按空格键给 SPI support 选项打上*号，然后按回车进入选项。



9) 选择 xilinx 的 spi 总线驱动, 选择 save, 然后 exit



测试方法步骤如下:

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ./ax-spi-drw.ko
cd ./build-ax_i2c_test-ZYNQ-Debug/
./ax_i2c_test /dev/ax_flash
```

IP 和路径根据实际情况调整。

串口工具中的调试结果如下:


```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ./ax-spi-driv.ko
ax_spi driv: loading out-of-tree module taints kernel.
flash probe
ax_qflash node find
Manufacture ID: 0xef
Device ID: 0x4019
[drm] load() is deferred & will be called again
root@ax_peta:/mnt# cd ./build-ax_i2c_test-ZYNQ-Debug/
root@ax_peta:/mnt/build-ax_i2c_test-ZYNQ-Debug# ./ax_i2c_test /dev/ax_flash
write data 5A, 55, AA
read data 5A, 55, AA
root@ax_peta:/mnt/build-ax_i2c_test-ZYNQ-Debug#
```

读写结果一致，试验成功。

第二十章 串口驱动

串口是常用的外设，和其他 mcu 通讯、和传感器 2G、GPS 等模块通讯，都经常会选择串口。在 Linux 系统下也不例外，我们通过串口工具来和开发板进行交互，无疑就是用到了串口。串口有很多电平标准，TTL、232、485 等等，但他们的驱动程序都是一样的。在嵌入式 Linux 系统中，串口被看成终端设备（tty），包括 3 个结构体：uart_driver、uart_port、uart_ops 定义在文件 include/serial_core.h 中。实现一个 uart 驱动程序只要实现这 3 个结构体即可。

uart 的驱动一般也是由芯片厂家提供。xilinx 提供的 uart 驱动在 drivers/tty/serial/xilinx_uartps.c 中。我们只要在设备树中添加相应的串口节点，就可以使用串口外设了。这章就来学习 Linux 中串口驱动框架，并对 xilinx 的串口驱动实现稍做了解。

20.1 uart 驱动框架

1) usb_driver

Linux 内核中使用 usb_driver 来表示，uart_driver 包含了串口设备名、串口驱动名、主次设备号等信息，并且包含了 tty_driver 使得底层串口驱动无需关心 tty_driver。

uart_driver 定义在文件 include/linux/serial_core.h 中，如下：

```
1. struct uart_driver {
2.     struct module *owner;
3.     const char *driver_name;
4.     const char *dev_name;
5.     int major;
6.     int minor;
7.     int nr;
8.     struct console *cons;
9.
10.    /*
11.     * these are private; the low level driver should not
12.     * touch these; they should be initialised to NULL
13.     */
14.    struct uart_state *state;
15.    struct tty_driver *tty_driver;
16. };
```

owner 一般为 THIS_MODULE。

driver_name 为驱动名。

dev_name 为设备名。

cons 为控制台。

定义好 uart_driver 后使用下面你的函数来向内核注册：

```
int uart_register_driver(struct uart_driver *drv)
```

drv：要注册的 uart_driver。

返回值：0 成功，负值失败。

相对的注销 uart_driver 使用下面的函数：

```
void uart_unregister_driver(struct uart_driver *drv)
```

drv 为要注销的 uart_driver。

2) uart_prot

uart_port 表示一个具体的 uart 端口，用于描述一个 uart 端口的 I/O 端口或 I/O 内存地址、FIFO 大小、端口类型等信息。定义在文件 include/linux/serial_core.h 中，如下：

```

117. struct uart_port {
118.     spinlock_t    lock;           /* port lock */
119.     unsigned long  iobase;        /* in/out[bwl] */
120.     unsigned char  __iomem *membase; /* read/write[bwl] */
121.     unsigned int    (*serial_in)(struct uart_port *, int);
122.     void            (*serial_out)(struct uart_port *, int, int);
123.     void            (*set_termios)(struct uart_port *,
124.                                     struct ktermios *new,
125.                                     struct ktermios *old);
126.     unsigned int    (*get_mctrl)(struct uart_port *);
127.     void            (*set_mctrl)(struct uart_port *, unsigned int);
128.     int             (*startup)(struct uart_port *port);
129.     void            (*shutdown)(struct uart_port *port);
130.     .....
237.     const struct uart_ops *ops;
238.     unsigned int    custom_divisor;
239.     unsigned int    line;          /* port index */
240.     unsigned int    minor;
241.     resource_size_t mapbase;       /* for ioremap */
242.     resource_size_t mapsize;
243.     struct device    *dev;         /* parent device */
244.     unsigned char    hub6;         /* this should be in the 8250 driver */
245.     unsigned char    suspended;
246.     unsigned char    irq_wake;
247.     unsigned char    unused[2];
248.     struct attribute_group *attr_group; /* port specific attributes */
249.     const struct attribute_group **tty_groups; /* all attributes (serial core use only) */
250.     struct serial_rs485 rs485;
251.     void            *private_data; /* generic platform data pointer */
252. };

```

其中最主要的就是 ops 串口驱动操作函数集。uart_port 定义后需要和 uart_driver 关联。使用下面的函数把 uart_port 添加到 uart_driver 中：

```
int uart_add_one_port(struct uart_driver *drv, struct uart_port *uport)
```

drv：注册目标 uart_port 对应的 uart_driver。

uport：要添加到 uart_driver 的 uart_port。

返回值：0 成功；负值失败。

相对的使用下面的函数把 uart_port 从 uart_driver 中删除：

```
int uart_remove_one_port(struct uart_driver *drv, struct uart_port *uport)
```

drv：卸载目标 uart_port 对应的 uart_driver。

uport：要卸载的 uart_port。

返回值：0 成功；负值失败。

3) uart_ops

uart_ops 是 uart_port 中重要的成员，是 uart 具体驱动函数的集合。内核使用串口收发数据最终都是调用 ops 中的函数。uart_ops 定义在文件 include/linux/serial_core.h 中，如下：

```

1. struct uart_ops {
2.     unsigned int    (*tx_empty)(struct uart_port *);
3.     void            (*set_mctrl)(struct uart_port *, unsigned int mctrl);
4.     unsigned int    (*get_mctrl)(struct uart_port *);
5.     void            (*stop_tx)(struct uart_port *);
6.     void            (*start_tx)(struct uart_port *);
7.     void            (*throttle)(struct uart_port *);

```

```

8.     void      (*unthrottle)(struct uart_port *);
9.     void      (*send_xchar)(struct uart_port *, char ch);
10.    void      (*stop_rx)(struct uart_port *);
11.    void      (*enable_ms)(struct uart_port *);
12.    void      (*break_ctl)(struct uart_port *, int ctl);
13.    int       (*startup)(struct uart_port *);
14.    void      (*shutdown)(struct uart_port *);
15.    void      (*flush_buffer)(struct uart_port *);
16.    void      (*set_termios)(struct uart_port *, struct ktermios *new,
17.                          struct ktermios *old);
18.    void      (*set_ldisc)(struct uart_port *, struct ktermios *);
19.    void      (*pm)(struct uart_port *, unsigned int state,
20.                  unsigned int oldstate);
21.
22.    /*
23.     * Return a string describing the type of the port
24.     */
25.    const char *(*type)(struct uart_port *);
26.
27.    /*
28.     * Release IO and memory resources used by the port.
29.     * This includes iounmap if necessary.
30.     */
31.    void      (*release_port)(struct uart_port *);
32.
33.    /*
34.     * Request IO and memory resources used by the port.
35.     * This includes iomapping the port if necessary.
36.     */
37.    int       (*request_port)(struct uart_port *);
38.    void      (*config_port)(struct uart_port *, int);
39.    int       (*verify_port)(struct uart_port *, struct serial_struct *);
40.    int       (*ioctl)(struct uart_port *, unsigned int, unsigned long);
41. #ifdef CONFIG_CONSOLE_POLL
42.    int       (*poll_init)(struct uart_port *);
43.    void      (*poll_put_char)(struct uart_port *, unsigned char);
44.    int       (*poll_get_char)(struct uart_port *);
45. #endif
46. };

```

uart_ops 中的函数需要底层开发人员去实现，是直接和寄存器打交道的部分。uart_ops 结构体中的这些函数的具体含义可以参考文档 Documentation/serial/driver。

总结一下，一个串口驱动要完成的主要工作为：

1. 定义 uart_driver、uart_ops、uart_port 等结构体变量并初始化。
2. 驱动模块加载时使用 uart_register_driver() 和 uart_add_one_port() 函数注册 uart_driver 并添加端口。驱动模块卸载时使用函数 uart_unregister_driver() 和 uart_remove_one_port() 来注销 uart_driver 并移除端口。
3. 根据具体硬件的 datasheet 实现 uart_ops 中的成员函数。

然后我们再对应到具体的驱动代码来看。看看 xilinx 的 uart 实现。

20.2 xilinx 的 uart 驱动

先看看设备树中 uart 的节点描述，打开文件 zynq-7000.dtsi。找到 uart 相关节点，如下：

```

1. uart0: serial@e0000000 {
2.     compatible = "xlnx,xuartps", "cdns,uart-r1p8";
3.     status = "disabled";
4.     clocks = <&clkc 23>, <&clkc 40>;
5.     clock-names = "uart_clk", "pclk";
6.     reg = <0xE0000000 0x1000>;
7.     interrupts = <0 27 4>;

```

```

8.  };
9.
10. uart1: serial@e0001000 {
11.     compatible = "xlnx,xuartps", "cdns,uart-r1p8";
12.     status = "disabled";
13.     clocks = <&clkc 24>, <&clkc 41>;
14.     clock-names = "uart_clk", "pclk";
15.     reg = <0xE0001000 0x1000>;
16.     interrupts = <0 50 4>;
17. };

```

节点中两个 uart 节点都是 ps 端的串口，zynq 的 ps 端就只有两路串口，如果需要更多，就需要借助 pl 端的资源。

首先根据 compatible 属性找到对应的驱动代码，为文件 drivers/tty/serial/xilinx_uartps.c。其中 of_device_id 如下：

```

1.  /* Match table for of_platform binding */
2.  static const struct of_device_id cdns_uart_of_match[] = {
3.      { .compatible = "xlnx,xuartps", },
4.      { .compatible = "cdns,uart-r1p8", },
5.      { .compatible = "cdns,uart-r1p12", .data = &zynqmp_uart_def },
6.      { .compatible = "xlnx,zynqmp-uart", .data = &zynqmp_uart_def },
7.      {}
8.  };

```

在驱动代码 xilinx_uartps.c 中 1614 行，会发现如下代码：

```

1.  static struct platform_driver cdns_uart_platform_driver = {
2.      .probe = cdns_uart_probe,
3.      .remove = cdns_uart_remove,
4.      .driver = {
5.          .name = CDNS_UART_NAME,
6.          .of_match_table = cdns_uart_of_match,
7.          .pm = &cdns_uart_dev_pm_ops,
8.      },
9.  };

```

可见，uart 本质上是一个 platform 驱动。

然后根据前面说的 uart 框架，来对应到这个驱动代码中去。

1) uart_driver

```

1.  static struct uart_driver cdns_uart_uart_driver = {
2.      .owner = THIS_MODULE,
3.      .driver_name = CDNS_UART_NAME,
4.      .dev_name = CDNS_UART_TTY_NAME,
5.      .major = CDNS_UART_MAJOR,
6.      .minor = CDNS_UART_MINOR,
7.      .nr = CDNS_UART_NR_PORTS,
8.      #ifdef CONFIG_SERIAL_XILINX_PS_UART_CONSOLE
9.      .cons = &cdns_uart_console,
10. #endif
11.  };
12.  .....
13.  static int __init cdns_uart_init(void)
14.  {
15.      int retval = 0;
16.
17.      /* Register the cdns_uart driver with the serial core */
18.      retval = uart_register_driver(&cdns_uart_uart_driver);
19.      if (retval)
20.          return retval;
21.      .....
22.      return retval;
23.  }

```

```
24.
25. static void __exit cdns_uart_exit(void)
26. {
27.     /* Unregister the platform driver */
28.     platform_driver_unregister(&cdns_uart_platform_driver);
29.     .....
30. }
```

可以找到 `uart_driver` 类型的变量 `cdns_uart_uart_driver`，并且初始化了。

之后在驱动入口函数和出口函数中，分别有对应的注册和注销方法。

2) uart_port

```
1. static struct uart_port cdns_uart_port[CDNS_UART_NR_PORTS];
2.
3. /**
4.  * cdns_uart_get_port - Configure the port from platform device resource info
5.  * @id: Port id
6.  *
7.  * Return: a pointer to a uart_port or NULL for failure
8.  */
9. static struct uart_port *cdns_uart_get_port(int id)
10. {
11.     struct uart_port *port;
12.
13.     /* Try the given port id if failed use default method */
14.     if (cdns_uart_port[id].mapbase != 0) {
15.         /* Find the next unused port */
16.         for (id = 0; id < CDNS_UART_NR_PORTS; id++)
17.             if (cdns_uart_port[id].mapbase == 0)
18.                 break;
19.     }
20.
21.     if (id >= CDNS_UART_NR_PORTS)
22.         return NULL;
23.
24.     port = &cdns_uart_port[id];
25.
26.     /* At this point, we've got an empty uart_port struct, initialize it */
27.     spin_lock_init(&port->lock);
28.     port->membase = NULL;
29.     port->irq = 0;
30.     port->type = PORT_UNKNOWN;
31.     port->iotype = UPIO_MEM32;
32.     port->flags = UPF_BOOT_AUTOCONF;
33.     port->ops = &cdns_uart_ops;
34.     port->fifosize = CDNS_UART_FIFO_SIZE;
35.     port->line = id;
36.     port->dev = NULL;
37.     return port;
38. }
39.
40. static int cdns_uart_probe(struct platform_device *pdev)
41. {
42.     int rc, id, irq;
43.     struct uart_port *port;
44.     struct resource *res;
45.     struct cdns_uart *cdns_uart_data;
46.     const struct of_device_id *match;
47.     .....
48.     /* Look for a serialN alias */
49.     id = of_alias_get_id(pdev->dev.of_node, "serial");
50.     if (id < 0)
51.         id = 0;
52. }
```

```
53.     /* Initialize the port structure */
54.     port = cdns_uart_get_port(id);
55.
56.     if (!port) {
57.         dev_err(&pdev->dev, "Cannot get uart_port structure\n");
58.         rc = -ENODEV;
59.         goto err_out_notif_unreg;
60.     }
61.
62.     /*
63.      * Register the port.
64.      * This function also registers this device with the tty layer
65.      * and triggers invocation of the config_port() entry point.
66.      */
67.     port->mapbase = res->start;
68.     port->irq = irq;
69.     port->dev = &pdev->dev;
70.     port->uartclk = clk_get_rate(cdns_uart_data->uartclk);
71.     port->private_data = cdns_uart_data;
72.     cdns_uart_data->port = port;
73.     platform_set_drvdata(pdev, port);
74.
75.     .....
76.     rc = uart_add_one_port(&cdns_uart_driver, port);
77.     if (rc) {
78.         dev_err(&pdev->dev,
79.             "uart_add_one_port() failed; err=%i\n", rc);
80.         goto err_out_pm_disable;
81.     }
82.
83.     .....
84. }
85.
```

在程序中，可以找到如上的代码片段，首先定义了 `uart_port` 型的数组

`cdns_uart_port[CDNS_UART_NR_PORTS]`。

又实现了函数 `cdns_uart_get_port` 来对 `uart_port` 实现初始化。

在 `probe` 函数中 73 行(实际源码中 1554 行)，把 `uart_port` 添加到设备驱动结构体的私有数据中，以便于在之后 `ops` 函数实现时调用。

同样在 `probe` 函数 76 行(实际源码 1561 行)，调用 `uart_add_one_port` 把 `uart_port` 添加到 `uart_driver` 中。

3) `uart_ops`

`uart_ops` 变量定义在 1081 行，名为 `cdns_uart_ops`，如下：

```
1. static const struct uart_ops cdns_uart_ops = {
2.     .set_mctrl = cdns_uart_set_mctrl,
3.     .get_mctrl = cdns_uart_get_mctrl,
4.     .start_tx   = cdns_uart_start_tx,
5.     .stop_tx    = cdns_uart_stop_tx,
6.     .stop_rx    = cdns_uart_stop_rx,
7.     .tx_empty   = cdns_uart_tx_empty,
8.     .break_ctl  = cdns_uart_break_ctl,
9.     .set_termios = cdns_uart_set_termios,
10.    .startup     = cdns_uart_startup,
11.    .shutdown    = cdns_uart_shutdown,
12.    .pm          = cdns_uart_pm,
13.    .type        = cdns_uart_type,
14.    .verify_port = cdns_uart_verify_port,
15.    .request_port = cdns_uart_request_port,
```



```
16.     .release_port  = cdns_uart_release_port,  
17.     .config_port   = cdns_uart_config_port,  
18. #ifdef CONFIG_CONSOLE_POLL  
19.     .poll_get_char  = cdns_uart_poll_get_char,  
20.     .poll_put_char  = cdns_uart_poll_put_char,  
21. #endif  
22. };
```

在 1112 行的 `uart_port` 初始化函数中，其中的 `ops` 就是赋值为 `cdns_uart_ops`。
`cdns_uart_ops` 中的函数，也就是 `uart` 具体的驱动函数了。

第二十一章 块设备驱动

Linux 设备被分为字符设备、块设备、网络设备。前面介绍的框架都是和字符设备结合起来实验的，这章换换口味来说说块设备。

21.1 块设备简介

块设备被区别于字符设备肯定是有他的特殊性。

先回顾一下字符设备，在之前的字符设备实验中，字符设备框架部分几乎可以保证通用，少有区别的部分只有 file_operations 中操作函数的实现。既然能保证通用性，又为何要引入块设备的概念呢？

块设备较于字符设备最突出的特征就是，他需要支持批量的数据读写操作。当然多次调用字符设备的读写也可以达成最终目的，但具体到硬件操作就会遇到几个问题：

- 1) 一些块设备如 flash 在写之前需要先擦除当前扇区
- 2) 一些块设备的机械结构需要按地址顺序读写才能达到最高效率

因此块设备的驱动中应该要有这些功能：

- 1) 有读出扇区数据的 buffer
- 2) 有读写操作队列，并能按地址优化排列读写顺序

而块设备框架的引入正是应对了这些问题。

21.2 内核中的块设备访问

从应用层到硬件，块设备的访问过程如下：

- 1) 应用程的 app 调用 read 等方法操作文件如 txt 等
- 2) 经由文件系统 ext2、ext3 等，把对文件的操作转化为对扇区的读写
- 3) 调用块设备驱动中对应的操作函数
- 4) 操作硬件

与字符设备的差异在于中间多了一个文件系统。这一步中，有一个关键的函数 ll_rw_block()，从这个函数开始，就进入设备层面了。

ll_rw_block()函数，定义在文件 fs/buffer.c 中。如下：

```
1. void ll_rw_block(int op, int op_flags, int nr, struct buffer_head *bhs[])
2. {
3.     int i;
4.
5.     for (i = 0; i < nr; i++) {
6.         struct buffer_head *bh = bhs[i];
7.
8.         if (!trylock_buffer(bh))
9.             continue;
10.        if (op == WRITE) {
11.            if (test_clear_buffer_dirty(bh)) {
```

```

12.         bh->b_end_io = end_buffer_write_sync;
13.         get_bh(bh);
14.         submit_bh(op, op_flags, bh);
15.         continue;
16.     }
17. } else {
18.     if (!buffer_uptodate(bh)) {
19.         bh->b_end_io = end_buffer_read_sync;
20.         get_bh(bh);
21.         submit_bh(op, op_flags, bh);
22.         continue;
23.     }
24. }
25. unlock_buffer(bh);
26. }
27. }

```

输入参数中的 buffer_head 结构体是缓冲区的描述符，存放缓冲区信息，定义如下：

```

1. struct buffer_head {
2.     unsigned long b_state;           //缓冲区状态
3.     struct buffer_head *b_this_page; //页缓冲区头
4.     struct page *b_page;             //存储缓冲区所在页
5.     sector_t b_blocknr;              //块号
6.     size_t b_size;                   //块大小
7.     char *b_data;                    //页数据缓冲区
8.
9.     struct block_device *b_bdev;      //块设备
10.
11.     bh_end_io_t *b_end_io;            //I/O 完成后执行函数指针
12.
13.     void *b_private;                  //b_end_io 的输入参数
14.
15.     struct list_head b_assoc_buffers; //映射链表
16.
17.     /* mapping this buffer is associated with */
18.     struct address_space *b_assoc_map;
19.     atomic_t b_count;                 //缓冲区计数
20. };

```

再看 ll_rw_block()函数中 14 和 21 行最终做的操作 submit_bh()函数。submit_bh()是对函数 submit_bh_wbc()的简化封装，submit_bh_wbc()定义如下(省略原有注释)：

```

1. int submit_bh(int rw, struct buffer_head * bh)
2. {
3.     struct bio *bio;                  //定义一个
4.     bio(block input output),也就是块设备 i/o
5.     BUG_ON(!buffer_locked(bh));
6.     BUG_ON(!buffer_mapped(bh));
7.     BUG_ON(!bh->b_end_io);
8.     BUG_ON(buffer_delay(bh));
9.     BUG_ON(buffer_unwritten(bh));
10.
11.     if (test_set_buffer_req(bh) && (op == REQ_OP_WRITE))
12.         clear_buffer_write_io_error(bh);

```

```

12.
13.     bio = bio_alloc(GFP_NOIO, 1); //分配 bio
14.
15.     if (wbc) {
16.         wbc_init_bio(wbc, bio);
17.         wbc_account_io(wbc, bh->b_page, bh->b_size);
18.     }
19.
20.     /* 构造 bio */
21.     bio->bi_sector = bh->b_blocknr * (bh->b_size >> 9); //存放逻辑块号
22.     bio->bi_bdev = bh->b_bdev; //存放对应的块设
    备
23.
24.     bio_add_page(bio, bh->b_page, bh->b_size, bh_offset(bh));
25.     BUG_ON(bio->bi_iter.bi_size != bh->b_size);
26.
27.     bio->bi_end_io = end_bio_bh_io_sync; //设置 i/o 回调函
    数
28.     bio->bi_private = bh; //回调函数参
    数, 指向哪个缓冲区
29.     bio->bi_flags |= bio_flags;
30.
31.     guard_bio_eod(op, bio);
32.
33.     if (buffer_meta(bh))
34.         op_flags |= REQ_META;
35.     if (buffer_prio(bh))
36.         op_flags |= REQ_PRIO;
37.     bio_set_op_attrs(bio, op, op_flags);
38.
39.     submit_bio(bio); //提交 bio
40.     return 0;
41. }

```

这里的工作主要就是使用 `buffer_head` 来构造一个 `bio`, `bio` 即 `block input/output`, 块设备输入输出。最终调用 `submit_bio()` 函数提交 `bio` 到内核队列。

再往下看, `submit_bio()` 函数最终调用了 `generic_make_request()`, 再往下调用 `__generic_make_request()`, 再到 `__make_request()`, 在 `__make_request()` 函数中的 `elv_merge()` 函数是内核中的电梯算法(`elevator merge`), 所谓的电梯算法, 也就是实现了我们上面说的按地址顺序优化读写顺序。

我们担心的事情内核已经帮我们完成了, 而块设备框架, 则在此基础上展开。

21.3 块设备框架

字符设备框架可以参考内核中先用的字符设备驱动代码 `drivers/block/zram.c` 或其他。

大致步骤如下:

- 1) 分配一个 `gendisk` 结构体
- 2) 分配一个队列 `request_queue_t`
- 3) 设置 `gendisk` 结构体的成员

- 4) 注册 gendisk 结构体
分别来看。

21.3.1 gendisk 结构体

gendisk 结构体定义在 genhd.h 中，如下：

```
1. struct gendisk {
2.     /* major, first_minor and minors are input parameters only,
3.      * don't use directly. Use disk_devt() and disk_max_parts().
4.      */
5.     int major;           /* major number of driver */
6.     int first_minor;
7.     int minors;          /* maximum number of minors, =1 for
8.                          * disks that can't be partition
9.     ed. */
10.    char disk_name[DISK_NAME_LEN]; /* name of major driver */
11.    char *(*devnode)(struct gendisk *gd, umode_t *mode);
12.
13.    unsigned int events;    /* supported events */
14.    unsigned int async_events; /* async events, subset of all */
15.
16.    /* Array of pointers to partitions indexed by partno.
17.     * Protected with matching bdev lock but stat and other
18.     * non-critical accesses use RCU. Always access through
19.     * helpers.
20.     */
21.    struct disk_part_tbl __rcu *part_tbl;
22.    struct hd_struct part0;
23.
24.    const struct block_device_operations *fops;
25.    struct request_queue *queue;
26.    void *private_data;
27.
28.    int flags;
29.    struct kobject *slave_dir;
30.
31.    struct timer_rand_state *random;
32.    atomic_t sync_io;      /* RAID */
33.    struct disk_events *ev;
34. #ifdef CONFIG_BLK_DEV_INTEGRITY
35.    struct kobject integrity_kobj;
36. #endif /* CONFIG_BLK_DEV_INTEGRITY */
37.    int node_id;
38.    struct badblocks *bb;
39. };
```

部分成员介绍：

1. major 为设备主设备号。
2. first_minor 起始次设备号。
3. minors 次设备号的数量，也称为分区数量，值为 1 表示无法分区。
4. disk_name 设备名称。
5. part 分区表的信息。
6. fops 块设备操作集合，块设备的操作函数与字符设备差别较大，碰到了再去研究。

7. queue 申请队列, 用于管理该设备 IO 申请队列的指针。
8. private_data 私有数据。
9. capacity 扇区数。
gendisk 的操作大概如下:
 - 1) 在块设备驱动程序中, 首先定义一个 gendisk 结构体变量, 在入口函数中使用函数 struct gendisk *alloc_disk(int minors)分配空间, minors 是分区数量。
 - 2) 分配队列 queue。
 - 3) 使用 register_blkdev 函数向内核申请块设备的主设备号。
 - 4) 使用函数 set_capacity()设置扇区数。
 - 5) 设置其他参数。
 - 6) 使用函数 add_disk()向内和注册 gendisk。
 - 7) 在出口函数中实现相反的操作:
使用 put_disk()和 del_gendisk()来注销, 释放 gendisk 结构体。
使用 kfree()释放磁盘扇区缓存。
使用 blk_cleanup_queue()清除内存中的申请队列。
使用 unregister_blkdev()卸载块设备。

21.3.2 request_queue

首先定义一个 request_queue 结构体变量。使用下面函数来分配:

```
struct request_queue *blk_init_queue(request_fn_proc *rfn, spinlock_t *lock)
```

rfn 是队列处理函数。

lock 是自旋锁。

使用 blk_init_queue 函数获取到 request_queue 之后复制给 ramblock_disk.queue 即可。

这里 rfn 函数的实现, 也就是块设备中的重点了。

块设备框架的使用, 到实验例程中去讲解。

21.4 实验

本章实验把内存模拟成块设备来操作, 重在熟悉块设备的框架和块设备驱动编写流程。

21.1.1 驱动程序

使用 petalinux 新建名为“ax-block-driv”驱动程序, 并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-block-driv.c 文件中输入下面的代码:

```
1. #include <linux/module.h>
2. #include <linux/errno.h>
3. #include <linux/interrupt.h>
4. #include <linux/mm.h>
5. #include <linux/fs.h>
6. #include <linux/kernel.h>
7. #include <linux/timer.h>
8. #include <linux/hdreg.h>
```

```
9. #include <linux/ioport.h>
10. #include <linux/init.h>
11. #include <linux/wait.h>
12. #include <linux/blkdev.h>
13. #include <linux/blkpg.h>
14. #include <linux/delay.h>
15. #include <linux/io.h>
16. #include <asm/mach/map.h>
17. #include <asm/uaccess.h>
18. #include <asm/dma.h>
19.
20. #define AX_BLOCK_SIZE    (1024*64)           //块设备大小
21. #define SECTOR_SIZE      512                 //扇区大小
22. #define SECTORS_NUM      AX_BLOCK_SIZE / SECTOR_SIZE //扇区数
23.
24. #define AX_BLOCK_NAME     "ax_block"          //设备节点名称
25.
26. #define AX_BLOCK_MAJOR    40
27. struct ax_block{
28.     struct gendisk        *block_disk;       //磁盘结构体
29.     struct request_queue  *block_request;    //申请队列
30.     unsigned char         *block_buf;        //磁盘地址
31. };
32. /* 声明设备结构体变量 */
33. struct ax_block ax_block_drv;
34. /* 定义一个自旋锁 */
35. static DEFINE_SPINLOCK(ax_block_lock);
36.
37. /* 块设备操作函数集，即使没有操作函数，也需要赋值 */
38. static struct block_device_operations ax_block_fops = {
39.     .owner    =    THIS_MODULE,
40. };
41.
42. /* 队列处理函数 */
43. static void ax_block_request(struct request_queue * q)
44. {
45.     struct request *req;
46.
47.     /* 轮询队列中的每个申请 */
48.     req = blk_fetch_request(q);
49.     while(req)
50.     {
51.         unsigned long start;
52.         unsigned long len;
53.         void *buffer = bio_data(req->bio);
54.         /* 获取首地址 */
55.         start = (int)blk_rq_pos(req) * SECTOR_SIZE + ax_block_drv.block_
buf;
56.         /* 获取长度 */
57.         len = blk_rq_cur_bytes(req);
58.
59.         if(rq_data_dir(req) == READ)
60.         {
61.             printk("ax_request read\n");
62.             memcpy(buffer, (char *)start, len);
63.         }
64.         else
65.         {
```



```
66.         printk("ax_request write\n");
67.         memcpy((char *)start, buffer, len);
68.     }
69.     /* 处理完的申请出列 */
70.     if (!__blk_end_request_cur(req, 0))
71.         req = blk_fetch_request(q);
72. }
73. }
74.
75. static int __init ax_block_init(void)
76. {
77.     /* 分配块设备 */
78.     ax_block_drv.block_disk = alloc_disk(1);
79.     /* 分配申请队列, 提供队列处理函数 */
80.     ax_block_drv.block_request = blk_init_queue(ax_block_request, &ax_block_lock);
81.     /* 设置申请队列 */
82.     ax_block_drv.block_disk->queue = ax_block_drv.block_request;
83.     /* 向内核注册块设备 */
84.     register_blkdev(AX_BLOCK_MAJOR, AX_BLOCK_NAME);
85.     /* 主设备号赋给块设备得主设备号字段 */
86.     ax_block_drv.block_disk->major = AX_BLOCK_MAJOR;
87.     /* 设置其他参数 */
88.     ax_block_drv.block_disk->first_minor = 0;
89.     ax_block_drv.block_disk->fops = &ax_block_fops;
90.     sprintf(ax_block_drv.block_disk->disk_name, AX_BLOCK_NAME);
91.     /* 设置扇区数 */
92.     set_capacity(ax_block_drv.block_disk, SECTORS_NUM);
93.     /* 获取缓存, 把内存模拟成块设备 */
94.     ax_block_drv.block_buf = kzalloc(AX_BLOCK_SIZE, GFP_KERNEL);
95.     /* 向内核注册 gendisk 结构体 */
96.     add_disk(ax_block_drv.block_disk);
97.     return 0;
98. }
99.
100. static void __exit ax_block_exit(void)
101. {
102.     /* 注销 gendisk 结构体 */
103.     put_disk(ax_block_drv.block_disk);
104.     /* 释放 gendisk 结构体 */
105.     del_gendisk(ax_block_drv.block_disk);
106.     /* 释放缓存 */
107.     kfree(ax_block_drv.block_buf);
108.     /* 清空内存中的队列 */
109.     blk_cleanup_queue(ax_block_drv.block_request);
110.     /* 卸载块设备 */
111.     unregister_blkdev(AX_BLOCK_MAJOR, AX_BLOCK_NAME);
112. }
113.
114. module_init(ax_block_init);
115. module_exit(ax_block_exit);
116.
117. /* 驱动描述信息 */
118. MODULE_AUTHOR("Alinx");
119. MODULE_ALIAS("block test");
120. MODULE_DESCRIPTION("BLOCK driver");
121. MODULE_VERSION("v1.0");
```

```
122. MODULE_LICENSE("GPL");
```

先看块设备框架，入口函数出口函数以及操作函数三个部分的结构和字符设备框架无异。多了一个在 76 行开始的驱动入口函数中：

79 行先使用函数 `alloc_disk` 分配块设备。

81 行使用 `blk_init_queue` 函数分配申请队列，并提供队列处理函数 `ax_block_request`。

`ax_block_request` 函数实现稍后再去看。

83 行把分配的队列设置到块设备结构体成员中。

85 行向内和注册块设备，需要附上设备号和设备名。87 行设置住设备设备号，89 行设置起始次设备号也是起始扇区号，90 行设置操作函数集合，91 行设置设备名称。

93 行设置扇区数，95 行使用 `kalloc` 函数获取缓存来模拟块设备。

97 行使用 `add_disk` 函数向内和注册 `gendisk` 结构体。

101 行开始的驱动出口函数中做相反的注销操作。

然后回头看 44 行队列处理函数中的内容。

内容也不多，可以看到最终的处理就是 63 行和 68 行 `memcpy`，把读的或者写的内容拷贝到 `buffer` 或者内存地址中。当然这里是应为使用内存所以可以直接使用 `memcpy` 方法，实际使用外设块设备，读写方法会复杂一些。

在 `memcpy` 的前一步，就是判断本次操作是读还是写，调用函数 `rq_data_dir` 函数，如果申请是读，就会返回 `READ`，函数参数是 `struct request` 类型的指针。注意我们的队列处理函数，输入参数是 `struct request_queue` 类型的指针，是申请的队列，使用函数 `blk_fetch_request` 函数可以从申请队列中获取队列最前的申请，如 49 行的操作。获取到的申请，就是我们后面处理的对象。

`memcpy` 的关键参数有三，数据来源，数据目标，和数据长度。这些都要通过申请结构体变量指针 `struct request *req` 来获取。54 行通过 `bio_data(req->bio)` 获取数据目标地址。56 行先使用 `blk_rq_pos(req)` 获取当前扇区，然后乘上扇区大小，再加上缓存的首地址，则是当前申请数据所在的首地址了。58 行使用方法 `blk_rq_cur_bytes(req)` 获取当前申请的数据长度。

在队列处理函数中，需要轮询队列中的每一个申请，方法就是使用函数 `blk_fetch_request(q)`，如果有申请，则返回正值。最后处理完了使用 `blk_end_request_cur(req, 0)` 函数把处理完成的申请移出队列。

另外在读和写之前加上 `printk` 来查看效果。

21.1.2 运行测试

测试方法步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ./ax-block-drv.ko
mkdosfs /dev/ax_block
cd /
```

```
mkdir test
mount /dev/ax_block /test/
cd /test/
vi test.txt
(编辑任意内容, 这里我输入了 test)
cd /
umount /test/
cat /dev/ax_block > /mnt/block_test.bin
```

IP 和路径根据实际情况调整。

然后到 ubuntu 虚拟机中，验证 block_test.bin，步骤如下：

```
alinx@ubuntu: /  
alinx@ubuntu:/$ sudo mount -o loop /home/alinx/work/block_test.bin /mnt  
[sudo] password for alinx:  
alinx@ubuntu:/$ cat /mnt/test.txt  
test  
alinx@ubuntu:/$
```

把 block_test.bin 挂载到/mnt 目录下,发现 mnt 目录中就有了我们之前创建的 test.txt 文件,并且内容也是之前输入的内容。

串口工具中的调试结果如下：

[illegible]

通过串口打应的 read 和 write 信息，也可以看出块设备内电梯算法的结果，都每次申请的

读和写不会交叉进行，提高了读写效率。

另外块设备的读写是异步操作，要确保每次申请的操作都完成，可以使用 `sync` 命令来完成当前未完成的异步操作。

第二十二章 网卡驱动

22.1 Linux 网卡驱动

注意这里说的是网卡驱动不是网络驱动。网络驱动是相当繁杂的。我们知道 OSI 七层网络模型从上往下为：应用层、表示层、会话层、传输层、网络层、数据链路层、物理层，而 Linux 中使用 TCP/IP 四层模型为：应用层、传输层、网际层、网络接口层。

OSI	Linux TCP/IP
应用层	应用层
表示层	
会话层	
传输层	传输层
网络层	网际层
数据链路层	网络接口层
物理层	

网卡驱动是被包含于网络驱动中的一环，属于网络接口层，包括数据链路层和物理层。也是网络驱动中唯一与硬件相关的部分，其他的都是纯软件的概念，对于底层驱动开发而言，只需要关心硬件相关的内容，因此这里我们来学习网卡驱动。

网卡驱动需要实现两个部分，一是和上层交互数据收发，二是结合具体硬件处理数据。网络接口层中又可以细分四层：网络协议接口层、网络设备接口层、设备驱动功能层、网络设备与媒介层。网络协议接口层负责对上层发送或接收数据，设备驱动层负责对硬件发送或接收数据，网络设备接口层则在中间起到数据承接的作用，网络设备层则负责实际的数据收发操作。

这里的关键就是网络设备接口层，Linux 使用 `net_device` 结构体来描述一个网络设备信息，并对不同得硬件做了概括和统一。

从功能得角度看，网卡设备需要实现的功能有三，初始化、发送和接受，我们以 `net_device` 结构体为切入点，以三个需要实现的功能为引来看 Linux 网卡驱动程序的实现方法。

22.1.1 初始化

`net_device` 结构体表示一个网卡，网卡的初始化，实际也就是定义一个 `net_device` 结构体变量，并初始化这个变量。先看一下 `net_device` 结构体的具体定义，在文件 `include/linux/netdevice.h` 中，如下(有省略和更改)：

```
1. struct net_device {
2.     char          name[IFNAMSIZ]; //网卡设备名称
3.     struct hlist_node  name_hlist;
4.     char          *ifalias;
5.
6.     unsigned long    mem_end;      //该设备的内存结束地址
7.     unsigned long    mem_start;    //该设备的内存起始地址
8.     unsigned long    base_addr;    //该设备的内存 I/O 基地址
```

```

9.     int          irq;                //该设备的中断号
10.
11.     atomic_t      carrier_changes;
12.
13.     unsigned long  state;            //网络设备和网络适配器的状态信息
14.
15.     .....
16.
17.     struct net_device_stats stats;    //用来保存统计信息的 net_device_stats 结构体
18.
19.     .....
20.
21.     const struct net_device_ops *netdev_ops;
22.                                     //网络设备操作函数
23.     .....
24.
25.     unsigned int   flags;            //flags 指网络接口标志,常见取值如下
26.                                     //IFF_UP:设备被激活并可以开始发送数据包时设置该
标志
27.                                     //IFF_AUTOMEDIA:设置设备可在多种媒介间切换
28.                                     //IFF_BROADCAST:允许广播
29.                                     //IFF_DEBUG:调试模式
30.                                     //IFF_LOOPBACK:回环
31.                                     //IFF_NOARP:无可执行 ARP,点对点接口就不需要运行
ARP
32.                                     //IFF_POINTOPOINT:点到点链路
33.     .....
34.
35.     unsigned int    mtu;              //最大数据包
36.     unsigned short  type;             //接口的硬件类型
37.     unsigned short  hard_header_len;
38.                                     //硬件帧头长度,一般被赋为 ETH_HLEN 值为 14
39.
40.     .....
41.     unsigned long    last_rx;         //接收数据包的时间戳,调用 netif_rx()后赋上
jiffies 即可
42.     unsigned char    *dev_addr;      //MAC 地址
43.
44.     .....
45. };

```

除了一些基本信息名称、地址之外,最为关键的成员就是 struct net_device_ops 类型的指针变量 netdev_ops, 他包含了网络设备的操作函数集我们需要实现的发包函数也包含在其中, 看一下他的具体定义, 也在这个文件中, 如下(有省略):

```

1. struct net_device_ops {
2.     .....
3.     int (*ndo_open)(struct net_device *dev);
4.     int (*ndo_stop)(struct net_device *dev);
5.     netdev_tx_t (*ndo_start_xmit)(struct sk_buff *skb,
6.                                   struct net_device *dev);
7.     .....
8. };

```

ndo_open 用于开启网络设备, ndo_stop 用于关闭网络设备。

ndo_start_xmit 函数就是关键的发包函数了。

再看一下 net_device 结构体中的 struct net_device_stats stats 成员, net_device_stats 结构体定义如下:

```
1. struct net_device_stats {
2.     unsigned long rx_packets;
3.     unsigned long tx_packets;
4.     unsigned long rx_bytes;
5.     unsigned long tx_bytes;
6.     unsigned long rx_errors;
7.     unsigned long tx_errors;
8.     unsigned long rx_dropped;
9.     unsigned long tx_dropped;
10.    unsigned long multicast;
11.    unsigned long collisions;
12.    unsigned long rx_length_errors;
13.    unsigned long rx_over_errors;
14.    unsigned long rx_crc_errors;
15.    unsigned long rx_frame_errors;
16.    unsigned long rx_fifo_errors;
17.    unsigned long rx_missed_errors;
18.    unsigned long tx_aborted_errors;
19.    unsigned long tx_carrier_errors;
20.    unsigned long tx_fifo_errors;
21.    unsigned long tx_heartbeat_errors;
22.    unsigned long tx_window_errors;
23.    unsigned long rx_compressed;
24.    unsigned long tx_compressed;
25.};
```

实际就是网络状态的统计信息合集,我们会在收发处理函数中去操作这个 stats 来记录状态。

可见 net_device 结构体变量是网卡设备的关键,网卡设备的初始化也就是 net_device 结构体变量的初始化,步骤如下:

- 1) 定义一个 net_device 结构体变量
- 2) 使用函数 alloc_netdev()分配一个 net_device 结构体变量
- 3) 初始化硬件寄存器
- 4) 设置 net_device 结构体变量成员变量
- 5) 使用 register_netdev()注册 net_device 结构体变量

alloc_netdev()是个宏定义,具体如下:

```
alloc_netdev(sizeof_priv, name, name_assign_type, setup)
```

第一个参数 sizeof_priv 为私有变量的大小。

第二个参数 name 为设备名。

第三个参数 name_assign_type 为设备来源名称。

第四个参数 setup 为 net_device 的 setup()函数指针, setup()函数接收的参数为 struct net_device 指针,用于预置 net_device 成员的值。

register_netdev()只有一个输入参数就是需要注册的 net_device 结构体变量指针。

22.1.2 发包

发包函数 ndo_start_xmit 需要我们自己构建,原型如下:

```
netdev_tx_t (*ndo_start_xmit)(struct sk_buff *skb, struct net_device *dev);
```

第一个输入参数 struct sk_buff 是只 socket(套接字)缓冲区,用于网络模型中各层之间的数据传递。struct sk_buff 定义在文件 include/linux/skbuff.h 中,是一个双向链表,如下(有省略):

```
1. struct sk_buff {
2.     union {
3.         struct {
```



```

4.      /* These two members must be first. */
5.      struct sk_buff      *next;  //指向下一个 sk_buff 结构体
6.      struct sk_buff      *prev;  //指向前一个 sk_buff 结构体
7.      .....
8.      };
9.      .....
10.     };
11.
12.     .....
13.
14.     unsigned int         len,      //数据包的总长度,包括线性数据和非线性数据
15.                           data_len; //非线性的数据长度
16.     __u16                 mac_len,  //mac 包头长度
17.                           hdr_len;
18.
19.     .....
20.
21.     __u32                 priority; //当前 sk_buff 结构体的优先级
22.
23.     .....
24.
25.     __be16                protocol; //存放上层的协议类型,可以通过 eth_type_trans()来获
    取
26.     __u16                 transport_header; //传输层头部的偏移值
27.     __u16                 network_header;  //网络层头部的偏移值
28.     __u16                 mac_header;      //MAC 数据链路层头部的偏移值
29.
30.     .....
31.
32.     /* public: */
33.
34.     /* These elements must be at the end, see alloc_skb() for details. */
35.     sk_buff_data_t        tail;          //指向缓冲区的数据包末尾
36.     sk_buff_data_t        end;           //指向缓冲区的末尾
37.     unsigned char         *head,         //指向缓冲区的协议头开始位置
38.                           *data;         //指向缓冲区的数据包开始位置
39.     .....
40. };

```

sk_buff 中个数据段排布空间如下:



数据段中又分为几段:

MAC 头	IP 头	数据类型	数据
-------	------	------	----

MAC 头用结构体 ethhdr 表示。

IP 头用结构体 iphdr 表示。

数据类型为一个字节。

发包函数 ndo_start_xmit 中具体要做的是如下:

- 1) 调用函数 netif_stop_queue()停止上层下传数据。
- 2) 通过硬件发送数据。
- 3) 调用函数 dev_kfree_skb()释放 sk_buff。
- 4) 发送成功进入中断时,更新统计信息,调用 netif_wake_queue()函数,使上层继续下发数据。

- 5) 发送超时则在 net_device_ops 中的 ndo_tx_timeout 函数中调用 netif_wake_queue() 函数, 使上层继续下发数据。

dev_kfree_skb() 函数原型如下:

```
#define dev_kfree_skb(a)    consume_skb(a)
void consume_skb(struct sk_buff *skb)
```

netif_wake_queue() 函数原型如下:

```
static inline void netif_wake_queue(struct net_device *dev)
```

netif_stop_queue() 函数原型如下:

```
static inline void netif_stop_queue(struct net_device *dev)
```

22.1.3 收包

收包一般放在中断中处理。处理过程为:

- 1) 使用 alloc_skb() 函数构造一个 sk_buff。
- 2) 使用 skb_reserve(skb_buff, 2) 把 sk_buff 里的数据包先后位移 2 字节以腾出 sk_buff 里的头部空间。
- 3) 读取网络设备硬件上接收到的数据, 把数据复制到 sk_buff 的成员 data 指针。
- 4) 使用 eth_type_trans() 函数来获取上层协议, 将返回值赋给 sk_buff 的成员 protocol。
- 5) 更新统计信息, 最后使用 netif_rx() 将 sk_buff 上传到上层协议。

22.2 实验

21.1.1 驱动程序

使用 petalinux 新建名为 "ax-netcard-drv" 驱动程序, 并执行 petalinux-config -c rootfs 命令选上新增的驱动程序。

在 ax-netcard-drv.c 文件中输入下面的代码:

```
1. #include <linux/module.h>
2. #include <linux/kernel.h>
3. #include <linux/types.h>
4. #include <linux/fcntl.h>
5. #include <linux/ioport.h>
6. #include <linux/in.h>
7. #include <linux/skbuff.h>
8. #include <linux/string.h>
9. #include <linux/init.h>
10. #include <linux/bitops.h>
11. #include <linux/ip.h>
12. #include <linux/netdevice.h>
13. #include <linux/etherdevice.h>
14. #include <asm/io.h>
15. #include <asm/irq.h>
16.
17. /* 定义一个 net_device 结构体变量 */
18. static struct net_device *ax_net_dev;
19. static const unsigned char mac_addr[6] = {0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F};
20. /* 模拟接收, 构造一个虚拟的 sk_buff 上报, 并更新统计信息 */
21. static void ax_net_rx(struct sk_buff *skb, struct net_device *dev)
22. {
23.     unsigned char *type;
24.     struct iphdr *ih;
25.     __be32 *saddr, *daddr, tmp;
```

```
26. unsigned char tmp_dev_addr[ETH_ALEN];
27. struct ethhdr *ethhdr;
28.
29. struct sk_buff *rx_skb;
30.
31. /* 交换接受和发送方的 mac 地址 */
32. ethhdr = (struct ethhdr *)skb->data;
33. memcpy(tmp_dev_addr, ethhdr->h_dest, ETH_ALEN);
34. memcpy(ethhdr->h_dest, ethhdr->h_source, ETH_ALEN);
35. memcpy(ethhdr->h_source, tmp_dev_addr, ETH_ALEN);
36.
37. /* 交换接受和发送方的 ip 地址 */
38. ih = (struct iphdr *) (skb->data + sizeof(struct ethhdr));
39. saddr = &ih->saddr;
40. daddr = &ih->daddr;
41.
42. tmp = *saddr;
43. *saddr = *daddr;
44. *daddr = tmp;
45.
46. type = skb->data + sizeof(struct ethhdr) + sizeof(struct iphdr);
47. /* 修改类型, 0 表示 reply */
48. *type = 0;
49.
50. ih->check = 0;
51. ih->check = ip_fast_csum((unsigned char *)ih, ih->ihl);
52.
53. /* 构造 sk_buff */
54. rx_skb = dev_alloc_skb(skb->len + 2);
55. skb_reserve(rx_skb, 2);
56. memcpy(skb_put(rx_skb, skb->len), skb->data, skb->len);
57.
58. rx_skb->dev = dev;
59. rx_skb->protocol = eth_type_trans(rx_skb, dev);
60. rx_skb->ip_summed = CHECKSUM_UNNECESSARY;
61. dev->stats.rx_packets++;
62. dev->stats.rx_bytes += skb->len;
63.
64. /* 提交 sk_buff */
65. netif_rx(rx_skb);
66. }
67.
68. static netdev_tx_t ax_net_tx(struct sk_buff *skb, struct net_device *dev)
69. {
70.     static int cnt = 0;
71.
72.     /* 停止上层数据下传队列 */
73.     netif_stop_queue(dev);
74.     /* 模拟接收, 以达到一个完成的发送接收过程 */
75.     ax_net_rx(skb, dev);
76.     /* 释放 skb */
77.     dev_kfree_skb(skb);
78.     /* 发送完成, 恢复上层数据下传队列 */
79.     netif_wake_queue(dev);
80.     /* 更新统计信息 */
81.     dev->stats.tx_packets++;
82.     dev->stats.tx_bytes += skb->len;
83.
84.     return NETDEV_TX_OK;
85. }
86.
87. /* 网卡设备操作函数集 */
88. static const struct net_device_ops ax_netdev_ops =
89. {
90.     .ndo_start_xmit = ax_net_tx,
```

```
91. };
92.
93. /* 驱动入口函数 */
94. static int __init ax_net_init(void)
95. {
96.     /* 分配 net_device 结构体 */
97.     ax_net_dev = alloc_netdev(0, "ax_net%d", NET_NAME_UNKNOWN, ether_setup);
98.     /* 设置操作函数集 */
99.     ax_net_dev->netdev_ops = &ax_netdev_ops;
100.
101.     /* 设置 MAC 地址 */
102.     ax_net_dev->dev_addr = mac_addr;
103.
104.     /* 设置 ping 功能 */
105.     ax_net_dev->flags      |= IFF_NOARP;
106.     ax_net_dev->features   |= NETIF_F_CSUM_MASK;
107.     /* 注册网卡驱动 */
108.     register_netdev(ax_net_dev);
109.
110.     return 0;
111. }
112.
113. /* 驱动出口函数 */
114. static void __exit ax_net_exit(void)
115. {
116.     unregister_netdev(ax_net_dev);
117.     free_netdev(ax_net_dev);
118. }
119.
120. module_init(ax_net_init);
121. module_exit(ax_net_exit);
122.
123. /* 驱动描述信息 */
124. MODULE_AUTHOR("Alinx");
125. MODULE_ALIAS("net card test");
126. MODULE_DESCRIPTION("NET CARD driver");
127. MODULE_VERSION("v1.0");
128. MODULE_LICENSE("GPL");
```

94 行的驱动入口函数中，先使用 `alloc_netdev()` 来分配一个 `net_device` 结构体，设置 `net_device` 结构体变量的成员，设置完成后使用 `register_netdev()` 向内核注册 `net_device` 结构体。设置 `net_device` 结构体变量成员主要是操作函数和 105 行和 106 行设置 ping 功能。

68 行的发包函数中，先调用 `netif_stop_queue(dev)` 函数停止上层发送数据包，之后如果是真实的网卡，应该要调用硬件接口来发送数据因为这里是模拟网卡，所以直接调用模拟接受的函数表示发送完成。接受函数 `ax_net_rx()` 之后再看。发送完成后释放 `skb_buff` 并调用 `netif_wake_queue()` 函数恢复上层发送数据包。最后更新统计信息。

31 行开始的模拟接受函数，本是应该放在接受中断中处理的。在接收函数中，修改 `sk_buff` 中的数据，使它变成一个接受 `sk_buff`，计提方法就是对调 `erhdr` 和 `iphdr` 结构体中发送、接收 mac 地址。使用 `ip_fast_csum` 函数重新获取 `iphdr` 校验。之后设置 flag，0x08 表示发送 ping 包，接受需要修改为 0。重新构造 `sk_buff`，重新赋值。使用 `eth_type_trans` 函数获取上层协议。最后调用 `netif_rx` 提交 `sk_buff`。

21.1.2 运行测试

测试方法步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
```

```
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ./ax-netcard-drv.ko
```

IP 和路径根据实际情况调整。

加载驱动后，使用 `ls /sys/class/net/` 命令查看网卡驱动有没有存在。

再使用命令 `ifconfig ax_net0 3.3.3.3` 设置 ip，之后就可以 ping 测试了。

之后使用 `ifconfig` 命令查看网卡信息，核对 mac 地址信息和收发包数的统计信息。

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt
root@ax_peta:/mnt# mkdir /tmp/qt
root@ax_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@ax_peta:/mnt# cd /tmp/qt
root@ax_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@ax_peta:/tmp/qt# cd /mnt
root@ax_peta:/mnt# insmod ./ax-netcard-drv.ko
ax_netcard_drv: loading out-of-tree module taints kernel.
ax_net0: mixed HW and IP checksum settings.
root@ax_peta:/mnt# ls /sys/class/net/
ax_net0  eth0  lo  sit0
root@ax_peta:/mnt# ifconfig ax_net0 3.3.3.3
root@ax_peta:/mnt# ping 3.3.3.4
PING 3.3.3.4 (3.3.3.4): 56 data bytes
64 bytes from 3.3.3.4: seq=0 ttl=64 time=0.186 ms
64 bytes from 3.3.3.4: seq=1 ttl=64 time=0.080 ms
64 bytes from 3.3.3.4: seq=2 ttl=64 time=0.067 ms
64 bytes from 3.3.3.4: seq=3 ttl=64 time=0.063 ms
64 bytes from 3.3.3.4: seq=4 ttl=64 time=0.063 ms
64 bytes from 3.3.3.4: seq=5 ttl=64 time=0.060 ms
^C
--- 3.3.3.4 ping statistics ---
6 packets transmitted, 6 packets received, 0% packet loss
round-trip min/avg/max = 0.060/0.086/0.186 ms
root@ax_peta:/mnt# ifconfig
ax_net0      Link encap:Ethernet  HWaddr 0A:0B:0C:0D:0E:0F
              inet addr:3.3.3.3  Bcast:3.255.255.255  Mask:255.0.0.0
              inet6 addr: fe80::80b:0ff:fe0d:e0f1lo/64 Scope:Link
              UP BROADCAST RUNNING NOARP MULTICAST  MTU:1500  Metric:1
              RX packets:17 errors:0 dropped:0 overruns:0 frame:0
              TX packets:17 errors:0 dropped:0 overruns:0 carrier:0
              collisions:0 txqueuelen:1000
              RX bytes:2343 (2.2 KiB)  TX bytes:2343 (2.2 KiB)

eth0         Link encap:Ethernet  HWaddr 00:0A:35:00:1E:53
              inet addr:192.168.1.56  Bcast:192.168.1.255  Mask:255.255.255.0
              inet6 addr: fe80::20a:35ff:fe00:1e53lo/64 Scope:Link
              UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
              RX packets:5093 errors:0 dropped:1 overruns:0 frame:0
              TX packets:2082 errors:0 dropped:0 overruns:0 carrier:0
              collisions:0 txqueuelen:1000
              RX bytes:6069490 (5.7 MiB)  TX bytes:208918 (204.0 KiB)
              Interrupt:29 Base address:0xb000

lo           Link encap:Local Loopback
              inet addr:127.0.0.1  Mask:255.0.0.0
              inet6 addr: ::1%1/128 Scope:Host
              UP LOOPBACK RUNNING  MTU:65536  Metric:1
              RX packets:0 errors:0 dropped:0 overruns:0 frame:0
              TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
              collisions:0 txqueuelen:1
              RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@ax_peta:/mnt#
```

第二十三章 DMA 驱动

23.1 DMA 简介

CPU 和内存是计算机组成中都是不可或缺的部分。和 CPU 相比，内存是非常慢速的拖油瓶，CPU 的速度被内存限制，同时又不得不等待内存处理而无法去处理其他事件。所以使用 CPU 搬运内存数据，是非常浪费资源的。因此专门用于搬运内存数据的器件 DMA 应运而生。

DMA 是 Direct Memory Access 的缩写，也就是直接内存读写，所谓的直接，也就是内存到内存，不通过 CPU。DMA 的可以支持内存到外设、外设到内存、内存到内存的数据交互，必要时节省很多 CPU 资源。当然，虽然 DMA 几乎不占用 CPU，但还是会占用系统总线的。

23.2 DMAC

这里的 DMAC 指的是 DMA controller 即 DMA 控制器。一个外设支持 DMA 不代表它能够主动发起 DMA 操作只能说至少他支持被动 DMA 数据读写。前面我们说 DMA 几乎不占用 CPU 资源，是因为 DMA 传输需要有 CPU 发起，之后就全部交由 DMAC 搬运数据了。DMAC 搬运过程中，涉及到以下关键概念。

23.2.1 DMA channels

大部分 DMAC 支持多路 DMA，也就是 DMA channels。DMA channel 相当于是 DMAC 提供的数据传输通道也就是数据传输的方法，解决了如何传输的问题。实际上 DMA channel 是一个抽象的概念，不是物理通道。前面我们说过，DMA 传输还是会占用总线资源的，总线资源十分有限，因此同时进行多路 DMA 传输不太现实。

抽象出 DMA channel 概念的目的实际是为了方便管理 DMA 的 client，让每个 client 都独占一格通道，但实际上传输时 DMAC 会进行先后仲裁，串行传输。

23.2.2 DMA request line

request line 指代 DMA 设备和 DMAC 之间的物理连线。这条线用于 DMA 设备通知 DMAC 是否可以开书传输，即解决了何时传输的问题。通常每个 DMA 数据收发节点(endpoint)都有 DMA request line 连接到 DMAC。

从主从的角度看，DMA channel 由 DMAC 提供，相当于主 cotroller，DMA request line 和 DMA 设备绑定相当于从 client。在系统中 DMA request line 的数量通常比 DMA channel 多，因为 DMA request line 不是每一刻都处于活跃状态。

23.2.3 transfer size、transfer wide、burst size

transfer wide 可以理解为单次传输数据的大小，和串口来比较的话，串口一次只能传一个字节，而 DMA 则可以选择一次能传输的数据大小。在这基础上的 transfer size 则是传输的次数，而不是单纯的总大小，也就是说 DMA 传输总长度实际上是 transfer size 乘上 transfer wide。

burst size 是指 DMAC 内部缓存大小。当 DMA 传输的源或目的是内存 memory 时，DMAC 会先读取数据到缓存，再传入或传出。

23.2.4 scatter-gather

DMA 操作必须是连续的物理内存，在实际应用中，难免会遇到处理物理内存不连续的数据

的情况，scatter-gather 指的就是把不连续的数据拷贝到连续的 buffer 中的操作。这个操作过程可以用软件实现，也有直接的硬件支持。具体的实现暂且不用关心，这里主要是强调 DMA 操作必须是连续的物理内存这件事。

23.3 PL330

PL330 是 ZYNQ 的 PS 端使用的 DMAC。PL330 的使用可参考 xilinx 的数据手册“ug585-Zynq-7000-TRM”的 chapter9（注意实际使用与手册中的介绍略有出入）。

为了方便使用，PL330 专门提供了完善的指令集，但在 C 语言环境下，无法直接去使用。很遗憾的是 xilinx 的手册中并没有说明指令集以外的用法，好在根据内核中现有的 PL330 驱动代码，结合手册中的汇编例程，也算是整理出了 C 语言用法的七七八八。

所谓的指令集可以理解成是寄存器的预设。那我们必然可以通过寄存器来完成指令集完成的操作。按照这个思路先看看一个完整的 DMA 操作需要指令。

Code
DMAMOV CCR, SB4 SS64 DB4 DS64
DMAMOV SAR, 0x1000
DMAMOV DAR, 0x4000
DMALP 16
DMALD
DMAST
DMALPEND
DMAEND

这是手册中一个内存到内存的 DMA 传输汇编示例，这款里的指令就是 PL330 提供的。

DMAMOV 指令是赋值指令。指令的第一个参数是赋值目标，DMAMOV 指令只能给 CCR、SAR、DAR 这三个目标赋值。第二个参数是赋给目标的值。那我们再看看这三个赋值目标又分别指代什么。在手册的附录 B 中找到 DMAC 相关的寄存器，不难找到这三组寄存器。

CCR 寄存器用于设置源读出数据的 size 和 wide、目的写入数据的 size 和 wide。size 和 wide 前面说过，DMA 传输的总长为 size*wide。

SAR 寄存器用于设置源的物理地址。DAR 寄存器用于设置目的物理地址。

也就是说通过 DMAMOV 指令，我们就已经完成源、目的、长度三个要素的设置，只需要触发 DMA 开始传输即可。再往下看。

DMALP 是设置 DMA 传输循环次数，和 DMALPEND 成对。循环做什么事？就是 DMALD、DMAST 两件事。

DMA 的传输，被 DMAC 分成了两步，先从源读出数据，再写入目的。CCR 寄存器也罢读出和写入的长度设置区分开来。而这里循环的两个指令 DMALD 和 DMAST 也就分别对应的读出和写入。

到这里 DMAC 使用的步骤就很开朗了，接下来就是如何用 C 语言通过寄存器去实现。这里我们参考 linux 内核源码目录中的/drivers/dma.pl330.c。在这个文件中尝试着搜索一下 DMAMOV 这个指令，果然真的找到了所用指令的宏，如下：

```
#define CMD_DMAADDH    0x54
#define CMD_DMAEND     0x00
#define CMD_DMAFLUSHP  0x35
#define CMD_DMAGO      0xa0
#define CMD_DMALD      0x04
#define CMD_DMALDP     0x25
#define CMD_DMALP      0x20
#define CMD_DMALPEND   0x28
#define CMD_DMAKILL    0x01
#define CMD_DMAMOV     0xbc
#define CMD_DMANOP     0x18
#define CMD_DMARMB     0x12
#define CMD_DMASEV     0x34
#define CMD_DMAST      0x08
#define CMD_DMASTP     0x29
#define CMD_DMASTZ     0x0c
#define CMD_DMAWFE     0x36
#define CMD_DMAWFP     0x30
#define CMD_DMAWMB     0x13
```

通过宏定义把每一个指令对应到一个值，但还无法看出这个值的具体含义。再看看在哪里调用了 DMAMOV 宏。可以找到这样一个函数。

```
static inline u32 _emit_MOV(unsigned dry_run, u8 buf[],
    enum dmamov_dst dst, u32 val)
{
    if (dry_run)
        return SZ_DMAMOV;

    buf[0] = CMD_DMAMOV;
    buf[1] = dst;
    *((__le32 *)&buf[2]) = cpu_to_le32(val);

    PL330_DBGCMD_DUMP(SZ_DMAMOV, "\tDMAMOV %s 0x%x\n",
        dst == SAR ? "SAR" : (dst == DAR ? "DAR" : "CCR"), val);

    return SZ_DMAMOV;
}
```

这个函数把指令宏的值，dst，val 三个值依次放到一个 buffer 的从 0 开始三个连续元素中。dst 看下面的代码，不难理解，他指代的就是 SAR、DAR、CCR 三个寄存器地址之一，而最后的 val 就是赋给寄存器的值。

同样的再去看一些其他的指令，几乎都是如此，指令、参数 1、参数 2 打包到一个 buffer 中。那我们大概能猜到指令组织的方法。但指令如此之多，也没能在寄存器手册中找到所有指令对应的寄存器，那组织的指令该如何执行？

阅读 pl330.c。最终，我们找到了 932 行的 _execute_DBGINSN() 函数。execute 是执行的意思，DBGINSN 我们能在手册中找到两个寄存器 DBGINSN0 和 DBGINSN1，但是他们的解释都比较暧昧，大概可以理解为，两个寄存器，就是用来执行指令的。

DBGINSN0 的高 16 位和 DBGINSN1 的全 32，再手册中都没有解释清楚，分析 pl330.c 最终发现，DBGINSN0 的 16~23 位是拼入了命令 DMAGO，DBGINSN1 中输入了我们指令 buffer 的首地

址。最终我们通过给寄存器 DBGCMD 的 bit0 写入 0，DMAC 就开始指令指令了。

接下来再对照源码去分析：

```
1.  #include <linux/module.h>
2.  #include <linux/kernel.h>
3.  #include <linux/fs.h>
4.  #include <linux/device.h>
5.  #include <linux/init.h>
6.  #include <linux/cdev.h>
7.  #include <linux/platform_device.h>
8.  #include <linux/miscdevice.h>
9.  #include <linux/ioport.h>
10. #include <linux/of.h>
11. #include <linux/uaccess.h>
12. #include <linux/interrupt.h>
13. #include <linux/of_irq.h>
14. #include <linux/irq.h>
15. #include <linux/timer.h>
16. #include <linux/dma-mapping.h>
17. #include <asm/io.h>
18. #include <asm/uaccess.h>
19. #include <asm/irq.h>
20. #include <asm/byteorder.h>
21.
22. //驱动个数
23. #define AX_DRV_CNT 1
24. //设备节点名称
25. #define AX_DRV_NAME "ax_dma"
26.
27. //AMBA 总线时钟使能
28. #define DMA_CLKEN_ADDR 0xF800012C
29. volatile u32 * clken;
30.
31. //基地址
32. #define DMA_BASE_ADDR 0xF8003000
33.
34. //DBGSTATUS
35. #define DMA_DBGSTATUS_OFFSET 0x00000D00
36. volatile u32 * dbgstatus;
37.
38. //DBGCMD
39. #define DMA_DBGCMD_OFFSET 0x00000D04
40. volatile u32 * dbgcmd;
41.
42. //DBGINST0
43. #define DMA_DBGINST0_OFFSET 0x00000D08
44. volatile u32 * dbginst0;
45.
46. //DBGINST1
47. #define DMA_DBGINST1_OFFSET 0x00000D0C
48. volatile u32 * dbginst1;
49.
50. //cache
51. #define DMA_CACH_OFFSET 0x00000E04
52. volatile u32 * cache;
53.
54. //DMA 中断使能
```

```
55. #define DMA_IRQ_EN_OFFSET      0x00000020
56. volatile u32 * irqen;
57.
58. //DMA 中断清除
59. #define DMA_IRQ_CLR_OFFSET      0x0000002C
60. volatile u32 * irqclr;
61.
62.
63. //指令集
64. #define CMD_DMAADDH      0x54
65. #define CMD_DMAEND      0x00
66. #define CMD_DMAFLUSHP    0x35
67. #define CMD_DMAGO       0xa0
68. #define CMD_DMALD       0x04
69. #define CMD_DMALDP      0x25
70. #define CMD_DMALP       0x20
71. #define CMD_DMALPEND    0x28
72. #define CMD_DMAKILL     0x01
73. #define CMD_DMAMOV      0xbc
74. #define CMD_DMANOP      0x18
75. #define CMD_DMARMB      0x12
76. #define CMD_DMASEV      0x34
77. #define CMD_DMAST       0x08
78. #define CMD_DMASTP      0x29
79. #define CMD_DMASTZ      0x0c
80. #define CMD_DMAWFE      0x36
81. #define CMD_DMAWFP      0x30
82. #define CMD_DMAWMB      0x13
83.
84. //指令长度
85. #define SZ_DMAADDH      3
86. #define SZ_DMAEND      1
87. #define SZ_DMAFLUSHP    2
88. #define SZ_DMAGO       6
89. #define SZ_DMALD       1
90. #define SZ_DMALDP      2
91. #define SZ_DMALP       2
92. #define SZ_DMALPEND    2
93. #define SZ_DMAKILL     1
94. #define SZ_DMAMOV      6
95. #define SZ_DMANOP      1
96. #define SZ_DMARMB      1
97. #define SZ_DMASEV      2
98. #define SZ_DMAST       1
99. #define SZ_DMASTP      2
100. #define SZ_DMASTZ      1
101. #define SZ_DMAWFE      2
102. #define SZ_DMAWFP      2
103. #define SZ_DMAWMB      1
104.
105. //指令队列总长
106. #define INSTR_Q_MAX     500
107.
108. //缓冲区大小
109. #define BUF_SIZE      (512*64)
110.
111. //DMAMOV 指令对应的三个目标地址下标
112. enum dmamov_dst {
```

```
113.     SAR = 0,
114.     CCR,
115.     DAR,
116. };
117.
118. //DMA 源缓冲区
119. static char *src;
120. static u32 src_phys;
121.
122. //DMA 目标缓冲区
123. static char *dst;
124. static u32 dst_phys;
125.
126. //DMA 指令缓冲区
127. static char *instr_q;
128. static u32 instr_q_phys;
129.
130.
131. struct ax_dma_drv {
132.     dev_t    devid;           //设备号
133.     struct cdev cdev;         //字符设备
134.     struct class *class;      //类
135.     struct device *device;     //设备
136.     int      major;           //主设备号
137. };
138. struct ax_dma_drv ax_dma;
139.
140. static irqreturn_t dma_irq(int irq, void *dev_id)
141. {
142.     u32 reg;
143.     //清除通道 1 中断标志
144.     iowrite32(0x000000ff, irqclr);
145.
146.     if(!memcmp(src, dst, 160))
147.     {
148.         printk("dma irq test ok\r\n");
149.     }
150.
151.     return IRQ_HANDLED;
152. }
153.
154. static int dma_open(struct inode *inode, struct file *file)
155. {
156.     printk("dma_open\r\n");
157.     return 0;
158. }
159.
160. static int dma_write(struct file *file, const char __user *buf,
size_t count, loff_t *ppos)
161. {
162.     //指令长度计数
163.     int instr_cnt = 0, loop_start = 0, lpcount;
164.     u32 reg;
165.     printk("dma_write\r\n");
166.
167.     memset(instr_q, 0x00, INSTR_Q_MAX);
168.     memset(src, 0xAA, 160);
169.     memset(dst, 0x55, 160);
```

```

170.
171.    //设置数据源地址
172.    instr_q[instr_cnt + 0] = (char)(CMD_DMAMOV);
173.    instr_q[instr_cnt + 1] = (char)(SAR);
174.    *((__le32 *)&instr_q[instr_cnt + 2]) = cpu_to_le32(src_phys)
;
175.    //指令总长计数
176.    instr_cnt += SZ_DMAMOV;
177.
178.    //设置数据目标地址
179.    instr_q[instr_cnt + 0] = (char)(CMD_DMAMOV);
180.    instr_q[instr_cnt + 1] = (char)(DAR);
181.    *((__le32 *)&instr_q[instr_cnt + 2]) = cpu_to_le32(dst_phys)
;
182.    //指令总长计数
183.    instr_cnt += SZ_DMAMOV;
184.
185.    //设置数据传输规则, 每个循环传输 burst_size * burst_len, 源和目
标地址变化规则等
186.    instr_q[instr_cnt + 0] = (char)(CMD_DMAMOV);
187.    instr_q[instr_cnt + 1] = (char)(CCR);
188.    //0x0005c017 -> 0000 0000 0000 0001 0111 0000 0001 0111
189.    //len = 2byte, instr_cnt = 8byte, inc = Incrementing-address
s
190.    //单次循环数据大小 = burst_size * burst_len = 2 * 8 = 16
191.    *((__le32 *)&instr_q[instr_cnt + 2]) = cpu_to_le32(0x0005c0
17);
192.    //指令总长计数
193.    instr_cnt += SZ_DMAMOV;
194.
195.    //循环装载数据, 输出 FIFO
196.    instr_q[instr_cnt + 0] = (char)(CMD_DMALP);
197.    instr_q[instr_cnt + 1] = (char)(100);    //循环次数
198.    instr_cnt += SZ_DMALP;
199.    loop_start = instr_cnt;
200.
201.    for(lpcount = 0; lpcount < 100; lpcount++)
202.    {
203.        //从源读数据
204.        instr_q[instr_cnt + 0] = (char)(CMD_DMALD);
205.        instr_cnt += SZ_DMALD;
206.        instr_q[instr_cnt + 0] = (char)(CMD_DMARMB);
207.        instr_cnt += SZ_DMARMB;
208.        //写数据到目标地址
209.        instr_q[instr_cnt + 0] = (char)(CMD_DMAST);
210.        instr_cnt += SZ_DMAST;
211.        instr_q[instr_cnt + 0] = (char)(CMD_DMAWMB);
212.        instr_cnt += SZ_DMAWMB;
213.    }
214.
215.    //申请中断
216.    instr_q[instr_cnt + 0] = (char)(CMD_DMASEV);
217.    instr_q[instr_cnt + 1] = (char)(1 << 3);
218.    instr_cnt += SZ_DMASEV;
219.
220.    //等待 dmac 空闲
221.    do {
222.        reg = ioread32(dbgstatus);

```

```
223.     } while((reg & 0x01) == 0x01);
224.
225.     iowrite32((0 << 24) | (CMD_DMAC0 << 16) | (0 << 8) | (0 <<
0), dbginst0);
226.     iowrite32(instr_q_phys, dbginst1);
227.     iowrite32(0, dbgcmd);
228.     printk("dma go\r\n");
229.
230.     return 0;
231. }
232.
233. static int dma_release(struct inode *inode, struct file *filp)
234. {
235.     printk("dma_release\r\n");
236.     return 0;
237. }
238.
239. static struct file_operations dma_lops=
240. {
241.     .owner    = THIS_MODULE,
242.     .open     = dma_open,
243.     .write    = dma_write,
244.     .release  = dma_release,
245. };
246.
247. static int dma_init(void)
248. {
249.     int err;
250.     u32 reg;
251.
252.     printk("dma_init\r\n");
253.
254.     //构建设备号
255.     alloc_chrdev_region(&ax_dma.devid, 0, AX_DRV_CNT, AX_DRV_NAME);
256.
257.     //注册设备
258.     cdev_init(&ax_dma.cdev, &dma_lops);
259.     cdev_add(&ax_dma.cdev, ax_dma.devid, AX_DRV_CNT);
260.
261.     //创建类
262.     ax_dma.class = class_create(THIS_MODULE, AX_DRV_NAME);
263.     if(IS_ERR(ax_dma.class))
264.     {
265.         return PTR_ERR(ax_dma.class);
266.     }
267.
268.     //创建设备
269.     ax_dma.device = device_create(ax_dma.class, NULL, ax_dma.devid, NULL, AX_DRV_NAME);
270.     if(IS_ERR(ax_dma.device))
271.     {
272.         return PTR_ERR(ax_dma.device);
273.     }
274.
275.     //注册中断
276.     err = request_irq(33, dma_irq, IRQF_TRIGGER_HIGH, "ax-dmac2", NULL);
```

```
277.     if(err < 0) printk("irq err=%d\n", err);
278.
279.     //分配 SRC 对应的缓冲区
280.     src = dma_alloc_coherent(NULL, BUF_SIZE, &src_phys, GFP_KERNEL);
281.     if (NULL == src)
282.     {
283.         printk("can't alloc buffer for src\n");
284.         return -ENOMEM;
285.     }
286.
287.     //分配 DST 对应的缓冲区
288.     dst = dma_alloc_coherent(NULL, BUF_SIZE, &dst_phys, GFP_KERNEL);
289.     if (NULL == dst)
290.     {
291.         dma_free_coherent(NULL, BUF_SIZE, src, src_phys);
292.         printk("can't alloc buffer for dst\n");
293.         return -ENOMEM;
294.     }
295.
296.     instr_q = dma_alloc_coherent(NULL, INSTR_Q_MAX, &instr_q_phys, GFP_KERNEL);
297.     if (NULL == instr_q)
298.     {
299.         dma_free_coherent(NULL, BUF_SIZE, src, src_phys);
300.         dma_free_coherent(NULL, BUF_SIZE, dst, dst_phys);
301.         printk("can't alloc buffer for instr_q\n");
302.         return -ENOMEM;
303.     }
304.
305.     //虚拟地址映射
306.     clken = ioremap(DMA_CLKEN_ADDR, 4);
307.     dbgstatus = ioremap(DMA_BASE_ADDR + DMA_DBGSTATUS_OFFSET, 4)
;
308.     dbgcmd = ioremap(DMA_BASE_ADDR + DMA_DBGCMD_OFFSET, 4)
;
309.     dbginst0 = ioremap(DMA_BASE_ADDR + DMA_DBGINST0_OFFSET, 4)
;
310.     dbginst1 = ioremap(DMA_BASE_ADDR + DMA_DBGINST1_OFFSET, 4)
;
311.     irqen = ioremap(DMA_BASE_ADDR + DMA_IRQ_EN_OFFSET, 4)
;
312.     irqclr = ioremap(DMA_BASE_ADDR + DMA_IRQ_CLR_OFFSET, 4)
;
313.     cache = ioremap(DMA_BASE_ADDR + DMA_CACH_OFFSET, 4)
;
314.
315.     //使能 AMBA 时钟
316.     reg = ioread32(clken);
317.     iowrite32(reg | 0x00000001, clken);
318.     //使能通道 1 中断
319.     iowrite32(0x000000ff, irqen);
320.     //读取 cache 寄存器以初始化 cache
321.     reg = ioread32(cache);
322.
323.     return 0;
324. }
```



```
325.
326. static void dma_exit(void)
327. {
328.     //释放中断
329.     free_irq(33, NULL);
330.     //删除设备
331.     cdev_del(&ax_dma.cdev);
332.     unregister_chrdev_region(ax_dma.major, AX_DRV_CNT);
333.     //注销类
334.     device_destroy(ax_dma.class, ax_dma.devid);
335.     class_destroy(ax_dma.class);
336.     //释放缓冲区
337.     dma_free_coherent(NULL, BUF_SIZE, src, src_phys);
338.     dma_free_coherent(NULL, BUF_SIZE, dst, dst_phys);
339.     dma_free_coherent(NULL, BUF_SIZE, instr_q, instr_q_phys);
340.     //失能 DMA 中断
341.     iowrite32(0x00000000, irqen);
342.     //释放虚拟地址
343.     iounmap(dbgstatus);
344.     iounmap(dbgcmd);
345.     iounmap(dbginst0);
346.     iounmap(dbginst1);
347.     iounmap(irqen);
348.     iounmap(irqclr);
349.     iounmap(clken);
350.     iounmap(cache);
351.     //test
352. }
353.
354. //驱动入口函数标记
355. module_init(dma_init);
356. //驱动出口函数标记
357. module_exit(dma_exit);
358.
359. /* 驱动描述信息 */
360. MODULE_AUTHOR("Alinx");
361. MODULE_ALIAS("dma");
362. MODULE_DESCRIPTION("DMA driver");
363. MODULE_VERSION("v1.0");
364. MODULE_LICENSE("GPL");
```

这个代码是以简单字符设备框架测试了 pl330 的 DMA 内存到内存的数据传输。

27~60 行是一些寄存器的定义，在驱动入口函数中 305~313 行，把这些寄存器地址转换成虚拟地址。相应的在出口函数 342~350 行释放虚拟地址占用。

63~103 行把 pl330.c 中的指令宏定义和指令长度搬运过来。112~116 行把 DMAMOV 赋值的三个目标编号枚举搬过来。

118~128 行定义了 3 个缓冲区分别源缓冲区、目标缓冲区和指令缓冲区。在入口函数的 280~303 行通过函数 dma_alloc_coherent() 分配了 3 个缓冲区，并获取了他们对应的物理地址。所谓分配缓冲区，目的是获取物理地址连续的内存方便内存到内存的 DMA 操作。顺便我们获取到了缓冲区对应的物理地址，pl330 的操作必须直接操作物理地址，包括指令的操作也是。在出口函数中 337~339 行对应释放缓冲区。

131~138 行定义一个设备结构体并定义变量。

看完了定义，我们再从入口函数开始看。

255~273 行是很常规的字符设备注册。

276 行注册了一个中断，中断号 33 是从设备树中使用函数 `irq_of_parse_and_map()` 获取的。在 petalinux 工程中查找 `zynq-7000.dtsi`，在这个文件中能找到如下节点

```
1.  dmac@f8003000 {
2.      compatible = "arm,pl330", "arm,primecell";
3.      reg = <0xf8003000 0x1000>;
4.      interrupt-parent = <&intc>;
5.      interrupt-names = "abort", "dma0", "dma1", "dma2", "dma3",
6.          "dma4", "dma5", "dma6", "dma7";
7.      interrupts = <0 13 4>,
8.          <0 14 4>, <0 15 4>,
9.          <0 16 4>, <0 17 4>,
10.         <0 40 4>, <0 41 4>,
11.         <0 42 4>, <0 43 4>;
12.     #dma-cells = <1>;
13.     #dma-channels = <8>;
14.     #dma-requests = <4>;
15.     clocks = <&clkc 27>;
16.     clock-names = "apb_pclk";
17. };
```

这里有 9 个中断号，分别对应停止、dma0~9，这里我们获取了通道 1 对应的虚拟中断号 33。实际上使用任意通道中断都可以，但要和后面的 DMASEV 指令对应，具体一会儿再说。

之后分配缓冲区和映射虚拟地址已经说过了。最后有几个寄存器操作。

316~317 行使能 AMBA 时钟，这个时钟默认是使能的，这里实际没有使能，暂时没必要去深究原因，既然没使能，就使能一下好了。

319 行使能中断，为了方便测试，这里把所有的中断都是能了。

321 行需要通过读取 cache 寄存器以初始化 cache。

再看操作函数的实现，write 函数是关键。write 函数实际可以分为两部分，一是拼接指令到指令缓冲区，二是通过设置 DBG 相关的寄存器来执行指令。

172~218 行拼接指令，首先设置 SAR、DAR、CCR，之后记录循环次数，然后在循环中 DMALD、DMAST。这些步骤是不是就和手册中汇编例程步骤很相似。

但这里多了一步申请中断，DMASEV 命令也很简单，就是触发对应通道的中断，表示 DMA 操作已经完成。注意这里触发中断的通道要和我们注册的中断通道一致。

指令评介完成后，先要不断查询 DMAC 状态，DMAC 空闲后再进行实际的发送操作。

225 行往 DBGINST0 寄存器中填值，低 16 位值得含义可参考手册，高 16 位中 16~23 位需要写入 DMAGO 指令，24~32 位选择通道。

226 行往 DBGINST1 寄存器中填值，值就是指令拼接 buffer 的首地址。

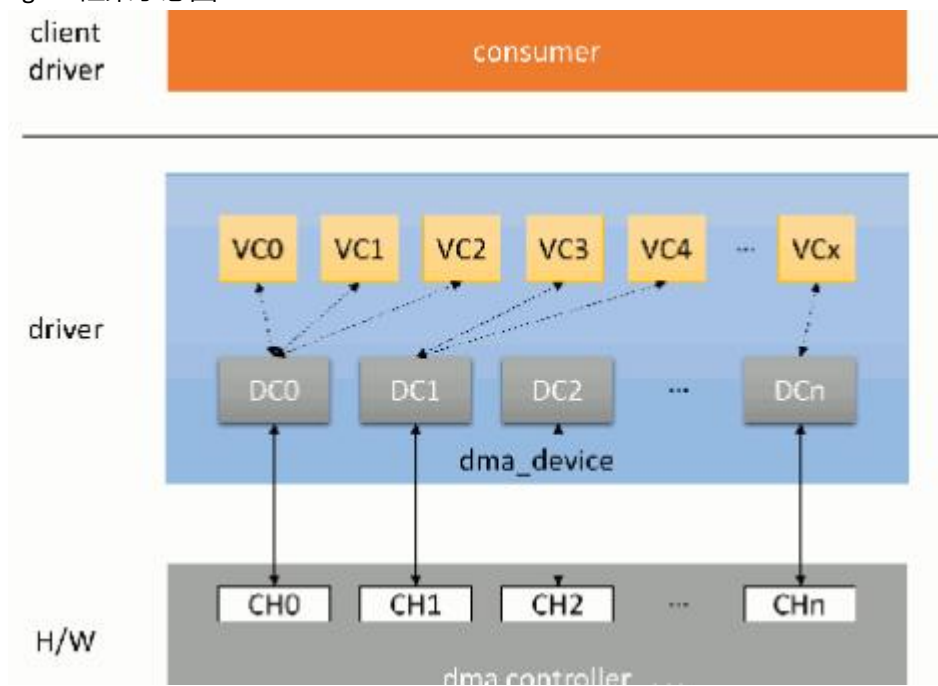
到这里实际上还是在设置，知道执行了 227 行的代码，往寄存器 DBGCMD 中写入 0，DMA 才开始真正的执行。

144 行需要在中断回掉函数中清除中断标志位，否则中断会一直触发，系统就无法使用了。

23.4 linux 中的 DMA

Linux DMA engine 框架提供了 DMA controller 和 DMA client 两个框架。分别对应 DMA 提供者和 DMA 使用者两个角度。pl330 是个 DMA 控制器，实际上就是站 DMA 提供者的角度。我们上面的例程中，使用 DMA 的对象实际可以具体到内存到内存，内存就是 DMA 的使用者。但是在上面的例程中并没有区分这样的概念，这就到了例程的局限性。实际上前面的例程中 pl330 的操作都可以抽出来，他们对于其他使用者来说，都是共通的。这也是 linux 系统一贯的设计思路。也是因为如此，原本并不复杂 DMA 在这样的设计下显得有些复杂。

DMA engine 框架示意图：



23.4.1 DMA controller 驱动框架

控制器的驱动也有现成的可用，这和前面的 I2C、SPI 之类的相似。我们主要还是去了解 client 的驱动，controller 驱动稍作了解。

DMA controller 框架抽象出 channel 对应 DMAC 的物理通道，又定义了虚拟的 channel，软件上可以实现多个虚拟 channel 对应一个物理通道。

介绍一下 DMA controller 框架中主要涉及到的数据结构：

1) struct dma_device

struct dma_device 定义在 include/linux/dmaengine.h 中。如下

```
1. struct dma_device {
2.
3.     unsigned int chancnt;
4.     unsigned int privatecnt;
5.     struct list_head channels;
6.     struct list_head global_node;
7.     struct dma_filter filter;
8.     dma_cap_mask_t cap_mask;
9.     unsigned short max_xor;
10.    unsigned short max_pq;
```

```

11.     enum dmaengine_alignment copy_align;
12.     enum dmaengine_alignment xor_align;
13.     enum dmaengine_alignment pq_align;
14.     enum dmaengine_alignment fill_align;
15.     #define DMA_HAS_PQ_CONTINUE (1 << 15)
16.
17.     int dev_id;
18.     struct device *dev;
19.
20.     u32 src_addr_widths;
21.     u32 dst_addr_widths;
22.     u32 directions;
23.     u32 max_burst;
24.     bool descriptor_reuse;
25.     enum dma_residue_granularity residue_granularity;
26.
27.     int (*device_alloc_chan_resources)(struct dma_chan *chan);
28.
29.     void (*device_free_chan_resources)(struct dma_chan *chan);
30.
31.     struct dma_async_tx_descriptor *(*device_prep_dma_memcpy)(
32.         struct dma_chan *chan, dma_addr_t dst, dma_addr_t src,
33.         size_t len, unsigned long flags);
34.     struct dma_async_tx_descriptor *(*device_prep_dma_xor)(
35.         struct dma_chan *chan, dma_addr_t dst, dma_addr_t *src,
36.         unsigned int src_cnt, size_t len, unsigned long flags);
37.     struct dma_async_tx_descriptor *(*device_prep_dma_xor_val)(
38.         struct dma_chan *chan, dma_addr_t *src, unsigned int sr
39.         c_cnt,
40.         size_t len, enum sum_check_flags *result, unsigned long
41.         flags);
42.     struct dma_async_tx_descriptor *(*device_prep_dma_pq)(
43.         struct dma_chan *chan, dma_addr_t *dst, dma_addr_t *src,
44.         unsigned int src_cnt, const unsigned char *scf,
45.         size_t len, unsigned long flags);
46.     struct dma_async_tx_descriptor *(*device_prep_dma_pq_val)(
47.         struct dma_chan *chan, dma_addr_t *pq, dma_addr_t *src,
48.         unsigned int src_cnt, const unsigned char *scf, size_t
49.         len,
50.         enum sum_check_flags *pqres, unsigned long flags);
51.     struct dma_async_tx_descriptor *(*device_prep_dma_memset)(
52.         struct dma_chan *chan, dma_addr_t dest, int value, size
53.         t len,
54.         unsigned long flags);
55.     struct dma_async_tx_descriptor *(*device_prep_dma_memset_sg)
56.     (
57.         struct dma_chan *chan, struct scatterlist *sg,
58.         unsigned int nents, int value, unsigned long flags);

```

```

53.     struct dma_async_tx_descriptor *(*device_prep_dma_interrupt)
(
54.         struct dma_chan *chan, unsigned long flags);
55.     struct dma_async_tx_descriptor *(*device_prep_dma_sg)(
56.         struct dma_chan *chan,
57.         struct scatterlist *dst_sg, unsigned int dst_nents,
58.         struct scatterlist *src_sg, unsigned int src_nents,
59.         unsigned long flags);
60.
61.     struct dma_async_tx_descriptor *(*device_prep_slave_sg)(
62.         struct dma_chan *chan, struct scatterlist *sgl,
63.         unsigned int sg_len, enum dma_transfer_direction direct
ion,
64.         unsigned long flags, void *context);
65.     struct dma_async_tx_descriptor *(*device_prep_dma_cyclic)(
66.         struct dma_chan *chan, dma_addr_t buf_addr, size_t buf_
len,
67.         size_t period_len, enum dma_transfer_direction directio
n,
68.         unsigned long flags);
69.     struct dma_async_tx_descriptor *(*device_prep_interleaved_d
ma)(
70.         struct dma_chan *chan, struct dma_interleaved_template
*xt,
71.         unsigned long flags);
72.     struct dma_async_tx_descriptor *(*device_prep_dma_imm_data)
(
73.         struct dma_chan *chan, dma_addr_t dst, u64 data,
74.         unsigned long flags);
75.
76.     int (*device_config)(struct dma_chan *chan,
77.         struct dma_slave_config *config);
78.     int (*device_pause)(struct dma_chan *chan);
79.     int (*device_resume)(struct dma_chan *chan);
80.     int (*device_terminate_all)(struct dma_chan *chan);
81.     void (*device_synchronize)(struct dma_chan *chan);
82.
83.     enum dma_status (*device_tx_status)(struct dma_chan *chan,
84.         dma_cookie_t cookie,
85.         struct dma_tx_state *txstate);
86.     void (*device_issue_pending)(struct dma_chan *chan);
87. };

```

channels:链表头。

cap_mask:表示 controller 的传输能力, 需要和后面 device_prep_dma_xxx 形式的回调函数对应。常见取值如下:

DMA_MEMCPY:可进行 memory copy。

DMA_MEMSET:可进行 memory set。

DMA_SG:可进行 scatter list 传输。

DMA_CYCLIC:可进行 cyclic 类的传输。

DMA_INTERLEAVE:可进行交叉传输。

src_addr_widths:表示 controller 支持哪些宽度的 src 类型。

dst_addr_widths:表示 controller 支持哪些宽度的 dst 类型。

directions:表示 controller 支持的传输方向取值参考枚举 dma_transfer_direction。

max_burst:最大的 burst 传输的 size。

descriptor_reuse:表示 controller 的传输描述能不能复用。

device_alloc_chan_resources:client 申请 channel 时会调用。

device_free_chan_resources:client 释放 channel 时会调用。

device_prep_dma_xxx:client 通过 dmaengine_prep_xxx 获取传输描述符时会调用。

device_config:client 调用 dmaengine_slave_configchannel 时会调用。

device_pause:client 调用 dmaengine_pause 时会调用。

device_resume:client 调用 dmaengine_resume 时会调用。

device_terminate_all:client 调用 dmaengine_terminate_xxx 时会调用。

device_issue_pending:client 调用 dma_async_issue_pending 启动传输时会调用。

DMA controller 驱动需要实现这些函数的具体处理内容, 相当于字符设备框架中的 ops 操作函数。

2) struct dma_chan

定义如下:

```

1.  struct dma_chan {
2.      struct dma_device *device;
3.      dma_cookie_t cookie;
4.      dma_cookie_t completed_cookie;
5.
6.      /* sysfs */
7.      int chan_id;
8.      struct dma_chan_dev *dev;
9.
10.     struct list_head device_node;
11.     struct dma_chan_percpu __percpu *local;
12.     int client_count;
13.     int table_count;
14.
15.     /* DMA router */
16.     struct dma_router *router;
17.     void *route_data;
18.
19.     void *private;
20. };

```

device:指向该 channel 所在的 dma controller。

cookie:client 以该 channel 为操作对象获取传输描述符时, dma controller 返回给 client 的最后一个 cookie。

completed_cookie:在当前 channel 上最后一次完成的传输的 cookie。

device_node:于将该 channel 添加到 dma_device 的 channel 列表中。

3) struct virt_dma_cha

定义在文件 drivers/dma/virt-dma.h 中如下:

```

1.  struct virt_dma_desc {

```

```

2.      struct dma_async_tx_descriptor tx;
3.      /* protected by vc.lock */
4.      struct list_head node;
5.  };
6.
7.  struct virt_dma_chan {
8.      struct dma_chan chan;
9.      struct tasklet_struct task;
10.     void (*desc_free)(struct virt_dma_desc *);
11.
12.     spinlock_t lock;
13.
14.     /* protected by vc.lock */
15.     struct list_head desc_allocated;
16.     struct list_head desc_submitted;
17.     struct list_head desc_issued;
18.     struct list_head desc_completed;
19.
20.     struct virt_dma_desc *cyclic;
21.
22. };

```

chan:一个 struct dma_chan 类型的变量，用于和 client 交互。

task:一个 tasklet，等待该虚拟 channel 的传输完成。

desc_allocated、desc_submitted、desc_issued、desc_completed:四个链表头，用于保存不同状态的虚拟 channel 描述符。

DMA controller 框架相关 API 如下:

1) struct dma_device 注册和注销

```

int dma_async_device_register(struct dma_device *device);
void dma_async_device_unregister(struct dma_device *device);

```

struct dma_device 变量初始化完成后，调用 dma_async_device_register 向内核注册。注册成功后 dma_device 会放在一个名称为 dma_device_list 的全局链表上，以便后面使用。

dma_async_device_unregister 是相对的注销函数。

2) cookie 相关接口

DMA cookie 表示 DMA engine 在数据传送中使用的一段连续内存。

```

static inline void dma_cookie_init(struct dma_chan *chan)
static inline dma_cookie_t dma_cookie_assign(struct dma_async_tx_descriptor *tx)
static inline void dma_cookie_complete(struct dma_async_tx_descriptor *tx)
static inline enum dma_status dma_cookie_status(struct dma_chan
*chan, dma_cookie_t cookie, struct dma_tx_state *state)

```

dma_cookie_init:初始化 channel 中的 cookie、completed_cookie。

dma_cookie_assign:为指针的传输描述分配一个 cookie。

dma_cookie_complete:当一个传输完成时，可调用该接口更新该传输所对应 channel 的 completed_cookie 字段。

dma_cookie_status:获取指定 channel 指定 cookie 的传输状态。

DMA controller 驱动实例可以参考 pl330.c。大致流程如下:

- 1) 定义 struct dma_device 变量并初始化、根据硬件实现必要的回调函数。
- 2) 根据 controller 支持的 channel 数，为每个 channel 定义一个 struct dma_chan 变量并初

始化后，将每个 channel 都添加到 dma_device 的 channels 链表中。

- 3) 调用 dma_async_device_register 向内核注册 dma_device。

23.4.2 DMA client 驱动框架

从源和目标的不同可以把 DMA 划分为四类：内存到内存、内存到外设、外设到内存、外设到外设。因为内存可以使用 memcpy、memset 等操作，linux engine 中把内存到内存这一部分分离出来单独提供了一套 API-Async TX API。而剩余的三类就共用一个结构 Slave-DMA API。这里的 Slave 指代的 client 视角，也就是 DMA 的使用者。

DMA client 框架中主要涉及到的数据结构如下：

- 1) struct dma_slave_config

定义在 include/linux/dmaengine.h 中如下：

```
1. struct dma_slave_config {
2.     enum dma_transfer_direction direction;
3.     phys_addr_t src_addr;
4.     phys_addr_t dst_addr;
5.     enum dma_slave_buswidth src_addr_width;
6.     enum dma_slave_buswidth dst_addr_width;
7.     u32 src_maxburst;
8.     u32 dst_maxburst;
9.     bool device_fc;
10.    unsigned int slave_id;
11. };
```

direction:传输方向，取值参考 enum dma_transfer_direction。

src_addr:传输方向是外设到内存或者外设到外设时，读取数据的位置。内存到设备类型的 channel 不需配置该参数。

dst_addr:传输方向是内存到外设或者外设到外设时，写入数据的位置。外设到内存类型的 channel 不需配置该参数。

src_addr_width:src 地址的宽度。

dst_addr_width:dst 地址的宽度。

src_maxburst:src 最大可传输的 burst size，单位是 src_addr_width。

dst_maxburst:dst 最大可传输的 burst size，单位是 dst_addr_width。

slave_id:外设对于 controller 的标识。

- 2) struct dma_async_tx_descriptor

定义如下：

```
1. struct dma_async_tx_descriptor {
2.     dma_cookie_t cookie;
3.     enum dma_ctrl_flags flags; /* not a 'long' to pack with
4.     dma_addr_t phys;
5.     struct dma_chan *chan;
6.     dma_cookie_t (*tx_submit)(struct dma_async_tx_descriptor
7.     int (*desc_free)(struct dma_async_tx_descriptor *tx);
8.     dma_async_tx_callback callback;
```

```

9.         void *callback_param;
10.        struct dmaengine_unmap_data *unmap;
11.        #ifdef CONFIG_ASYNC_TX_ENABLE_CHANNEL_SWITCH
12.        struct dma_async_tx_descriptor *next;
13.        struct dma_async_tx_descriptor *parent;
14.        spinlock_t lock;
15.    #endif
16.    };

```

cookie:一个整型数,用于追踪本次传输。一般情况下,controller 会在内部维护一个递增的 number,当 client 获取传输描述时都会将该 number 赋给 cookie,然后递增。

flags:enum dma_ctrl_flags 枚举。

chan:对应的 channel。

tx_submit:controller 提供的回调函数,用于把改描述符提交到待传输列表。

desc_free:controller 提供的回调函数,用于释放描述符。

callback:传输完成的回调函数

callback_param:callback 的参数。

使用 Slave-DMA API 的 DMA client 实现步骤大致如下:

1) 申请 DMA channel

```

struct dma_chan *dma_request_chan(struct device *dev, const char *name);
void dma_release_channel(struct dma_chan *chan);

```

dma_request_chan 向内核申请 dma_chan。

dma_release_channel 则是相对的释放 dma_chan。

2) 设置 DMA channel 参数

设置参数使用下面的函数

```

int dmaengine_slave_config(struct dma_chan *chan, struct dma_slave_config *config)

```

首先把配置内容用 struct dma_slave_config 表示,在使用该函数提交配置信息。

3) 获取描述符

在 DMA 启动传输之前,client 需要把传输的信息(src、dst 等)告知 controller,controller 确认后就会返回一个描述符给 client,client 使用这个描述符控制跟踪这次传输。

根据不同的传输类型,使用下面三个接口来获取描述符

```

struct dma_async_tx_descriptor *dmaengine_prep_slave_sg(
    struct dma_chan *chan, struct scatterlist *sgl,
    unsigned int sg_len, enum dma_data_direction direction,
    unsigned long flags);
struct dma_async_tx_descriptor *dmaengine_prep_dma_cyclic(
    struct dma_chan *chan, dma_addr_t buf_addr, size_t buf_len,
    size_t period_len, enum dma_data_direction direction);
struct dma_async_tx_descriptor *dmaengine_prep_interleaved_dma(
    struct dma_chan *chan, struct dma_interleaved_template *xt,
    unsigned long flags);

```

4) 提交启动传输

使用下面的函数提交描述符

```

dma_cookie_t dmaengine_submit(struct dma_async_tx_descriptor *desc)

```

使用下面的函数启动传输

```
void dma_async_issue_pending(struct dma_chan *chan);
```

5) 等待传输结束

等待传输可以通过回调函数，也可以通过 `dma_async_is_tx_complete` 等函数查询传输是否完成。另外还可以使用 `maengine_pause`、`dmaengine_resume` 函数，暂停、终止传输。

client 简单的 demo 如下：

```
1.  #include<linux/dmaengine.h>
2.  #include<linux/dma-mapping.h>
3.  #include<linux/types.h>
4.  #include<linux/slab.h>
5.  #include<linux/module.h>
6.  #include<linux/init.h>
7.  #include<linux/fs.h>
8.  #include<linux/sched.h>
9.  #include <linux/miscdevice.h>
10. #include<linux/device.h>
11. #include<linux/string.h>
12. #include<linux/errno.h>
13. #include<linux/types.h>
14. #include<linux/slab.h>
15. #include<asm/uaccess.h>
16. #include <asm/delay.h>
17.
18. #define DEVICE_NAME "ax_dma"
19. #define PL_ADDR_SRC 0x40000000
20.
21. #define MAX_SIZE (512*64)
22. static char *src;
23. static char *dst;
24. dma_addr_t dma_src;
25. dma_addr_t dma_dst;
26.
27. struct dma_chan *chan;
28. struct dma_device *dev;
29. struct dma_async_tx_descriptor *tx = NULL;
30. enum dma_ctrl_flags flags;
31. dma_cookie_t cookie;
32.
33. void dma_cb(void *dma_async_param)
34. {
35.     if(!memcmp(src, dst, MAX_SIZE))
36.     {
37.         printk("dma irq test ok\r\n");
38.     }
39. }
40.
41. static int dma_open(struct inode *inode, struct file *file)
42. {
43.     printk("dma_open\r\n");
44.     return 0;
45. }
46.
47. static int dma_release(struct inode *indoe, struct file *file)
48. {
```

```
49.     printk("dma_release\r\n");
50.     return 0;
51. }
52.
53. static ssize_t dma_read(struct file *filp, char __user *buf, si
ze_t size, loff_t *ppos)
54. {
55.     int ret = 0;
56.
57.     tx = dev->device_prep_dma_memcpy(chan, dma_dst, dma_src, MA
X_SIZE, flags);
58.     if (!tx){
59.         printk(KERN_INFO "Failed to prepare DMA memcpy");
60.     }
61.
62.     tx->callback = dma_cb;
63.     tx->callback_param = NULL;
64.     cookie = tx->tx_submit(tx); //submit the desc
65.     if (dma_submit_error(cookie)){
66.         printk("DMA tx submit failed");
67.     }
68.     dma_async_issue_pending(chan); //begin dma transfer
69.
70.     return ret;
71. }
72.
73. static struct file_operations ax_fops =
74. {
75.     .owner    = THIS_MODULE,
76.     .open     = dma_open,
77.     .read     = dma_read,
78.     .release  = dma_release,
79. };
80.
81. static struct miscdevice dma_misc =
82. {
83.     .minor = MISC_DYNAMIC_MINOR,
84.     .name  = DEVICE_NAME,
85.     .fops  = &ax_fops,
86. };
87.
88. static int __init dma_init( void )
89. {
90.     int ret=0;
91.     int i = 0;
92.     dma_cap_mask_t mask;
93.
94.     ret = misc_register(&dma_misc);
95.     if(ret)
96.     {
97.         printk("misc_register failed!\n");
98.         return 0;
99.     }
100.
101.     src = (char *)ioremap(PL_ADDR_SRC, MAX_SIZE);
102.     dma_src = PL_ADDR_SRC;
103.
104.     dst = dma_alloc_coherent(NULL, size, &dma_dst, GFP_KERNEL);
```

```
105.     dma_cap_zero(mask);
106.     dma_cap_set(DMA_SLAVE, mask);
107.     chan = dma_request_channel(mask, NULL, NULL);
108.     flags = DMA_CTRL_ACK | DMA_PREP_INTERRUPT;
109.     dev = chan->device;
110.
111.     memset(src, 0x5A, MAX_SIZE);
112.
113.     return 0;
114. }
115.
116. static void __exit dma_exit( void )
117. {
118.     misc_deregister(&dma_misc);
119.     iounmap(PL_ADDR_SRC);
120.     dma_free_coherent(NULL, MAX_SIZE, dst, &dma_dst);
121.     dma_release_channel(chan);
122. }
123.
124. //驱动入口函数标记
125. module_init(dma_init);
126. //驱动出口函数标记
127. module_exit(dma_exit);
128.
129. /* 驱动描述信息 */
130. MODULE_AUTHOR("Alinx");
131. MODULE_ALIAS("dma");
132. MODULE_DESCRIPTION("DMA driver");
133. MODULE_VERSION("v1.0");
134. MODULE_LICENSE("GPL");
```

第二十四章 多点触摸屏驱动

多点触摸屏我们并不陌生，也就是电容屏，现在的手机、pad 都是多点触摸屏。很多应用场景单点触摸屏已经不能适应了，比如手机游戏就需要同时触摸多个点。当然这些都不是我们要关心的，我们只需要弄清楚他在 linux 中驱动的实现方法。

24.1 input 子系统中的多点触摸屏

多点触摸屏也是输入设备，前面我们讲的 linux 中 input 子系统中，也有对多点触摸屏的支持。回想一下 input 子系统，input 子系统中，通过 input_handler 帮我们实现了 fops 中的方法，我们需要做的就是结合具体硬件，有数据时通过接口发送数据。实现一个输入设备驱动，步骤大致如下：

- 1.构造 input_device
- 2.设置 input_device 参数
- 3.向内核注册 input_device
- 4.当硬件接收到数据时通过 input_event 上报数据

结合上我们这里的硬件设备多点触摸屏，大致步骤如下：

- 1) 分配 input_dev 结构体

```
struct input_dev *ts_dev;  
ts_dev = input_allocate_device();
```

- 2) 设置 input_dev 结构体

设置能够产生的事件类型：

```
set_bit(EV_SYN, ts_dev->evbit); //设置按键事件  
set_bit(EV_ABS, ts_dev->evbit); //设置滑动事件
```

设置这类事件类型中能产生的具体事件：

```
set_bit(ABS_MT_TRACKING_ID, ts_dev->absbit);  
set_bit(ABS_MT_POSITION_X, ts_dev->absbit);  
set_bit(ABS_MT_POSITION_Y, ts_dev->absbit);
```

设置事件范围

```
input_set_abs_params(ts_dev, ABS_MT_TRACKING_ID, 0, MTP_MAX_ID, 0, 0);  
input_set_abs_params(ts_dev, ABS_MT_POSITION_X, 0, MTP_MAX_X, 0, 0);  
input_set_abs_params(ts_dev, ABS_MT_POSITION_Y, 0, MTP_MAX_Y, 0, 0);
```

- 3) 注册 input_dev 结构体

```
input_register_device(ts_dev);
```

- 4) 硬件操作

注册中断：

```
request_irq();
```

上报事件：

```
input_report_abs(ts_dev, ABS_MT_POSITION_X, mtp_events[i].x);  
input_report_abs(ts_dev, ABS_MT_POSITION_Y, mtp_events[i].y);  
input_report_abs(ts_dev, ABS_MT_TRACKING_ID, mtp_events[i].id);  
input_mt_sync(ts_dev);
```

24.2 多点触摸屏与 SoC 的接口

触摸屏一般会通过 IIC 或者 SPI 连接到 SoC，软件上把屏幕看成是一个平面坐标系，当我们触摸到屏幕上某个位置，触摸屏就会通过 IIC 或 SPI 发送这个点的坐标到 SoC，SoC 如何处理这个坐标这就是后话了。对于多点触摸屏来说也只是一次发送多个点而已。

当存在“多个”的概念时，就必然多了一个需要关心的属性，那就是多个之间的关系。这里我们上报也就有了两种模式，一是不关心触点间的关系直接上报位置，二是上位置的同时加上触点关系先后顺序等。

只上报触点位置：

```
ABS_MT_POSITION_X x0
ABS_MT_POSITION_Y y0
SYN_MT_REPORT
ABS_MT_POSITION_X x1
ABS_MT_POSITION_Y y1
SYN_MT_REPORT
```

上报触点位置和关系：

```
ABS_MT_TRACKING_ID 0
ABS_MT_POSITION_X x0
ABS_MT_POSITION_Y y0
ABS_MT_TRACKING_ID 1
ABS_MT_POSITION_X x1
ABS_MT_POSITION_Y y1
SYN_MT_REPORT
```

我们的开发板平台是通过 IIC 和多点触摸屏互联的，也就是说我们的多点触摸屏驱动会是 IIC 和 input 子系统的混合体。

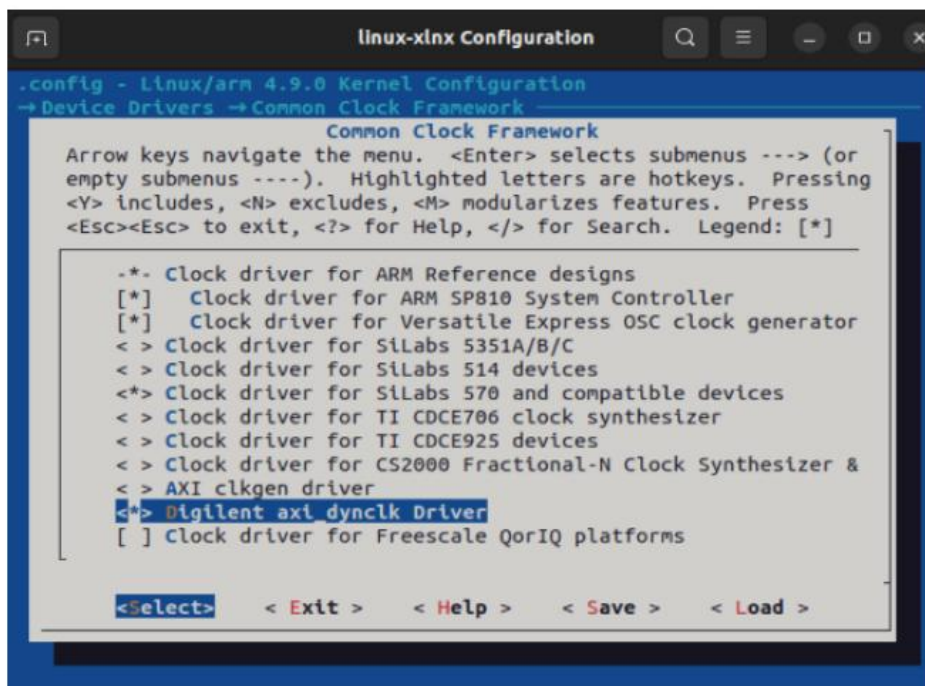
IIC 框架比较简单，我们就直接通过代码去看。

24.3 vivado 工程与 petalinux

ALINX 的平台上是通过 PL 端的扩展口和触摸屏相连的，因此需要使用 vivado 软件配置工程，然后重新制作 petalinux 工程。重建 vivado 工程和 petalinux 工程我们在 pwm 章节中已经经历过一次了。

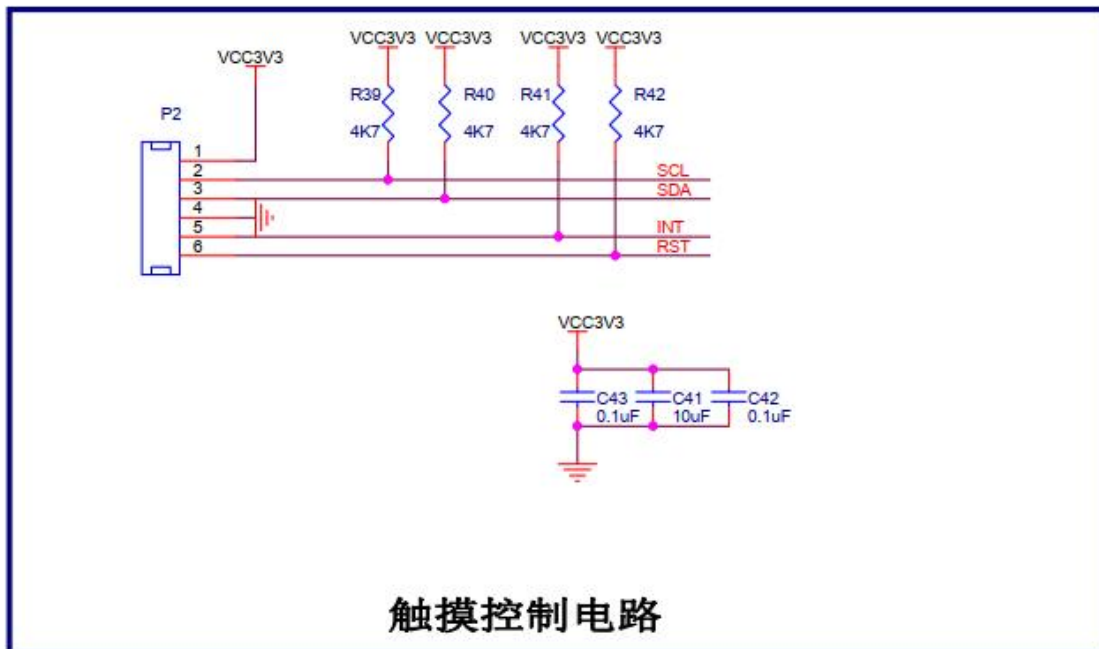
vivado 工程配置方法步骤以及 petalinux 的设置可参考 pdf 教程“course_s4_ZYNQ 那些事儿 -Linux 实验篇 V1.06”的第十四章，这里就不再重复说明了。有一点注意，我们这里的驱动都是当成模块来加载的，所以重新制作 petalinux 工程，我们只需要做其中的一步，选择 Digilent axi_dynclock Driver 即可。

- 4) 在 Device Drivers > Common Clock Framework 中选择 `<*> Digilent axi_dynclk Driver`



24.4 实验

24.4.1 原理图



触摸屏的电路很简单，一是传递数据的 I2C(SCL、SDA)，二是提醒数据的中断 INT，三是复位信号 RST。

24.4.2 设备树

把触摸屏看作是一个 IIC 的 client，把设备树节点放在 `&i2c0` 下。设备树修改方法可参考前面

的章节，别忘了要在新得 petalinux 工程中操作哦。

```

&i2c0 {
    clock-frequency = <100000>;
    alinx_an071@5D {
        compatible = "alinx,an071";
        reg = <0x5D>;
        interrupt-parent = <&intc>;
        interrupts = <0 29 4>;
    };
};

```

24.4.3 多点触摸屏驱动程序

在新的 petalinux 工程中新建驱动文件 ax-touchscreen-drv:

```
petalinux-create -t modules --name ax-touchscreen-drv --enable
```

在文件 ax-touchscreen-drv.c 中输入下面的代码:

```

1. #include <linux/kernel.h>
2. #include <linux/hrtimer.h>
3. #include <linux/i2c.h>
4. #include <linux/input.h>
5. #include <linux/module.h>
6. #include <linux/delay.h>
7. #include <linux/i2c.h>
8. #include <linux/proc_fs.h>
9. #include <linux/string.h>
10. #include <asm/uaccess.h>
11. #include <linux/vmalloc.h>
12. #include <linux/interrupt.h>
13. #include <linux/io.h>
14. #include <linux/gpio.h>
15. #include <linux/irq.h>
16. #include <linux/uaccess.h>
17.
18. #define SCREEN_MAX_X          800
19. #define SCREEN_MAX_Y          480
20. //*****PART1:ON/OFF define*****
21. #define GTP_DEBUG_ON          0
22. #define GTP_DEBUG_ARRAY_ON    0
23. #define GTP_DEBUG_FUNC_ON     0
24.
25. struct goodix_ts_data {
26.     spinlock_t irq_lock;
27.     struct i2c_client *client;
28.     struct input_dev *input_dev;
29.     struct hrtimer timer;
30.     struct work_struct work;
31.     s32 irq_is_disable;
32.     s32 use_irq;
33.     u16 abs_x_max;
34.     u16 abs_y_max;
35.     u8  int_trigger_type;
36.
37.     u8  gtp_rawdiff_mode;
38. };
39.
40. //*****PART2:TODO define *****
41. #define GTP_MAX_HEIGHT        800
42. #define GTP_MAX_WIDTH         480
43. #define GTP_INT_TRIGGER       0 // 0: Rising 1: Falling
44. #define GTP_MAX_TOUCH         5
45. //*****PART3:OTHER define*****
46. #define GTP_DRIVER_VERSION     "V2.4<2014/11/28>"
47. #define GTP_I2C_NAME           "Goodix-TS"

```

```

48. #define GTP_POLL_TIME          10
49. #define GTP_ADDR_LENGTH       2
50. #define GTP_CONFIG_MIN_LENGTH 186
51. #define GTP_CONFIG_MAX_LENGTH 240
52. #define FAIL                    0
53. #define SUCCESS                 1
54. #define SWITCH_OFF             0
55. #define SWITCH_ON              1
56.
57. // Registers define
58. #define GTP_READ_COOR_ADDR      0x814E
59. #define GTP_REG_SLEEP          0x8040
60. #define GTP_REG_SENSOR_ID      0x814A
61. #define GTP_REG_CONFIG_DATA    0x8047
62. #define GTP_REG_VERSION        0x8140
63.
64. #define RESOLUTION_LOC         3
65. #define TRIGGER_LOC            8
66.
67. // Log define
68. #define GTP_INFO(fmt,arg...)    printk("<<-GTP-INFO->> "fmt"\n",##arg)
69. #define GTP_ERROR(fmt,arg...)  printk("<<-GTP-ERROR->> "fmt"\n",##arg)
70. #define GTP_DEBUG(fmt,arg...)  do{\
71.                                if(GTP_DEBUG_ON)\
72.                                printk("<<-GTP-DEBUG->> [%d]"fmt"\n",__LINE
__, ##arg);\
73.                                }while(0)
74.
75. #define GTP_DEBUG_FUNC()        do{\
76.                                if(GTP_DEBUG_FUNC_ON)\
77.                                printk("<<-GTP-FUNC->> Func:%s@Line:%d\n",_
_func__,__LINE__);\
78.                                }while(0)
79.
80. static const char *goodix_ts_name = "goodix-ts";
81. static const char *goodix_input_phys = "input/ts";
82. static struct workqueue_struct *goodix_wq;
83. struct i2c_client * i2c_connect_client = NULL;
84.
85. u8 config[GTP_CONFIG_MAX_LENGTH + GTP_ADDR_LENGTH]
86.     = {GTP_REG_CONFIG_DATA >> 8, GTP_REG_CONFIG_DATA & 0xff};
87.
88. static s8 gtp_i2c_test(struct i2c_client *client);
89.
90. struct edt_i2c_chip_data
91. {
92.     int max_support_points;
93. };
94.
95. s32 gtp_i2c_read(struct i2c_client *client, u8 *buf, s32 len)
96. {
97.     struct i2c_msg msgs[2];
98.     s32 ret=-1;
99.     s32 retries = 0;
100.
101.     GTP_DEBUG_FUNC();
102.
103.     msgs[0].flags = !I2C_M_RD;
104.     msgs[0].addr = client->addr;
105.     msgs[0].len = GTP_ADDR_LENGTH;
106.     msgs[0].buf = &buf[0];
107.
108.     msgs[1].flags = I2C_M_RD;
109.     msgs[1].addr = client->addr;
110.     msgs[1].len = len - GTP_ADDR_LENGTH;
111.     msgs[1].buf = &buf[GTP_ADDR_LENGTH];
112.
113.     while(retries < 5)
114.     {
115.         ret = i2c_transfer(client->adapter, msgs, 2);

```

```

116.         if(ret == 2)break;
117.         retries++;
118.     }
119.     if((retries >= 5))
120.     {
121.         GTP_ERROR("I2C Read: 0x%04X, %d bytes failed, errcode: %d! Process reset.",
122.         (((u16)(buf[0] << 8)) | buf[1]), len-2, ret);
123.     }
124.     return ret;
125. }
126. s32 gtp_i2c_write(struct i2c_client *client,u8 *buf,s32 len)
127. {
128.     struct i2c_msg msg;
129.     s32 ret = -1;
130.     s32 retries = 0;
131.
132.     GTP_DEBUG_FUNC();
133.
134.     msg.flags = !I2C_M_RD;
135.     msg.addr = client->addr;
136.     msg.len = len;
137.     msg.buf = buf;
138.     //msg.scl_rate = 300 * 1000;    // for Rockchip, etc
139.
140.     while(retries < 5)
141.     {
142.         ret = i2c_transfer(client->adapter, &msg, 1);
143.         if (ret == 1)break;
144.         retries++;
145.     }
146.     if((retries >= 5))
147.     {
148.         GTP_ERROR("I2C Write: 0x%04X, %d bytes failed, errcode: %d! Process reset.",
149.         (((u16)(buf[0] << 8)) | buf[1]), len-2, ret);
150.     }
151.     return ret;
152. }
153. void gtp_irq_disable(struct goodix_ts_data *ts)
154. {
155.     unsigned long irqflags;
156.
157.     GTP_DEBUG_FUNC();
158.
159.     spin_lock_irqsave(&ts->irq_lock, irqflags);
160.     if (!ts->irq_is_disable)
161.     {
162.         ts->irq_is_disable = 1;
163.         disable_irq_nosync(ts->client->irq);
164.     }
165.     spin_unlock_irqrestore(&ts->irq_lock, irqflags);
166. }
167.
168. void gtp_irq_enable(struct goodix_ts_data *ts)
169. {
170.     unsigned long irqflags = 0;
171.
172.     GTP_DEBUG_FUNC();
173.
174.     spin_lock_irqsave(&ts->irq_lock, irqflags);
175.     if (ts->irq_is_disable)
176.     {
177.         enable_irq(ts->client->irq);
178.         ts->irq_is_disable = 0;
179.     }
180.     spin_unlock_irqrestore(&ts->irq_lock, irqflags);
181. }
182.
183. static void gtp_touch_down(struct goodix_ts_data* ts,s32 id,s32 x,s32 y,s32 w)

```

```

184. {
185.     input_report_abs(ts->input_dev, ABS_X, x);
186.     input_report_abs(ts->input_dev, ABS_Y, y);
187.     input_event(ts->input_dev, EV_KEY, BTN_TOUCH, 1);
188.     input_report_abs(ts->input_dev, ABS_PRESSURE, 1);
189. }
190.
191. static void gtp_touch_up(struct goodix_ts_data* ts, s32 id,s32 x,s32 y,s32 w)
192. {
193.     input_report_key(ts->input_dev, BTN_TOUCH, 0);
194.     input_report_abs(ts->input_dev, ABS_PRESSURE, 0);
195.     GTP_DEBUG("ID:%d, X:%d, Y:%d, W:%d", id, x, y, w);
196. }
197.
198. static void goodix_ts_work_func(struct work_struct *work)
199. {
200.     u8 end_cmd[3] = {GTP_READ_COOR_ADDR >> 8, GTP_READ_COOR_ADDR & 0xFF, 0};
201.     u8 point_data[2 + 1 + 8 * GTP_MAX_TOUCH + 1]={GTP_READ_COOR_ADDR >> 8, GTP_READ_COOR_ADDR & 0xFF};
202.     u8 touch_num = 0;
203.     u8 finger = 0;
204.     static u16 pre_touch = 0;
205.     static u8 pre_key = 0;
206.
207.     u8 key_value = 0;
208.     u8* coor_data = NULL;
209.     s32 input_x = 0;
210.     s32 input_y = 0;
211.     s32 input_w = 0;
212.     s32 id = 0;
213.     s32 i = 0;
214.     s32 ret = -1;
215.     struct goodix_ts_data *ts = NULL;
216.
217.     GTP_DEBUG_FUNC();
218.     ts = container_of(work, struct goodix_ts_data, work);
219.
220.     ret = gtp_i2c_read(ts->client, point_data, 12);
221.     if (ret < 0)
222.     {
223.         GTP_ERROR("I2C transfer error. errno:%d\n ", ret);
224.         if (ts->use_irq)
225.         {
226.             gtp_irq_enable(ts);
227.         }
228.         return;
229.     }
230.
231.     finger = point_data[GTP_ADDR_LENGTH];
232.
233.     if (finger == 0x00)
234.     {
235.         if (ts->use_irq)
236.         {
237.             gtp_irq_enable(ts);
238.         }
239.         return;
240.     }
241.
242.     if((finger & 0x80) == 0)
243.     {
244.         goto exit_work_func;
245.     }
246.
247.     touch_num = finger & 0x0f;
248.     if (touch_num > GTP_MAX_TOUCH)
249.     {
250.         goto exit_work_func;
251.     }
252.

```

```
253.     if (touch_num > 1)
254.     {
255.         u8 buf[8 * GTP_MAX_TOUCH] = {(GTP_READ_COOR_ADDR + 10) >> 8, (GTP_READ_COOR
256.         _ADDR + 10) & 0xff};
257.         ret = gtp_i2c_read(ts->client, buf, 2 + 8 * (touch_num - 1));
258.         memcpy(&point_data[12], &buf[2], 8 * (touch_num - 1));
259.     }
260.
261.     pre_key = key_value;
262.
263.     if (touch_num)
264.     {
265.         for (i = 0; i < touch_num; i++)
266.         {
267.             coor_data = &point_data[i * 8 + 3];
268.
269.             id = coor_data[0] & 0x0F;
270.             input_x = coor_data[1] | (coor_data[2] << 8);
271.             input_y = coor_data[3] | (coor_data[4] << 8);
272.             input_w = coor_data[5] | (coor_data[6] << 8);
273.
274.             {
275.                 gtp_touch_down(ts, id, input_x, input_y, input_w);
276.             }
277.         }
278.     }
279.     else if (pre_touch)
280.     {
281.         {
282.             for (i = 0; i < pre_touch; i++)
283.             {
284.                 coor_data = &point_data[i * 8 + 3];
285.
286.                 id = coor_data[0] & 0x0F;
287.                 input_x = coor_data[1] | (coor_data[2] << 8);
288.                 input_y = coor_data[3] | (coor_data[4] << 8);
289.                 input_w = coor_data[5] | (coor_data[6] << 8);
290.                 gtp_touch_up(ts, id, input_x, input_y, input_w);
291.             }
292.         }
293.     }
294.
295.     pre_touch = touch_num;
296.
297.     input_sync(ts->input_dev);
298.
299. exit_work_func:
300.     if(!ts->gtp_rawdiff_mode)
301.     {
302.         ret = gtp_i2c_write(ts->client, end_cmd, 3);
303.         if (ret < 0)
304.         {
305.             GTP_INFO("I2C write end_cmd error!");
306.         }
307.     }
308.     if (ts->use_irq)
309.     {
310.         gtp_irq_enable(ts);
311.     }
312. }
313.
314. static enum hrtimer_restart goodix_ts_timer_handler(struct hrtimer *timer)
315. {
316.     struct goodix_ts_data *ts = container_of(timer, struct goodix_ts_data, timer);
317.
318.     GTP_DEBUG_FUNC();
319.
320.     queue_work(goodix_wq, &ts->work);
```

```
321.     hrtimer_start(&ts->timer, ktime_set(0, (GTP_POLL_TIME+6)*1000000), HRTIMER_MODE
    _REL);
322.     return HRTIMER_NORESTART;
323. }
324.
325. static irqreturn_t goodix_ts_irq_handler(int irq, void *dev_id)
326. {
327.     struct goodix_ts_data *ts = dev_id;
328.
329.     GTP_DEBUG_FUNC();
330.
331.     gtp_irq_disable(ts);
332.
333.     queue_work(goodix_wq, &ts->work);
334.
335.     return IRQ_HANDLED;
336. }
337.
338. static s32 gtp_init_panel(struct goodix_ts_data *ts)
339. {
340.     if ((ts->abs_x_max == 0) && (ts->abs_y_max == 0))
341.     {
342.         ts->abs_x_max = (config[RESOLUTION_LOC + 1] << 8) + config[RESOLUTION_LOC];
343.         ts->abs_y_max = (config[RESOLUTION_LOC + 3] << 8) + config[RESOLUTION_LOC +
    2];
344.         ts->int_trigger_type = (config[TRIGGER_LOC]) & 0x03;
345.     }
346.
347.     GTP_INFO("X_MAX: %d, Y_MAX: %d, TRIGGER: 0x%02x", ts->abs_x_max, ts->abs_y_max, t
    s->int_trigger_type);
348.
349.     msleep(10);
350.     return 0;
351. }
352.
353. s32 gtp_read_version(struct i2c_client *client, u16* version)
354. {
355.     s32 ret = -1;
356.     u8 buf[8] = {GTP_REG_VERSION >> 8, GTP_REG_VERSION & 0xff};
357.
358.     GTP_DEBUG_FUNC();
359.
360.     ret = gtp_i2c_read(client, buf, sizeof(buf));
361.     if (ret < 0)
362.     {
363.         GTP_ERROR("GTP read version failed");
364.         return ret;
365.     }
366.
367.     if (version)
368.     {
369.         *version = (buf[7] << 8) | buf[6];
370.     }
371.     if (buf[5] == 0x00)
372.     {
373.         GTP_INFO("IC Version: %c%c%c_%02x%02x", buf[2], buf[3], buf[4], buf[7], buf
    [6]);
374.     }
375.     else
376.     {
377.         GTP_INFO("IC Version: %c%c%c%c_%02x%02x", buf[2], buf[3], buf[4], buf[5], b
    uf[7], buf[6]);
378.     }
379.     return ret;
380. }
381.
382. static s8 gtp_i2c_test(struct i2c_client *client)
383. {
384.     u8 test[3] = {GTP_REG_CONFIG_DATA >> 8, GTP_REG_CONFIG_DATA & 0xff};
```



```
385.     u8 retry = 0;
386.     s8 ret = -1;
387.
388.     GTP_DEBUG_FUNC();
389.
390.     while(retry++ < 5)
391.     {
392.         ret = gtp_i2c_read(client, test, 3);
393.         if (ret > 0)
394.         {
395.             return ret;
396.         }
397.         GTP_ERROR("GTP i2c test failed time %d.", retry);
398.         msleep(10);
399.     }
400.     return ret;
401. }
402.
403. static s8 gtp_request_irq(struct goodix_ts_data *ts)
404. {
405.     s32 ret = -1;
406.     unsigned long irq_flags;
407.
408.     GTP_DEBUG_FUNC();
409.     GTP_DEBUG("INT trigger type:%x", ts->int_trigger_type);
410.
411.     irq_flags = irq_get_trigger_type(ts->client->irq);
412.     if (irq_flags == IRQF_TRIGGER_NONE) irq_flags = IRQF_TRIGGER_HIGH;
413.     irq_flags |= IRQF_ONESHOT;
414.
415.     ret = devm_request_threaded_irq(&ts->client->dev, ts->client->irq, NULL, goodix
_ts_irq_handler, irq_flags, ts->client->name, ts);
416.     if (ret)
417.     {
418.         GTP_ERROR("Request IRQ failed!ERRNO:%d.", ret);
419.
420.         hrtimer_init(&ts->timer, CLOCK_MONOTONIC, HRTIMER_MODE_REL);
421.         ts->timer.function = goodix_ts_timer_handler;
422.         hrtimer_start(&ts->timer, ktime_set(1, 0), HRTIMER_MODE_REL);
423.         return -1;
424.     }
425.     else
426.     {
427.         gtp_irq_disable(ts);
428.         ts->use_irq = 1;
429.         return 0;
430.     }
431. }
432.
433. static s8 gtp_request_input_dev(struct goodix_ts_data *ts)
434. {
435.     s8 ret = -1;
436.
437.     GTP_DEBUG_FUNC();
438.
439.     ts->input_dev = input_allocate_device();
440.     if (ts->input_dev == NULL)
441.     {
442.         GTP_ERROR("Failed to allocate input device.");
443.         return -ENOMEM;
444.     }
445.
446.     __set_bit(EV_ABS, ts->input_dev->evbit);
447.     __set_bit(EV_KEY, ts->input_dev->evbit);
448.     __set_bit(EV_SYN, ts->input_dev->evbit);
449.
450.     __set_bit(BTN_TOUCH, ts->input_dev->keybit);
451.     __set_bit(ABS_X, ts->input_dev->absbit);
452.     __set_bit(ABS_Y, ts->input_dev->absbit);
453.     __set_bit(ABS_PRESSURE, ts->input_dev->absbit);
```

```
454.     input_set_abs_params(ts->input_dev, ABS_X, 0, SCREEN_MAX_X, 0, 0);
455.     input_set_abs_params(ts->input_dev, ABS_Y, 0, SCREEN_MAX_Y, 0, 0);
456.
457.     ts->input_dev->name = goodix_ts_name;
458.     ts->input_dev->phys = goodix_input_phys;
459.     ts->input_dev->id.bustype = BUS_I2C;
460.     ts->input_dev->dev.parent = &ts->client->dev;
461.
462.     input_set_drvdata(ts->input_dev, ts);
463.
464.     ret = input_register_device(ts->input_dev);
465.     if (ret)
466.     {
467.         GTP_ERROR("Register %s input device failed", ts->input_dev->name);
468.         return -ENODEV;
469.     }
470.
471.     return 0;
472. }
473.
474. static int goodix_ts_probe(struct i2c_client *client, const struct i2c_device_id *i
    d)
475. {
476.     s32 ret = -1;
477.     struct goodix_ts_data *ts;
478.     u16 version_info;
479.
480.     GTP_DEBUG_FUNC();
481.
482.     GTP_INFO("GTP Driver Version: %s", GTP_DRIVER_VERSION);
483.     GTP_INFO("GTP I2C Address: 0x%02x", client->addr);
484.
485.     i2c_connect_client = client;
486.
487.     if (!i2c_check_functionality(client->adapter, I2C_FUNC_I2C))
488.     {
489.         GTP_ERROR("I2C check functionality failed.");
490.         return -ENODEV;
491.     }
492.     ts = kzalloc(sizeof(*ts), GFP_KERNEL);
493.     if (ts == NULL)
494.     {
495.         GTP_ERROR("Alloc GFP_KERNEL memory failed.");
496.         return -ENOMEM;
497.     }
498.
499.     INIT_WORK(&ts->work, goodix_ts_work_func);
500.     ts->client = client;
501.     spin_lock_init(&ts->irq_lock);
502.
503.     i2c_set_clientdata(client, ts);
504.     ts->gtp_rawdiff_mode = 0;
505.
506.     ret = gtp_i2c_test(client);
507.     if (ret < 0)
508.     {
509.         GTP_ERROR("I2C communication ERROR!");
510.     }
511.
512.     ret = gtp_read_version(client, &version_info);
513.     if (ret < 0)
514.     {
515.         GTP_ERROR("Read version failed.");
516.     }
517.
518.     ret = gtp_init_panel(ts);
519.     if (ret < 0)
520.     {
521.         GTP_ERROR("GTP init panel failed.");
522.         ts->abs_x_max = GTP_MAX_WIDTH;
```

```
523.         ts->abs_y_max = GTP_MAX_HEIGHT;
524.         ts->int_trigger_type = GTP_INT_TRIGGER;
525.     }
526.
527.     ret = gtp_request_input_dev(ts);
528.     if (ret < 0)
529.     {
530.         GTP_ERROR("GTP request input dev failed");
531.     }
532.
533.     ret = gtp_request_irq(ts);
534.     if (ret < 0)
535.     {
536.         GTP_INFO("GTP works in polling mode.");
537.     }
538.     else
539.     {
540.         GTP_INFO("GTP works in interrupt mode.");
541.     }
542.
543.     if (ts->use_irq)
544.     {
545.         gtp_irq_enable(ts);
546.     }
547.
548.     return 0;
549. }
550.
551. static int goodix_ts_remove(struct i2c_client *client)
552. {
553.     struct goodix_ts_data *ts = i2c_get_clientdata(client);
554.
555.     GTP_DEBUG_FUNC();
556.
557.     if (ts)
558.     {
559.         if (ts->use_irq)
560.         {
561.             //free_irq(client->irq, ts);
562.         }
563.         else
564.         {
565.             hrtimer_cancel(&ts->timer);
566.         }
567.     }
568.
569.     GTP_INFO("GTP driver removing...");
570.     i2c_set_clientdata(client, NULL);
571.     input_unregister_device(ts->input_dev);
572.     kfree(ts);
573.
574.     return 0;
575. }
576.
577. static int __maybe_unused alinx_ts_suspend(struct device *dev)
578. {
579.     struct i2c_client *client = to_i2c_client(dev);
580.
581.     if (device_may_wakeup(dev)) enable_irq_wake(client->irq);
582.
583.     return 0;
584. }
585.
586. static int __maybe_unused alinx_ts_resume(struct device *dev)
587. {
588.     struct i2c_client *client = to_i2c_client(dev);
589.
590.     if (device_may_wakeup(dev)) disable_irq_wake(client->irq);
591.
592.     return 0;
```

```

593. }
594.
595. static SIMPLE_DEV_PM_OPS(alinx_ts_pm_ops, alinx_ts_suspend, alinx_ts_resume);
596.
597. static const struct edt_i2c_chip_data alinx_data =
598. {
599.     .max_support_points = 5,
600. };
601.
602.
603. static const struct of_device_id goodix_match_table[] = {
604.     {.compatible = "goodix,gt9xx", .data = &alinx_data },
605.     { },
606. };
607.
608. static const struct i2c_device_id goodix_ts_id[] = {
609.     { .name = GTP_I2C_NAME, .driver_data = (long)&alinx_data },
610.     { }
611. };
612.
613. static struct i2c_driver goodix_ts_driver = {
614.     .probe      = goodix_ts_probe,
615.     .remove     = goodix_ts_remove,
616.     .id_table   = goodix_ts_id,
617.     .driver     = {
618.         .name     = GTP_I2C_NAME,
619.         .owner    = THIS_MODULE,
620.         .of_match_table = goodix_match_table,
621.         .pm       = &alinx_ts_pm_ops,
622.     },
623. };
624.
625. static int goodix_ts_init(void)
626. {
627.     s32 ret;
628.
629.     GTP_DEBUG_FUNC();
630.     GTP_INFO("GTP driver installing...");
631.     goodix_wq = create_singlethread_workqueue("goodix_wq");
632.     if (!goodix_wq)
633.     {
634.         GTP_ERROR("Creat workqueue failed.");
635.         return -ENOMEM;
636.     }
637.
638.     ret = i2c_add_driver(&goodix_ts_driver);
639.     return ret;
640. }
641.
642. static void __exit goodix_ts_exit(void)
643. {
644.     GTP_DEBUG_FUNC();
645.     GTP_INFO("GTP driver exited.");
646.     i2c_del_driver(&goodix_ts_driver);
647.     if (goodix_wq)
648.     {
649.         destroy_workqueue(goodix_wq);
650.     }
651. }
652.
653. module_init(goodix_ts_init);
654. module_exit(goodix_ts_exit);
655.
656. MODULE_DESCRIPTION("GTP Series Driver");
657. MODULE_LICENSE("GPL");

```

驱动程序还是从入口函数开始看起。

625 行 goodix_ts_init() 函数中初始化了一个工作队列，用于添加中断处理的内容。

638 行通过 `i2c_add_driver` 添加的一个 `i2c` 驱动, 之后就是 `i2c` 框架的内容, 先看到注册的内容 `goodix_ts_driver`。

`goodix_ts_driver` 的定义在 613 行, 主要看一下 `probe` 函数, 跳转到 474 行 `goodix_ts_probe()` 函数实现。

499 行添加一个队列项到队列中, 队列项 `goodix_ts_work_func()` 实际也就是中断要处理的内容。198 行的函数实现中, 主要是对 `i2c` 返回的数据解析并处理, 具体的定义可参考触摸屏的手册。解析后我们在 `gtp_touch_down`、`gtp_touch_up` 函数中做触摸对应的操作。

527 行调用的 `gtp_request_input_dev()` 函数, 内容是注册一个输入子系统, 并且设置了点击事件滑动事件等。

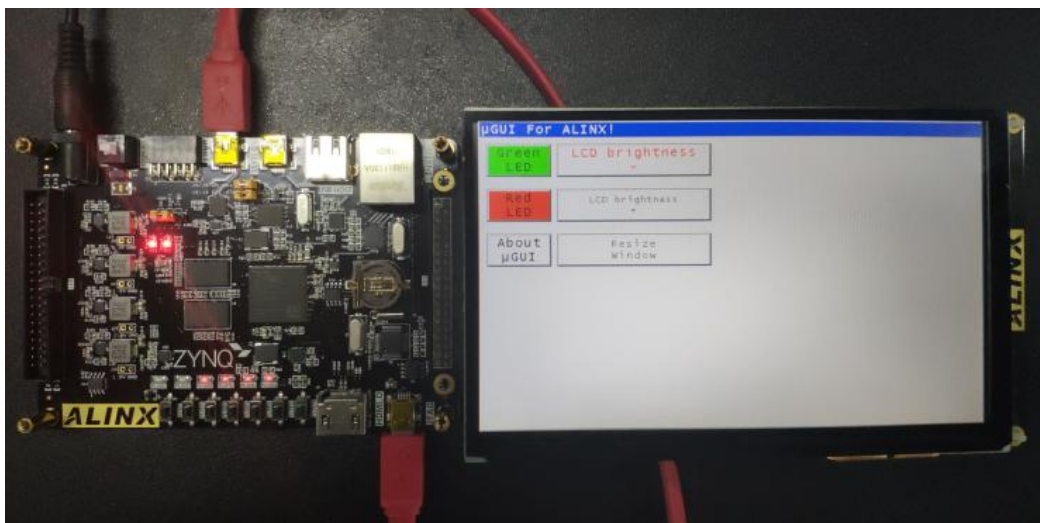
代码中还有一些参数设置就不细说了。总结下来, 就是通过 `i2c` 设置并获取触摸屏的信息, 再通过输入子系统, 把触摸行为反馈给系统, 以响应其他程序的需求。

我们把 21 行的 `GTP_DEBUG_ON` 定义为 1, 编译后在板端加载驱动, 点击屏幕, 就能看到串口中断返回触摸坐标了。

但这还体现不出 `input` 子系统的作用, 下一章我们结合 `lcd` 显示, 再确认最终的测试结果。

24.4.4 运行测试

在加载驱动之前, 需要先将 AN970 触摸屏插到板卡上。



测试方法步骤如下:

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
insmod ./ax-touchscreen-drv.ko
```

```
ax_peta:/mnt# insmod ax-touchscreen-drv.ko
ax_touchscreen_drv: loading out-of-tree module taints kernel.
<<-GTP-INFO->> GTP driver installing...
<<-GTP-INFO->> GTP Driver Version: V2.4<2014/11/28>
<<-GTP-INFO->> GTP I2C Address: 0x14
<<-GTP-INFO->> IC Version: 911_1060
<<-GTP-INFO->> X_MAX: 0, Y_MAX: 0, TRIGGER: 0x00
input: goodix-ts as /devices/soc0/axi/e0004000.i2c/i2c-0/0-005d/input/input0
<<-GTP-DEBUG->> [409]INT trigger type:0
<<-GTP-INFO->> GTP works in interrupt mode.
```

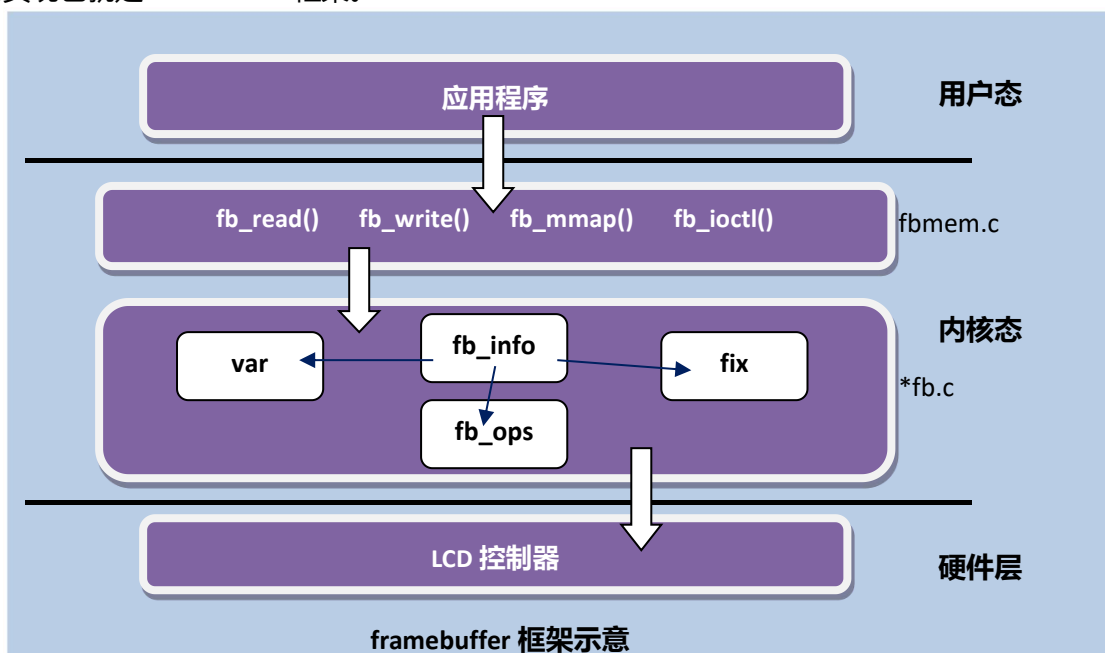
在加载完驱动后，点击屏幕，控制台会打印点击的坐标信息。

```
ax_peta:/mnt# <<-GTP-DEBUG->> [195]ID:0, X:81, Y:58, W:20
<<-GTP-DEBUG->> [195]ID:0, X:79, Y:124, W:21
<<-GTP-DEBUG->> [195]ID:0, X:94, Y:159, W:25
<<-GTP-DEBUG->> [195]ID:0, X:280, Y:121, W:23
<<-GTP-DEBUG->> [195]ID:0, X:290, Y:137, W:26
<<-GTP-DEBUG->> [195]ID:0, X:245, Y:190, W:29
<<-GTP-DEBUG->> [195]ID:0, X:271, Y:106, W:30
<<-GTP-DEBUG->> [195]ID:0, X:277, Y:124, W:25
<<-GTP-DEBUG->> [195]ID:0, X:295, Y:117, W:28
<<-GTP-DEBUG->> [195]ID:0, X:361, Y:81, W:27
<<-GTP-DEBUG->> [195]ID:0, X:408, Y:68, W:36
```

第二十五章 LCD 驱动

25.1 framebuffer 框架

Lcd 显示原理通俗的来说一般是这样:lcd 通过若干数据线连接到主控芯片(除了数据线之外,还有时钟线、同步信号线等),主控芯片分配一块内存空间用于存放显示画面数据,通过数据线发送画面数据到 lcd 屏幕显示。这一块内存空间就是帧缓冲 framebuffer。Linux 中把 framebuffer 抽象成一个设备,对 lcd 的操作最终被简化成了 framebuffer 的操作,所以 lcd 驱动在 Linux 内核中的实现也就是 framebuffer 框架。



framebuffer 框架本质上是一个字符设备,和之前讲的 misc 设备如出一辙,是对字符设备的再封装,并且创建了一个类/sys/class/graphics。对于应用层来说,使用 framebuffer 的方法大概是这样的:报头文件 linux/fb.h、打开设备文件/dev/fbn、mmap 映射、填充 framebuffer。

framebuffer 框架相关的代码在/driver/video/fbdev/core/fbmem.c 中,与具体硬件操作相关的代码则是/driver/video/fbdev 目录下*fb.c 的这些文件。fbmem.c 中提供了与用户层对接的操作函数接口以及并和*fb.c 关联,而*fb.c 则负责接口函数的具体实现。驱动开发者需要完成的也就是*fb.c 部分,xilinx 也提供了 xilinuxfb.c。参考这些已经实现的*fb.c,大概能推测实现一个 lcd 驱动的步骤大致如下:

- 1) 使用 framebuffer_alloc 函数注册一个 fb_info 结构体变量

fb_info 结构体定义在/include/linux/fb.h 中,用于描述 fb 设备。我们主要需要关心的是:可变参数 struct fb_var_screeninfo var、不可变参数 struct fb_fix_screeninfo fix、操作函数集 struct fb_ops *fbops。也是我们设置 fb_info 结构体变量的重点。

- 2) 设置 fb_info 结构体变量,包括实现 fb_ops 操作函数集

设置固定参数显存字节大小、类型、lcd 行长等。设置可变参数水平分片率、垂直分辨率、虚拟水平分辨率、虚拟垂直分辨率、像素深度、RGB 等。其他参数根据需求设置。操作函数根

据也需要实现。

3) 完成硬件相关操作

映射相关寄存器并完成硬件初始化，需要结合具体硬件。

4) 使用 register_framebuffer 函数向内核注册 fb_info 结构体变量

这一步也就可以理解为关联*fb.c 和 fbmem.c 中的操作函数接口了。

实际上 ZYNQ 中没用 framebuffer 框架，但鉴于 framebuffer 框架对于 lcd 来说还比较常用，所以这里就只简单的带过一下。当然 ZYNQ 也可以使用 fb 来实现 lcd 驱动，就拿我们的开发板来说，lcd 屏幕的接口是连接到 PL 端的，只要把 PL 端的资源配置成相应的 IO 即可当然这也是我理论上的说法，没有实际试验过，有兴趣的朋友可以自己尝试。

25.2 VDMA

VDMA 是 xilinx 提供的一个 Ip 核，是 ZYNQ 设备上显示设备工作的关键。

VDMA 好处都有啥？VDMA 可以方便的实现双缓冲或者多缓冲机制。双缓冲机制解决至少说是改善了画面闪烁、撕裂等不良效果。前面我们说了帧缓存，帧缓存中的内容会直接显示到屏幕上，当帧缓存中的数据没有准备好时，屏幕上的图像就会缺损，而双缓冲则是再帧缓存的基础上在家了一个后缓冲区 back buffer。数据现在 back buffer 中准备好，再交换前后缓冲区的显示。也就是原先的前缓冲区变成的后缓冲区，准备下一个显示的数据，而后缓冲区则变成了前缓冲区，显示图像。

硬件上 VDMA 通过 AXI_HP 接口访问 DDR。本质上 VDMA 是一个搬运数据的 IP，为数据今出 DDR 提供了便捷，VDMA 数据接口分读写通道，我们可以通过通道将 AXI-Stream 数据流写入 DDR，也可以将 DDR 中的数据以 AXI-Stream 数据流格式读出。数据进入 DDR 也就达到了我们需要将数据存入帧缓存的目的。VDMA 可以控制 32 个帧缓存，所以能够亲送实现多缓冲。

VDMA 主要由控制和状态寄存器、数据搬运模块和行缓冲分构成。数据进出 DDR 要经过行缓冲，然后由数据搬运模块读写数据。VDMA 的工作状态可以通过状态寄存器读取。

VDMA 有以下几种接口：

AXI-lite： PS 通过该接口来配置 VDMA；

AXI Memory Map write：映射到存储器写；

AXI Memory Map read：映射到存储器读；

AXI Stream Write(S2MM)：AXI Stream 视频流写入图像；

AXI Stream Read(MM2S)：AXI Stream 视频流读出图像。

VDMA 也提供了寄存器操作，具体细节可以参考 xilinx 官方提供的说明。VDMA 在 vivado 中的使用可以参考 cause4 中的触摸屏实验。

在 Linux 中使用 VDMA 实际上 xilinx 已经实现好了，在文件/drivers/dma/xilinx/xilinx_dma.c 中，实现了 zynq 支持的 axidma、cdma 以及 vdma。我们可以直接使用这个驱动，使用方法我们来看一下设备树中这一段：

```

&amba_pl {
    ax_lcd_encoder {
        compatible = "ax-drm-encoder";

        clocks = <&axi_dyncclk_0>;
        clock-names = "clk";

        port@0 {
            #address-cells = <1>;
            #size-cells = <0>;
            reg = <0>;
            hdmi_ep: endpoint {
                remote-endpoint = <&pl_disp_ep>;
            };
        };

    };

    xlnx_pl_disp {
        compatible = "xlnx,pl-disp";
        dmas = <&axi_vdma_0 0>;
        dma-names = "dma0";

        xlnx,vformat = "AR24";
        xlnx,bridge = <&v_tc_0>;

        port@0 {
            reg = <0>;
            pl_disp_ep: endpoint {
                remote-endpoint = <&hdmi_ep>;
            };
        };

    };
};

```

这段设备树在 couse4 的触摸屏例程中截取，它的作用之后再说，先看到 16 行的 `dmas = <&axi_vdma_0 0>`，这显然是对 `axi_vdma_0` 这个节点的引用，那儿这个节点在哪里？在 petalinux 工程中搜索 `axi_vdma_0`，可以找到一个 `pl.dtsi` 的设备树头文件，其中我们找到了如下节点：

```

axi_vdma_0: dma@43000000 {
    #dma-cells = <1>;
    clock-names = "s_axi_lite_aclk", "m_axi_mm2s_aclk", "m_axi_mm2s_aclk";
    clocks = <&clkc 15>, <&clkc 15>, <&clkc 15>;
    compatible = "xlnx,axi-vdma-1.00.a";
    interrupt-parent = <&intc>;
    interrupts = <0 29 4>;
    reg = <0x43000000 0x10000>;
    xlnx,addrwidth = <0x20>;
    xlnx,flush-fsync = <0x1>;
    xlnx,num-fstores = <0x1>;
    dma-channel@43000000 {
        compatible = "xlnx,axi-vdma-mm2s-channel";
        interrupts = <0 29 4>;
        xlnx,datawidth = <0x18>;
        xlnx,device-id = <0x0>;
        xlnx,genlock-mode ;
        xlnx,include-dre ;
    };
};

```

不过找到这个节点，除了能通过 `compatible` 属性找到 `vdma` 驱动的源码所在(`/drivers/dma/xilinx/xilinx_dma.c`)，对于我们使用来说没有什么意义，除非有兴趣分析 `vdma` 驱动实现的源码。

回头再看看引用节点的这段设备树，先前我们说 `vdma` 是实现 `zynq` 显示的关键，那这段设备树对应的驱动，就应该是 `zynq` 上取代 `fb` 框架的实际 `lcd` 驱动了。

25.3 DRM 框架

通过上面截取的设备树 `compatible = "xlnx,drm"`，不难发现 ZYNQ 上使用了 `drm` 框架来实现 `lcd` 显示。`framebuffer` 框架无法处理基于 GPU 的 3D 加速需求，也无法处理多个程序协同访问视频卡的情况。DRM(Direct Rendering Manager)应运而生，DRM 独占视屏卡的访问权限，由它来负责初始化并维护命令队列、视屏卡等硬件资源。要使用 GPU 的程序向 DRM 发送请求，DRM 仲裁避免冲突。同时 DRM 也负责 GPU 切换的问题。

通俗的讲 DRM 是 Linux 内核用来管理显示输出、buffer 分配的。DRM 由两个部分组成：一是内核态的子系统，这个子系统对硬件操作进行了一层框架封装。二是提供了一个用户态 `libdrm` 库，应用程序可以直接操作库里的 API 如 `ioctl` 或者是用 `framebuffer` 提供的接口进行显示相关操作。`libdrm` 库的具体使用就不展开说了。

DRM 涵盖了很多以前需要在用户空间处理的问题，如图形执行管理器 GEM、内核模式设置 KMS，这些都是属于 DRM 子系统。

GEM(Graphic Execution Manager)，主要负责显存的分配。

KMS(Kernel Mode Setting)，主要负责画面更新和设置显示参数(包括分辨率、刷新率等)。

KMS 特性中又包含这些块：Framebuffer、CRTC、Planes、Encoder、Connector。

Framebuffer 和前面 fb 框架中提到的其实是一个意思，驱动和应用都可以访问，使用前需要格式化，设置分辨率、色彩等选项。它只用于描述显存信息，不负责显存的分配，

CRTC 是 RGB 信号发生源，阴极射像管上下文，对内连接 frame buffer，对外连接 encoder。他会扫描 fb 中的内容并叠加上 planes 中的内容传送到 encoder。每个 CRTC 至少要有有一个 plane。

planes 也是一段内存地址，和 framebuffer 的区别是，它给视屏提供了高速通道，可以叠加在 framebuffer 上下，并且 planes 可以存在多个。

encoder 也就是编码器，他的工作就是把像素数据转换为显示器所需要的信号，不同的显示设备需要不同的电信号，如 DVID、VGA、MIPI 等。

connector 通常对应硬件上的连接器如 VGA、HDMI 等，同时他还保存与连接器相连的显示设备的相关信息如连接状态、DPMS 状态等。

DRM 驱动框架，也就是对上述这些特性的封装，这里只是简单过了一遍，DRM 的具体实现可以说相当复杂不是这里三言两语能说清楚的。好在 `xilinx` 也为我们提供了一套完整的 `drm` 驱动实现，在 `/driver/gpu/drm/xilinx` 文件夹中，我们可以直接使用。使用的方法也就是在设备树中添加相应的节点，具体方法可以参考内何种文档\Documentation\devicetree\bindings\drm\xilinx\xilinx_drm.txt。

我们再回头看上一节中节选的设备树代码，我们之前关注的 `axi_vdma_0` 这个节点，他是 `plane0` 这个节点下的属性，`plane0` 是我们上面说到的视频通道，也就是说 VDMA 对应的缓存通道 0 对应到了 `plane0` 视频通道。

节选设备树中的第 11 行设置编码器，这里为了对应我们的液晶屏修改成了我们定制的编码器驱动，这个几点在第三行，属性只有 `compatible`，是驱动中的 `probe` 能够执行。定制的编码器驱动代码如下：

```

1. // SPDX-License-Identifier: GPL-2.0
2. /*
3.  * Xilinx FPGA SDI Tx Subsystem driver.
4.  *
5.  * Copyright (c) 2017 Xilinx Pvt., Ltd
6.  *
7.  * Contacts: Saurabh Sengar <saurabhs@xilinx.com>
8.  */
9.
10. #include <drm/drm_atomic_helper.h>
11. #include <drm/drm_crtc_helper.h>
12. #include <drm/drmP.h>
13. #include <drm/drm_probe_helper.h>
14. #include <linux/clk.h>
15. #include <linux/component.h>
16. #include <linux/device.h>
17. #include <linux/of_device.h>
18. #include <linux/of_graph.h>
19. #include <linux/phy/phy.h>
20. #include <video/videomode.h>
21.
22. /* SDI register offsets */
23. #define XSDI_TX_RST_CTRL      0x00
24. #define XSDI_TX_MDL_CTRL     0x04
25. #define XSDI_TX_GLBL_IER     0x0C
26. #define XSDI_TX_ISR_STAT     0x10
27. #define XSDI_TX_IER_STAT     0x14
28. #define XSDI_TX_ST352_LINE   0x18
29. #define XSDI_TX_ST352_DATA_CH0 0x1C
30. #define XSDI_TX_VER          0x3C
31. #define XSDI_TX_SYS_CFG      0x40
32. #define XSDI_TX_STS_SB_TDATA 0x60
33. #define XSDI_TX_AXI4S_STS1   0x68
34. #define XSDI_TX_AXI4S_STS2   0x6C
35. #define XSDI_TX_ST352_DATA_DS2 0x70
36.
37. /* MODULE_CTRL register masks */
38. #define XSDI_TX_CTRL_M      BIT(7)
39. #define XSDI_TX_CTRL_INS_CRC BIT(12)
40. #define XSDI_TX_CTRL_INS_ST352 BIT(13)
41. #define XSDI_TX_CTRL_OVR_ST352 BIT(14)
42. #define XSDI_TX_CTRL_INS_SYNC_BIT BIT(16)
43. #define XSDI_TX_CTRL_USE_ANC_IN BIT(18)
44. #define XSDI_TX_CTRL_INS_LN   BIT(19)
45. #define XSDI_TX_CTRL_INS_EDH  BIT(20)
46. #define XSDI_TX_CTRL_MODE     0x7
47. #define XSDI_TX_CTRL_MUX      0x7
48. #define XSDI_TX_CTRL_MODE_SHIFT 4
49. #define XSDI_TX_CTRL_M_SHIFT  7
50. #define XSDI_TX_CTRL_MUX_SHIFT 8
51. #define XSDI_TX_CTRL_ST352_F2_EN_SHIFT 15
52. #define XSDI_TX_CTRL_420_BIT   BIT(21)
53. #define XSDI_TX_CTRL_INS_ST352_CHROMA BIT(23)
54. #define XSDI_TX_CTRL_USE_DS2_3GA BIT(24)
55.
56. /* TX_ST352_LINE register masks */
57. #define XSDI_TX_ST352_LINE_MASK GENMASK(10, 0)
58. #define XSDI_TX_ST352_LINE_F2_SHIFT 16
59.
60. /* ISR STAT register masks */
61. #define XSDI_GTTX_RSTDONE_INTR BIT(0)
62. #define XSDI_TX_CE_ALIGN_ERR_INTR BIT(1)
63. #define XSDI_AXI4S_VID_LOCK_INTR BIT(8)
64. #define XSDI_OVERFLOW_INTR     BIT(9)
65. #define XSDI_UNDERFLOW_INTR    BIT(10)
66. #define XSDI_IER_EN_MASK      (XSDI_GTTX_RSTDONE_INTR | \
67.                                XSDI_TX_CE_ALIGN_ERR_INTR | \
68.                                XSDI_OVERFLOW_INTR | \
69.                                XSDI_UNDERFLOW_INTR)
70.
71. /* RST_CTRL_OFFSET masks */
72. #define XSDI_TX_CTRL_EN      BIT(0)
73. #define XSDI_TX_BRIDGE_CTRL_EN BIT(8)
74. #define XSDI_TX_AXI4S_CTRL_EN BIT(9)
75. /* STS_SB_TX_TDATA masks */
76. #define XSDI_TX_TDATA_GT_RESETDONE BIT(2)
77.
78. #define XSDI_TX_MUX_SD_HD_3GA 0
79. #define XSDI_TX_MUX_3GB      1

```

```

80. #define XSDI_TX_MUX_8STREAM_6G_12G 2
81. #define XSDI_TX_MUX_4STREAM_6G 3
82. #define XSDI_TX_MUX_16STREAM_12G 4
83.
84. #define SDI_MAX_DATASTREAM 8
85. #define PIXELS_PER_CLK 2
86. #define XSDI_CH_SHIFT 29
87. #define XST352_PROG_PIC BIT(6)
88. #define XST352_PROG_TRANS BIT(7)
89. #define XST352_2048_SHIFT BIT(6)
90. #define XST352_YUV420_MASK 0x03
91. #define ST352_BYTE3 0x00
92. #define ST352_BYTE4 0x01
93. #define GT_TIMEOUT 50
94. /* SDI modes */
95. #define XSDI_MODE_HD 0
96. #define XSDI_MODE_SD 1
97. #define XSDI_MODE_3GA 2
98. #define XSDI_MODE_3GB 3
99. #define XSDI_MODE_6G 4
100. #define XSDI_MODE_12G 5
101.
102. #define SDI_TIMING_PARAMS_SIZE 48
103.
104. /**
105.  * enum payload_line_1 - Payload Ids Line 1 number
106.  * @PAYLD_LN1_HD_3_6_12G: line 1 HD,3G,6G or 12G mode value
107.  * @PAYLD_LN1_SDPAL: line 1 SD PAL mode value
108.  * @PAYLD_LN1_SDNTSC: line 1 SD NTSC mode value
109.  */
110. enum payload_line_1 {
111.     PAYLD_LN1_HD_3_6_12G = 10,
112.     PAYLD_LN1_SDPAL = 9,
113.     PAYLD_LN1_SDNTSC = 13
114. };
115.
116. /**
117.  * enum payload_line_2 - Payload Ids Line 2 number
118.  * @PAYLD_LN2_HD_3_6_12G: line 2 HD,3G,6G or 12G mode value
119.  * @PAYLD_LN2_SDPAL: line 2 SD PAL mode value
120.  * @PAYLD_LN2_SDNTSC: line 2 SD NTSC mode value
121.  */
122. enum payload_line_2 {
123.     PAYLD_LN2_HD_3_6_12G = 572,
124.     PAYLD_LN2_SDPAL = 322,
125.     PAYLD_LN2_SDNTSC = 276
126. };
127.
128. /**
129.  * struct xlnx_sdi - Core configuration SDI Tx subsystem device structure
130.  * @encoder: DRM encoder structure
131.  * @connector: DRM connector structure
132.  * @dev: device structure
133.  * @base: Base address of SDI subsystem
134.  * @mode_flags: SDI operation mode related flags
135.  * @wait_event: wait event
136.  * @event_received: wait event status
137.  * @enable_st352_chroma: Able to send ST352 packets in Chroma stream.
138.  * @enable_anc_data: Enable/Disable Ancillary Data insertion for Audio
139.  * @sdi_mode: configurable SDI mode parameter, supported values are:
140.  *     0 - HD
141.  *     1 - SD
142.  *     2 - 3GA
143.  *     3 - 3GB
144.  *     4 - 6G
145.  *     5 - 12G
146.  * @sdi_mod_prop_val: configurable SDI mode parameter value
147.  * @sdi_data_strm: configurable SDI data stream parameter
148.  * @sdi_data_strm_prop_val: configurable number of SDI data streams
149.  *     value currently supported are 2, 4 and 8
150.  * @sdi_420_in: Specifying input bus color format parameter to SDI
151.  * @sdi_420_in_val: 1 for yuv420 and 0 for yuv422
152.  * @sdi_420_out: configurable SDI out color format parameter
153.  * @sdi_420_out_val: 1 for yuv420 and 0 for yuv422
154.  * @is_frac_prop: configurable SDI fractional fps parameter
155.  * @is_frac_prop_val: configurable SDI fractional fps parameter value
156.  * @bridge: bridge structure
157.  * @height_out: configurable bridge output height parameter
158.  * @height_out_prop_val: configurable bridge output height parameter value

```

```

159. * @width_out: configurable bridge output width parameter
160. * @width_out_prop_val: configurable bridge output width parameter value
161. * @in_fmt: configurable bridge input media format
162. * @in_fmt_prop_val: configurable media bus format value
163. * @out_fmt: configurable bridge output media format
164. * @out_fmt_prop_val: configurable media bus format value
165. * @en_st352_c_prop: configurable ST352 payload on Chroma stream parameter
166. * @en_st352_c_val: configurable ST352 payload on Chroma parameter value
167. * @use_ds2_3ga_prop: Use DS2 instead of DS3 in 3GA mode parameter
168. * @use_ds2_3ga_val: Use DS2 instead of DS3 in 3GA mode parameter value
169. * @video_mode: current display mode
170. * @axi_clk: AXI Lite interface clock
171. * @sditx_clk: SDI Tx Clock
172. * @vidin_clk: Video Clock
173. */
174. struct xlnx_sdi {
175.     struct drm_encoder encoder;
176.     struct drm_connector connector;
177.     struct device *dev;
178.     void __iomem *base;
179.     u32 mode_flags;
180.     wait_queue_head_t wait_event;
181.     bool event_received;
182.     bool enable_st352_chroma;
183.     bool enable_anc_data;
184.     struct drm_property *sdi_mode;
185.     u32 sdi_mod_prop_val;
186.     struct drm_property *sdi_data_strm;
187.     u32 sdi_data_strm_prop_val;
188.     struct drm_property *sdi_420_in;
189.     bool sdi_420_in_val;
190.     struct drm_property *sdi_420_out;
191.     bool sdi_420_out_val;
192.     struct drm_property *is_frac_prop;
193.     bool is_frac_prop_val;
194.     struct xlnx_bridge *bridge;
195.     struct drm_property *height_out;
196.     u32 height_out_prop_val;
197.     struct drm_property *width_out;
198.     u32 width_out_prop_val;
199.     struct drm_property *in_fmt;
200.     u32 in_fmt_prop_val;
201.     struct drm_property *out_fmt;
202.     u32 out_fmt_prop_val;
203.     struct drm_property *en_st352_c_prop;
204.     bool en_st352_c_val;
205.     struct drm_property *use_ds2_3ga_prop;
206.     bool use_ds2_3ga_val;
207.     struct drm_display_mode video_mode;
208.     struct clk *axi_clk;
209.     struct clk *sditx_clk;
210.     struct clk *vidin_clk;
211. };
212.
213. #define connector_to_sdi(c) container_of(c, struct xlnx_sdi, connector)
214. #define encoder_to_sdi(e) container_of(e, struct xlnx_sdi, encoder)
215.
216. static const struct drm_display_mode alinx_lcd_001_mode = {
217.     .clock = 33260,
218.     .hdisplay = 800,
219.     .hsync_start = 800 + 40,
220.     .hsync_end = 800 + 40 + 128,
221.     .htotal = 800 + 40 + 128 + 88,
222.     .vdisplay = 480,
223.     .vsync_start = 480 + 10,
224.     .vsync_end = 480 + 10 + 2,
225.     .vtotal = 480 + 10 + 2 + 33,
226.     .vrefresh = 60,
227.     .flags = DRM_MODE_FLAG_NHSYNC | DRM_MODE_FLAG_NVSYNC,
228.     .type = 0,
229.     .name = "800x480",
230. };
231.
232. static int xlnx_sdi_atomic_set_property(struct drm_connector *connector,
233.     struct drm_connector_state *state,
234.     struct drm_property *property, uint64_t val)
235. {
236.     struct xlnx_sdi *sdi = connector_to_sdi(connector);
237.

```



```
238.     if (property == sdi->sdi_mode)
239.         sdi->sdi_mod_prop_val = (unsigned int)val;
240.     else if (property == sdi->sdi_data_strm)
241.         sdi->sdi_data_strm_prop_val = (unsigned int)val;
242.     else if (property == sdi->sdi_420_in)
243.         sdi->sdi_420_in_val = val;
244.     else if (property == sdi->sdi_420_out)
245.         sdi->sdi_420_out_val = val;
246.     else if (property == sdi->is_frac_prop)
247.         sdi->is_frac_prop_val = !!val;
248.     else if (property == sdi->height_out)
249.         sdi->height_out_prop_val = (unsigned int)val;
250.     else if (property == sdi->width_out)
251.         sdi->width_out_prop_val = (unsigned int)val;
252.     else if (property == sdi->in_fmt)
253.         sdi->in_fmt_prop_val = (unsigned int)val;
254.     else if (property == sdi->out_fmt)
255.         sdi->out_fmt_prop_val = (unsigned int)val;
256.     else if (property == sdi->en_st352_c_prop)
257.         sdi->en_st352_c_val = !!val;
258.     else if (property == sdi->use_ds2_3ga_prop)
259.         sdi->use_ds2_3ga_val = !!val;
260.     else
261.         return -EINVAL;
262.     return 0;
263. }
264.
265. static int xlnx_sdi_atomic_get_property(struct drm_connector *connector,
266.                                         const struct drm_connector_state *state,
267.                                         struct drm_property *property, uint64_t *val)
268. {
269.     struct xlnx_sdi *sdi = connector_to_sdi(connector);
270.
271.     if (property == sdi->sdi_mode)
272.         *val = sdi->sdi_mod_prop_val;
273.     else if (property == sdi->sdi_data_strm)
274.         *val = sdi->sdi_data_strm_prop_val;
275.     else if (property == sdi->sdi_420_in)
276.         *val = sdi->sdi_420_in_val;
277.     else if (property == sdi->sdi_420_out)
278.         *val = sdi->sdi_420_out_val;
279.     else if (property == sdi->is_frac_prop)
280.         *val = sdi->is_frac_prop_val;
281.     else if (property == sdi->height_out)
282.         *val = sdi->height_out_prop_val;
283.     else if (property == sdi->width_out)
284.         *val = sdi->width_out_prop_val;
285.     else if (property == sdi->in_fmt)
286.         *val = sdi->in_fmt_prop_val;
287.     else if (property == sdi->out_fmt)
288.         *val = sdi->out_fmt_prop_val;
289.     else if (property == sdi->en_st352_c_prop)
290.         *val = sdi->en_st352_c_val;
291.     else if (property == sdi->use_ds2_3ga_prop)
292.         *val = sdi->use_ds2_3ga_val;
293.     else
294.         return -EINVAL;
295.
296.     return 0;
297. }
298.
299. static int xlnx_sdi_drm_add_modes(struct drm_connector *connector)
300. {
301.     int num_modes = 0;
302.     struct drm_display_mode *mode;
303.     struct drm_device *dev = connector->dev;
304.
305.     mode = drm_mode_duplicate(dev, &xlnx_lcd_001_mode);
306.     drm_mode_probed_add(connector, mode);
307.     num_modes++;
308.
309.     return num_modes;
310. }
311.
312. static enum drm_connector_status xlnx_sdi_detect(struct drm_connector *connector, bool force)
313. {
314.     return connector_status_connected;
315. }
316.
```



```

317. static void xlnx_sdi_connector_destroy(struct drm_connector *connector)
318. {
319.     drm_connector_unregister(connector);
320.     drm_connector_cleanup(connector);
321.     connector->dev = NULL;
322. }
323.
324. static const struct drm_connector_funcs xlnx_sdi_connector_funcs = {
325.     .detect = xlnx_sdi_detect,
326.     .fill_modes = drm_helper_probe_single_connector_modes,
327.     .destroy = xlnx_sdi_connector_destroy,
328.     .atomic_duplicate_state = drm_atomic_helper_connector_duplicate_state,
329.     .atomic_destroy_state = drm_atomic_helper_connector_destroy_state,
330.     .reset = drm_atomic_helper_connector_reset,
331.     .atomic_set_property = xlnx_sdi_atomic_set_property,
332.     .atomic_get_property = xlnx_sdi_atomic_get_property,
333. };
334.
335. static struct drm_encoder *
336. xlnx_sdi_best_encoder(struct drm_connector *connector)
337. {
338.     return &(connector_to_sdi(connector)->encoder);
339. }
340.
341. static int xlnx_sdi_get_modes(struct drm_connector *connector)
342. {
343.     return xlnx_sdi_drm_add_modes(connector);
344. }
345.
346. static struct drm_connector_helper_funcs xlnx_sdi_connector_helper_funcs = {
347.     .get_modes = xlnx_sdi_get_modes,
348.     .best_encoder = xlnx_sdi_best_encoder,
349. };
350.
351. static void xlnx_sdi_drm_connector_create_property(struct drm_connector *base_connector)
352. {
353.     struct drm_device *dev = base_connector->dev;
354.     struct xlnx_sdi *sdi = connector_to_sdi(base_connector);
355.
356.     sdi->is_frac_prop = drm_property_create_bool(dev, 0, "is_frac");
357.     sdi->sdi_mode = drm_property_create_range(dev, 0,
358.         "sdi_mode", 0, 5);
359.     sdi->sdi_data_strm = drm_property_create_range(dev, 0,
360.         "sdi_data_stream", 2, 8);
361.     sdi->sdi_420_in = drm_property_create_bool(dev, 0, "sdi_420_in");
362.     sdi->sdi_420_out = drm_property_create_bool(dev, 0, "sdi_420_out");
363.     sdi->height_out = drm_property_create_range(dev, 0,
364.         "height_out", 2, 4096);
365.     sdi->width_out = drm_property_create_range(dev, 0,
366.         "width_out", 2, 4096);
367.     sdi->in_fmt = drm_property_create_range(dev, 0,
368.         "in_fmt", 0, 16384);
369.     sdi->out_fmt = drm_property_create_range(dev, 0,
370.         "out_fmt", 0, 16384);
371.     if (sdi->enable_st352_chroma) {
372.         sdi->en_st352_c_prop = drm_property_create_bool(dev, 0,
373.             "en_st352_c");
374.         sdi->use_ds2_3ga_prop = drm_property_create_bool(dev, 0,
375.             "use_ds2_3ga");
376.     }
377. }
378.
379. static void xlnx_sdi_drm_connector_attach_property(struct drm_connector *base_connector)
380. {
381.     struct xlnx_sdi *sdi = connector_to_sdi(base_connector);
382.     struct drm_mode_object *obj = &base_connector->base;
383.
384.     if (sdi->sdi_mode)
385.         drm_object_attach_property(obj, sdi->sdi_mode, 0);
386.
387.     if (sdi->sdi_data_strm)
388.         drm_object_attach_property(obj, sdi->sdi_data_strm, 0);
389.
390.     if (sdi->sdi_420_in)
391.         drm_object_attach_property(obj, sdi->sdi_420_in, 0);
392.
393.     if (sdi->sdi_420_out)
394.         drm_object_attach_property(obj, sdi->sdi_420_out, 0);
395.

```

```
396.     if (sdi->is_frac_prop)
397.         drm_object_attach_property(obj, sdi->is_frac_prop, 0);
398.
399.     if (sdi->height_out)
400.         drm_object_attach_property(obj, sdi->height_out, 0);
401.
402.     if (sdi->width_out)
403.         drm_object_attach_property(obj, sdi->width_out, 0);
404.
405.     if (sdi->in_fmt)
406.         drm_object_attach_property(obj, sdi->in_fmt, 0);
407.
408.     if (sdi->out_fmt)
409.         drm_object_attach_property(obj, sdi->out_fmt, 0);
410.
411.     if (sdi->en_st352_c_prop)
412.         drm_object_attach_property(obj, sdi->en_st352_c_prop, 0);
413.
414.     if (sdi->use_ds2_3ga_prop)
415.         drm_object_attach_property(obj, sdi->use_ds2_3ga_prop, 0);
416. }
417.
418. static int xlnx_sdi_create_connector(struct drm_encoder *encoder)
419. {
420.     struct xlnx_sdi *sdi = encoder_to_sdi(encoder);
421.     struct drm_connector *connector = &sdi->connector;
422.     int ret;
423.
424.     connector->interlace_allowed = true;
425.     connector->doublescan_allowed = true;
426.
427.     ret = drm_connector_init(encoder->dev, connector,
428.                             &xlnx_sdi_connector_funcs,
429.                             DRM_MODE_CONNECTOR_Unknown);
430.     if (ret) {
431.         dev_err(sdi->dev, "Failed to initialize connector with drm\n");
432.         return ret;
433.     }
434.
435.     drm_connector_helper_add(connector, &xlnx_sdi_connector_helper_funcs);
436.     drm_connector_register(connector);
437.     drm_connector_attach_encoder(connector, encoder);
438.     xlnx_sdi_drm_connector_create_property(connector);
439.     xlnx_sdi_drm_connector_attach_property(connector);
440.
441.     return 0;
442. }
443.
444. static void xlnx_sdi_set_display_enable(struct xlnx_sdi *sdi)
445. {
446.
447. }
448.
449. static void xlnx_sdi_encoder_atomic_mode_set(struct drm_encoder *encoder,
450.                                              struct drm_crtc_state *crtc_state,
451.                                              struct drm_connector_state *connector_state)
452. {
453.     struct xlnx_sdi *sdi = encoder_to_sdi(encoder);
454.
455.     sdi->video_mode = alinx_lcd_001_mode;
456. }
457.
458. static void xlnx_sdi_commit(struct drm_encoder *encoder)
459. {
460.     struct xlnx_sdi *sdi = encoder_to_sdi(encoder);
461.     long ret;
462.
463.     dev_dbg(sdi->dev, "%s\n", __func__);
464.     sdi->event_received = false;
465. }
466.
467. static void xlnx_sdi_disable(struct drm_encoder *encoder)
468. {
469.
470. }
471.
472. static const struct drm_encoder_helper_funcs xlnx_sdi_encoder_helper_funcs = {
473.     .atomic_mode_set = xlnx_sdi_encoder_atomic_mode_set,
474.     .enable = xlnx_sdi_commit,
```

```
475.     .disable      = xlnx_sdi_disable,
476. };
477.
478. static const struct drm_encoder_funcs xlnx_sdi_encoder_funcs = {
479.     .destroy = drm_encoder_cleanup,
480. };
481.
482. static int xlnx_sdi_bind(struct device *dev, struct device *master,
483.     void *data)
484. {
485.     struct xlnx_sdi *sdi = dev_get_drvdata(dev);
486.     struct drm_encoder *encoder = &sdi->encoder;
487.     struct drm_device *drm_dev = data;
488.     int ret;
489.
490.     encoder->possible_crtcs = 1;
491.
492.     drm_encoder_init(drm_dev, encoder, &xlnx_sdi_encoder_funcs,
493.         DRM_MODE_ENCODER_TMDS, NULL);
494.
495.     drm_encoder_helper_add(encoder, &xlnx_sdi_encoder_helper_funcs);
496.
497.     ret = xlnx_sdi_create_connector(encoder);
498.     if (ret) {
499.         dev_err(sdi->dev, "fail creating connector, ret = %d\n", ret);
500.         drm_encoder_cleanup(encoder);
501.     }
502.     return ret;
503. }
504.
505. static void xlnx_sdi_unbind(struct device *dev, struct device *master,
506.     void *data)
507. {
508.     struct xlnx_sdi *sdi = dev_get_drvdata(dev);
509.
510.     drm_encoder_cleanup(&sdi->encoder);
511.     drm_connector_cleanup(&sdi->connector);
512. }
513.
514. static const struct component_ops xlnx_sdi_component_ops = {
515.     .bind = xlnx_sdi_bind,
516.     .unbind = xlnx_sdi_unbind,
517. };
518.
519. static int xlnx_sdi_probe(struct platform_device *pdev)
520. {
521.     struct device *dev = &pdev->dev;
522.     struct resource *res;
523.     struct xlnx_sdi *sdi;
524.     struct device_node *vpss_node;
525.     int ret, irq;
526.     struct device_node *ports, *port;
527.     u32 nports = 0, portmask = 0;
528.
529.     sdi = devm_kzalloc(dev, sizeof(*sdi), GFP_KERNEL);
530.     if (!sdi)
531.         return -ENOMEM;
532.
533.     sdi->dev = dev;
534.
535.     platform_set_drvdata(pdev, sdi);
536.
537.
538.     /* in case all "port" nodes are grouped under a "ports" node */
539.     ports = of_get_child_by_name(sdi->dev->of_node, "ports");
540.     if (!ports) {
541.         dev_dbg(dev, "Searching for port nodes in device node.\n");
542.         ports = sdi->dev->of_node;
543.     }
544.
545.     for_each_child_of_node(ports, port) {
546.         struct device_node *endpoint;
547.         u32 index;
548.
549.         if (!port->name || of_node_cmp(port->name, "port")) {
550.             dev_dbg(dev, "port name is null or node name is not port!\n");
551.             continue;
552.         }
553.     }
```

```

554.     endpoint = of_get_next_child(port, NULL);
555.     if (!endpoint) {
556.         dev_err(dev, "No remote port at %s\n", port->name);
557.         of_node_put(endpoint);
558.         ret = -EINVAL;
559.         goto err_disable_vidin_clk;
560.     }
561.
562.     of_node_put(endpoint);
563.
564.     ret = of_property_read_u32(port, "reg", &index);
565.     if (ret) {
566.         dev_err(dev, "reg property not present - %d\n", ret);
567.         goto err_disable_vidin_clk;
568.     }
569.
570.     portmask |= (1 << index);
571.
572.     nports++;
573. }
574.
575.
576. /* initialize the wait queue for GT reset event */
577. init_waitqueue_head(&sdi->wait_event);
578.
579. /* video mode properties needed by audio driver are shared to audio
580.  * driver through a pointer in platform data. This will be used in
581.  * audio driver. The solution may be needed to modify/extend to avoid
582.  * probable error scenarios
583.  */
584.
585. sdi->video_mode = alinx_lcd_001_mode;
586.
587.
588. pdev->dev.platform_data = &sdi->video_mode;
589.
590. ret = component_add(dev, &xlnx_sdi_component_ops);
591. if (ret < 0)
592.     goto err_disable_vidin_clk;
593.
594. dev_info(sdi->dev, "alinx lcd probed\n");
595.
596. return ret;
597.
598.err_disable_vidin_clk:
599.
600.err_disable_sditx_clk:
601.
602.err_disable_axi_clk:
603.
604.
605.     return ret;
606. }
607.
608.static int xlnx_sdi_remove(struct platform_device *pdev)
609. {
610.     struct xlnx_sdi *sdi = platform_get_drvdata(pdev);
611.
612.     component_del(&pdev->dev, &xlnx_sdi_component_ops);
613.     clk_disable_unprepare(sdi->vidin_clk);
614.     clk_disable_unprepare(sdi->sditx_clk);
615.     clk_disable_unprepare(sdi->axi_clk);
616.
617.     return 0;
618. }
619.
620.static const struct of_device_id xlnx_sdi_of_match[] = {
621.     { .compatible = "ax-drm-encoder" },
622.     { }
623. };
624.MODULE_DEVICE_TABLE(of, xlnx_sdi_of_match);
625.
626.static struct platform_driver sdi_tx_driver = {
627.     .probe = xlnx_sdi_probe,
628.     .remove = xlnx_sdi_remove,
629.     .driver = {
630.         .name = "alinx_lcd",
631.         .of_match_table = xlnx_sdi_of_match,
632.     },

```

```
633. };  
634.  
635. module_platform_driver(sdi_tx_driver);  
636.  
637. MODULE_AUTHOR("alinx");  
638. MODULE_DESCRIPTION("alinx lcd");  
639. MODULE_LICENSE("GPL v2");
```

在上一章实验的基础上，把我们节选出来的那段设备树代码添加到 system-user.dtsi 中。编译上面这个驱动代码为一个 ko 模块，在系统中载入这个模块和上一章的触摸模块。运行 crous e4 中的 QT 程序(别忘了运行 QT 程序前需要配置环境)，点击上面的按钮，这样就是一套完整触摸屏的实验了。此外还可以尝试使用 xilinx 自带的 encoder 看看效果，或者更改我们解码器驱动程序中的参数再运行查看效果，加深对 encoder 的理解。

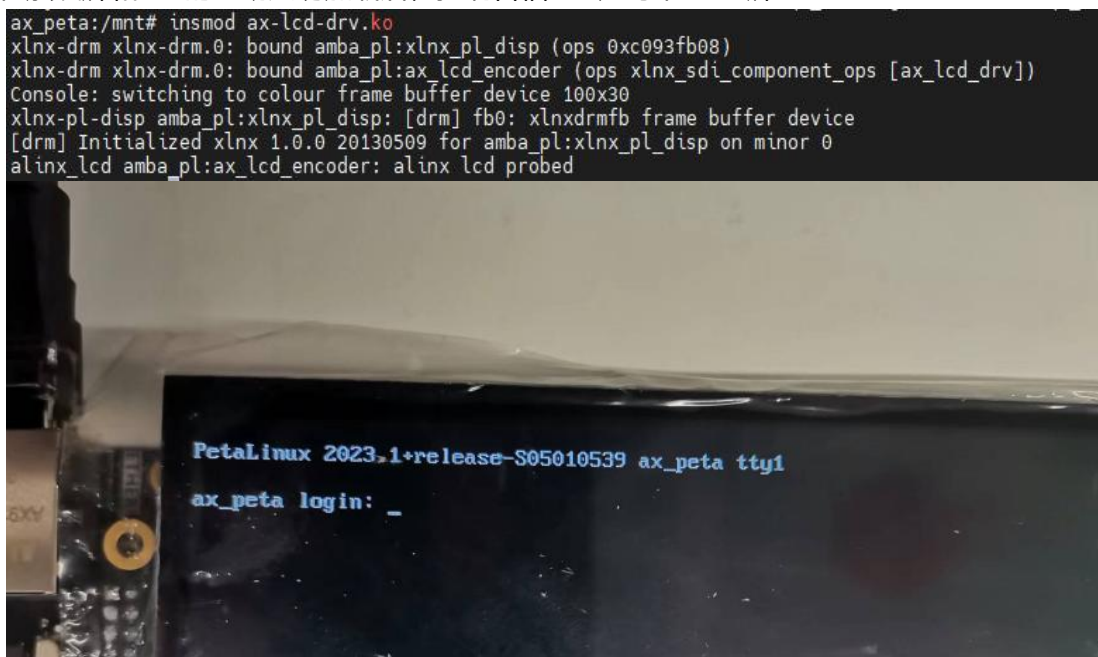
25.3 实验

25.3.1 运行测试

测试方法步骤如下：

```
mount -t nfs -o nolock 192.168.1.107:/home/alinx/work /mnt  
cd /mnt  
mkdir /tmp/qt  
mount qt_lib.img /tmp/qt  
cd /tmp/qt  
source ./qt_env_set.sh  
cd /mnt  
insmod ./ax-touchscreen-drv.ko  
insmod ./ax-lcd-drv.ko  
./qt_test
```

在触摸屏和 lcd 的驱动成功加载后，控制台信息会显示在 LCD 屏上。



运行 Qt 程序后，在 LCD 屏幕上会显示出几个按键，每个按键点击后，都会呈现出按下的效

果，同时串口也会打印出对应的坐标信息。

